

宝贵建议请发送至：wangzhenyang@itcast.cn



黑马程序员
itheima.com 上海

- 编程，始于黑马

Android 面试宝典

Version 2.5

By 阳哥

1. 内容介绍	11
2. JavaSE 基础 (★★)	11
一、Java 面向对象思想	12
1、面向对象都有哪些特性以及你对这些特性的理解	12
二、Java 中的多态	13
1、Java 中实现多态的机制是什么？	13
三、Java 的异常处理	13
1、Java 中异常分为哪些种类	13
2、调用下面的方法，得到的返回值是什么	14
四、Java 的数据类型	14
1、Java 的基本数据类型都有哪些各占几个字节	14
2、String 是基本数据类型吗？可以被继承吗？	15
五、Java 的 IO	15
1、Java 中有几种类型的流	15
2、字节流如何转为字符流	15
3、如何将一个 java 对象序列化到文件里	15
六、Java 的集合	16
1、HashMap 排序题，上机题。(本人主要靠这道题入职的第一家公司)	16
2、集合的安全性问题	18
3、ArrayList 内部用什么实现的？(2015-11-24)	19
4、并发集合和普通集合如何区别？(2015-11-24)	24
七、Java 的多线程	25

1、 多线程的两种创建方式.....	26
2、 在 java 中 wait 和 sleep 方法的不同？.....	26
3、 synchronized 和 volatile 关键字的作用.....	26
4、 分析线程并发访问代码解释原因.....	27
5、 什么是线程池，如何使用？.....	28
6、 请叙述一下您对线程池的理解？（ 2015-11-25 ）.....	29
7、 线程池的启动策略？（ 2015-11-25 ）.....	29
8、 如何控制某个方法允许并发访问线程的个数？（ 2015-11-30 ）.....	30
3. JavaSE 高级（★★）.....	32
一、 Java 中的反射.....	32
1、 说说你对 Java 中反射的理解.....	32
二、 Java 中的动态代理.....	32
1、 写一个 ArrayList 的动态代理类（笔试题）.....	32
2、 动静代理的区别，什么场景使用？（ 2015-11-25 ）.....	33
三、 Java 中的设计模式&回收机制.....	33
1、 你所知道的设计模式有哪些.....	33
2、 单例设计模式.....	33
3、 工厂设计模式.....	34
4、 建造者模式（ Builder ）.....	38
5、 适配器设计模式.....	39
6、 装饰模式（ Decorator ）.....	41
7、 策略模式（ strategy ）.....	42

8、 观察者模式 (Observer)	43
9、 JVM 垃圾回收机制和常见算法	45
10、 谈谈 JVM 的内存结构和内存分配	46
11、 Java 中引用类型都有哪些？ (重要)	47
四、 Java 的类加载器 (2015-12-2)	50
1、 Java 的类加载器的种类都有哪些？	50
2、 类什么时候被初始化？	50
4. Android 基础 (★★★)	51
一、 Android 基本常识	51
1、 写 10 个简单的 linux 命令	51
2、 书写出 android 工程的目录结构	52
3、 什么是 ANR 如何避免它？	53
4、 谈谈 Android 的优点和不足之处	53
5、 一条最长的短信息约占多少 byte?	54
6、 sim 卡的 EF 文件有何作用？ (不用看，废弃)	54
7、 如何判断是否有 SD 卡？	57
8、 dvm 的进程和 Linux 的进程, 应用程序的进程是否为同一个概念？	57
9、 Android 程序与 Java 程序的区别？	58
10、 启动应用后，改变系统语言，应用的语言会改变么？	58
11、 请介绍下 adb、ddms、aapt 的作用	58
12、 ddms 和 traceview 的区别	59
13、 补充知识：TraceView 的使用	59

14、 Android 中数据存储方式有哪些？	63
15、 DVM 和 JVM 的区别？	64
16、 谈一谈 Android 的安全机制	64
17、 Android 的四大组件都需要在清单文件中注册吗？	64
18、 在 Android 中进程的级别有哪些？	65
二、 Activity	65
1、 什么是 Activity?	65
2、 请描述一下 Activity 生命周期	65
3、 Activity 的状态都有哪些？	66
4、 如何保存 Activity 的状态？	66
5、 两个 Activity 之间跳转时必然会执行的是哪几个方法？（重要）	67
6、 横竖屏切换时 Activity 的生命周期	67
7、 如何将一个 Activity 设置成窗口的样式？	68
8、 如何退出 Activity？如何安全退出已调用多个 Activity 的 Application？	68
9、 请描述一下 Activity 的启动模式都有哪些以及各自的特点	69
10、 一个启动模式为 singleTop 的 activity，如果再试图启动会怎样？ 面试官想问的是 onNewIntent() (2015-11-24)	79
三、 Service	79
1、 Service 是否在 main thread 中执行, service 里面是否能执行耗时的操作?	79
2、 Activity 怎么和 Service 绑定，怎么在 Activity 中启动自己对应的 Service？	80
3、 请描述一下 Service 的生命周期	80
4、 什么是 IntentService？有何优点？	81

5、说说 Activity、Intent、Service 是什么关系.....	84
6、Service 和 Activity 在同一个线程吗.....	84
7、Service 里面可以弹吐司么？.....	84
8、如何让一个 Service 成为前置进程？.....	85
9、Service 的 onStartCommand 方法有几种返回值？各代表什么意思？.....	85
10、Service 的 onRebind (Intent) 方法在什么情况下会执行？.....	86
11、Activity 调用 Service 中的方法都有哪些方式？.....	86
12、Activity 如何给 Service 发送 Message？(2015.10.18)	89
13、Service 如何给 Activity 发送 Message？(2015.10.18)	90
四、BroadcastReceiver.....	93
1、请描述一下 BroadcastReceiver.....	94
2、在 manifest 和代码中如何注册和使用 BroadcastReceiver.....	94
3、BroadcastReceiver 的生命周期.....	95
4、如何让自己的广播只让指定的 app 接收 (2015.09.02)	95
5、什么是最终广播接收者？.....	96
6、广播的优先级对无序广播生效吗？.....	96
7、动态注册的广播优先级谁高？.....	96
8、如何判断当前 BroadcastReceiver 接收到的是有序广播还是无序广播？(2015-10-16)	97
五、ContentProvider&数据库.....	97
1、请介绍下 ContentProvider 是如何实现数据共享的？.....	97
2、为什么要用 ContentProvider？它和 sql 的实现上有什么差别？.....	97
3、说说 ContentProvider、ContentResolver、ContentObserver 之间的关系.....	98

4、 如何访问 asserts 资源目录下的数据库？	98
5、 如何在高并发下进行数据库查询？（2015-11-25）	99
六、 Android 中的布局	99
1、 Android 中常用的布局都有哪些	99
2、 谈谈 UI 中， Padding 和 Margin 有什么区别？	99
3、 使用权重如何让一个控件的宽度为父控件的 1/3？	100
4、 Android 中布局的优化措施都有哪些？	100
5、 android:layout_gravity 和 android:gravity 的区别？	101
七、 ListView	101
1、 ListView 如何提高其效率？	101
2、 ViewHolder 为什么要声明为静态类？	102
3、 在 Activity 中使用 Handler 的时候如何去除警告信息？	102
4、 谈谈 ListView 中的 MVC 思想？	102
5、 ListView 使用了哪些设计模式？	102
6、 当 ListView 数据集改变后，如何更新 ListView？	102
7、 ListView 如何实现分页加载	103
8、 ListView 可以显示多种类型的条目吗？	103
9、 ListView 如何定位到指定位置	103
10、 如何在 ScrollView 中如何嵌入 ListView	104
11、 ListView 中如何优化图片	106
12、 ListView 中图片错位的问题是如何产生的	107
13、 scrollView 嵌套 listview 方式除了测量还有什么方法？（2015-11-29）	107

八、 JNI&NDK	109
1、 在 Android 中如何调用 C 语言	109
2、 请介绍一下 NDK	110
3、 JNI 调用常用的两个参数	110
九、 Android 中的网络访问	110
1、 Android 中如何访问网络	110
2、 如何解析服务器传来的 JSON 文件	111
3、 如何解析服务器传来的 XML 格式数据	113
4、 如何从网络上加载一个图片显示到界面	115
5、 如何播放网络视频	115
6、 常见的访问网络 API 都有哪些？	115
十、 Intent	116
1、 Intent 传递数据时，可以传递哪些类型数据？	116
2、 Serializable 和 Parcelable 的区别	116
3、 请描述一下 Intent 和 IntentFilter	117
4、 under what condition could the code sample below crash your application?How would you modify the code to avoid this potential problem?Explain your answer?(重要)	119
5、 what are Activity and Fragment?where and why should you use one over the other?	120
6、 can you use an intent to provide data to a ContentProvider ? if not ,what would be the proper mechanism for doing this?	120
十一、 Fragment	120
1、 Fragment 跟 Activity 之间是如何传值的？	120

2、 描述一下 Fragment 的生命周期.....	121
3、 Fragment 的 replace 和 add 方法的区别 (2015.8.30)	121
4、 Fragment 如何实现类似 Activity 栈的压栈和出栈效果的？ (2015.8.30)	122
5. Android 高级 (★★★)	124
一、 Android 性能优化.....	124
1、 如何对 Android 应用进行性能分析.....	124
2、 什么情况下会导致内存泄露.....	126
3、 如何避免 OOM 异常.....	129
4、 Android 中如何捕获未捕获的异常(重要).....	131
5、 Android 性能优化博客.....	133
二、 Android 屏幕适配.....	135
1、 屏幕适配方式都有哪些.....	135
2、 屏幕适配的处理技巧都有哪些.....	140
3、 dp 和 px 之间的关系.....	143
三、 AIDL.....	144
1、 什么是 AIDL 以及如何使用.....	144
四、 自定义控件.....	144
1、 如何自定义一个控件.....	145
2、 请描述一下 View 的绘制流程.....	146
五、 Android 中的事件处理.....	148
1、 Handler 机制.....	148
2、 事件分发机制.....	148

3、在 Android 中主线程如何给子线程发 Message ? (2015-12-1) 重要	151
六、Android 签名	153
1、简单描述下 Android 数字签名	153
2、使用 Eclipse 如何生成数字签名	154
七、Android 中的动画	154
1、Android 中的动画有哪几类，它们的特点和区别是什么	155
2、如何修改 Activity 进入和退出动画	155
八、其他知识 (集大众智慧)	155
1、AsyncTask 如何使用	156
2、都使用过哪些框架、平台	160
3、Glide 原理 (2015-11-29)	160
4、Android 四大著名图片处理框架 (2015-11-29)	162
5、都使用过哪些自定义控件	172
6、Android 程序员进阶必备网站 (2015-11-29)	172
7、volley 的原理 (2015-11-29)	178
8、okhttp 的原理 (2015-11-29)	183
9、ViewPagerindicator 的原理 (2015-11-29)	184
10、slidingmenu 的原理 (2015-11-29)	184
6. Android 项目 (★★★)	185
1、如何让 LinearLayout 自动换行如下图的颜色分类所示。	185
2、ImageLoader 在项目中的使用	186
3、Java 和 javaScript 互相调用 (webview 和 js 的互相调用)	187

4、 PopupWindow 弹出层在项目中的使用.....	188
5、 Notification 在 Android 中的使用.....	189
6、 带索引的 ListView 在 Android 中的应用.....	190
7、 随手势滑动而消失 Activity 的使用.....	190
8、 TouchGallery 在 Android 中的应用.....	192
9、 TextView 显示富文本.....	192
10、 CircleImageView 实现圆形图片.....	193
11、 网易新闻客户端频道管理的实现 (2015-11-25)	194
12、 Android 瀑布流的实现.....	195
13、 监听键盘事件 (2015-11-29)	198
14、 可以按照字母排序的 ListView (2015-11-29)	201
15、 省市区三级联动.....	202
16、 购物客户端二级菜单.....	203
17、 二维码扫描.....	204
18、 微信图片选择器.....	204
7. 面试实战记录 (★★)	206
挑战公司 No.1 : 上海创梯信息科技有限公司.....	206
挑战公司 No.2 : 上海复深蓝信息技术有限公司.....	210
挑战公司 No.3 : 中阜投融资产管理股份有限公司---中投融.....	221
挑战公司 No.4 : 上海游竞网络科技有限公司---PLU.....	228
挑战公司 No.5 : 一号店旗下壹药网.....	236
挑战公司 No.6 : 聚信租赁.....	246

Android 面试宝典-V2.5

1. 内容介绍

该面试宝典不仅收录了本人亲身面试遇到的问题，还收录了从黑马学子那里收集过来的问题。在以后的工作中本人也会不断的更新和充实该面试宝典，当然也希望大家能够多多奉献比较优质的面试题。

该面试宝典不仅展示了常见的面试问题以及回答技巧，还详细讲解了每一道题所包含的知识点，让黑马学子不仅知其然，更知其所以然。

本人的面试实战记录发布在黑马论坛：<http://bbs.itheima.com/thread-196394-1-1.html>

大家可以访问上面的网址，通过阳哥的实战记录略微感知一下真实面试的情况，从中学习一些面试技巧以便让自己在未来的面试中能够得心应手，顺利拿到自己喜欢的 offer。

注意：该面试宝典仅供参考，本人无法对其所有内容的正确性和准确性做保证。由于本人的知识水平有限加之编写时间仓促因此难免有不少 bug 的存在，希望大家见谅。

如果您有不错的知识或者面试题，您可以发送面试题到 wangzhenyang@itcast.cn，本人将不胜感激。让天下没有难学的知识，希望你我的努力能帮到更多的莘莘学子。

2. JavaSE 基础 (★★)

有人可能会问对于我们学 Android 的同学来讲，面试还会问 Java 基础吗？答案是会的，但是不会太多，因此我

给了两颗星的重要程度。一般笔试的时候出现 java 基础题的概率比较大，口头面试的时候比较少，比如自己在面试的时候一些对基础知识比较看重的面试官会深究着 Java 基础去问，比如问你异常的类型以及处理方式，集合的体系结构等等。

一、Java 面向对象思想

1、面向对象都有哪些特性以及你对这些特性的理解

◆ 继承：继承是从已有类得到继承信息创建新类的过程。提供继承信息的类被称为父类（超类、基类）；得到继承信息的类被称为子类（派生类）。继承让变化中的软件系统有了一定的延续性，同时继承也是封装程序中可变因素的重要手段。

◆ 封装：通常认为封装是把数据和操作数据的方法绑定起来，对数据的访问只能通过已定义的接口。面向对象的本质就是将现实世界描绘成一系列完全自治、封闭的对象。我们在类中编写的方法就是对实现细节的一种封装；我们编写一个类就是对数据和数据操作的封装。可以说，封装就是隐藏一切可隐藏的东西，只向外界提供最简单的编程接口。

◆ 多态性：多态性是指允许不同子类型的对象对同一消息作出不同的响应。简单的说就是用同样的对象引用调用同样的方法但是做了不同的事情。多态性分为编译时的多态性和运行时的多态性。如果将对象的方法视为对象向外界提供的服务，那么运行时的多态性可以解释为：当 A 系统访问 B 系统提供的服务时，B 系统有多种提供服务的方式，但一切对 A 系统来说都是透明的。方法重载（overload）实现的是编译时的多态性（也称为前绑定），而方法重写（override）实现的是运行时的多态性（也称为后绑定）。运行时的多态是面向对象最精髓的东西，要实现多态需要做两件事：1. 方法重写（子类继承父类并重写父类中已有的或抽象的方法）；2. 对象造型（用父类型引用引用子类型对象，这样同样的引用调用同样的方法就会根据子类对象的不同而表现出不同的行为）。

◆ 抽象：抽象是将一类对象的共同特征总结出来构造类的过程，包括数据抽象和行为抽象两方面。抽象只关注对象有哪些属性和行为，并不关注这些行为的细节是什么。

二、Java 中的多态

1、Java 中实现多态的机制是什么？

靠的是父类或接口定义的引用变量可以指向子类或具体实现类的实例对象，而程序调用的方法在运行期才动态绑定，就是引用变量所指向的具体实例对象的方法，也就是内存里正在运行的那个对象的方法，而不是引用变量的类型中定义的方法。

三、Java 的异常处理

1、Java 中异常分为哪些种类

1) 按照异常需要处理的时机分为编译时异常也叫 `CheckedException` 和运行时异常也叫 `RuntimeException`。只有 java 语言提供了 `Checked` 异常，Java 认为 `Checked` 异常都是可以被处理的异常，所以 Java 程序必须显式处理 `Checked` 异常。如果程序没有处理 `Checked` 异常，该程序在编译时就会发生错误无法编译。这体现了 Java 的设计哲学：没有完善错误处理的代码根本没有机会被执行。对 `Checked` 异常处理方法有两种：

- 1 当前方法知道如何处理该异常，则用 `try...catch` 块来处理该异常。
- 2 当前方法不知道如何处理，则在定义该方法是声明抛出该异常。

运行时异常只有当代码在运行时才发行的异常，编译时不需要 `try catch`。Runtime 如除数是 0 和数组下标越界等，其产生频繁，处理麻烦，若显示申明或者捕获将会对程序的可读性和运行效率影响很大。所以由系统自动检测并将它们交给缺省的异常处理程序。当然如果你有处理要求也可以显示捕获它们。

2、调用下面的方法，得到的返回值是什么

```
1 public int getNum(){
2     try {
3         int a = 1/0;
4         return 1;
5     } catch (Exception e) {
6         return 2;
7     } finally{
8         //int b = 1/0;
9         return 3;
10    }
```

代码在走到第 3 行的时候抛了一个 `MathException`，这时第四行的代码就不会执行了，代码直接跳转到 `catch` 语句中，走到第 6 行的时候，异常机制有这么一个原则如果在 `catch` 中遇到了 `return` 或者异常等能使该函数终止的话那么用 `finally` 就必须先执行完 `finally` 代码块里面的代码然后再返回值。因此代码又跳到第 8 行，可惜第 8 行是一个 `return` 语句，那么这个时候方法就结束了，因此第 6 行的返回结果就无法被真正返回。如果 `finally` 仅仅是处理了一个释放资源的操作，那么该道题最终返回的结果就是 2。

因此上面返回值是 3。

四、Java 的数据类型

1、Java 的基本数据类型都有哪些各占几个字节

Java 有 8 种基本数据类型

byte 1

char 2

short 2

int 4

float 4

double 8

long 8

boolean 1 (boolean 类型比较特别可能只占一个 bit , 多个 boolean 可能共同占用一个字节)

2、String 是基本数据类型吗？可以被继承吗？

String 是引用类型，底层用 char 数组实现的。因为 String 是 final 类，在 java 中被 final 修饰的类不能被继承，因此 String 当然不可以被继承。

五、Java 的 IO

1、Java 中有几种类型的流

字节流和字符流。字节流继承于 InputStream 和 OutputStream，字符流继承于 InputStreamReader 和 OutputStreamWriter。

2、字节流如何转为字符流

字节输入流转字符输入流通过 InputStreamReader 实现，该类的构造函数可以传入 InputStream 对象。

字节输出流转字符输出流通过 OutputStreamWriter 实现，该类的构造函数可以传入 OutputStream 对象。

3、如何将一个 java 对象序列化到文件里

在 java 中能够被序列化的类必须先实现 Serializable 接口，该接口没有任何抽象方法只是起到一个标记作用。

```
//对象输出流
ObjectOutputStream objectOutputStream = new ObjectOutputStream(new
FileOutputStream(new File("D://obj")));
objectOutputStream.writeObject(new User("zhangsan", 100));
objectOutputStream.close();
//对象输入流
ObjectInputStream objectInputStream = new ObjectInputStream(new
FileInputStream(new File("D://obj")));
User user = (User)objectInputStream.readObject();
System.out.println(user);
objectInputStream.close();
```

六、Java 的集合

1、HashMap 排序题，上机题。（本人主要靠这道题入职的第一家公司）

已知一个 `HashMap<Integer, User>` 集合，`User` 有 `name (String)` 和 `age (int)` 属性。请写一个方法实现对 `HashMap` 的排序功能，该方法接收 `HashMap<Integer, User>` 为形参，返回类型为 `HashMap<Integer, User>`，要求对 `HashMap` 中的 `User` 的 `age` 倒序进行排序。排序时 `key=value` 键值对不得拆散。

Tips：要做出这道题必须对集合的体系结构非常的熟悉。`HashMap` 本身就是不可排序的，但是该道题偏偏让给 `HashMap` 排序，那我们就得想在 API 中有没有这样的 `Map` 结构是有序的，`LinkedHashMap`，对的，就是他，他是 `Map` 结构，也是链表结构，有序的，更可喜的是他是 `HashMap` 的子类，我们返回 `LinkedHashMap<Integer, User>` 即可，还符合面向接口（父类编程的思想）。

但凡是对集合的操作，我们应该保持一个原则就是能用 JDK 中的 API 就有 JDK 中的 API，比如排序算法我们不应该去用冒泡或者选择，而是首先想到用 `Collections` 集合工具类。

```
public class HashMapTest {
    public static void main(String[] args) {
        HashMap<Integer, User> users = new HashMap<>();
        users.put(1, new User("张三", 25));
        users.put(3, new User("李四", 22));
        users.put(2, new User("王五", 28));
        System.out.println(users);
        HashMap<Integer, User> sortHashMap = sortHashMap(users);
        System.out.println(sortHashMap);
        /**
         * 控制台输出内容
         * {1=User [name=张三, age=25], 2=User [name=王五, age=28], 3=User [name=李四,
age=22]}
         * {2=User [name=王五, age=28], 1=User [name=张三, age=25], 3=User [name=李四,
age=22]}
         */
    }

    public static HashMap<Integer, User> sortHashMap(HashMap<Integer, User> map) {
        // 首先拿到 map 的键值对集合
        Set<Entry<Integer, User>> entrySet = map.entrySet();

        // 将 set 集合转为 List 集合，为什么，为了使用工具类的排序方法
        List<Entry<Integer, User>> list = new ArrayList<Entry<Integer,
User>>(entrySet);
    }
}
```

```
// 使用 Collections 集合工具类对 list 进行排序，排序规则使用匿名内部类来实现
Collections.sort(list, new Comparator<Entry<Integer, User>>() {

    @Override
    public int compare(Entry<Integer, User> o1, Entry<Integer, User> o2) {
        //按照要求根据 User 的 age 的倒序进行排
        return o2.getValue().getAge()-o1.getValue().getAge();
    }
});
//创建一个新的有序的 HashMap 子类的集合
LinkedHashMap<Integer, User> linkedHashMap = new LinkedHashMap<Integer,
User>();
//将 List 中的数据存储在 LinkedHashMap 中
for(Entry<Integer, User> entry : list){
    linkedHashMap.put(entry.getKey(), entry.getValue());
}
//返回结果
return linkedHashMap;
}
```

2、集合的安全性问题

请问 ArrayList、HashSet、HashMap 是线程安全的吗？如果不是我想要线程安全的集合怎么办？

我们都看过上面那些集合的源码（如果没有那就看看吧），每个方法都没有加锁，显然都是线程不安全的。话又说过来如果他们安全了也就没第二问了。

在集合中 Vector 和 Hashtable 倒是线程安全的。你打开源码会发现其实就是把各自核心方法添加上了 **synchronized** 关键字。

Collections 工具类提供了相关的 API，可以让上面那 3 个不安全的集合变为安全的。

```
// Collections.synchronizedCollection(c)
// Collections.synchronizedList(list)
// Collections.synchronizedMap(m)
// Collections.synchronizedSet(s)
```

上面几个函数都有对应的返回值类型，传入什么类型返回什么类型。打开源码其实实现原理非常简单，就是将集合的核心方法添加上了 **synchronized** 关键字。

3、ArrayList 内部用什么实现的？（2015-11-24）

（回答这样的问题，不要只回答个皮毛，可以再介绍一下 ArrayList 内部是如何实现数组的增加和删除的，因为数组在创建的时候长度是固定的，那么就有个问题我们往 ArrayList 中不断的添加对象，它是如何管理这些数组呢？）

ArrayList 内部是用 Object[]实现的。接下来我们分别分析 ArrayList 的构造、add、remove、clear 方法的实现原理。

一、构造函数

1) 空参构造

```
/**
 * Constructs a new {@code ArrayList} instance with zero initial capacity.
 */
public ArrayList() {
    array = EmptyArray.OBJECT;
}
```

array 是一个 Object[]类型。当我们 new 一个空参构造时系统调用了 EmptyArray.OBJECT 属性，EmptyArray 仅仅是一个系统的类库，该类源码如下：

```
public final class EmptyArray {
    private EmptyArray() {}

    public static final boolean[] BOOLEAN = new boolean[0];
    public static final byte[] BYTE = new byte[0];
    public static final char[] CHAR = new char[0];
    public static final double[] DOUBLE = new double[0];
    public static final int[] INT = new int[0];

    public static final Class<?>[] CLASS = new Class[0];
    public static final Object[] OBJECT = new Object[0];
    public static final String[] STRING = new String[0];
    public static final Throwable[] THROWABLE = new Throwable[0];
    public static final StackTraceElement[] STACK_TRACE_ELEMENT = new StackTraceElement[0];
}
```


也就是说当我们 new 一个空参 ArrayList 的时候，系统内部使用了一个 new Object[0]数组。

2) 带参构造 1

```
/**
 * Constructs a new instance of {@code ArrayList} with the specified
 * initial capacity.
 *
 * @param capacity
 *         the initial capacity of this {@code ArrayList}.
 */
public ArrayList(int capacity) {
    if (capacity < 0) {
        throw new IllegalArgumentException("capacity < 0: " + capacity);
    }
    array = (capacity == 0 ? EmptyArray.OBJECT : new Object[capacity]);
}
```

该构造函数传入一个 int 值，该值作为数组的长度值。如果该值小于 0，则抛出一个运行时异常。如果等于 0，则使用一个空数组，如果大于 0，则创建一个长度为该值的新数组。

3) 带参构造 2

```
/**
 * Constructs a new instance of {@code ArrayList} containing the elements of
 * the specified collection.
 *
 * @param collection
 *         the collection of elements to add.
 */
public ArrayList(Collection<? extends E> collection) {
    if (collection == null) {
        throw new NullPointerException("collection == null");
    }

    Object[] a = collection.toArray();
    if (a.getClass() != Object[].class) {
        Object[] newArray = new Object[a.length];
        System.arraycopy(a, 0, newArray, 0, a.length);
        a = newArray;
    }
    array = a;
    size = a.length;
}
```

如果调用构造函数的时候传入了一个 Collection 的子类，那么先判断该集合是否为 null，为 null 则抛出空指针异常。如果不是则将该集合转换为数组 a，然后将该数组赋值为成员变量 array，将该数组的长度作为成员变量 size。这里面它先判断 a.getClass 是否等于 Object[].class，其实一般都是相等的，我也暂时没想明白为什么多加了这个判断，toArray 方法是 Collection 接口定义的，因此其所有的子类都有这样的方法，list 集合的 toArray 和 Set 集合的 toArray 返回的都是 Object[] 数组。

这里讲些题外话，其实在看 Java 源码的时候，作者的很多意图都很费人心思，我能知道他的目标是啥，但是不知道他为何这样写。比如对于 ArrayList，array 是他的成员变量，但是每次在方法中使用该成员变量的时候作者都会重新在方法中开辟一个局部变量，然后给局部变量赋值为 array，然后再使用，有人可能说这是为了防止并发修改 array，毕竟 array 是成员变量，大家都可以使用因此需要将 array 变为局部变量，然后再使用，这样的说法并不是都成立的，也许有时候就是老外们写代码的一个习惯而已。

二、add 方法

add 方法有两个重载，这里只研究最简单的那个。

```
/**
 * Adds the specified object at the end of this {@code ArrayList}.
 *
 * @param object
 *         the object to add.
 * @return always true
 */
@Override public boolean add(E object) {
    Object[] a = array;
    int s = size;
    if (s == a.length) {
        Object[] newArray = new Object[s +
            (s < (MIN_CAPACITY_INCREMENT / 2) ?
                MIN_CAPACITY_INCREMENT : s >> 1)];
        System.arraycopy(a, 0, newArray, 0, s);
        array = a = newArray;
    }
    a[s] = object;
    size = s + 1;
    modCount++;
}
```

```
return true;
}
```

- 1、首先将成员变量 array 赋值给局部变量 a，将成员变量 size 赋值给局部变量 s。
- 2、判断集合的长度 s 是否等于数组的长度（如果集合的长度已经等于数组的长度了，说明数组已经满了，该重新分配新数组了），重新分配数组的时候需要计算新分配内存的空间大小，如果当前的长度小于 MIN_CAPACITY_INCREMENT/2（这个常量值是 12，除以 2 就是 6，也就是如果当前集合长度小于 6）则分配 12 个长度，如果集合长度大于 6 则分配当前长度 s 的一半长度。这里面用到了三元运算符和位运算， $s >> 1$ ，意思就是将 s 往右移 1 位，相当于 $s=s/2$ ，只不过位运算是效率最高的运算。
- 3、将新添加的 object 对象作为数组的 a[s]个元素。
- 4、修改集合长度 size 为 s+1
- 5、modCount++，该变量是父类中声明的，用于记录集合修改的次数，记录集合修改的次数是为了防止在用迭代器迭代集合时避免并发修改异常，或者说用于判断是否出现并发修改异常的。
- 6、return true，这个返回值意义不大，因为一直返回 true，除非报了一个运行时异常。

三、remove 方法

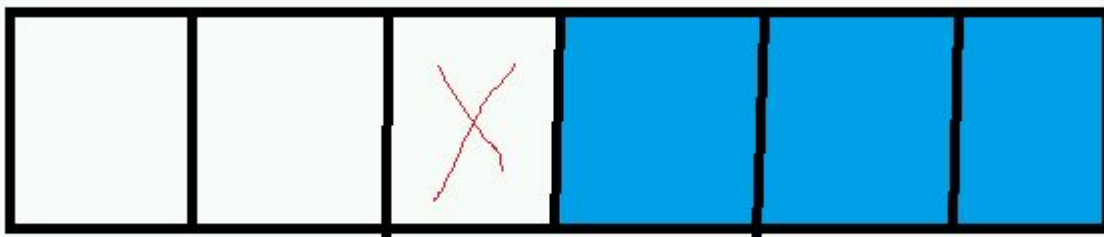
remove 方法有两个重载，我们只研究 remove (int index) 方法。

```
/**
 * Removes the object at the specified location from this list.
 *
 * @param index
 *         the index of the object to remove.
 * @return the removed object.
 * @throws IndexOutOfBoundsException
 *         when {@code location < 0 || location >= size()}
 */
@Override public E remove(int index) {
    Object[] a = array;
    int s = size;
    if (index >= s) {
        throwIndexOutOfBoundsException(index, s);
    }
    @SuppressWarnings("unchecked")
```

```
E result = (E) a[index];
System.arraycopy(a, index + 1, a, index, --s - index);
a[s] = null; // Prevent memory leak
size = s;
modCount++;
return result;
}
```

- 1、先将成员变量 array 和 size 赋值给局部变量 a 和 s。
- 2、判断形参 index 是否大于等于集合的长度，如果成了则抛出运行时异常
- 3、获取数组中脚标为 index 的对象 result，该对象作为方法的返回值
- 4、调用 System 的 arraycopy 函数，拷贝原理如下图所示。

```
System.arraycopy(a, index + 1, a, index, --s - index);
```



假设index=2，那么要删除的对象如图打叉部分，我们只需要将该数组后面蓝色区域整体往前移动一位位置即可。上面的代码完成就是这项工作。

5、接下来就是很重要的一个工作，因为删除了一个元素，而且集合整体向前移动了一位，因此需要将集合最后一个元素设置为 null，否则就可能内存泄露。

- 6、重新给成员变量 array 和 size 赋值
- 7、记录修改次数
- 8、返回删除的元素（让用户再看最后一眼）

四、clear 方法

```
/**
 * Removes all elements from this {@code ArrayList}, leaving it empty.
 *
 * @see #isEmpty
 * @see #size
 */
```

```
*/  
@Override public void clear() {  
    if (size != 0) {  
        Arrays.fill(array, 0, size, null);  
        size = 0;  
        modCount++;  
    }  
}
```

如果集合长度不等于 0，则将所有数组的值都设置为 null，然后将成员变量 size 设置为 0 即可，最后让修改记录加 1。

4、并发集合和普通集合如何区别？（2015-11-24）

并发集合常见的有 ConcurrentHashMap、ConcurrentLinkedQueue、ConcurrentLinkedDeque 等。并发集合位于 java.util.concurrent 包下，是 jdk1.5 之后才有的，主要作者是 Doug Lea (<http://baike.baidu.com/view/3141057.htm>) 完成的。

在 java 中有普通集合、同步（线程安全）的集合、并发集合。普通集合通常性能最高，但是不保证多线程的安全性和并发的可靠性。线程安全集合仅仅是给集合添加了 synchronized 同步锁，严重牺牲了性能，而且对并发的效率就更低了，并发集合则通过复杂的策略不仅保证了多线程的安全又提高的并发时的效率。

参考阅读：

ConcurrentHashMap 是线程安全的 HashMap 的实现，默认构造同样有 initialCapacity 和 loadFactor 属性，不过还多了一个 concurrencyLevel 属性，三属性默认值分别为 16、0.75 及 16。其内部使用锁分段技术，维持这锁 Segment 的数组，在 Segment 数组中又存放着 Entity[] 数组，内部 hash 算法将数据较均匀分布在不同锁中。

put 操作：并没有在此方法上加上 synchronized，首先对 key.hashCode 进行 hash 操作，得到 key 的 hash 值。hash 操作的算法和 map 也不同，根据此 hash 值计算并获取其对应的数组中的 Segment 对象（继承自 ReentrantLock），接着调用此 Segment 对象的 put 方法来完成当前操作。

ConcurrentHashMap 基于 concurrencyLevel 划分出了多个 Segment 来对 key-value 进行存储，从而避免每次

put 操作都得锁住整个数组。在默认的情况下，最佳情况下可允许 16 个线程并发无阻塞的操作集合对象，尽可能地减少并发时的阻塞现象。

get(key)

首先对 key.hashCode 进行 hash 操作，基于其值找到对应的 Segment 对象，调用其 get 方法完成当前操作。而 Segment 的 get 操作首先通过 hash 值和对象数组大小减 1 的值进行按位与操作来获取数组上对应位置的 HashEntry。在这个步骤中，可能会因为对象数组大小的改变，以及数组上对应位置的 HashEntry 产生不一致性，那么 ConcurrentHashMap 是如何保证的？

对象数组大小的改变只有在 put 操作时有可能发生，由于 HashEntry 对象数组对应的变量是 volatile 类型的，因此可以保证如 HashEntry 对象数组大小发生改变，读操作可看到最新的对象数组大小。

在获取到了 HashEntry 对象后，怎么能保证它及其 next 属性构成的链表上的对象不会改变呢？这点 ConcurrentHashMap 采用了一个简单的方式，即 HashEntry 对象中的 hash、key、next 属性都是 final 的，这也就意味着没办法插入一个 HashEntry 对象到基于 next 属性构成的链表中间或末尾。这样就可以保证当获取到 HashEntry 对象后，其基于 next 属性构建的链表是不会发生变化的。

ConcurrentHashMap 默认情况下采用将数据分为 16 个段进行存储，并且 16 个段分别持有各自不同的锁 Segment，锁仅用于 put 和 remove 等改变集合对象的操作，基于 volatile 及 HashEntry 链表的不变性实现了读取的不加锁。这些方式使得 ConcurrentHashMap 能够保持极好的并发支持，尤其是对于读远比插入和删除频繁的 Map 而言，而它采用的这些方法也可谓是对于 Java 内存模型、并发机制深刻掌握的体现。

推荐博客地址：<http://m.oschina.net/blog/269037>

七、Java 的多线程

1、多线程的两种创建方式

`java.lang.Thread` 类的实例就是一个线程但是它需要调用 `java.lang.Runnable` 接口来执行，由于线程类本身就是调用的 `Runnable` 接口所以你可以继承 `java.lang.Thread` 类或者直接实现 `Runnable` 接口来重写 `run()` 方法实现线程。

2、在 java 中 wait 和 sleep 方法的不同？

最大的不同是在等待时 `wait` 会释放锁，而 `sleep` 一直持有锁。`wait` 通常被用于线程间交互，`sleep` 通常被用于暂停执行。

3、synchronized 和 volatile 关键字的作用

一旦一个共享变量（类的成员变量、类的静态成员变量）被 `volatile` 修饰之后，那么就具备了两层语义：

1) 保证了不同线程对这个变量进行操作时的可见性，即一个线程修改了某个变量的值，这新值对其他线程来说是立即可见的。

2) 禁止进行指令重排序。

`volatile` 本质是在告诉 jvm 当前变量在寄存器（工作内存）中的值是不确定的，需要从主存中读取；

`synchronized` 则是锁定当前变量，只有当前线程可以访问该变量，其他线程被阻塞住。

1.volatile 仅能使用在变量级别；

`synchronized` 则可以使用在变量、方法、和类级别的

2.volatile 仅能实现变量的修改可见性，并不能保证原子性；

`synchronized` 则可以保证变量的修改可见性和原子性

3.volatile 不会造成线程的阻塞；

`synchronized` 可能会造成线程的阻塞。

4.volatile 标记的变量不会被编译器优化；

synchronized 标记的变量可以被编译器优化

4、分析线程并发访问代码解释原因

```
public class Counter {
    private volatile int count = 0;
    public void inc(){
        try {
            Thread.sleep(3);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        count++;
    }
    @Override
    public String toString() {
        return "[count=" + count + "]";
    }
}
//-----华丽的分割线-----
public class VolatileTest {
    public static void main(String[] args) {
        final Counter counter = new Counter();
        for(int i=0;i<1000;i++){
            new Thread(new Runnable() {
                @Override
                public void run() {
                    counter.inc();
                }
            }).start();
        }
        System.out.println(counter);
    }
}
```

上面的代码执行完后输出的结果确定为 1000 吗？

答案是不一定，或者不等于 1000。这是为什么吗？

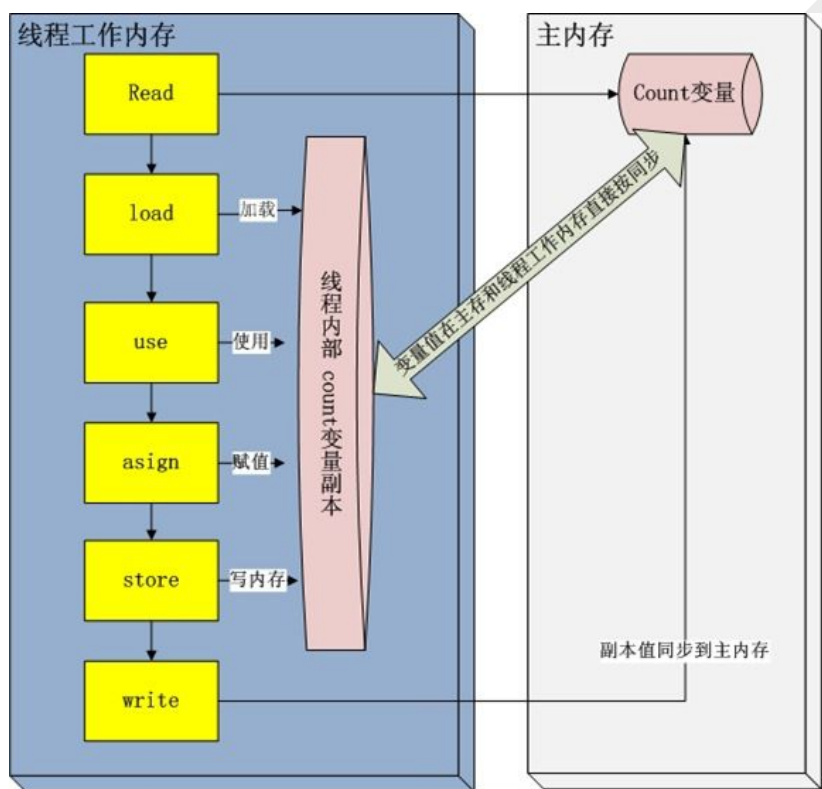
在 java 的内存模型中每一个线程运行时都有一个线程栈，线程栈保存了线程运行时候变量值信息。当线程访问某一个对象时候值的时候，首先通过对象的引用找到对应堆内存的变量的值，然后把堆内存变量的具体值 load 到线

程本地内存中，建立一个变量副本，之后线程就不再和对象在堆内存变量值有任何关系，而是直接修改副本变量的值，在修改完之后的某一个时刻（线程退出之前），自动把线程变量副本的值回写到对象在堆中变量。这样在堆中的对象的价值就产生变化了。

也就是说上面主函数中开启了 1000 个子线程，每个线程都有一个变量副本，每个线程修改变量只是临时修改了自己的副本，当线程结束时再将修改的值写入在主内存中，这样就出现了线程安全问题。因此结果就不可能等于 1000 了，一般都会小于 1000。

上面的解释用一张图表示如下：

（图片来自网络，非本人所绘）



5、什么是线程池，如何使用？

线程池就是事先将多个线程对象放到一个容器中，当使用的时候就不用 new 线程而是直接去池中拿线程即可，节省了开辟子线程的时间，提高的代码执行效率。

在 JDK 的 `java.util.concurrent.Executors` 中提供了生成多种线程池的静态方法。

```
ExecutorService newCachedThreadPool = Executors.newCachedThreadPool();
ExecutorService newFixedThreadPool = Executors.newFixedThreadPool(4);
ScheduledExecutorService newScheduledThreadPool =
    Executors.newScheduledThreadPool(4);
ExecutorService newSingleThreadExecutor = Executors.newSingleThreadExecutor();
```

然后调用他们的 `execute` 方法即可。

6、请叙述一下您对线程池的理解？（2015-11-25）

（如果问到了这样的问题，可以展开的说一下线程池如何用、线程池的好处、线程池的启动策略）

合理利用线程池能够带来三个好处。

第一：降低资源消耗。通过重复利用已创建的线程降低线程创建和销毁造成的消耗。

第二：提高响应速度。当任务到达时，任务可以不需要等到线程创建就能立即执行。

第三：提高线程的可管理性。线程是稀缺资源，如果无限制的创建，不仅会消耗系统资源，还会降低系统的稳定性，使用线程池可以进行统一的分配，调优和监控。

7、线程池的启动策略？（2015-11-25）

官方对线程池的执行过程描述如下：

```
1.  /*
2.      * Proceed in 3 steps:
3.      *
4.      * 1. If fewer than corePoolSize threads are running, try to
5.      * start a new thread with the given command as its first
6.      * task. The call to addWorker atomically checks runState and
7.      * workerCount, and so prevents false alarms that would add
8.      * threads when it shouldn't, by returning false.
9.      *
10.     * 2. If a task can be successfully queued, then we still need
11.     * to double-check whether we should have added a thread
12.     * (because existing ones died since last checking) or that
13.     * the pool shut down since entry into this method. So we
14.     * recheck state and if necessary roll back the enqueueing if
```

```
15.      * stopped, or start a new thread if there are none.
16.      *
17.      * 3. If we cannot queue task, then we try to add a new
18.      * thread. If it fails, we know we are shut down or saturated
19.      * and so reject the task.
20.      */
```

1、线程池刚创建时，里面没有一个线程。任务队列是作为参数传进来的。不过，就算队列里面有任务，线程池也不会马上执行它们。

2、当调用 `execute()` 方法添加一个任务时，线程池会做如下判断：

- a. 如果正在运行的线程数量小于 `corePoolSize`，那么马上创建线程运行这个任务；
- b. 如果正在运行的线程数量大于或等于 `corePoolSize`，那么将这个任务放入队列。
- c. 如果这时候队列满了，而且正在运行的线程数量小于 `maximumPoolSize`，那么还是要创建线程运行这个任务；
- d. 如果队列满了，而且正在运行的线程数量大于或等于 `maximumPoolSize`，那么线程池会抛出异常，告诉调用者“我不能再接受任务了”。

3、当一个线程完成任务时，它会从队列中取下一个任务来执行。

4、当一个线程无事可做，超过一定的时间（`keepAliveTime`）时，线程池会判断，如果当前运行的线程数大于 `corePoolSize`，那么这个线程就被停掉。所以线程池的所有任务完成后，它最终会收缩到 `corePoolSize` 的大小。

8、如何控制某个方法允许并发访问线程的个数？（2015-11-30）

```
21. package com.yange;
22.
23. import java.util.concurrent.Semaphore;
24. /**
25.  *
26.  * @author wzy 2015-11-30
27.  *
28.  */
29. public class SemaphoreTest {
```

```
30. /*
31.  * permits the initial number of permits available. This value may be negative,
32.  * in which case releases must occur before any acquires will be granted.
33.  * fair true if this semaphore will guarantee first-in first-out granting of
34.  * permits under contention, else false
35.  */
36. static Semaphore semaphore = new Semaphore(5,true);
37. public static void main(String[] args) {
38.     for(int i=0;i<100;i++){
39.         new Thread(new Runnable() {
40.
41.             @Override
42.             public void run() {
43.                 test();
44.             }
45.         }).start();
46.     }
47.
48. }
49.
50. public static void test(){
51.     try {
52.         //申请一个请求
53.         semaphore.acquire();
54.     } catch (InterruptedException e1) {
55.         e1.printStackTrace();
56.     }
57.     System.out.println(Thread.currentThread().getName()+"进来了");
58.     try {
59.         Thread.sleep(1000);
60.     } catch (InterruptedException e) {
61.         e.printStackTrace();
62.     }
63.     System.out.println(Thread.currentThread().getName()+"走了");
64.     //释放一个请求
65.     semaphore.release();
66. }
67. }
68.
```

可以使用 Semaphore 控制，第 16 行的构造函数创建了一个 Semaphore 对象，并且初始化了 5 个信号。这样的效果是控件 test 方法最多只能有 5 个线程并发访问，对于 5 个线程时就排队等待，走一个来一下。第 33 行，请求一个信号（消费一个信号），如果信号被用完了则等待，第 45 行释放一个信号，释放的信号新的线程就可以使用了。

3. JavaSE 高级 (★★)

一、Java 中的反射

1、说说你对 Java 中反射的理解

Java 中的反射首先是能够获取到 Java 中要反射类的字节码，获取字节码有三种方法，
1.Class.forName(className) 2.类名.class 3.this.getClass()。然后将字节码中的方法，变量，构造函数等映射成相应的 Method、Filed、Constructor 等类，这些类提供了丰富的方法可以被我们所使用。

二、Java 中的动态代理

1、写一个 ArrayList 的动态代理类 (笔试题)

```
final List<String> list = new ArrayList<String>();

List<String> proxyInstance = (List<String>)
Proxy.newProxyInstance(list.getClass().getClassLoader(),
list.getClass().getInterfaces(), new InvocationHandler() {

    @Override
    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
        return method.invoke(list, args);
    }
});
proxyInstance.add("你好");
System.out.println(list);
```

2、动静代理的区别，什么场景使用？（2015-11-25）

静态代理通常只代理一个类，动态代理是代理一个接口下的多个实现类。

静态代理事先知道要代理的是什么，而动态代理不知道要代理什么东西，只有在运行时才知道。

动态代理是实现 JDK 里的 InvocationHandler 接口的 invoke 方法，但注意的是代理的是接口，也就是你的业务类必须要实现接口，通过 Proxy 里的 newProxyInstance 得到代理对象。

还有一种动态代理 CGLIB，代理的是类，不需要业务类继承接口，通过派生的子类来实现代理。通过在运行时，动态修改字节码达到修改类的目的。

AOP 编程就是基于动态代理实现的，比如著名的 Spring 框架、Hibernate 框架等等都是动态代理的使用例子。

三、Java 中的设计模式&回收机制

1、你所知道的设计模式有哪些

Java 中一般认为有 23 种设计模式，我们不需要所有的都会，但是其中常用的几种设计模式应该去掌握。下面列出了所有的设计模式。需要掌握的设计模式我单独列出来了，当然能掌握的越多越好。

总体来说设计模式分为三大类：

创建型模式，共五种：**工厂方法模式**、**抽象工厂模式**、**单例模式**、**建造者模式**、**原型模式**。

结构型模式，共七种：**适配器模式**、**装饰器模式**、**代理模式**、**外观模式**、**桥接模式**、**组合模式**、**享元模式**。

行为型模式，共十一种：**策略模式**、**模板方法模式**、**观察者模式**、**迭代子模式**、**责任链模式**、**命令模式**、**备忘录模式**、**状态模式**、**访问者模式**、**中介者模式**、**解释器模式**。

2、单例设计模式

最好理解的一种设计模式，分为懒汉式和饿汉式。

◆ 饿汉式：

```
public class Singleton {  
    // 直接创建对象  
    public static Singleton instance = new Singleton();  
  
    // 私有化构造函数  
    private Singleton() {  
    }  
  
    // 返回对象实例  
    public static Singleton getInstance() {  
        return instance;  
    }  
}
```

◆ 懒汉式：

```
public class Singleton {  
    // 声明变量  
    private static volatile Singleton singleton2 = null;  
  
    // 私有构造函数  
    private Singleton2() {  
    }  
  
    // 提供对外方法  
    public static Singleton2 getInstance() {  
        if (singleton2 == null) {  
            synchronized (Singleton2.class) {  
                if (singleton == null) {  
                    singleton = new Singleton();  
                }  
            }  
        }  
        return singleton;  
    }  
}
```

3、工厂设计模式

工厂模式分为工厂方法模式和抽象工厂模式。

◆ 工厂方法模式

工厂方法模式分为三种：普通工厂模式，就是建立一个工厂类，对实现了同一接口的一些类进行实例的创建。

多个工厂方法模式，是对普通工厂方法模式的改进，在普通工厂方法模式中，如果传递的字符串出错，则不能正确创建对象，而多个工厂方法模式是提供多个工厂方法，分别创建对象。

静态工厂方法模式，将上面的多个工厂方法模式里的方法置为静态的，不需要创建实例，直接调用即可。

◆ 普通工厂模式

```
public interface Sender {
    public void Send();
}

public class MailSender implements Sender {

    @Override
    public void Send() {
        System.out.println("this is mail sender!");
    }
}

public class SmsSender implements Sender {

    @Override
    public void Send() {
        System.out.println("this is sms sender!");
    }
}

public class SendFactory {
    public Sender produce(String type) {
        if ("mail".equals(type)) {
            return new MailSender();
        } else if ("sms".equals(type)) {
            return new SmsSender();
        } else {
            System.out.println("请输入正确的类型!");
            return null;
        }
    }
}
```

◆ 多个工厂方法模式

该模式是对普通工厂方法模式的改进，在普通工厂方法模式中，如果传递的字符串出错，则不能正确创建对象，

而多个工厂方法模式是提供多个工厂方法，分别创建对象。

```
public class SendFactory {
    public Sender produceMail(){
        return new MailSender();
    }

    public Sender produceSms(){
        return new SmsSender();
    }
}

public class FactoryTest {
    public static void main(String[] args) {
        SendFactory factory = new SendFactory();
        Sender sender = factory.produceMail();
        sender.send();
    }
}
```

◆ 静态工厂方法模式，将上面的多个工厂方法模式里的方法置为静态的，不需要创建实例，直接调用即可。

```
public class SendFactory {
    public static Sender produceMail(){
        return new MailSender();
    }

    public static Sender produceSms(){
        return new SmsSender();
    }
}

public class FactoryTest {
    public static void main(String[] args) {
        Sender sender = SendFactory.produceMail();
        sender.send();
    }
}
```

◆ 抽象工厂模式

工厂方法模式有一个问题就是，类的创建依赖工厂类，也就是说，如果想要拓展程序，必须对工厂类进行修改，这违背了闭包原则，所以，从设计角度考虑，有一定的问题，如何解决？就用到抽象工厂模式，创建多个工厂类，这样一旦需要增加新的功能，直接增加新的工厂类就可以了，不需要修改之前的代码。

```
public interface Provider {
    public Sender produce();
}

-----

public interface Sender {
    public void send();
}

-----

public class MailSender implements Sender {

    @Override
    public void send() {
        System.out.println("this is mail sender!");
    }
}

-----

public class SmsSender implements Sender {

    @Override
    public void send() {
        System.out.println("this is sms sender!");
    }
}

-----

public class SendSmsFactory implements Provider {

    @Override
    public Sender produce() {
        return new SmsSender();
    }
}
```

```
public class SendMailFactory implements Provider {
```

```
    @Override
    public Sender produce() {
        return new MailSender();
    }
}
```

```
public class Test {
    public static void main(String[] args) {
        Provider provider = new SendMailFactory();
        Sender sender = provider.produce();
        sender.send();
    }
}
```

4、建造者模式 (Builder)

工厂类模式提供的是创建单个类的模式，而建造者模式则是将各种产品集中起来进行管理，用来创建复合对象，

所谓复合对象就是指某个类具有不同的属性，其实建造者模式就是前面抽象工厂模式和最后的 Test 结合起来得到的。

```
public class Builder {
    private List<Sender> list = new ArrayList<Sender>();

    public void produceMailSender(int count) {
        for (int i = 0; i < count; i++) {
            list.add(new MailSender());
        }
    }

    public void produceSmsSender(int count) {
        for (int i = 0; i < count; i++) {
            list.add(new SmsSender());
        }
    }
}
```

```
public class TestBuilder {
    public static void main(String[] args) {
        Builder builder = new Builder();
        builder.produceMailSender(10);
    }
}
```

5、适配器设计模式

适配器模式将某个类的接口转换成客户端期望的另一个接口表示，目的是消除由于接口不匹配所造成的类的兼容性问题。主要分为三类：类的适配器模式、对象的适配器模式、接口的适配器模式。

◆ 类的适配器模式

```
public class Source {
    public void method1() {
        System.out.println("this is original method!");
    }
}

-----

public interface Targetable {
    /* 与原类中的方法相同 */
    public void method1();
    /* 新类的方法 */
    public void method2();
}

public class Adapter extends Source implements Targetable {
    @Override
    public void method2() {
        System.out.println("this is the targetable method!");
    }
}

public class AdapterTest {
    public static void main(String[] args) {
        Targetable target = new Adapter();
        target.method1();
        target.method2();
    }
}
```

◆ 对象的适配器模式

基本思路和类的适配器模式相同，只是将 Adapter 类作修改，这次不继承 Source 类，而是持有 Source 类的实例，以达到解决兼容性的问题。

```
public class Wrapper implements Targetable {
    private Source source;

    public Wrapper(Source source) {
        super();
        this.source = source;
    }

    @Override
    public void method2() {
        System.out.println("this is the targetable method!");
    }

    @Override
    public void method1() {
        source.method1();
    }
}

-----

public class AdapterTest {

    public static void main(String[] args) {
        Source source = new Source();
        Targetable target = new Wrapper(source);
        target.method1();
        target.method2();
    }
}
```

◆ 接口的适配器模式

接口的适配器是这样的：有时我们写的一个接口中有多个抽象方法，当我们写该接口的实现类时，必须实现该接口的所有方法，这明显有时比较浪费，因为并不是所有的方法都是我们需要的，有时只需要某一些，此处为了解决这个问题，我们引入了接口的适配器模式，借助于一个抽象类，该抽象类实现了该接口，实现了所有的方法，而我们不

和原始的接口打交道，只和该抽象类取得联系，所以我们写一个类，继承该抽象类，重写我们需要的方法就行。

6、装饰模式 (Decorator)

顾名思义，装饰模式就是给一个对象增加一些新的功能，而且是动态的，要求装饰对象和被装饰对象实现同一个接口，装饰对象持有被装饰对象的实例。

```
public interface Sourceable {
    public void method();
}

-----

public class Source implements Sourceable {
    @Override
    public void method() {
        System.out.println("the original method!");
    }
}

-----

public class Decorator implements Sourceable {
    private Sourceable source;
    public Decorator(Sourceable source) {
        super();
        this.source = source;
    }

    @Override
    public void method() {
        System.out.println("before decorator!");
        source.method();
        System.out.println("after decorator!");
    }
}

-----

public class DecoratorTest {
    public static void main(String[] args) {
        Sourceable source = new Source();
        Sourceable obj = new Decorator(source);
        obj.method();
    }
}
```

7、策略模式 (strategy)

策略模式定义了一系列算法，并将每个算法封装起来，使他们可以相互替换，且算法的变化不会影响到使用算法的客户。需要设计一个接口，为一系列实现类提供统一的方法，多个实现类实现该接口，设计一个抽象类（可有可无，属于辅助类），提供辅助函数。策略模式的决定权在用户，系统本身提供不同算法的实现，新增或者删除算法，对各种算法做封装。因此，策略模式多用在算法决策系统中，外部用户只需要决定用哪个算法即可。

```
public interface ICalculator {
    public int calculate(String exp);
}

-----

public class Minus extends AbstractCalculator implements ICalculator {

    @Override
    public int calculate(String exp) {
        int arrayInt[] = split(exp, "-");
        return arrayInt[0] - arrayInt[1];
    }
}

-----

public class Plus extends AbstractCalculator implements ICalculator {

    @Override
    public int calculate(String exp) {
        int arrayInt[] = split(exp, "\\+");
        return arrayInt[0] + arrayInt[1];
    }
}

-----

public class AbstractCalculator {
    public int[] split(String exp, String opt) {
        String array[] = exp.split(opt);
        int arrayInt[] = new int[2];
        arrayInt[0] = Integer.parseInt(array[0]);
        arrayInt[1] = Integer.parseInt(array[1]);
        return arrayInt;
    }
}
```

```
public class StrategyTest {
    public static void main(String[] args) {
        String exp = "2+8";
        ICalculator cal = new Plus();
        int result = cal.calculate(exp);
        System.out.println(result);
    }
}
```

8、观察者模式 (Observer)

观察者模式很好理解，类似于邮件订阅和 RSS 订阅，当我们浏览一些博客或 wiki 时，经常会看到 RSS 图标，就这意思是，当你订阅了该文章，如果后续有更新，会及时通知你。其实，简单来讲就一句话：当一个对象变化时，其它依赖该对象的对象都会收到通知，并且随着变化！对象之间是一种一对多的关系。

```
public interface Observer {
    public void update();
}

public class Observer1 implements Observer {
    @Override
    public void update() {
        System.out.println("observer1 has received!");
    }
}

public class Observer2 implements Observer {
    @Override
    public void update() {
        System.out.println("observer2 has received!");
    }
}

public interface Subject {
    /*增加观察者*/
    public void add(Observer observer);

    /*删除观察者*/
    public void del(Observer observer);
}
```



```
/*通知所有的观察者*/
public void notifyObservers();

/*自身的操作*/
public void operation();
}

public abstract class AbstractSubject implements Subject {

    private Vector<Observer> vector = new Vector<Observer>();

    @Override
    public void add(Observer observer) {
        vector.add(observer);
    }

    @Override
    public void del(Observer observer) {
        vector.remove(observer);
    }

    @Override
    public void notifyObservers() {
        Enumeration<Observer> enumo = vector.elements();
        while (enumo.hasMoreElements()) {
            enumo.nextElement().update();
        }
    }
}

public class MySubject extends AbstractSubject {

    @Override
    public void operation() {
        System.out.println("update self!");
        notifyObservers();
    }
}

public class ObserverTest {
    public static void main(String[] args) {
        Subject sub = new MySubject();
        sub.add(new Observer1());
    }
}
```

```
        sub.add(new Observer2());  
        sub.operation();  
    }  
}
```

9、JVM 垃圾回收机制和常见算法

理论上来讲 Sun 公司只定义了垃圾回收机制规则而不局限于其实现算法，因此不同厂商生产的虚拟机采用的算法也不尽相同。

GC (Garbage Collector) 在回收对象前首先必须发现那些无用的对象，如何去发现定位这些无用的对象？常用的**搜索算法**如下：

1) 引用计数器算法 (废弃)

引用计数器算法是给每个对象设置一个计数器，当有地方引用这个对象的时候，计数器+1，当引用失效的时候，计数器-1，当计数器为 0 的时候，JVM 就认为对象不再被使用，是“垃圾”了。

引用计数器实现简单，效率高；但是不能解决循环引用问题（A 对象引用 B 对象，B 对象又引用 A 对象，但是 A,B 对象已不被任何其他对象引用），同时每次计数器的增加和减少都带来了许多额外的开销，所以在 JDK1.1 之后，这个算法已经不再使用了。

2) 根搜索算法 (使用)

根搜索算法是通过一些“GC Roots”对象作为起点，从这些节点开始往下搜索，搜索通过的路径成为引用链 (Reference Chain)，当一个对象没有被 GC Roots 的引用链连接的时候，说明这个对象是不可用的。

GC Roots 对象包括：

- a) 虚拟机栈（栈帧中的本地变量表）中的引用的对象。
- b) 方法区域中的类静态属性引用的对象。
- c) 方法区域中常量引用的对象。

d) 本地方法栈中 JNI (Native 方法) 的引用的对象。

通过上面的算法搜索到无用对象之后，就是回收过程，**回收算法**如下：

1) 标记—清除算法 (Mark-Sweep) (DVM 使用的算法)

标记—清除算法包括两个阶段：“标记”和“清除”。在标记阶段，确定所有要回收的对象，并做标记。清除阶段紧随标记阶段，将标记阶段确定不可用的对象清除。标记—清除算法是基础的收集算法，标记和清除阶段的效率不高，而且清除后会产生大量的不连续空间，这样当程序需要分配大内存对象时，可能无法找到足够的连续空间。

2) 复制算法 (Copying)

复制算法是把内存分成大小相等的两块，每次使用其中一块，当垃圾回收的时候，把存活的对象复制到另一块上，然后把这块内存整个清理掉。复制算法实现简单，运行效率高，但是由于每次只能使用其中的一半，造成内存的利用率不高。现在的 JVM 用复制方法收集新生代，由于新生代中大部分对象 (98%) 都是朝生夕死的，所以两块内存的比例不是 1:1(大概是 8:1)。

3) 标记—整理算法 (Mark-Compact)

标记—整理算法和标记—清除算法一样，但是标记—整理算法不是把存活对象复制到另一块内存，而是把存活对象往内存的一端移动，然后直接回收边界以外的内存。标记—整理算法提高了内存的利用率，并且它适合在收集对象存活时间较长的老年代。

4) 分代收集 (Generational Collection)

分代收集是根据对象的存活时间把内存分为新生代和老年代，根据各个代对象的存活特点，每个代采用不同的垃圾回收算法。新生代采用复制算法，老年代采用标记—整理算法。垃圾算法的实现涉及大量的程序细节，而且不同的虚拟机平台实现的方法也各不相同。

10、谈谈 JVM 的内存结构和内存分配

a) Java 内存模型

Java 虚拟机将其管辖的内存大致分三个逻辑部分：方法区(Method Area)、Java 栈和 Java 堆。

- 1、方法区是静态分配的，编译器将变量绑定在某个存储位置上，而且这些绑定不会在运行时改变。

常数池，源代码中的命名常量、String 常量和 static 变量保存在方法区。

- 2、Java Stack 是一个逻辑概念，特点是后进先出。一个栈的空间可能是连续的，也可能是不连续的。

最典型的 Stack 应用是方法的调用，Java 虚拟机每调用一次方法就创建一个方法帧（frame），退出该方法则对应的 方法帧被弹出(pop)。栈中存储的数据也是运行时确定的。

- 3、Java 堆分配(heap allocation)意味着以随意的顺序，在运行时进行存储空间分配和收回的内存管理模型。

堆中存储的数据常常是大小、数量和生命期在编译时无法确定的。Java 对象的内存总是在 heap 中分配。

我们每天都在写代码，每天都在使用 JVM 的内存。

b) java 内存分配

- 1、基础数据类型直接在栈空间分配;
- 2、方法的形式参数，直接在栈空间分配，当方法调用完成后从栈空间回收;
- 3、引用数据类型，需要用 new 来创建，既在栈空间分配一个地址空间，又在堆空间分配对象的类变量;
- 4、方法的引用参数，在栈空间分配一个地址空间，并指向堆空间的对象区，当方法调用完后从栈空间回收;
- 5、局部变量 new 出来时，在栈空间和堆空间中分配空间，当局部变量生命周期结束后，栈空间立刻被回收，堆空间区域等待 GC 回收;
- 6、方法调用时传入的实际参数，先在栈空间分配，在方法调用完成后从栈空间释放;
- 7、字符串常量在 DATA 区域分配，this 在堆空间分配;
- 8、数组既在栈空间分配数组名称，又在堆空间分配数组实际的大小！

11、Java 中引用类型都有哪些？（重要）

Java 中对象的引用分为四种级别，这四种级别由高到低依次为：强引用、软引用、弱引用和虚引用。

◆ 强引用 (StrongReference)

这个就不多说，我们写代码天天在用的就是强引用。如果一个对象被被人拥有强引用，那么垃圾回收器绝不会回收它。当内存空间不足，Java 虚拟机宁愿抛出 OutOfMemoryError 错误，使程序异常终止，也不会靠随意回收具有强引用的对象来解决内存不足问题。

Java 的对象是位于 heap 中的，heap 中对象有强可及对象、软可及对象、弱可及对象、虚可及对象和不可到达对象。应用的强弱顺序是强、软、弱、和虚。对于对象是属于哪种可及的对象，由他的最强的引用决定。如下代码：

```
String abc=new String("abc"); //1
SoftReference<String> softRef=new SoftReference<String>(abc); //2
WeakReference<String> weakRef = new WeakReference<String>(abc); //3
abc=null; //4
softRef.clear();//5
```

第一行在 heap 堆中创建内容为“abc”的对象，并建立 abc 到该对象的强引用，该对象是强可及的。

第二行和第三行分别建立对 heap 中对象的软引用和弱引用，此时 heap 中的 abc 对象已经有 3 个引用，显然此时 abc 对象仍是强可及的。

第四行之后 heap 中对象不再是强可及的，变成软可及的。

第五行执行之后变成弱可及的。

◆ 软引用 (SoftReference)

如果一个对象只具有软引用，那么如果内存空间足够，垃圾回收器就不会回收它，如果内存空间不足了，就会回收这些对象的内存。只要垃圾回收器没有回收它，该对象就可以被程序使用。软引用可用来实现内存敏感的高速缓存。

软引用可以和一个引用队列 (ReferenceQueue) 联合使用，如果软引用所引用的对象被垃圾回收，Java 虚拟机就会把这个软引用加入到与之关联的引用队列中。

软引用是主要用于内存敏感的高速缓存。在 jvm 报告内存不足之前会清除所有的软引用，这样以来 gc 就有

可能收集软可及的对象，可能解决内存吃紧问题，避免内存溢出。什么时候会被收集取决于 gc 的算法和 gc 运行时可用内存的大小。当 gc 决定要收集软引用时执行以下过程,以上面的 softRef 为例：

- 1 首先将 softRef 的 referent (abc) 设置为 null，不再引用 heap 中的 new String("abc")对象。
- 2 将 heap 中的 new String("abc")对象设置为可结束的(finalizable)。
- 3 当 heap 中的 new String("abc")对象的 finalize()方法被运行而且该对象占用的内存被释放，softRef 被添加到它的 ReferenceQueue(如果有的话)中。

注意:对 ReferenceQueue 软引用和弱引用可以有可无，但是虚引用必须有。

被 Soft Reference 指到的对象，即使没有任何 Direct Reference，也不会被清除。一直要到 JVM 内存不足且没有 Direct Reference 时才会清除，SoftReference 是用来设计 object-cache 之用的。如此一来 SoftReference 不但可以把对象 cache 起来，也不会造成内存不足的错误（OutOfMemoryError）。

◆ 弱引用（WeakReference）

如果一个对象只具有弱引用，那该类就是可有可无的对象，因为只要该对象被 gc 扫描到了随时都会把它干掉。

弱引用与软引用的区别在于：只具有弱引用的对象拥有更短暂的生命周期。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，都会回收它的内存。不过，由于垃圾回收器是一个优先级很低的线程，因此不一定会很快发现那些只具有弱引用的对象。

弱引用可以和一个引用队列（ReferenceQueue）联合使用，如果弱引用所引用的对象被垃圾回收，Java 虚拟机就会把这个弱引用加入到与之关联的引用队列中。

◆ 虚引用（PhantomReference）

"虚引用"顾名思义，就是形同虚设，与其他几种引用都不同，虚引用并不会决定对象的生命周期。如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收。虚引用主要用来跟踪对象被垃圾回收的活动。

虚引用与软引用和弱引用的一个区别在于：虚引用必须和引用队列（ReferenceQueue）联合使用。当垃圾

回收器准备回收一个对象时，如果发现它还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中。程序可以通过判断引用队列中是否已经加入了虚引用，来了解被引用的对象是否将要被垃圾回收。程序如果发现某个虚引用已经被加入到引用队列，那么就可以在所引用的对象的内存被回收之前采取必要的行动。

建立虚引用之后通过 `get` 方法返回结果始终为 `null`，通过源代码你会发现，虚引用通向会把引用的对象写进 `referent`，只是 `get` 方法返回结果为 `null`。先看一下和 `gc` 交互的过程再说一下他的作用。

- 1 不把 `referent` 设置为 `null`，直接把 `heap` 中的 `new String("abc")` 对象设置为可结束的(`finalizable`)。
- 2 与软引用和弱引用不同，先把 `PhantomReference` 对象添加到它的 `ReferenceQueue` 中，然后在释放虚可及的对象。

四、Java 的类加载器（2015-12-2）

1、Java 的类加载器的种类都有哪些？

- 1、根类加载器(Bootstrap) --C++写的，看不到源码
- 2、扩展类加载器 (Extension) --加载位置： `jre\lib\ext` 中
- 3、系统(应用)类加载器(System\App) --加载位置： `classpath` 中
- 4、自定义加载器(必须继承 `ClassLoader`)

2、类什么时候被初始化？

- 1) 创建类的实例，也就是 `new` 一个对象
- 2) 访问某个类或接口的静态变量，或者对该静态变量赋值
- 3) 调用类的静态方法
- 4) 反射 (`Class.forName("com.lyj.load")`)

5) 初始化一个类的子类 (会首先初始化子类的父类)

6) JVM 启动时标明的启动类，即文件名和类名相同的那个类

只有这 6 中情况才会导致类的初始化。

类的初始化步骤：

1) 如果这个类还没有被加载和链接，那先进行加载和链接

2) 假如这个类存在直接父类，并且这个类还没有被初始化 (注意：在一个类加载器中，类只能初始化一次)，那就初始化直接的父类 (不适用于接口)

3) 加入类中存在初始化语句 (如 static 变量和 static 块)，那就依次执行这些初始化语句。

4. Android 基础 (★★★★)

一、Android 基本常识

1、写 10 个简单的 linux 命令

mkdir 创建文件夹

rmdir 删除文件夹

rm 删除文件

mv 移动文件

cp 拷贝文件

cat 查看文件

tail 查看文件尾部

more 分页查看文件

cd 切换当前目录

ls 列出文件清单

reboot 重启

date 显示日期

cal 显示日历

ps 查看系统进程相当于 windows 的任务管理器

ifconfig 配置网络

2、书写出 android 工程的目录结构

src 源文件

gen 生成的文件 R 文件就在此

android.jar 依赖的 android sdk

assets 资源文件

bin 生成的字节码 apk 在此

libs 依赖 jar 和 so

res 资源文件

drawable

drawable-hdpi

layout

menu

values

AndroidManifest.xml

project.properties

3、什么是 ANR 如何避免它？

在 Android 上，如果你的应用程序有一段时间响应不够灵敏，系统会向用户显示一个对话框，这个对话框称作应用程序无响应（ANR：Application Not Responding）对话框。用户可以选择让程序继续运行，但是，他们在使用你的应用程序时，并不希望每次都要处理这个对话框。因此，在程序里对响应性能的设计很重要，这样，系统不会显示 ANR 给用户。

不同的组件发生 ANR 的时间不一样，主线程（Activity、Service）是 5 秒，BroadcastReceiver 是 10 秒。

解决方案：

将所有耗时操作，比如访问网络，Socket 通信，查询大量 SQL 语句，复杂逻辑计算等都放在子线程中去，然后通过 handler.sendMessage、runOnUiThread、AsyncTask 等方式更新 UI。无论如何都要确保用户界面操作的流畅度。如果耗时操作需要让用户等待，那么可以在界面上显示进度条。

4、谈谈 Android 的优点和不足之处

优点：

- 1、开放性，开源，免费，可定制
- 2、挣脱运营商束缚
- 3、丰富的硬件选择
- 4、不受任何限制的开发商
- 5、无缝结合的 Google 应用

缺点：

- 1、安全问题、隐私问题
- 2、同质化严重
- 3、运营商对 Android 手机仍然有影响

4、山寨化严重

5、过分依赖开发商，缺乏标准配置

5、一条最长的短信息约占多少 byte?

在国内的三大运营商通常情况下中文 70(包括标点)，英文 160 个。对于国外的其他运行商具体多长需要看运营商类型了。

android 内部是通过如下代码进行判断具体一个短信多少 byte 的。

```
ArrayList<String> android.telephony.SmsManager.divideMessage(String text)
```

6、sim 卡的 EF 文件有何作用？（不用看，废弃）

基本文件 EF(Elementary File)是 SIM 卡文件系统的一部分。

文件	文件标识符	文件缩写	中文名称	文件作用
MF	3F00	根目录	备注：所有非 ETSI GSM 协议中规定的应用文件由各厂家自行定义在根目录下（如：PIN1，PIN2…）	
EF _{ICCID}	2FE2	ICCID	SIM 卡唯一的识别号	包含运营商、卡商、发卡时间、省市代码等信息
DF _{GSM}	7F20	GSM 目录	备注：根据 ETSIGSM09.91 的规定 Phase2(或以上)的 SIM 卡中应该有 7F21 并指向 7F20, 用以兼容 Phase1 的手机	
EF _{LP} 语言选择	6F05	LP	语言选择文件	包含一种或多种语言编码
EF _{IMSI}	6F07	IMSI	国际移动用户识别符	包含 SIM 卡所对应的号段，比如 46000 代表 135—139 号段、46002 代表 1340—1348
EF _{Kc} 语音加密密钥	6F20	Kc	计算密钥	用于 SIM 卡的加密、解密
EF _{PLMNsel} 网络选择表	6F30	PLMNsel	公共陆地网选择	决定 SIM 卡选择哪种网络，在这里应该选择中国移动的网络

EF _{HPLMN} 归属地网络选择表	6F31	HPLMN	两次搜索 PLMN 的时间间隔	两次搜索中国移动的网络的时间间隔
EF _{ACMmax} 最大计费额	6F37	ACMmax	包含累积呼叫表的最大值	全部的 ACM 数据存在 SIM 卡中，此处取最大值
EF _{SST} SIM 卡服务表	6F38	SST	SIM 卡服务列表	指出 SIM 卡可以提供服务的种类，哪些业务被激活哪些业务没有开通
EF _{ACM} 累加计费计数器	6F39	ACM	累计呼叫列表	当前的呼叫和以前的呼叫的单位总和
EF _{GID1} 分组识别 1	6F3E	GID1	1 级分组识别文件	包含特定的 SIM-ME 组合的标识符，可以识别一组特定的 SIM 卡
EF _{GID2} 分组识别 2	6F3F	GID2	2 级分组识别文件	包含特定的 SIM-ME 组合的标识符，可以识别一组特定的 SIM 卡
EF _{PUCT} 单位价格/货币表	6F41	PUCT	呼叫单位的价格和货币表	PUCT 是与计费通知有关的信息，ME 用这个信息结合 EFACM，以用户选择的货币来计算呼叫费用
EF _{CBMI} 小区广播识别号	6F45	CBMI	小区广播信息标识符	规定了用户希望 MS 采纳的小区广播消息内容的类型
EF _{SPN} 服务提供商	6F46	SPN	服务提供商名称	包含服务提供商的名称和 ME 显示的相应要求
EF _{CBMID}	6F48	CBMID	数据下载的小区广播消息识别符	移动台将收到的 CBMID 传送给 SIM 卡
EF _{SUME}	6F54	SUME	建立菜单单元	建立 SIM 卡中的菜单
EF _{BCCH} 广播信道	6F74	BCCH	广播控制信道	由于 BCCH 的存储，在选择小区时，MS 可以缩小对 BCCH 载波的搜索范围
EF _{ACC} 访问控制级别	6F78	ACC	访问控制级别	SIM 卡有 15 个级别，10 个普通级别，5 个高级级别
EF _{FPLMN} 禁止网络号	6F7B	FPLMN	禁用的 PLMN	禁止选择除中国移动以外的其他运营商，比如中国联通、中国卫通等
EF _{LOCI} 位置信息	6F7E	LOCI	位置信息	存储临时移动用户识别符、位置区信息等内容

EF _{AD} 管理数据	6FAD	AD	管理数据	包括关于不同类型 SIM 卡操作模式的信息。例如：常规模式（PLMN 用户用于 GSM 网络操作），型号认证模式（允许 ME 在无线设备的认证期间的特殊应用）；小区测试模式（在小区商用之前，进行小区测试），制造商特定模式（允许 ME 制造商在维护阶段进行特定的性能自动测试）
EF _{PHASE} 阶段	6FAE	PHASE	阶段标识	标识 SIM 卡所处的阶段信息，比如是普通 SIM 卡还是 STK 卡等
DF _{TELECOM}	7F10	电信目录		
EF _{ADN} 缩位拨号	6F3A	AND	电话簿	用于将电话记录存放在 SIM 卡中
EF _{FDN} 固定拨号	6F3B	FDN	固定拨号	包括固定拨号（FDN）和/或补充业务控制字串（SSC），还包括相关网络/承载能力的识别符和扩展记录的识别符，以及有关的 α 识别符
EF _{SMS} 短消息	6F3C	SMS	短消息	用于将短消息记录存放在 SIM 卡中
EF _{CCP} 能力配置参数	6F3D	CCP	能力配置参数	包括所需要的网络和承载能力的参数，以及当采用一个缩位拨号号码，固定拨号号码，MSISDN、最后拨号号码、服务拨号号码或禁止拨号方式等，建立呼叫时相关的 ME 配置
EF _{MSISDN} 电话号码	6F40	MSISDN	移动基站国际综合业务网号	存放用户的手机号
EF _{SMSP} 短信息参数	6F42	SMSP	短消息业务参数	包括短信中心号码等信息
EF _{SMSS} 短信息状态	6F43	SMSS	短消息状态	这个标识是用来控制流量的
EF _{LND} 最后拨号	6F44	LND	最后拨叫号码	存储最后拨叫号码

EF _{Ext1} 扩展文件 1	6F4A	EXT1	扩展文件 1	包括 AND, MSISDN 或 LND 的扩展数据
EF _{Ext2} 扩展文件 2	6F4B	EXT2	扩展文件 2	包含 FDN 的扩展数据

7、如何判断是否有 SD 卡？

通过如下方法：

```
Environment.getExternalStorageState().equals(Environment.MEDIA_MOUNTED)
```

如果返回 true 就是有 sdcard，如果返回 false 则没有。

8、dvm 的进程和 Linux 的进程，应用程序的进程是否为同一个概念？

dvm 指 dalvik 的虚拟机。每一个 Android 应用程序都拥有一个独立的 Dalvik 虚拟机实例,应用程序都在它自己的进程中运行。而每一个 dvm 都是在 Linux 中的一个进程，所以说可以近似认为是同一个概念。

什么是 android DVM: Dalvik 是 Google 公司自己设计用于 Android 平台的 Java 虚拟机,每一个 Dalvik 应用作为一个独立的 Linux 进程执行。独立的进程可以防止在虚拟机崩溃的时候所有程序都被关闭。

Dalvik 和 Java 虚拟机的区别

1：Dalvik 主要是完成对象生命周期管理，堆栈管理，线程管理，安全和异常管理，以及垃圾回收等等重要功能。

2：Dalvik 负责进程隔离和线程管理，每一个 Android 应用在底层都会对应一个独立的 Dalvik 虚拟机实例，其代码在虚拟机的解释下得以执行。

3：不同于 Java 虚拟机运行 java 字节码，Dalvik 虚拟机运行的是其专有的文件格式 Dex

4: dex 文件格式可以减少整体文件尺寸，提高 I/O 操作的类查找速度。

5: odex 是为了在运行过程中进一步提高性能，对 dex 文件的进一步优化。

6：所有的 Android 应用的线程都对应一个 Linux 线程，虚拟机因而可以更多的依赖操作系统的线程调度和管理机制

7：有一个特殊的虚拟机进程 Zygote，他是虚拟机实例的孵化器。它在系统启动的时候就会产生，它会完成虚拟机的初始化，库的加载，预制类库和初始化的操作。如果系统需要一个新的虚拟机实例，它会迅速复制自身，以最快的数据提供给系统。对于一些只读的系统库，所有虚拟机实例都和 Zygote 共享一块内存区域。

9、Android 程序与 Java 程序的区别？

Android 程序用 android sdk 开发,java 程序用 javasdk 开发.

Android SDK 引用了大部分的 Java SDK，少数部分被 Android SDK 抛弃，比如说界面部分，java.awt swing package 除了 java.awt.font 被引用外，其他都被抛弃，在 Android 平台开发中不能使用。android sdk 添加工具 jar httpclient, pull opengl

10、启动应用后，改变系统语言，应用的语言会改变么？

这个一般是不会的，一般需要重启应用才能改变应用语言。但是对应应用来说如果做了国际化处理则支持如果没有处理那系统语言再更改也是无用的。

11、请介绍下 adb、ddms、aapt 的作用

adb 是 Android Debug Bridge ,Android 调试桥的意思，ddms 是 Dalvik Debug Monitor Service，dalvik 调试监视服务。aapt 即 Android Asset Packaging Tool，在 SDK 的 build-tools 目录下。该工具可以查看，创建，更新 ZIP 格式的文档附件(zip, jar, apk)。也可将资源文件编译成二进制文件，尽管我们没有直接使用过该工具，但是开发工具会使用这个工具打包 apk 文件构成一个 Android 应用程序。

Android 的主要调试工具是 adb(Android debugging bridge)，ddms 是一个在 adb 基础上的一个图形化工具。

adb，它是一个命令行工具。而 ddms 功能与 adb 相同，只是它有一个图形化界面。对不喜欢命令操作方式的人

来说是一个不错的选择。

12、ddms 和 traceview 的区别

ddms 原意是：davik debug monitor service。简单的说 ddms 是一个程序执行查看器，在里面可以看见线程和堆栈等信息，traceView 是程序性能分析器。traceview 是 ddms 中的一部分内容。

13、补充知识：TraceView 的使用

一、TraceView 简介

Traceview 是 Android 平台特有的数据采集和分析工具，它主要用于分析 Android 中应用程序的 hotspot（瓶颈）。Traceview 本身只是一个数据分析工具，而数据的采集则需要使用 Android SDK 中的 Debug 类或者利用 DDMS 工具。二者的用法如下：

开发者在一些关键代码段开始前调用 Android SDK 中 Debug 类的 startMethodTracing 函数，并在关键代码段结束前调用 stopMethodTracing 函数。这两个函数运行过程中将采集运行时间内该应用所有线程（注意，只能是 Java 线程）的函数执行情况，并将采集数据保存到/mnt/sdcard/下的一个文件中。开发者然后需要利用 SDK 中的 Traceview 工具来分析这些数据。

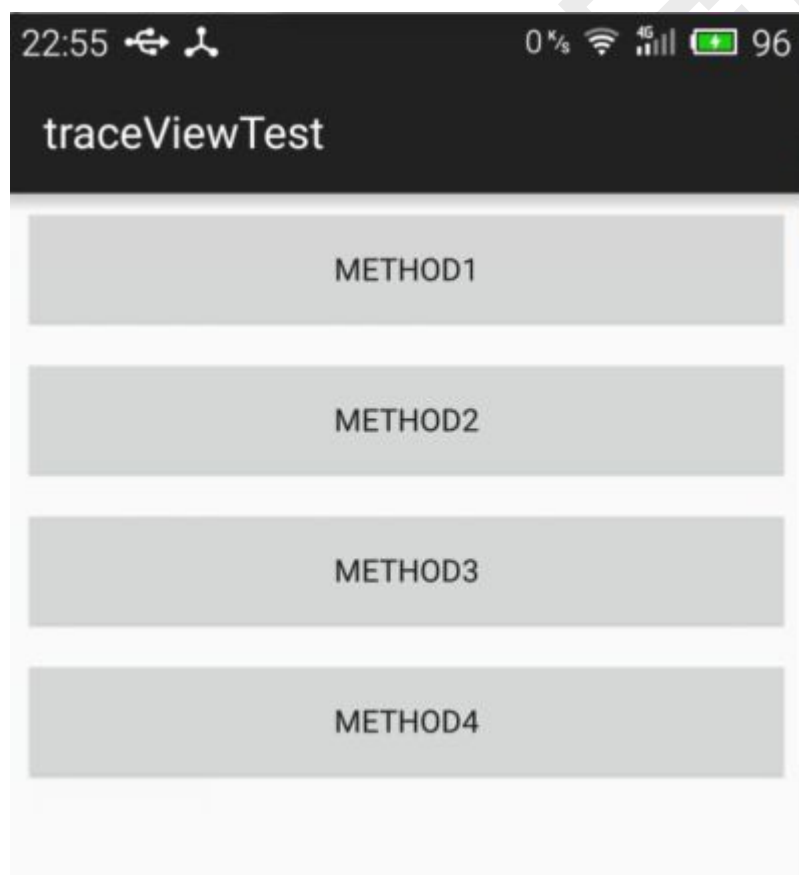
借助 Android SDK 中的 DDMS 工具。DDMS 可采集系统中某个正在运行的进程的函数调用信息。对开发者而言，此方法适用于没有目标应用源代码的情况。DDMS 工具中 Traceview 的使用如下图所示。



点击上图中所示按钮即可以采集目标进程的数据。当停止采集时，DDMS 会自动触发 Traceview 工具来浏览采集数据。

下面，我们通过一个示例程序介绍 Traceview 的使用。

实例程序如下图所示：界面有 4 个按钮，对应四个方法。



点击不同的方法会进行不同的耗时操作。

```
public class MainActivity extends ActionBarActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }

    public void method1(View view) {
        int result = jisuan();
        System.out.println(result);
    }
}
```

```
private int jisuan() {
    for (int i = 0; i < 10000; i++) {
        System.out.println(i);
    }
    return 1;
}

public void method2(View view) {
    SystemClock.sleep(2000);
}

public void method3(View view) {
    int sum = 0;
    for (int i = 0; i < 1000; i++) {
        sum += i;
    }
    System.out.println("sum=" + sum);
}

public void method4(View view) {
    Toast.makeText(this, "" + new Date(), 0).show();
}
}
```

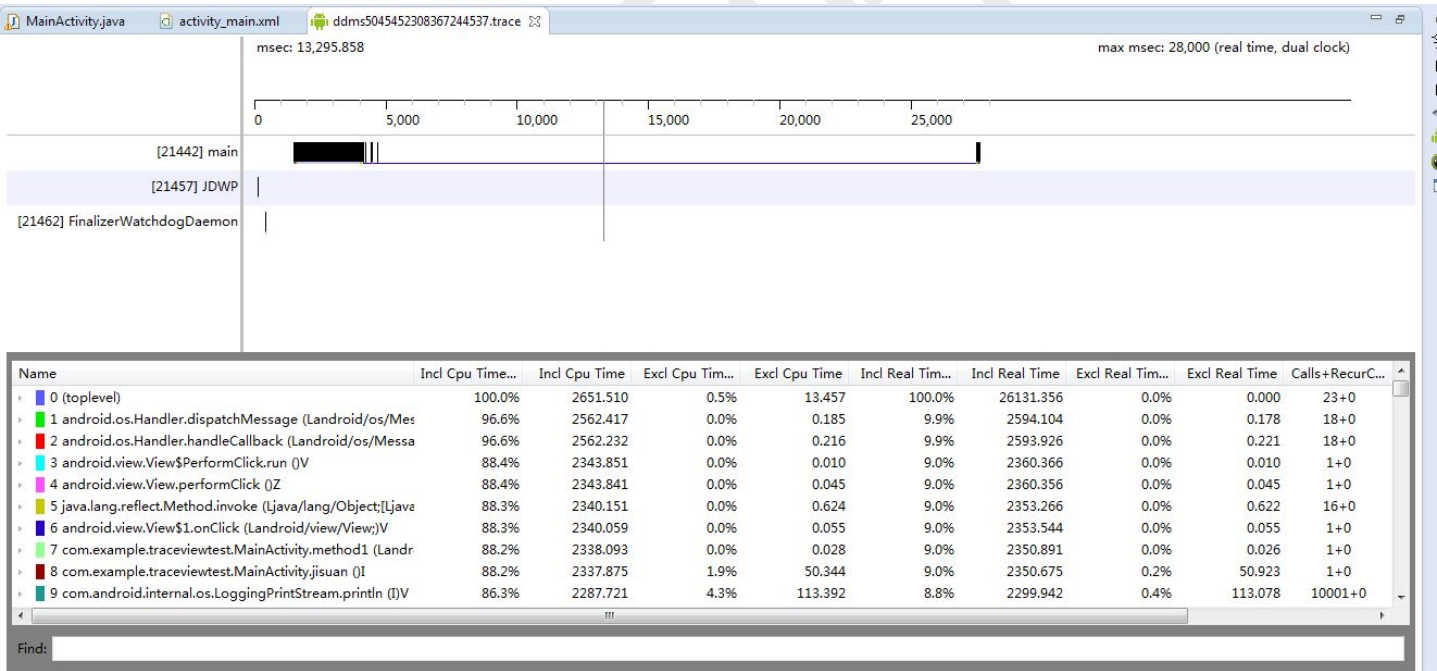
我们分别点击按钮一次，要求找出最耗时的方法。点击前通过 DDMS 启动 Start Method Profiling 按钮。



然后依次点击 4 个按钮，都执行后再次点击上图中红框中按钮，停止收集数据。

接下来我们开始对数据进行分析。

当我们停止收集数据的时候会出现如下分析图表。该图表分为 2 大部分，上面分不同的行，每一行代表一个线程的执行耗时情况。main 线程对应行的内容非常丰富，而其他线程在这段时间内干得工作则要少得多。图表的下半部分是具体的每个方法执行的时间情况。显示方法执行情况的前提是先选中某个线程。



我们主要是分析 main 线程。

上面方法指标参数所代表的意思如下：

列名	描述
Name	该线程运行过程中所调用的函数名

Incl Cpu Time	某函数占用的 CPU 时间，包含内部调用其它函数的 CPU 时间
Excl Cpu Time	某函数占用的 CPU 时间，但不含内部调用其它函数所占用的 CPU 时间
Incl Real Time	某函数运行的真实时间（以毫秒为单位），内含调用其它函数所占用的真实时间
Excl Real Time	某函数运行的真实时间（以毫秒为单位），不含调用其它函数所占用的真实时间
Call+Recur Calls/Total	某函数被调用次数以及递归调用占总调用次数的百分比
Cpu Time/Call	某函数调用 CPU 时间与调用次数的比。相当于该函数平均执行时间
Real Time/Call	同 CPU Time/Call 类似，只不过统计单位换成了真实时间

我们为了找到最耗时的操作，那么可以通过点击 Incl Cpu Time，让其按照时间的倒序排列。我点击后效果如下图：

Name	Incl Cpu Time...	Incl Cpu ...	Excl Cpu Tim...	Excl Cpu Time	Incl Real Tim...	Incl Real Time	Excl Real Tim...	Excl Real Time	Calls+RecurC...
0 (toplevel)	100.0%	2651.510	0.5%	13.457	100.0%	26131.356	0.0%	0.000	23+0
1 android.os.Handler.dispatchMessage (Landroid/os/Mes	96.6%	2562.417	0.0%	0.185	9.9%	2594.104	0.0%	0.178	18+0
2 android.os.Handler.handleCallback (Landroid/os/Messa	96.6%	2562.232	0.0%	0.216	9.9%	2593.926	0.0%	0.221	18+0
3 android.view.View\$PerformClick.run ()V	88.4%	2343.851	0.0%	0.010	9.0%	2360.366	0.0%	0.010	1+0
4 android.view.View.performClick ()Z	88.4%	2343.841	0.0%	0.045	9.0%	2360.356	0.0%	0.045	1+0
5 java.lang.reflect.Method.invoke (Ljava/lang/Object;[Jjav	88.3%	2340.151	0.0%	0.624	9.0%	2353.266	0.0%	0.622	16+0
6 android.view.View\$1.onClick (Landroid/view/View;)V	88.3%	2340.059	0.0%	0.055	9.0%	2353.544	0.0%	0.055	1+0
7 com.example.traceviewtest.MainActivity.method1 (Landr	88.2%	2338.093	0.0%	0.028	9.0%	2350.891	0.0%	0.026	1+0
8 com.example.traceviewtest.MainActivity.jisuan ()I	88.2%	2337.875	1.9%	50.344	9.0%	2350.675	0.2%	50.923	1+0
9 com.android.internal.os.LoggingPrintStream.println (I)V	86.3%	2287.721	4.3%	113.392	8.8%	2299.942	0.4%	113.078	10001+0
10 com.android.internal.os.LoggingPrintStream.flush (Z)V	52.1%	1382.477	8.9%	234.993	5.3%	1388.300	0.9%	235.907	10001+0
11 java.lang.StringBuilder.append (Ljava/lang/StringBuilc	29.9%	791.852	2.6%	69.036	3.1%	798.386	0.3%	69.077	10001+0
12 java.lang.IntegralToString.appendInt (Ljava/lang/Abstr	27.3%	722.816	2.6%	67.797	2.8%	729.309	0.3%	67.760	10001+0
13 java.lang.IntegralToString.convertInt (Ljava/lang/Abstre	24.7%	655.084	5.5%	146.132	2.5%	659.603	0.6%	147.898	10002+0
14 java.lang.StringBuilder.substring (I)Ljava/lang/String;	13.7%	363.810	2.6%	70.150	1.4%	365.069	0.3%	69.746	10001+0
15 java.lang.ThreadLocal.get ()Ljava/lang/Object;	11.6%	308.482	7.6%	201.578	1.2%	311.870	0.8%	203.362	9984+0

通过分析发现：method1 最耗时，耗时 2338 毫秒。

0 android.view.View\$1.onClick (Landroid/view/View;)V
7 com.example.traceviewtest.MainActivity.method1 (Landr
8 com.example.traceviewtest.MainActivity.jisuan ()I

那么有了上面的信息我们可以进入我们的 method1 方法查看分析我们的代码了。

14、Android 中数据存储方式有哪些？

- 文件存储
- xml，SharedPreferences
- SQLiteDatabase

- d) ContentProvider
- e) 网络

15、DVM 和 JVM 的区别？

- a) dvm 执行的是.dex 文件，而 jvm 执行的是.class。Android 工程编译后的所有.class 字节码会被 dex 工具抽取到一个.dex 文件中。
- b) dvm 是基于寄存器的虚拟机 而 jvm 执行是基于虚拟栈的虚拟机。寄存器存取速度比栈快的多，dvm 可以根据硬件实现最大的优化，比较适合移动设备。
- c) .class 文件存在很多的冗余信息，dex 工具会去除冗余信息，并把所有的.class 文件整合到.dex 文件中。减少了 I/O 操作，提高了类的查找速度。

16、谈一谈 Android 的安全机制

- 1、Android 是基于 Linux 内核的，因此 Linux 对文件权限的控制同样适用于 Android

在 Android 中每个应用都有自己的/data/data/包名 文件夹，该文件夹只能该应用访问，而其他应用则无权访问。

- 2、Android 的权限机制保护了用户的合法权益

如果我们的代码想拨打电话、发送短信、访问通信录、定位、访问 sdcard 等所有可能侵犯用户权益的行为都是必须要在 AndroidManifest.xml 中进行声明的，这样就给了用户一个知情权。

- 3、Android 的代码混淆保护了开发者的劳动成果

17、Android 的四大组件都需要在清单文件中注册吗？

Activity、Service、ContentProvider 如果要使用则必须在 AndroidManifest.xml 中进行注册，而 BroadcastReceiver 则有两种注册方式，静态注册和动态注册。其中静态注册就是指在 AndroidManifest.xml 中进行

注册，而动态注册时通过代码注册。

18、在 Android 中进程的级别有哪些？

- a) Foreground process
- b) Visible process
- c) Service process
- d) Background process
- e) Empty process

二、Activity

1、什么是 Activity?

四大组件之一,通常一个用户交互界面对应一个 activity。activity 是 Context 的子类,同时实现了 window.callback 和 KeyEvent.callback, 可以处理与窗体用户交互的事件。

常见的 Activity 类型有 FragmentActivity, ListActivity, TabActivity 等。

如果界面有共同的特点或者功能的时候,还会自己定义一个 BaseActivity。

2、请描述一下 Activity 生命周期

Activity 从创建到销毁有多种状态，从一种状态到另一种状态时会激发相应的回调方法，这些回调方法包括：

onCreate onStart onResume onPause onStop onDestroy

其实这些方法都是两两对应的，onCreate 创建与 onDestroy 销毁；

onStart 可见与 onStop 不可见；onResume 可编辑（即焦点）与 onPause；

这 6 个方法是相对应的，那么就只剩下一个 onRestart 方法了，这个方法在什么时候调用呢？

答案就是：在 Activity 被 onStop 后，但是没有被 onDestroy，在再次启动此 Activity 时就调用 onRestart（而不再调用 onCreate）方法；

如果被 onDestroy 了，则是调用 onCreate 方法。

3、Activity 的状态都有哪些？

- a) foreground activity
- b) visible activity
- c) background activity
- d) empty process

4、如何保存 Activity 的状态？

Activity 的状态通常情况下系统会自动保存的，只有当我们需要保存额外的数据时才需要使用到这样的功能。

一般来说，调用 onPause()和 onStop()方法后的 activity 实例仍然存在于内存中，activity 的所有信息和状态数据不会消失，当 activity 重新回到前台之后，所有的改变都会得到保留。

但是当系统内存不足时，调用 onPause()和 onStop()方法后的 activity 可能会被系统摧毁，此时内存中就不会存有该 activity 的实例对象了。如果之后这个 activity 重新回到前台，之前所作的改变就会消失。为了避免此种情况的发生，我们可以覆写 onSaveInstanceState()方法。onSaveInstanceState()方法接受一个 Bundle 类型的参数，开发者可以将状态数据存储到这个 Bundle 对象中，这样即使 activity 被系统摧毁，当用户重新启动这个 activity 而调用它的 onCreate()方法时，上述的 Bundle 对象会作为实参传递给 onCreate()方法，开发者可以从 Bundle 对象中取出保存的数据，然后利用这些数据将 activity 恢复到被摧毁之前的状态。

需要注意的是，onSaveInstanceState()方法并不是一定会被调用的，因为有些场景是不需要保存状态数据的。比如用户按下 BACK 键退出 activity 时，用户显然想要关闭这个 activity，此时是没有必要保存数据以供下次恢复的，也

就是 `onSaveInstanceState()` 方法不会被调用。如果调用 `onSaveInstanceState()` 方法，调用将发生在 `onPause()` 或 `onStop()` 方法之前。

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    // TODO Auto-generated method stub
    super.onSaveInstanceState(outState);
}
```

5、两个 Activity 之间跳转时必然会执行的是哪几个方法？（重要）

一般情况下比如说有两个 activity, 分别叫 A,B, 当在 A 里面激活 B 组件的时候, A 会调用 `onPause()` 方法, 然后 B 调用 `onCreate()`, `onStart()`, `onResume()`。

这个时候 B 覆盖了窗体, A 会调用 `onStop()` 方法。如果 B 是个透明的, 或者是对话框的样式, 就不会调用 A 的 `onStop()` 方法。

如下图，打开一个 MainActivity，然后点击 MainActivity 中的按钮跳转到 SecondActivity 的日志。

Text
MainActivity onCreate
MainActivity onStart
MainActivity onResume
MainActivity onPause
SecondActivity onCreate
SecondActivity onStart
SecondActivity onResume
MainActivity onStop

6、横竖屏切换时 Activity 的生命周期

此时的生命周期跟清单文件里的配置有关系。

1、不设置 Activity 的 `android:configChanges` 时，切屏会重新调用各个生命周期

默认首先销毁当前 activity，然后重新加载。

如下图，当横竖屏切换时先执行 onPause/onStop 方法

```
Text
SecondActivity onPause
SecondActivity onStop
SecondActivity onCreate
SecondActivity onStart
SecondActivity onResume
```

2、设置 Activity 的 android:configChanges="orientation|keyboardHidden|screenSize"时，切屏不会重新调用各个生命周期，只会执行 onConfigurationChanged 方法。

通常在游戏开发，屏幕的朝向都是写死的。

7、如何将一个 Activity 设置成窗口的样式？

只需要给我们的 Activity 配置如下属性即可。

```
android:theme="@android:style/Theme.Dialog"
```

8、如何退出 Activity？如何安全退出已调用多个 Activity 的 Application？

- 1、通常情况用户退出一个 Activity 只需按返回键，我们写代码想退出 activity 直接调用 finish()方法就行。
- 2、记录打开的 Activity：

每打开一个 Activity，就记录下来。在需要退出时，关闭每一个 Activity 即可。

```
//伪代码
List<Activity> lists ;// 在 application 全局的变量里面
lists = new ArrayList<Activity>();
lists.add(this);
for(Activity activity: lists)
{
    activity.finish();
}
lists.remove(this);
```

3、发送特定广播：

在需要结束应用时，发送一个特定的广播，每个 Activity 收到广播后，关闭即可。

//给某个 activity 注册接受接受广播的意图

```
registerReceiver(receiver, filter)
```

//如果过接受到的是 关闭 activity 的广播 就调用 finish()方法 把当前的 activity finish()掉

4、递归退出

在打开新的 Activity 时使用 startActivityForResult，然后自己加标志，在 onActivityResult 中处理，递归关闭。

5、其实 也可以通过 intent 的 flag 来实现 intent.setFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP)激活一个新的 activity。此时如果该任务栈中已经有该 Activity，那么系统会把这个 Activity 上面的所有 Activity 干掉。其实相当于给 Activity 配置的启动模式为 SingleTop。

9、请描述一下 Activity 的启动模式都有哪些以及各自的特点

启动模式(launchMode)在多个 Activity 跳转的过程中扮演着重要的角色，它可以决定是否生成新的 Activity 实例，是否重用已存在的 Activity 实例，是否和其他 Activity 实例公用一个 task 里。这里简单介绍一下 task 的概念，task 是一个具有栈结构的对象，一个 task 可以管理多个 Activity，启动一个应用，也就创建一个与之对应的 task。

Activity 一共有以下四种 launchMode：

- ◆ 1.standard
- ◆ 2.singleTop
- ◆ 3.singleTask
- ◆ 4.singleInstance

我们可以在 AndroidManifest.xml 配置<activity>的 android:launchMode 属性为以上四种之一即可。

下面我们结合实例——介绍这四种 launchMode：

8.1 standard

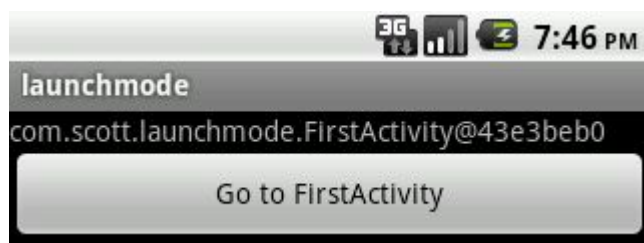
standard 模式是默认的启动模式，不用为<activity>配置 android:launchMode 属性即可，当然也可以指定值为 standard。

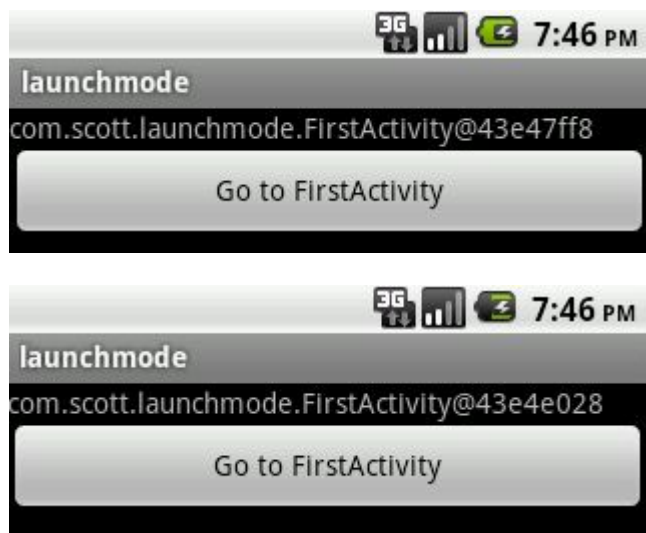
我们将创建一个 Activity，命名为 FirstActivity，来演示一下标准的启动模式。FirstActivity 代码如下：

```
public class FirstActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.first);
        TextView textView = (TextView) findViewById(R.id.tv);
        textView.setText(this.toString());
        Button button = (Button) findViewById(R.id.bt);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(FirstActivity.this,
                FirstActivity.class);
                startActivity(intent);
            }
        });
    }
}
```

FirstActivity 界面中的 TextView 用于显示当前 Activity 实例的序列号，Button 用于跳转到下一个 FirstActivity 界面。

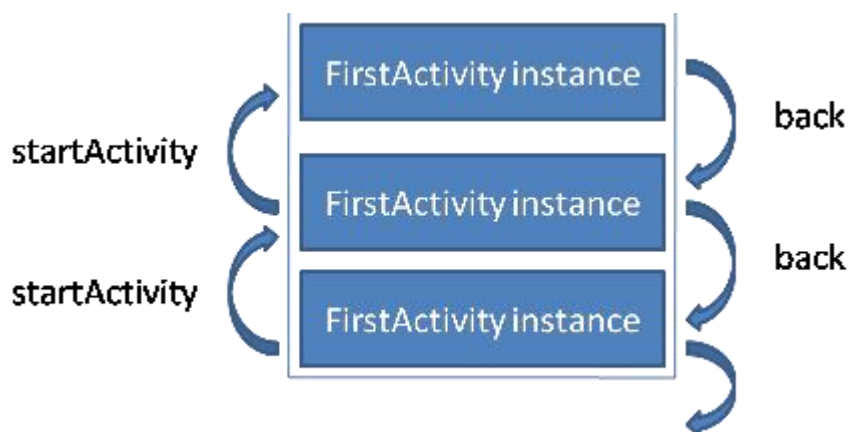
然后我们连续点击几次按钮，将会出现下面的现象：





我们注意到都是 FirstActivity 的实例，但序列号不同，并且我们需要连续按后退键两次，才能回到第一个 FirstActivity。

standard 模式的原理如下图所示：

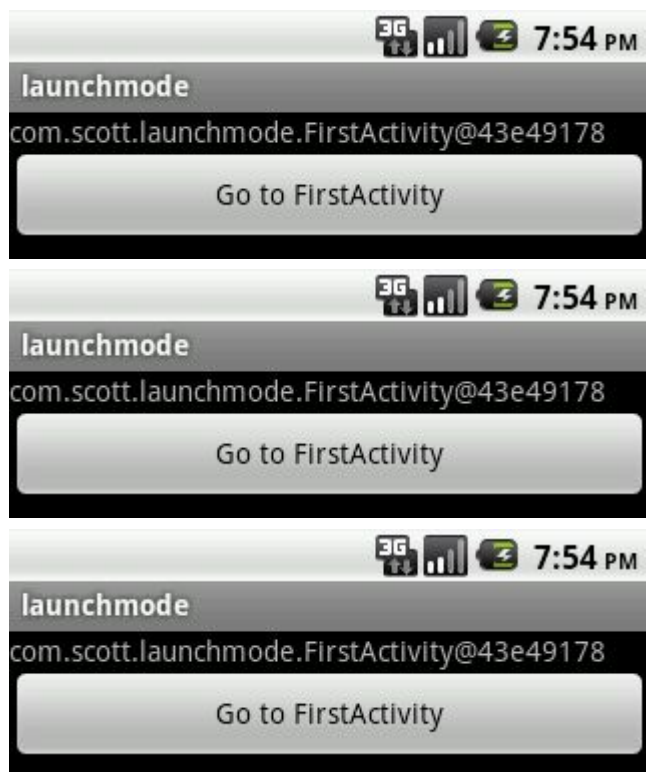


如图所示，每次跳转系统都会生成一个新的 FirstActivity 实例，并且放于栈结构的顶部，当我们按下后退键时，才能看到原来的 FirstActivity 实例。

这就是 standard 启动模式，不管有没有已存在的实例，都生成新的实例。

8.2 singleTop

我们在上面的基础上为 <activity> 指定属性 **android:launchMode="singleTop"**，系统就会按照 singleTop 启动模式处理跳转行为。我们重复上面几个动作，将会出现下面的现象：



我们看到这个结果跟 standard 有所不同，三个序列号是相同的，也就是说使用的都是同一个 FirstActivity 实例；如果按一下后退键，程序立即退出，说明当前栈结构中只有一个 Activity 实例。singleTop 模式的原理如下图所示：



正如下图所示，跳转时系统会先在栈结构中寻找是否有一个 FirstActivity 实例正位于栈顶，如果有则不再生成新的，而是直接使用。也许朋友们会有疑问，我只看到栈内只有一个 Activity，如果是多个 Activity 怎么办，如果不是在栈顶会如何？我们接下来再通过一个示例来证实一下大家的疑问。

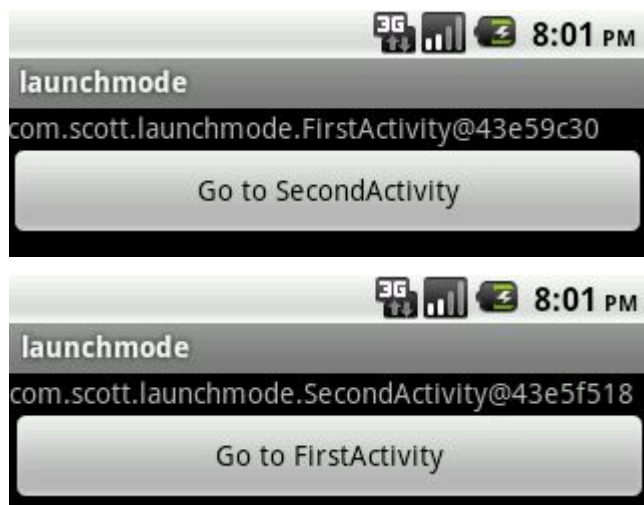
我们再新建一个 Activity 命名为 SecondActivity，如下：

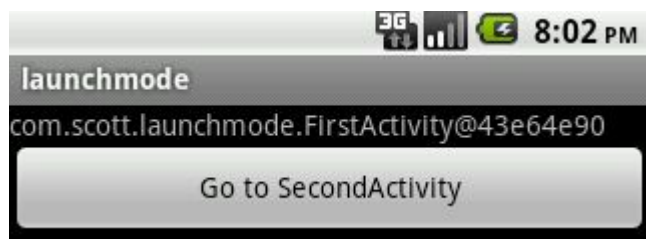
```
public class SecondActivity extends Activity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.second);
        TextView textView = (TextView) findViewById(R.id.tv);
        textView.setText(this.toString());
        Button button = (Button) findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent intent = new Intent(SecondActivity.this,
                FirstActivity.class);
                startActivity(intent);
            }
        });
    }
}
```

然后将之前的 FirstActivity 跳转代码改为：

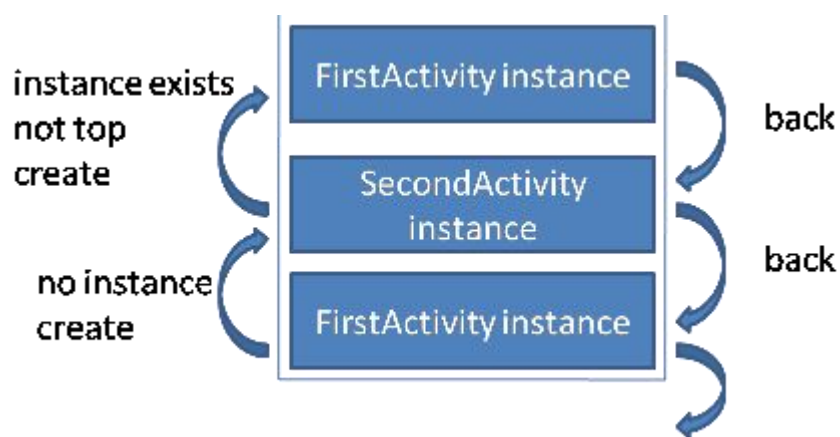
```
Intent intent = new Intent(FirstActivity.this, SecondActivity.class);
startActivity(intent);
```

这时候，FirstActivity 会跳转到 SecondActivity，SecondActivity 又会跳转到 FirstActivity。演示结果如下：





我们看到，两个 FirstActivity 的序号是不同的，证明从 SecondActivity 跳转到 FirstActivity 时生成了新的 FirstActivity 实例。原理图如下：

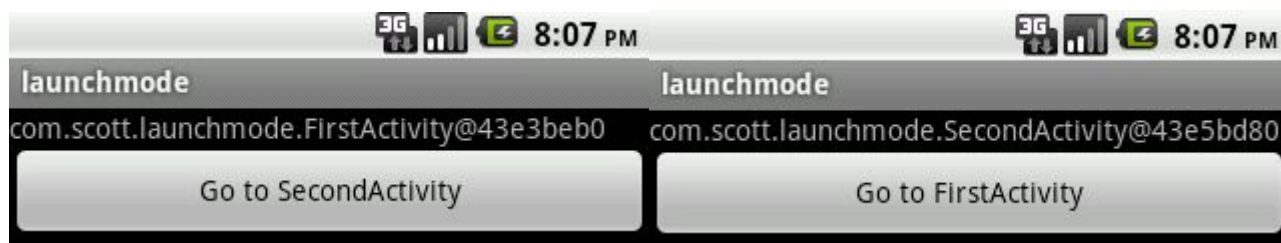


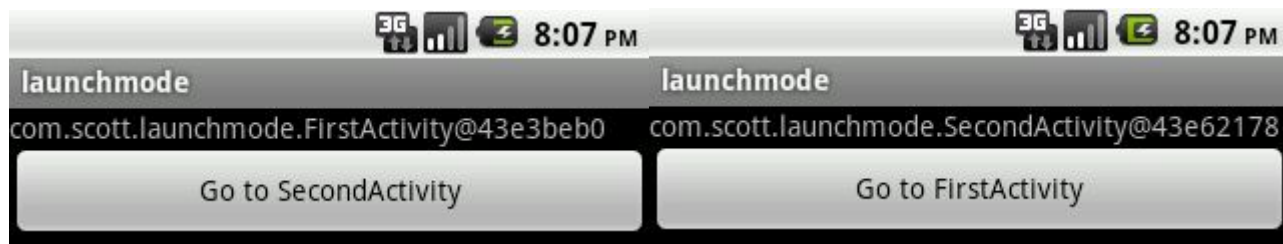
我们看到，当从 SecondActivity 跳转到 FirstActivity 时，系统发现存在有 FirstActivity 实例,但不是位于栈顶，于是重新生成一个实例。

这就是 singleTop 启动模式，如果发现有对应的 Activity 实例正位于栈顶，则重复利用，不再生成新的实例。

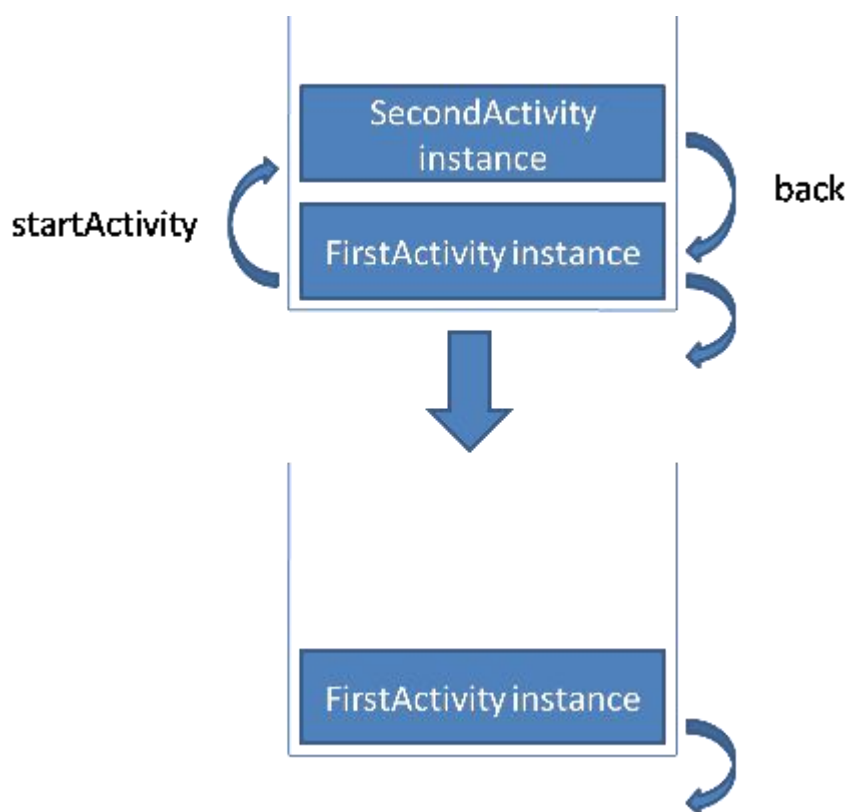
8.3 singleTask

在上面的基础上我们修改 FirstActivity 的属性 **android:launchMode="singleTask"**。演示的结果如下：





我们注意到，在上面的过程中，FirstActivity 的序列号是不变的，SecondActivity 的序列号却不是唯一的，说明从 SecondActivity 跳转到 FirstActivity 时，没有生成新的实例，但是从 FirstActivity 跳转到 SecondActivity 时生成了新的实例。singleTask 模式的原理图如下图所示：



在图中的下半部分是 SecondActivity 跳转到 FirstActivity 后的栈结构变化的结果，我们注意到，SecondActivity 消失了，没错，在这个跳转过程中系统发现有存在的 FirstActivity 实例，于是不再生成新的实例，而是将 FirstActivity 之上的 Activity 实例统统出栈，将 FirstActivity 变为栈顶对象，显示到幕前。也许朋友们有疑问，如果将 SecondActivity 也设置为 singleTask 模式，那么 SecondActivity 实例是不是可以唯一呢？在我们这个示例中是不可能的，因为每次从 SecondActivity 跳转到 FirstActivity 时，SecondActivity 实例都被迫出栈，下次等 FirstActivity 跳转到

SecondActivity 时，找不到存在的 SecondActivity 实例，于是必须生成新的实例。但是如果我们有 ThirdActivity，让 SecondActivity 和 ThirdActivity 互相跳转，那么 SecondActivity 实例就可以保证唯一。

这就是 singleTask 模式，如果有对应的 Activity 实例，则使此 Activity 实例之上的其他 Activity 实例统统出栈，使此 Activity 实例成为栈顶对象，显示到幕前。

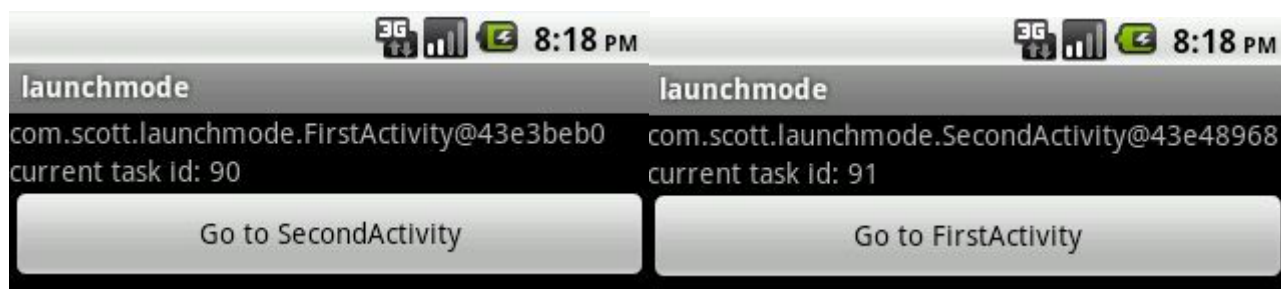
8.4 singleInstance

这种启动模式比较特殊，因为它会启用一个新的栈结构，将 Activity 放置于这个新的栈结构中，并保证不再有其他 Activity 实例进入。

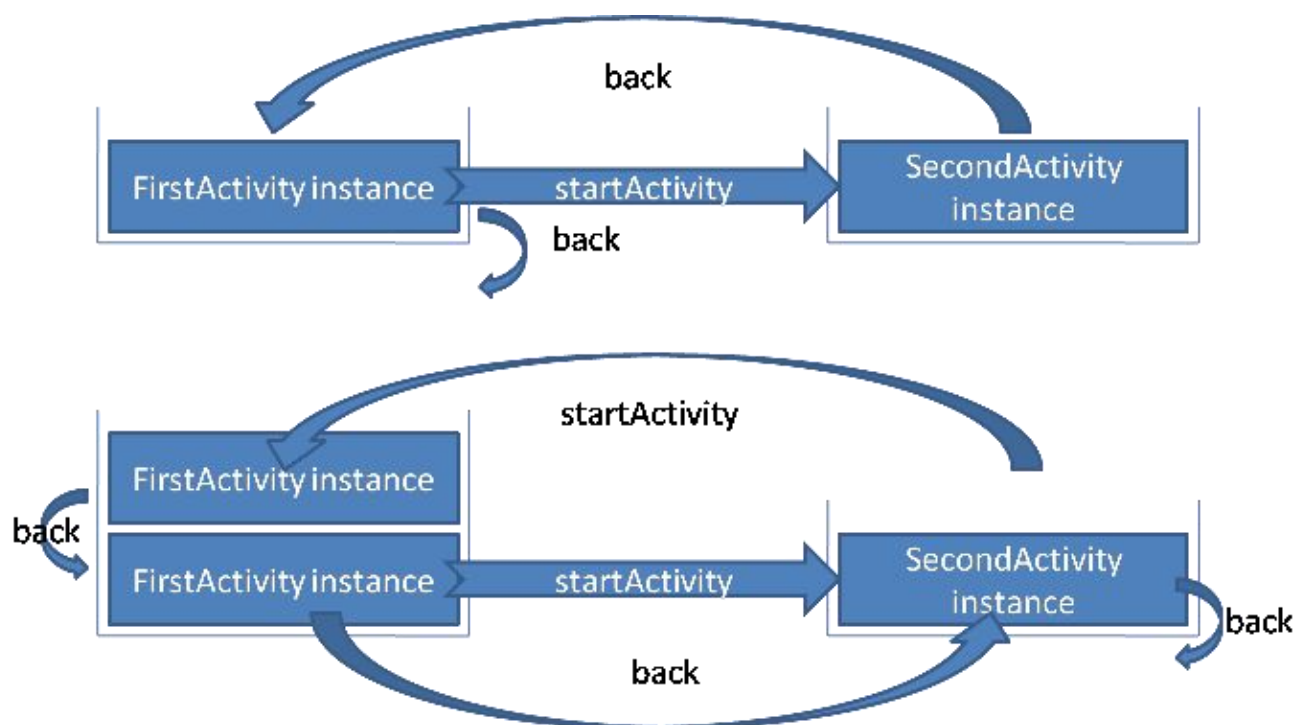
我们修改 FirstActivity 的 launchMode="standard"，SecondActivity 的 launchMode="singleInstance"，由于涉及到了多个栈结构，我们需要在每个 Activity 中显示当前栈结构的 id，所以我们为每个 Activity 添加如下代码：

```
TextView taskIdView = (TextView) findViewById(R.id.taskIdView);  
taskIdView.setText("current task id: " + this.getTaskId());
```

然后再演示一下这个流程：

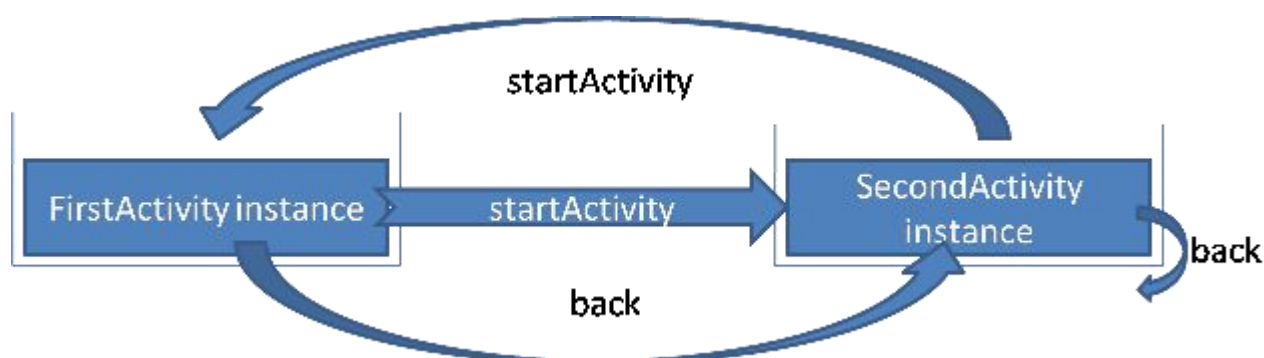


我们发现这两个 Activity 实例分别被放置在不同的栈结构中，关于 singleInstance 的原理图如下



我们看到从 FirstActivity 跳转到 SecondActivity 时，重新启用了一个新的栈结构，来放置 SecondActivity 实例，然后按下后退键，再次回到原始栈结构；图中下半部分显示的在 SecondActivity 中再次跳转到 FirstActivity，这个时候系统会在原始栈结构中生成一个 FirstActivity 实例，然后回退两次，注意，并没有退出，而是回到了 SecondActivity，为什么呢？是因为从 SecondActivity 跳转到 FirstActivity 的时候，我们的起点变成了 SecondActivity 实例所在的栈结构，这样一来，我们需要“回归”到这个栈结构。

如果我们修改 FirstActivity 的 launchMode 值为 singleTop、singleTask、singleInstance 中的任意一个，流程将会如图所示：



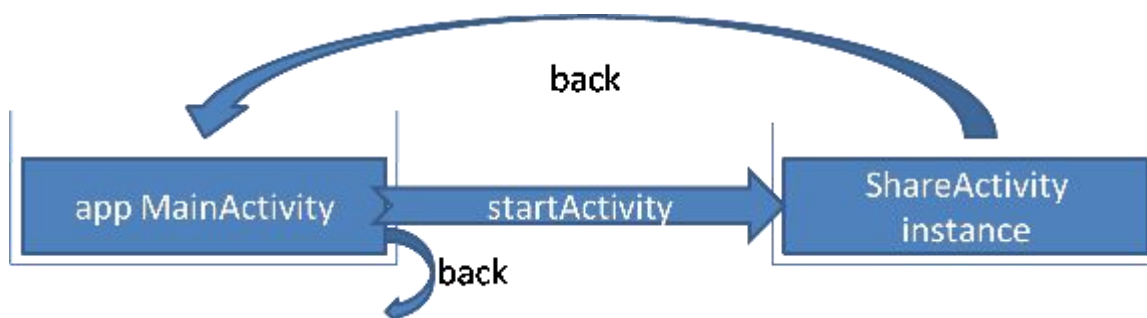
singleInstance 启动模式可能是最复杂的一种模式，为了帮助大家理解，我举一个例子，假如我们有一个 share 应用，其中的 ShareActivity 是入口 Activity，也是可供其他应用调用的 Activity，我们把这个 Activity 的启动模式设置为 singleInstance，然后在其他应用中调用。我们编辑 ShareActivity 的配置：

```
<activity
    android:name=".ShareActivity"
    android:launchMode="singleInstance" >
    <intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
    <intent-filter>
    <action android:name="android.intent.action.SINGLE_INSTANCE_SHARE" />
    <category android:name="android.intent.category.DEFAULT" />
    </intent-filter>
</activity>
```

然后我们在其他应用中这样启动该 Activity：

```
Intent intent = new Intent("android.intent.action.SINGLE_INSTANCE_SHARE");
startActivity(intent);
```

当我们打开 ShareActivity 后再按后退键回到原来界面时，ShareActivity 作为一个独立的个体存在，如果这时我们打开 share 应用，无需创建新的 ShareActivity 实例即可看到结果，因为系统会自动查找，存在则直接利用。大家可以在 ShareActivity 中打印一下 taskId，看看效果。关于这个过程，原理图如下：



10、一个启动模式为 singleTop 的 activity ,如果再试图启动会怎样？ 面试官想问的是 onNewIntent() (2015-11-24)

Activity 有一个 onNewIntent(Intent intent)回调方法，该方法我们几乎很少使用，导致已经将其忽略掉。该方法的官方解释如下：

```
This is called for activities that set launchMode to "singleTop" in their package, or if a client used the Intent.FLAG_ACTIVITY_SINGLE_TOP flag when calling startActivity. In either case, when the activity is re-launched while at the top of the activity stack instead of a new instance of the activity being started, onNewIntent() will be called on the existing instance with the Intent that was used to re-launch it.
```

```
An activity will always be paused before receiving a new intent, so you can count on onResume being called after this method.
```

```
Note that getIntent still returns the original Intent. You can use setIntent to update it to this new Intent.
```

上文大概意思如下：

该方法被启动模式设置为“singleTop”的 Activity 回调，或者当通过设置 Intent.FLAG_ACTIVITY_SINGLE_TOP 的 Intent 启动 Activity 时被回调。在任何情况下，只要当栈顶的 Activity 被重新启动时没有重新创建一个新的 Activity 实例而是依然使用该 Activity 对象，那么 onNewIntent(Intent)方法就会被回调。

当一个 Activity 接收到新 Intent 的时候会处于暂停状态，因此你可以统计到 onResume()方法会被再次执行，当然这个执行是在 onNewIntent 之后的。

注意：如果我们在 Activity 中调用了 getIntent () 方法，那么返回的 Intent 对象还是老的 Intent (也就是第一次启动该 Activity 时的传入的 Intent 对象)，但是如果想让 getIntent () 返回最新的 Intent，那么我们可以通过 setIntent(Intent)方法设置。

三、Service

1、Service 是否在 main thread 中执行, service 里面是否能执行耗时的操作?

默认情况,如果没有显示的指 service 所运行的进程, Service 和 activity 是运行在当前 app 所在进程的 main

thread(UI 主线程)里面。

service 里面不能执行耗时的操作(网络请求,拷贝数据库,大文件)

特殊情况 ,可以在清单文件配置 service 执行所在的进程 ,让 service 在另外的进程中执行

```
<service
    android:name="com.baidu.location.f"
    android:enabled="true"
    android:process=":remote" >
</service>
```

2、Activity 怎么和 Service 绑定，怎么在 Activity 中启动自己对应的 Service ？

Activity 通过 `bindService(Intent service, ServiceConnection conn, int flags)`跟 Service 进行绑定，当绑定成功的时候 Service 会将代理对象通过回调的形式传给 `conn`，这样我们就拿到了 Service 提供的服务代理对象。

在 Activity 中可以通过 `startService` 和 `bindService` 方法启动 Service。一般情况下如果想获取 Service 的服务对象那么肯定需要通过 `bindService ( )` 方法，比如音乐播放器，第三方支付等。如果仅仅只是为了开启一个后台任务那么可以使用 `startService ( )` 方法。

3、请描述一下 Service 的生命周期

Service 有绑定模式和非绑定模式，以及这两种模式的混合使用方式。不同的使用方法生命周期方法也不同。

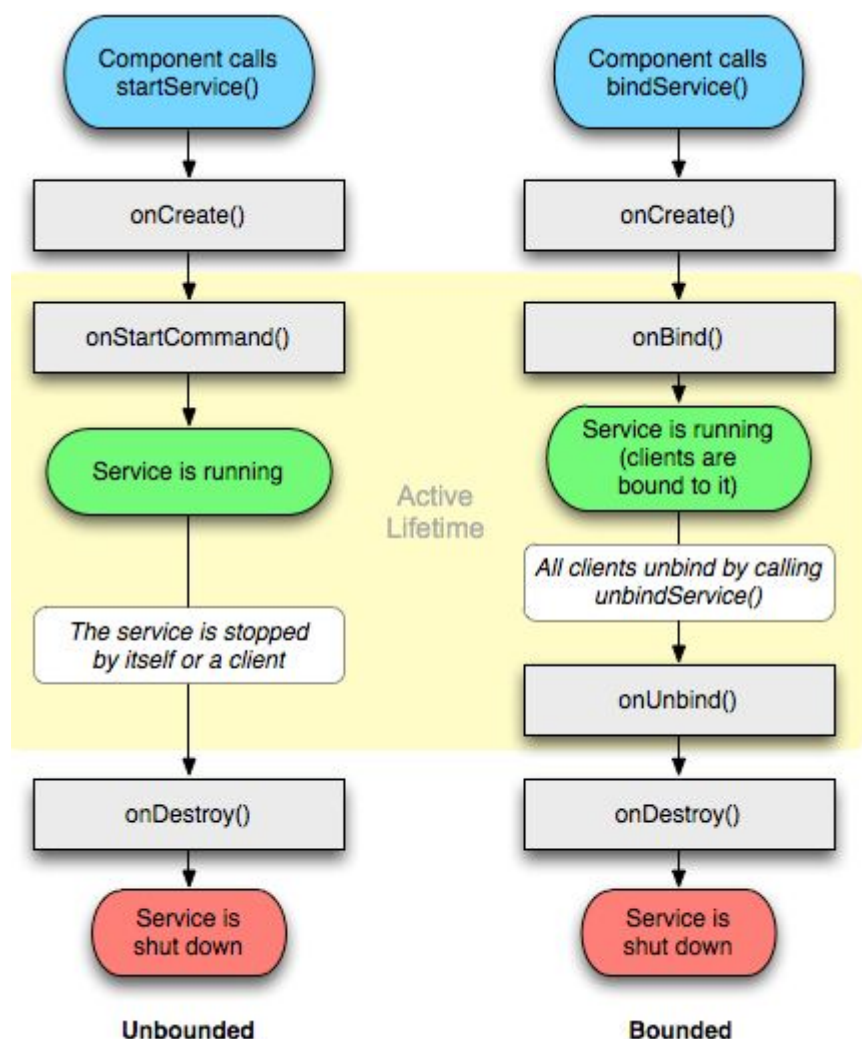
非绑定模式 :当第一次调用 `startService` 的时候执行的方法依次为 `onCreate()`、`onStartCommand()` (`onStart()`)
当 Service 关闭的时候调用 `onDestory` 方法。

绑定模式：第一次 `bindService ( )` 的时候，执行的方法为 `onCreate()`、`onBind( )` 解除绑定的时候会执行 `onUnbind()`、`onDestory()`。

上面的两种生命周期是在相对单纯的模式下的情形。我们在开发的过程中还必须注意 Service 实例只会有一个，也就是说如果当前要启动的 Service 已经存在了那么就不会再次创建该 Service 当然也不会调用 `onCreate ( )` 方法。

一个 Service 可以被多个客户进行绑定，只有所有的绑定对象都执行了 `onBind()` 方法后该 Service 才会销毁，不过如果有一个客户执行了 `onStart()` 方法，那么这个时候如果所有的 bind 客户都执行了 `unBind()` 该 Service 也不会销毁。

Service 的生命周期图如下所示，帮助大家记忆。



4、什么是 IntentService ？有何优点？

我们通常只会使用 Service，可能 IntentService 对大部分同学来说都是第一次听说。那么看了下面的介绍相信你就不会再陌生了。如果你还是不了解那么在面试的时候你就坦诚说没用过或者不了解等。并不是所有的问题都需要回答上来的。

一、IntentService 简介

IntentService 是 Service 的子类，比普通的 Service 增加了额外的功能。先看 Service 本身存在两个问题：

Service 不会专门启动一条单独的进程，Service 与它所在应用位于同一个进程中；

Service 也不是专门一条新线程，因此不应该在 Service 中直接处理耗时的任务；

二、IntentService 特征

会创建独立的 worker 线程来处理所有的 Intent 请求；

会创建独立的 worker 线程来处理 onHandleIntent()方法实现的代码，无需处理多线程问题；

所有请求处理完成后，IntentService 会自动停止，无需调用 stopSelf()方法停止 Service；

为 Service 的 onBind()提供默认实现，返回 null；

为 Service 的 onStartCommand 提供默认实现，将请求 Intent 添加到队列中；

三、使用 IntentService

本人写了一个 IntentService 的使用例子供参考。该例子中一个 MainActivity 一个 MyIntentService，这两个类都是四大组件当然需要在清单文件中注册。这里只给出核心代码：

MainActivity.java:

```
public void click(View view){
    Intent intent = new Intent(this, MyIntentService.class);
    intent.putExtra("start", "MyIntentService");
    startService(intent);
}
```

MyIntentService.java


```
public class MyIntentService extends IntentService {

    private String ex = "";
    private Handler mHandler = new Handler() {

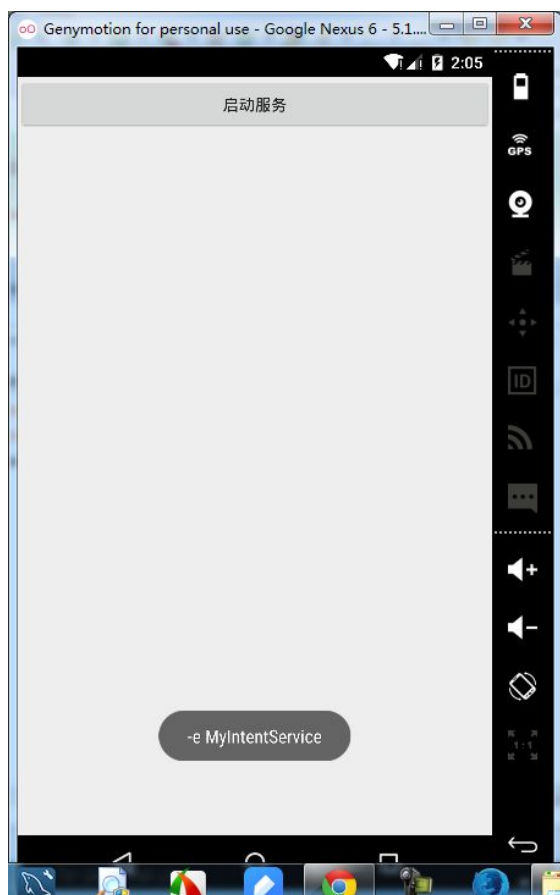
        public void handleMessage(android.os.Message msg) {
            Toast.makeText(MyIntentService.this, "-e " + ex,
                Toast.LENGTH_LONG).show();
        }
    };

    public MyIntentService(){
        super("MyIntentService");
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        ex = intent.getStringExtra("start");
        return super.onStartCommand(intent, flags, startId);
    }

    @Override
    protected void onHandleIntent(Intent intent) {
        /**
         * 模拟执行耗时任务
         * 该方法是在子线程中执行的，因此需要用到 handler 跟主线程进行通信
         */
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        mHandler.sendEmptyMessage(0);
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

运行后效果如下：



5、说说 Activity、Intent、Service 是什么关系

他们都是 Android 开发中使用频率最高的类。其中 Activity 和 Service 都是 Android 四大组件之一。他俩都是 Context 类的子类 ContextWrapper 的子类，因此他俩可以算是兄弟关系吧。不过兄弟俩各有各的本领，Activity 负责用户界面的显示和交互，Service 负责后台任务的处理。Activity 和 Service 之间可以通过 Intent 传递数据，因此可以把 Intent 看作是通信使者。

6、Service 和 Activity 在同一个线程吗

对于同一 app 来说默认情况下是在同一个线程中的，main Thread（UI Thread）。

7、Service 里面可以弹吐司么？

可以的。弹吐司有个条件就是得有一个 Context 上下文，而 Service 本身就是 Context 的子类，因此在 Service

里面弹吐司是完全可以的。比如我们在 Service 中完成下载任务后可以弹一个吐司通知用户。

8、如何让一个 Service 成为前置进程？

在启动该 Service 的时候可以在添加上如下方法：

```
@Override
public int onStartCommand(Intent intent, int flags, int startId) {
    Notification notification = new Notification(R.drawable.ic_launcher, "服务正在运行", System.currentTimeMillis());
    Intent notificationIntent = new Intent(this, MainActivity.class);
    PendingIntent pendingIntent = PendingIntent.getActivity(this, 0, notificationIntent, 0);
    notification.setLatestEventInfo(this, "消息标题", "消息内容", pendingIntent);
    startForeground(1, notification);
    return super.onStartCommand(intent, flags, startId);
}
```

9、Service 的 onStartCommand 方法有几种返回值？各代表什么意思？

有四种返回值，不同值代表的意思如下：

START_STICKY：如果 service 进程被 kill 掉，保留 service 的状态为开始状态，但不保留递送的 intent 对象。随后系统会尝试重新创建 service，由于服务状态为开始状态，所以创建服务后一定会调用 `onStartCommand(Intent,int,int)` 方法。如果在此期间没有任何启动命令被传递到 service 那么参数 Intent 将为 null。

START_NOT_STICKY：“非粘性的”。使用这个返回值时，如果在执行完 `onStartCommand` 后，服务被异常 kill 掉，系统不会自动重启该服务。

START_REDELIVER_INTENT：重传 Intent。使用这个返回值时，如果在执行完 `onStartCommand` 后，服务被异常 kill 掉，系统会自动重启该服务，并将 Intent 的值传入。

START_STICKY_COMPATIBILITY：START_STICKY 的兼容版本，但不保证服务被 kill 后一定能重启。

10、Service 的 onRebind (Intent) 方法在什么情况下会执行？

如果在 onUnbind () 方法返回 true 的情况下会执行，否则不执行。

官方解释如下：

```
Called when new clients have connected to the service, after it had previously been notified that all had disconnected in its onUnbind. This will only be called if the implementation of onUnbind was overridden to return true.
```

11、Activity 调用 Service 中的方法都有哪些方式？

Activity 调用 Service 中的方法主要是通过绑定服务的模式实现的，绑定服务又分为三种方式。如下所示：

一、Extending the Binder class

通过 Binder 接口的形式实现，当 Activity 绑定 Service 成功的时候 Activity 会在 ServiceConnection 的类的 onServiceConnected () 回调方法中获取到 Service 的 onBind () 方法 return 过来的 Binder 的子类。

二、Using a Messenger

这是官方给出的另外一种沟通方式。原文如下：

```
If you need your interface to work across different processes, you can create an interface for the service with a Messenger. In this manner, the service defines a Handler that responds to different types of Message objects. This Handler is the basis for a Messenger that can then share an IBinder with the client, allowing the client to send commands to the service using Message objects. Additionally, the client can define a Messenger of its own so the service can send messages back.
```

```
This is the simplest way to perform interprocess communication (IPC), because the Messenger queues all requests into a single thread so that you don't have to design your service to be thread-safe.
```

```
Here's a summary of how to use a Messenger:
```

- The service implements a Handler that receives a callback for each call from a client.
- The Handler is used to create a Messenger object (which is a reference to the Handler).
- The Messenger creates an IBinder that the service returns to clients from onBind().
- Clients use the IBinder to instantiate the Messenger (that references the service's Handler), which the client uses to send Message objects to the service.
- The service receives each Message in its Handler—specifically, in the handleMessage() method.

```
In this way, there are no "methods" for the client to call on the service. Instead, the client delivers
```

"messages" (Message objects) that the service receives in its Handler.

Here's a simple example service that uses a Messenger interface:

```
1. public class MessengerService extends Service {
2.     /** Command to the service to display a message */
3.     static final int MSG_SAY_HELLO = 1;
4.
5.     /**
6.      * Handler of incoming messages from clients.
7.      */
8.     class IncomingHandler extends Handler {
9.         @Override
10.        public void handleMessage(Message msg) {
11.            switch (msg.what) {
12.                case MSG_SAY_HELLO:
13.                    Toast.makeText(getApplicationContext(),
14. "hello!", Toast.LENGTH_SHORT).show();
15.                    break;
16.                default:
17.                    super.handleMessage(msg);
18.            }
19.        }
20.    }
21.
22.    /**
23.     * Target we publish for clients to send messages to IncomingHandler.
24.     */
25.    final Messenger mMessenger = new Messenger(new IncomingHandler());
26.
27.    /**
28.     * When binding to the service, we return an interface to our messenger
29.     * for sending messages to the service.
30.     */
31.    @Override
32.    public IBinder onBind(Intent intent) {
33.        Toast.makeText(getApplicationContext(),
34. "binding", Toast.LENGTH_SHORT).show();
35.        return mMessenger.getBinder();
36.    }
37. }
```

Notice that the `handleMessage()` method in the Handler is where the service receives the incoming Message and decides what to do, based on the `what` member.

All that a client needs to do is create a Messenger based on the IBinder returned by the service and send a message using `send()`. For example, here's a simple activity that binds to the service and delivers

the MSG_SAY_HELLO message to the service:

```
1. public class ActivityMessenger extends Activity {
2.     /** Messenger for communicating with the service. */
3.     Messenger mService = null;
4.
5.     /** Flag indicating whether we have called bind on the service. */
6.     boolean mBound;
7.
8.     /**
9.      * Class for interacting with the main interface of the service.
10.     */
11.     private ServiceConnection mConnection = new ServiceConnection() {
12.         public void onServiceConnected(ComponentName className, IBinder service) {
13.             // This is called when the connection with the service has been
14.             // established, giving us the object we can use to
15.             // interact with the service. We are communicating with the
16.             // service using a Messenger, so here we get a client-side
17.             // representation of that from the raw IBinder object.
18.             mService = new Messenger(service);
19.             mBound = true;
20.         }
21.
22.         public void onServiceDisconnected(ComponentName className) {
23.             // This is called when the connection with the service has been
24.             // unexpectedly disconnected -- that is, its process crashed.
25.             mService = null;
26.             mBound = false;
27.         }
28.     };
29.
30.     public void sayHello(View v) {
31.         if (!mBound) return;
32.         // Create and send a message to the service, using a supported 'what' value
33.         Message msg = Message.obtain(null, MessengerService.MSG_SAY_HELLO, 0, 0);
34.         try {
35.             mService.send(msg);
36.         } catch (RemoteException e) {
37.             e.printStackTrace();
38.         }
39.     }
40.
41.     @Override
42.     protected void onCreate(Bundle savedInstanceState) {
43.         super.onCreate(savedInstanceState);
```

```
44.         setContentView(R.layout.main);
45.     }
46.
47.     @Override
48.     protected void onStart() {
49.         super.onStart();
50.         // Bind to the service
51.         bindService(new Intent(this, MessengerService.class), mConnection,
52.             Context.BIND_AUTO_CREATE);
53.     }
54.
55.     @Override
56.     protected void onStop() {
57.         super.onStop();
58.         // Unbind from the service
59.         if (mBound) {
60.             unbindService(mConnection);
61.             mBound = false;
62.         }
63.     }
64. }
```

Notice that this example does not show how the service can respond to the client. If you want the service to respond, then you need to also create a Messenger in the client. Then when the client receives the `onServiceConnected()` callback, it sends a Message to the service that includes the client's Messenger in the `replyTo` parameter of the `send()` method.

三、Using AIDL

aidl 比较适合当客户端和服务端不在同一个应用下的场景。

12、Activity 如何给 Service 发送 Message ? (2015.10.18)

该题和下一题 都是关于 Activity 和 Service 直接收发消息的知识 非常的有意思 但是也非常不好回答。Activity 给 Service 还算稍微简单一点，但是 Service 给 Activity 发送消息就有点难度了，不是常人能所了解的。如果你没有看过 Android 官方关于 Bound Service 的解释，估计很难应付。

Activity 如何给 Service 发送 Message 见《Activity 调用 Service 中的方法都有哪些方式？》题目中 Activity 跟 Service 绑定的第二种方式：Using a Messenger。

13、Service 如何给 Activity 发送 Message ? (2015.10.18)

Service 和 Activity 如果想互发 Message 就必须使用使用 Messenger 机制。

Service 如何给 Activity 发送 Message 见《Activity 调用 Service 中的方法都有哪些方式？》题目中 Activity 跟 Service 绑定的第二种方式：Using a Messenger。

考虑到官方只给出了 Activity 给 Service 发送 Message 的代码，在这里我给出一个 Activity 跟 Service 之间互相发送 Message 通信的示例代码：

1. MainActivity 代码

```
1. package com.example.serviceAndActivity;
2.
3. import android.os.Bundle;
4. import android.os.Handler;
5. import android.os.IBinder;
6. import android.os.Message;
7. import android.os.Messenger;
8. import android.os.RemoteException;
9. import android.app.Activity;
10. import android.app.Service;
11. import android.content.ComponentName;
12. import android.content.Intent;
13. import android.content.ServiceConnection;
14. import android.util.Log;
15. import android.view.View;
16. import android.widget.Toast;
17. /**
18.  * Activity 和 Service 互发 Message
19.  *
20.  * @author wzy 2015-11-25
21.  *
22.  */
23. public class MainActivity extends Activity {
24.     private Messenger messenger;
25.     //将该 Handler 发送 Service
26.     private Messenger mOutMessenger = new Messenger(new OutgoingHandler());
27.
28.     @Override
29.     protected void onCreate(Bundle savedInstanceState) {
30.         super.onCreate(savedInstanceState);
```

```
31.     setContentView(R.layout.activity_main);
32. }
33. // 绑定服务
34. public void click1(View view) {
35.     Intent intent = new Intent(this, MessengerService.class);
36.     ServiceConnection conn = new MyServiceConnection();
37.     bindService(intent, conn, Service.BIND_AUTO_CREATE);
38. }
39.
40. // 发送消息
41. public void click2(View view) throws RemoteException {
42.     if (messenger == null) {
43.         Toast.makeText(this, "服务不可用！", Toast.LENGTH_SHORT).show();
44.         return;
45.     }
46.     Message message = new Message();
47.     message.obj="长江长江我是黄河";
48.     message.what =0;
49.     messenger.send(message);
50.
51. }
52.
53. class OutgoingHandler extends Handler{
54.     @Override
55.     public void handleMessage(Message msg) {
56.         Log.d("tag", msg.toString());
57.     }
58. }
59.
60. class MyServiceConnection implements ServiceConnection {
61.
62.     @Override
63.     public void onServiceConnected(ComponentName name, IBinder service) {
64.         Toast.makeText(MainActivity.this, "连接成功！", Toast.LENGTH_SHORT).show();
65.         messenger = new Messenger(service);
66.         Message message=new Message();
67.         message.what = 1;
68. //Activity 绑定 Service 的时候给 Service 发送一个消息，该消息的 obj 属性是一个 Messenger 对象
69.         message.obj = mOutMessenger;
70.         try {
71.             messenger.send(message);
72.         } catch (RemoteException e) {
73.             e.printStackTrace();
74.         }
```



```
75.     }
76.
77.     @Override
78.     public void onServiceDisconnected(ComponentName name) {
79.         Toast.makeText(MainActivity.this, "连接已经断开！", Toast.LENGTH_SHORT).show();
80.     }
81.
82. }
83.
84. }
```

2. MessengerService 代码

```
1. package com.example.serviceAndActivity;

import android.app.Service;
import android.content.Intent;
import android.os.Handler;
import android.os.IBinder;
import android.os.Message;
import android.os.Messenger;
import android.os.RemoteException;
import android.util.Log;
/**
 * 该 Service 接收到 Activity 的消息后会在返回一条消息
 *
 * @author wzy 2015-11-25
 *
 */
public class MessengerService extends Service {

    private Messenger messenger = new Messenger(new IncomingHandler());

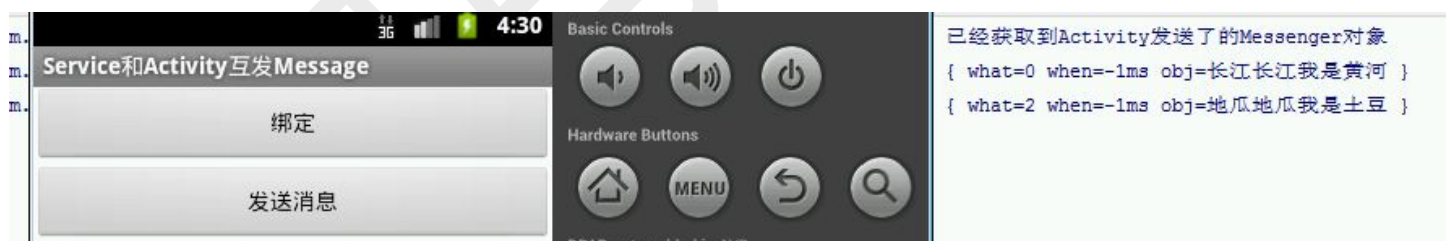
    private Messenger mActivityMessenger ;

    @Override
    public IBinder onBind(Intent intent) {
        IBinder binder = messenger.getBinder();
        return binder;
    }

    //1.定义一个 Handler 对象，该 Handler 处理 Activity 发送过来的消息
    class IncomingHandler extends Handler{
        @Override
        public void handleMessage(Message msg) {
            switch (msg.what) {
```

```
case 0:
    Log.d("tag", msg.toString());
    if (mActivityMessenger!=null) {
        Message message= new Message();
        message.what = 2;
        message.obj="地瓜地瓜我是土豆";
        try {
            mActivityMessenger.send(message);
        } catch (RemoteException e) {
            e.printStackTrace();
        }
    }
    break;
case 1:
    mActivityMessenger = (Messenger) msg.obj;
    Log.d("tag", "已经获取到 Activity 发送了的 Messenger 对象");
    break;
default:
    break;
}
}
}
```

运行界面以及日志如下图所示：



四、BroadcastReceiver

1、请描述一下 BroadcastReceiver

BroadcastReceiver 是 Android 四大组件之一，主要用于接收系统或者 app 发送的广播事件。

广播分两种：有序广播和无序广播。

内部通信实现机制：通过 Android 系统的 Binder 机制实现通信。

无序广播：完全异步，逻辑上可以被任何广播接收者接收到。优点是效率较高。缺点是一个接收者不能将处理结果传递给下一个接收者，并无法终止广播 intent 的传播。

有序广播：按照被接收者的优先级顺序，在被接收者中依次传播。比如有三个广播接收者 A, B, C, 优先级是 $A > B > C$ 。那这个消息先传给 A, 再传给 B, 最后传给 C。每个接收者有权终止广播，比如 B 终止广播，C 就无法接收到。此外 A 接收到广播后可以对结果对象进行操作，当广播传给 B 时，B 可以从结果对象中取得 A 存入的数据。

在通过 Context.sendOrderedBroadcast(intent, receiverPermission, resultReceiver, scheduler, requestCode, initData, initialExtras) 时我们可以指定 resultReceiver 广播接收者，这个接收者我们可以认为是最终接收者，通常情况下如果比他优先级更高的接收者如果没有终止广播，那么他的 onReceive 会被执行两次，第一次是正常的按照优先级顺序执行，第二次是作为最终接收者接收。如果比他优先级高的接收者终止了广播，那么他依然能接收到广播。

在我们的项目中经常使用广播接收者接收系统通知，比如开机启动、sd 挂载、低电量、外播电话、锁屏等。

如果我们做的是播放器，那么监听到用户锁屏后我们应该将我们的播放之暂停等。

2、在 manifest 和代码中如何注册和使用 BroadcastReceiver

在清单文件中注册广播接收者称为静态注册，在代码中注册称为动态注册。静态注册的广播接收者只要 app 在系统中运行则一直可以接收到广播消息，动态注册的广播接收者当注册的 Activity 或者 Service 销毁了那么就接收不到广播了。

静态注册：在清单文件中进行如下配置

```
<receiver android:name=".BroadcastReceiver1" >
    <intent-filter>
        <action android:name="android.intent.action.CALL" >
        </action>
    </intent-filter>
</receiver>
```

动态注册：在代码中进行如下注册

```
receiver = new BroadcastReceiver();
IntentFilter intentFilter = new IntentFilter();
intentFilter.addAction(CALL_ACTION);
context.registerReceiver(receiver, intentFilter);
```

3、BroadcastReceiver 的生命周期

- a. 广播接收者的生命周期非常短暂的，在接收到广播的时候创建，onReceive()方法结束之后销毁；
- b. 广播接收者中不要做一些耗时的工作，否则会弹出 Application No Response 错误对话框；
- c. 最好也不要再在广播接收者中创建子线程做耗时的工作，因为广播接收者被销毁后进程就成为了空进程，很容易被系统杀掉；
- d. 耗时的较长的工作最好放在服务中完成；

4、如何让自己的广播只让指定的 app 接收 (2015.09.02)

- 1、自己的应用（假设名称为应用 A）在发送广播的时候给自己发送的广播添加自定义权限，假设权限名为：

`com.itheima.android.permission`，然后需要在应用 A 的 AndroidManifest.xml 中声明如下权限：

```
<permission android:name="com.itheima.android.permission"
android:protectionLevel="normal"></permission>
<uses-permission android:name="com.itheima.android.permission"/>
```

- 2、其他应用（假设名称为应用 B）如果想接收该广播，那么就必须知道应用 A 广播使用的权限。然后在应用 B 的清单文件中如下配置：

```
<uses-permission android:name="com.itheima.android.permission"/>
```

或者在应用 AndroidManifest.xml 中的 <receiver> 标签中进行如下配置：

```
<receiver android:name="com.itheima.android.broadcastReceiver.MyReceiver"
android:permission="com.itheima.android.permission">
    <intent-filter>
        <action android:name="com.itheima.mybroadcast"></action>
    </intent-filter>
</receiver>
```

每个权限通过 **protectionLevel** 来标识保护级别：

normal：低风险权限，只要申请了就可以使用（在 AndroidManifest.xml 中添加 <uses-permission> 标签），安装时不需要用户确认；

dangerous：高风险权限，安装时需要用户的确认才可使用；

signature：只有当申请权限的应用程序的数字签名与声明此权限的应用程序的数字签名相同时（如果是申请系统权限，则需要与系统签名相同），才能将权限授给它；

signatureOrSystem：签名相同，或者申请权限的应用为系统应用（在 **system image** 中）。

上述四类权限级别同样可用于自定义权限中。如果开发者需要对自己的应用程序（或部分应用）进行访问控制，则可以通过在 AndroidManifest.xml 中添加 <permission> 标签，将其属性中的 **protectionLevel** 设置为上述四类级别中的某一种来实现。

5、什么是最终广播接收者？

最终广播是我们自己应用发送有序广播时通过 ContextWrapper.sendOrderedBroadcast() 方法指定的当前应用下的广播，该广播可能会被执行两次，第一次是作为普通广播按照优先级接收广播，第二次是作为 final receiver 必须接收一次。

6、广播的优先级对无序广播生效吗？

生效的。

7、动态注册的广播优先级谁高？

谁先注册谁优先级高。

8、如何判断当前 BroadcastReceiver 接收到的是有序广播还是无序广播？

(2015-10-16)

在 BroadcastReceiver 类中 onReceive () 方法中，可以调用 boolean b = isOrderedBroadcast () ;该方法是 BroadcastReceiver 类中提供的方法，用于告诉我们当前的接收到的广播是否为有序广播。

五、ContentProvider&数据库

1、请介绍下 ContentProvider 是如何实现数据共享的？

在 Android 中如果想将自己应用的数据（一般多为数据库中的数据）提供给第三发应用，那么我们只能通过 ContentProvider 来实现了。

ContentProvider 是应用程序之间共享数据的接口。使用的时候首先自定义一个类继承 ContentProvider，然后覆写 query、insert、update、delete 等方法。因为其是四大组件之一因此必须在 AndroidManifest 文件中进行注册。

```
<provider
    android:exported="true"
    android:name="com.itheima.contenProvider.provider.PersonContentPro
vider"android:authorities="com.itheima.person" />
```

第三方可以通过 ContentResolver 来访问该 Provider。

2、为什么要用 ContentProvider？它和 sql 的实现上有什么差别？

ContentProvider 屏蔽了数据存储的细节,内部实现对用户完全透明,用户只需要关心操作数据的 uri 就可以了，

ContentProvider 可以实现不同 app 之间共享。

Sql 也有增删改查的方法，但是 sql 只能查询本应用下的数据库。而 ContentProvider 还可以去增删改查本地文件.xml 文件的读取等。

3、说说 ContentProvider、ContentResolver、ContentObserver 之间的关系

ContentProvider 内容提供者，用于对外提供数据

ContentResolver.notifyChange(uri)发出消息

ContentResolver 内容解析者，用于获取内容提供者提供的数据

ContentObserver 内容监听器，可以监听数据的改变状态

ContentResolver.registerContentObserver()监听消息。

4、如何访问 asserts 资源目录下的数据库？

//1. 获取到 assert 目录下的 db 文件

```
AssetManager assetManager = getAssets();
InputStream is = assetManager.open("myuser.db");
//将文件拷贝到
/data/data/com.itheima.android.asserts.sqlite/databases/myuser.db
//如果 databases 目录不存在则创建
File file = new
File("/data/data/com.itheima.android.asserts.sqlite/databases");
if (!file.exists()) {
    file.mkdirs();
}
FileOutputStream fos = new FileOutputStream(new File(file, "myuser.db"));
byte[] buff = new byte[1024*8];
int len=-1;
while((len=is.read(buff))!=-1){
    fos.write(buff, 0, len);
}
fos.close();
is.close();

//访问数据库
```

```
SQLiteDatabase database = openOrCreateDatabase("myuser.db", MODE_PRIVATE,
null);
String sql = "select c_name from t_user";
Cursor cursor = database.rawQuery(sql, null);
while(cursor.moveToNext()){
    String string = cursor.getString(0);
    Log.d("tag", string);
}
cursor.close();
database.close();
}
```

5、如何在高并发下进行数据库查询？（2015-11-25）

（这个问题的回答很广泛，可以自由发挥）

比如：不要关联多表查询，减少链接时间，创建索引、将查询到的数据采用缓存策略等等。

六、Android 中的布局

1、Android 中常用的布局都有哪些

FrameLayout

RelativeLayout

LinearLayout

AbsoluteLayout

TableLayout

GrildeLayout(Android 4.0 推出)

2、谈谈 UI 中，Padding 和 Margin 有什么区别？

android:padding 和 android:layout_margin 的区别，其实概念很简单，padding 是站在父 view 的角度描述问题，它规定它里面的内容必须与这个父 view 边界的距离。margin 则是站在自己的角度描述问题，规定自己和其他（上

下左右) 的 view 之间的距离, 如果同一级只有一个 view, 那么它的效果基本上就和 padding 一样了。

3、使用权重如何让一个控件的宽度为父控件的 1/3 ?

可以在水平方向的 LinearLayout 中设置 weightSum 为 3, 然后让其子控件的 weight 为 1, 那么该子控件就是父控件的 1/3。

4、Android 中布局的优化措施都有哪些 ?

1、尽可能减少布局的嵌套层级

可以使用 sdk 提供的 hierarchyviewer 工具分析视图树, 帮助我们发现没有用到的布局。

2、不用设置不必要的背景, 避免过度绘制

比如父控件设置了背景色, 子控件完全将父控件给覆盖的情况下, 那么父控件就没有必要设置背景。

3、使用 <include> 标签复用相同的布局代码

4、使用 <merge> 标签减少视图层次结构

该标签主要有两种用法:

1) 因为所有的 Activity 视图的根节点都是 FrameLayout, 因此如果我们的自定义的布局也是 FragmentLayout 的时候那么可以使用 merge 替换。

2) 当应用 Include 或者 ViewStub 标签从外部导入 xml 结构时, 可以将被导入的 xml 用 merge 作为根节点表示, 这样当被嵌入父级结构中后可以很好的将它所包含的子集融合到父级结构中, 而不会出现冗余的节点。

<merge> 只能作为 xml 布局的根元素。

5、通过 <ViewStub> 实现 View 的延迟加载

布局如下:

```
<ViewStub
    android:id="@+id/vs"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:inflatedId="@+id/my_view"
    android:layout="@layout/my_layout" />
```

核心代码：

```
public void loadVS(View view){
    ViewStub vs = (ViewStub) findViewById(R.id.vs);
    View inflate = vs.inflate();
    int inflatedId = vs.getInflatedId();
    int id = inflate.getId();
    Toast.makeText(this, "inflatedId="+inflatedId+"***"+id,
        Toast.LENGTH_SHORT).show();
}
```

5、android:layout_gravity 和 android:gravity 的区别？

第一个是让该布局在其父控件中的布局方式，第二个是该布局布置其子对象的布局方式。

七、ListView

1、ListView 如何提高其效率？

- ① 复用 convertView
- ② 自定义静态类 ViewHolder
- ③ 使用分页加载

④ 使用 WeakReference 引用 ImageView 对象

2、ViewHolder 为什么要声明为静态类？

非静态内部类拥有外部类对象的强引用，因此为了避免对外部类（外部类很可能是 Activity）对象的引用，那么最好将内部类声明为 static 的。

3、在 Activity 中使用 Handler 的时候如何去除警告信息？

可以使用弱引用。

4、谈谈 ListView 中的 MVC 思想？

M : model

V : view

C : Controller

5、ListView 使用了哪些设计模式？

1、适配器

2、观察者

3、享元设计模式

6、当 ListView 数据集改变后，如何更新 ListView？

使用该 ListView 的 adapter 的 notifyDataSetChanged()方法。该方法会使 ListView 重新绘制。

7、ListView 如何实现分页加载

① 设置 ListView 的滚动监听器：setOnScrollListener(new OnScrollListener{....})

在监听器中有两个方法：滚动状态发生变化的方法(onScrollStateChanged)和 listView 被滚动时调用的方法(onScroll)

② 在滚动状态发生改变的方法中，有三种状态：

手指按下移动的状态：SCROLL_STATE_TOUCH_SCROLL: // 触摸滑动

惯性滚动（滑翔（fling）状态）：SCROLL_STATE_FLING: // 滑翔

静止状态：SCROLL_STATE_IDLE: // 静止

对不同的状态进行处理：

分批加载数据，只关心静止状态：关心最后一个可见的条目，如果最后一个可见条目就是数据适配器（集合）里的最后一个，此时可加载更多的数据。在每次加载的时候，计算出滚动的数量，当滚动的数量大于等于总数量的时候，可以提示用户无更多数据了。

8、ListView 可以显示多种类型的条目吗？

这个当然可以的，ListView 显示的每个条目都是通过 baseAdapter 的 getView(int position, View convertView, ViewGroup parent)来展示的，理论上我们完全可以让每个条目都是不同类型的 view，除此之外 adapter 还提供了 getViewTypeCount（）和 getItemViewType(int position)两个方法。在 getView 方法中我们可以根据不同的 viewtype 加载不同的布局文件。

9、ListView 如何定位到指定位置

可以通过 ListView 提供的 lv.setSelection(48);方法。

10、如何在 ScrollView 中如何嵌入 ListView

通常情况下我们不会在 ScrollView 中嵌套 ListView，但是如果面试官非让我嵌套的话也是可以的。

在 ScrollView 添加一个 ListView 会导致 listview 控件显示不全，通常只会显示一条，这是因为两个控件的滚动事件冲突导致。所以需要通过 listview 中的 item 数量去计算 listview 的显示高度，从而使其完整展示，如下提供一个方法供大家参考。

如下图，在 ScrollView 中嵌套了 ListView，ListView 展现的是物流状态。

09:21 0.09% 0.09% 0.09% 0.09% 84



物流查询

订单编号 13283790161 已发货

配送公司： 晟邦物流-上海 4006666066

2015-10-10 23:05:11
配送中（客户推迟收货），晟邦物流-上海 4006666066 晟邦物流-上海
2015-10-10 22:34:53
货物已由[上海宽容S10]的小件员[宽容]反馈为[分站 滞留],电话[4006666066],原因:客户更改派送时间(通 用) 晟邦物流-上海
2015-10-10 22:32:56
货物由[上海宽容S10]的派件员[宽容]正在派件..电话 [4006666066] 晟邦物流-上海
2015-10-10 17:07:06
配送中，晟邦物流-上海 4006666066 上海市
2015-10-10 05:25:20

猜你喜欢



```

lv = (ListView) findViewById(R.id.lv);
adapter = new MyAdapter();
lv.setAdapter(adapter);
setListViewHeightBasedOnChildren(lv);

-----
public void setListViewHeightBasedOnChildren(ListView listView) {
    ListAdapter listAdapter = listView.getAdapter();
    if (listAdapter == null) {
        return;
    }
    int totalHeight = 0;
    for (int i = 0; i < listAdapter.getCount(); i++) {
        View listItem = listAdapter.getView(i, null, listView);
        listItem.measure(0, 0);
        totalHeight += listItem.getMeasuredHeight();
    }
    ViewGroup.LayoutParams params = listView.getLayoutParams();
    params.height = totalHeight + (listView.getDividerHeight() *
(listAdapter.getCount() - 1));
    params.height += 5; // if without this statement, the listview will be a
                        // little short
    listView.setLayoutParams(params);
}

```

布局文件片段:

```

1. <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
2.     android:layout_width="match_parent"
3.     android:layout_height="match_parent"
4.     android:orientation="vertical" >
5.     <ScrollView
6.         android:id="@+id/sv"
7.         android:layout_width="match_parent"
8.         android:layout_height="0dp"
9.         android:layout_weight="1" >
10.
11.         <LinearLayout
12.             android:layout_width="match_parent"
13.             android:layout_height="match_parent"
14.             android:orientation="vertical" >
15.
16.             <ListView
17.                 android:id="@+id/lv"

```

```
18.         android:layout_width="match_parent"
19.         android:layout_height="wrap_content" >
20.     </ListView>
21. </LinearLayout>
22. </ScrollView>
23.
24. </LinearLayout>
```

注意:如果直接将 ListView 放到 ScrollView 中,那么上面的代码依然是没有效果的.必须将 ListView 放到 LinearLayout 等其他容器中才行.

11、ListView 中如何优化图片

图片的优化策略比较多。

1、处理图片的方式：

如果 ListView 中自定义的 Item 中有涉及到大量图片的，一定要对图片进行细心的处理，因为图片占的内存是 ListView 项中最头疼的，处理图片的方法大致有以下几种：

- ①、不要直接拿路径就去循环 BitmapFactory.decodeFile;使用 Options 保存图片大小、不要加载图片到内存去。
- ②、对图片一定要经过边界压缩尤其是比较大的图片，如果你的图片是后台服务器处理好的那就不需要了
- ③、在 ListView 中取图片时也不要直接拿个路径去取图片，而是以 WeakReference (使用 WeakReference 代替强引用。比如可以使用 WeakReference mContextRef)、SoftReference、WeakHashMap 等的来存储图片信息。
- ④、在 getView 中做图片转换时，产生的中间变量一定及时释放

2、异步加载图片基本思想：

- 1)、先从内存缓存中获取图片显示 (内存缓冲)
- 2)、获取不到的话从 SD 卡里获取 (SD 卡缓冲)
- 3)、都获取不到的话从网络下载图片并保存到 SD 卡同时加入内存并显示 (视情况看是否要显示)

原理：

优化一：先从内存中加载，没有则开启线程从 SD 卡或网络中获取，这里注意从 SD 卡获取图片是放在子线程里执行的，否则快速滚屏的话会不够流畅。

优化二：于此同时，在 adapter 里有个 busy 变量，表示 listview 是否处于滑动状态，如果是滑动状态则仅从内存中获取图片，没有的话无需再开启线程去外存或网络获取图片。

优化三：ImageLoader 里的线程使用了线程池，从而避免了过多线程频繁创建和销毁，如果每次总是 new 一个线程去执行这是非常不可取的，好一点的用的 AsyncTask 类，其实内部也是用到了线程池。在从网络获取图片时，先是将其保存到 sd 卡，然后再加载到内存，这么做的好处是在加载到内存时可以做个压缩处理，以减少图片所占内存。

12、ListView 中图片错位的问题是如何产生的

图片错位问题的本质源于我们的 listview 使用了缓存 convertView，假设一种场景，一个 listview 一屏显示九个 item，那么在拉出第十个 item 的时候，事实上该 item 是重复使用了第一个 item，也就是说在第一个 item 从网络中下载图片并最终要显示的时候，其实该 item 已经不在当前显示区域内了，此时显示的后果将可能在第十个 item 上输出图像，这就导致了图片错位的问题。所以解决之道在于可见则显示，不可见则不显示。

13、scrollView 嵌套 listview 方式除了测量还有什么方法？（2015-11-29）

1、手动设置 ListView 高度

经过测试发现，在 xml 中直接指定 ListView 的高度，是可以解决这个问题的，但是 ListView 中的数据是可变的，实际高度还需要实际测量。

于是手动代码设置 ListView 高度的方法就诞生了。

```
/**
```

```
 * 动态设置 ListView 的高度
```



```
* @param listView
*/

public static void setListViewHeightBasedOnChildren(ListView listView) {

    if(listView == null) return;

    ListAdapter listAdapter = listView.getAdapter();

    if (listAdapter == null) {

        // pre-condition

        return;

    }

    int totalHeight = 0;

    for (int i = 0; i < listAdapter.getCount(); i++) {

        View listItem = listAdapter.getView(i, null, listView);

        listItem.measure(0, 0);

        totalHeight += listItem.getMeasuredHeight();

    }

    ViewGroup.LayoutParams params = listView.getLayoutParams();

    params.height = totalHeight + (listView.getDividerHeight() * (listAdapter.getCount() - 1));

    listView.setLayoutParams(params);

}
```

2、使用单个 ListView 取代 ScrollView 中所有内容

如果满足头布局 and 脚布局的 UI 设计，直接使用 listview 替代 scrollview

3、使用 LinearLayout 取代 ListView

既然 ListView 不能适应 ScrollView，那就换一个可以适应 ScrollView 的控件，干嘛非要吊死在 ListView 这一棵树上呢？

而 LinearLayout 是最好的选择。但如果我仍想继续使用已经定义好的 Adapter 呢？我们只需要自定义一个类继承自 LinearLayout，为其加上对 BaseAdapter 的适配。

4、自定义可适应 ScrollView 的 ListView

这个方法和上面的方法是异曲同工，方法 3 是自定义了 LinearLayout 以取代 ListView 的功能，但如果我脾气就是倔，就是要用 ListView 怎么办？

那就只好自定义一个类继承自 ListView，通过重写其 onMeasure 方法，达到对 ScrollView 适配的效果。

5、参考博客

<http://bbs.anzhuo.cn/thread-982250-1-1.html>

八、JNI&NDK

1、在 Android 中如何调用 C 语言

当我们的 Java 需要调用 C 语言的时候可以通过 JNI 的方式，Java Native Interface。Android 提供了对 JNI 的支持，因此我们在 Android 中可以使用 JNI 调用 C 语言。在 Android 开发目录的 libs 目录下添加 xxx.so 文件，不过 xxx.so 文件需要放在对应的 CPU 架构名目录下，比如 armeabi，x86 等。

在 Java 代码需要通过 `System.loadLibrary(libName)`；加载 so 文件。同时 C 语言中的方法在 java 中必须以 native 关键字来声明。普通 Java 方法调用这个 native 方法接口，虚拟机内部自动调用 so 文件中对应的方法。

2、请介绍一下 NDK

1.NDK 是一系列工具的集合

NDK 提供了一系列的工具，帮助开发者快速开发 C (或 C++) 的动态库，并能自动将 so 和 java 应用一起打包成 apk。NDK 集成了交叉编译器，并提供了相应的 mk 文件隔离 CPU、平台、ABI 等差异，开发人员只需要简单修改 mk 文件（指出“哪些文件需要编译”、“编译特性要求”等），就可以创建出 so。

2.NDK 提供了一份稳定、功能有限的 API 头文件声明

Google 明确声明该 API 是稳定的，在后续所有版本中都稳定支持当前发布的 API。从该版本的 NDK 中看出，这些 API 支持的功能非常有限，包含有：C 标准库 (libc)、标准数学库 (libm)、压缩库 (libz)、Log 库 (liblog)。

3、JNI 调用常用的两个参数

JNIEnv *env, jobject obj

第一个是指向虚拟机对象的指针，是一个二级指针。里面封装了很多方法和中间变量供我们使用。

第二个代表着调用该方法的 Java 对象的 C 语言表示。

九、Android 中的网络访问

1、Android 中如何访问网络

Android 提供了 org.apache.http.HttpURLConnection 和 java.net.HttpURLConnection 两个连接网络对象。使用哪个都行，具体要看企业领导的要求了。

除此之外一般我比较喜欢使用 xUtils 中的 HttpUtils 功能，该模块底层使用的就是 org.apache.http.client.HttpClient，使用起来非常方便。

2、如何解析服务器传来的 JSON 文件

在 Android 中内置了 JSON 的解析 API，在 org.json 包中包含了如下几个类：JSONArray、JSONObject、JSONStringer、JSONTokener 和一个异常类 JSONException。

1、JSON 解析步骤

1)、读取网络文件数据并转为一个 json 字符串

```
InputStream in = conn.getInputStream();
```

```
String jsonStr = DataUtil.Stream2String(in); //将流转换成字符串的工具类
```

2)、将字符串传入相应的 JSON 构造函数中

①、通过构造函数将 json 字符串转换成 json 对象

```
JSONObject jsonObject = new JSONObject(jsonStr);
```

②、通过构造函数将 json 字符串转换成 json 数组：

```
JSONArray array = new JSONArray(jsonStr);
```

3)、解析出 JSON 中的数据信息：

①、从 json 对象中获取你所需要的键所对应的值

```
JSONObject json = jsonObject.getJSONObject("weatherinfo");
```

```
String city = json.getString("city");
```

```
String temp = json.getString("temp");
```

②、遍历 JSON 数组，获取数组中每一个 json 对象，同时可以获取 json 对象中键对应的值

```
for (int i = 0; i < array.length(); i++) {
```

```
    JSONObject obj = array.getJSONObject(i);
```

```
    String title = obj.getString("title");
```

```
String description=obj.getString("description");  
  
}
```

2、生成 JSON 对象和数组

1) 生成 JSON :

方法 1、创建一个 map，通过构造方法将 map 转换成 json 对象

```
Map<String, Object> map = new HashMap<String, Object>();  
  
map.put("name", "zhangsan");  
  
map.put("age", 24);  
  
JSONObject json = new JSONObject(map);
```

方法 2、创建一个 json 对象，通过 put 方法添加数据

```
JSONObject json=new JSONObject();  
  
json.put("name", "zhangsan");  
  
json.put("age", 24);
```

2) 生成 JSON 数组 :

创建一个 list，通过构造方法将 list 转换成 json 对象

```
Map<String, Object> map1 = new HashMap<String, Object>();  
  
map1.put("name", "zhangsan");  
  
map1.put("age", 24);  
  
Map<String, Object> map2 = new HashMap<String, Object>();  
  
map2.put("name", "lisi");  
  
map2.put("age", 25);  
  
List<Map<String, Object>> list=new ArrayList<Map<String,Object>>();
```

```
list.add(map1);

list.add(map2);

JSONArray array=new JSONArray(list);

System.out.println(array.toString());
```

3、如何解析服务器传来的 XML 格式数据

Android 为我们提供了原生的 XML 解析和生成支持。

1、XML 解析

获取解析器: `Xml.newPullParser()`

设置输入流: `setInput()`

获取当前事件类型: `getEventType()`

解析下一个事件, 获取类型: `next()`

获取标签名: `getName()`

获取属性值: `getAttributeValue()`

获取下一个文本: `nextText()`

获取当前文本: `getText()`

5 种事件类型: `START_DOCUMENT`, `END_DOCUMENT`, `START_TAG`, `END_TAG`, `TEXT`

示例代码：

```
public List<Person> getPersons(InuptStream in){

    XmlPullParser parser=Xml.newPullParser();//获取解析器

    parser.setInput(in,"utf-8");

    for(int type=){    //循环解析
```

```
}
```

```
}
```

2、XML 生成

获取生成工具: `Xml.newSerializer()`

设置输出流: `setOutput()`

开始文档: `startDocument()`

结束文档: `endDocument()`

开始标签: `startTag()`

结束标签: `endTag()`

属性: `attribute()`

文本: `text()`

示例代码：

```
XmlSerializer serial=Xml.newSerializer();//获取 xml 序列化工具
```

```
serial.setOutput(out,"utf-8");
```

```
serial.startDocument("utf-8",true);
```

```
serial.startTag(null,"persons");
```

```
for(Person p:persons){
```

```
    serial.startTag(null,"persons");
```

```
    serial.attribute(null,"id",p.getId().toString());
```

```
    serial.startTag(null,"name");
```

```
    serial.attribute(null,"name",p.getName().toString());
```

```
    serial.endTag(null,"name");
```

```
serial.startTag(null,"age");

serial.attribute(null,"age",p.getAge().toString());

serial.endTag(null,"age");

serial.endTag(null,"persons");

}
```

4、如何从网络上加载一个图片显示到界面

可以通过 `BitmapFactory.decodeStream(inputStream);` 方法将图片转换为 bitmap，然后通过 `imageView.setImageBitmap(bitmap);` 将该图片设置到 `ImageView` 中。这是原生的方法，还可以使用第三方开源的工具来实现，比如使用 `SmartImageView` 作为 `ImageView` 控件，然后直接设置一个 url 地址即可。也可以使用 `xUtils` 中的 `BitmapUtils` 工具。

5、如何播放网络视频

除了使用 `Android` 提供的 `MediaPlayer` 和 `VideoView` 外通常还可以使用第三方开源万能播放器，`VitamioPlayer`。该播放器兼容性好，支持几乎所有主流视频格式。

6、常见的访问网络 API 都有哪些？

`Android` 系统自带的 `URLConnection/HttpClient`。

`java.net.HttpURLConnection;`

`org.apache.http.client.HttpClient;`

`OKhttp;`

<http://blog.csdn.net/lmj623565791/article/details/47911083>

`xUtils :`

<https://github.com/wyouflf/xUtils>

十、Intent

1、Intent 传递数据时，可以传递哪些类型数据？

Intent 可以传递的数据类型非常的丰富，java 的基本数据类型和 String 以及他们的数组形式都可以，除此之外还可以传递实现了 Serializable 和 Parcelable 接口的对象。

2、Serializable 和 Parcelable 的区别

- 1.在使用内存的时候，Parcelable 类比 Serializable 性能高，所以推荐使用 Parcelable 类。
- 2.Serializable 在序列化的时候会产生大量的临时变量，从而引起频繁的 GC。
- 3.Parcelable 不能使用在要将数据存储在磁盘上的情况。尽管 Serializable 效率低点，但在这种情况下，还是建

议你用 Serializable 。

实现：

- 1 Serializable 的实现，只需要继承 Serializable 即可。这只是给对象打了一个标记，系统会自动将其序列化。
- 2 Parcelabel 的实现，需要在类中添加一个静态成员变量 CREATOR，这个变量需要继承 Parcelable.Creator 接

口。

```
public class MyParcelable implements Parcelable {
    private int mData;

    public int describeContents() {
        return 0;
    }

    public void writeToParcel(Parcel out, int flags) {
        out.writeInt(mData);
    }

    public static final Parcelable.Creator<MyParcelable> CREATOR
        = new Parcelable.Creator<MyParcelable>() {
        public MyParcelable createFromParcel(Parcel in) {
            return new MyParcelable(in);
        }

        public MyParcelable[] newArray(int size) {
            return new MyParcelable[size];
        }
    };

    private MyParcelable(Parcel in) {
        mData = in.readInt();
    }
}
```

3、请描述一下 Intent 和 IntentFilter

Android 中通过 Intent 对象来表示一条消息，一个 Intent 对象不仅包含有这个消息的目的地，还可以包含消息的内容，这好比一封 Email，其中不仅应该包含收件地址，还可以包含具体的内容。对于一个 Intent 对象，消息“目的地”是必须的，而内容则是可选项。

通过 Intent 可以实现各种系统组件的调用与激活。

IntentFilter: 可以理解为邮局或者是一个信笺的分拣系统...

这个分拣系统通过 3 个参数来识别

Action: 动作 view

Data: 数据 uri uri

Category：而外的附加信息

Action 匹配

Action 是一个用户定义的字符串，用于描述一个 Android 应用程序组件，一个 IntentFilter 可以包含多个 Action。在 AndroidManifest.xml 的 Activity 定义时可以在其 <intent-filter> 节点指定一个 Action 列表用于标示 Activity 所能接受的“动作”，例如：

```
<intent-filter >

<action android:name="android.intent.action.MAIN" />

<action android:name="cn.itheima.action" />

.....

</intent-filter>
```

如果我们在启动一个 Activity 时使用这样的 Intent 对象：

```
Intent intent =new Intent();

intent.setAction("cn.itheima.action");
```

那么所有的 Action 列表中包含了“cn.itheima”的 Activity 都将会匹配成功。

Android 预定义了一系列的 Action 分别表示特定的系统动作。这些 Action 通过常量的方式定义在 android.content.Intent 中，以“ACTION_”开头。我们可以在 Android 提供的文档中找到它们的详细说明。

URI 数据匹配

一个 Intent 可以通过 URI 携带外部数据给目标组件。在 <intent-filter> 节点中，通过 <data/> 节点匹配外部数据。

mimeType 属性指定携带外部数据的数据类型，scheme 指定协议，host、port、path 指定数据的位置、端口、

和路径。如下：

```
<data android:mimeType="mimeType" android:scheme="scheme"
android:host="host" android:port="port" android:path="path"/>
```

电话的 uri tel: 12345

http://www.baidu.com

自己定义的 uri itcast://cn.itcast/person/10

如果在 Intent Filter 中指定了这些属性，那么只有所有的属性都匹配成功时 URI 数据匹配才会成功。

Category 类别匹配

<intent-filter >节点中可以为组件定义一个 Category 类别列表，当 Intent 中包含这个列表的所有项目时

Category 类别匹配才会成功。

4、under what condition could the code sample below crash your application?How would you modify the code to avoid this potential problem?Explain your answer?(重要)

```
Intent sendIntent = new Intent();
sendIntent.setAction(Intent.ACTION_SEND);
sendIntent.putExtra(Intent.EXTRA_TEXT,textMessage);
sendIntent.setType(HTTP.PLAIN_TEXT_TYEP);/"text/plain" MIME type
context.startActivity(sendIntent);
```

answer:

if startActivity(Intent) method is not called from in Activity ,for example,from service or receiver,it will crash your application.

I can add a flag "Intent_ACTIVITY_NEW_TASK" on sendIntent to avoid this error.

5、 what are Activity and Fragment?where and why should you use one over the other?

6、 can you use an intent to provide data to a ContentProvider ? if not ,what would be the proper mechanism for doing this?

SQLiteDatabase is often used to provide data to a ContentProvider.

十一、Fragment

1、Fragment 跟 Activity 之间是如何传值的？

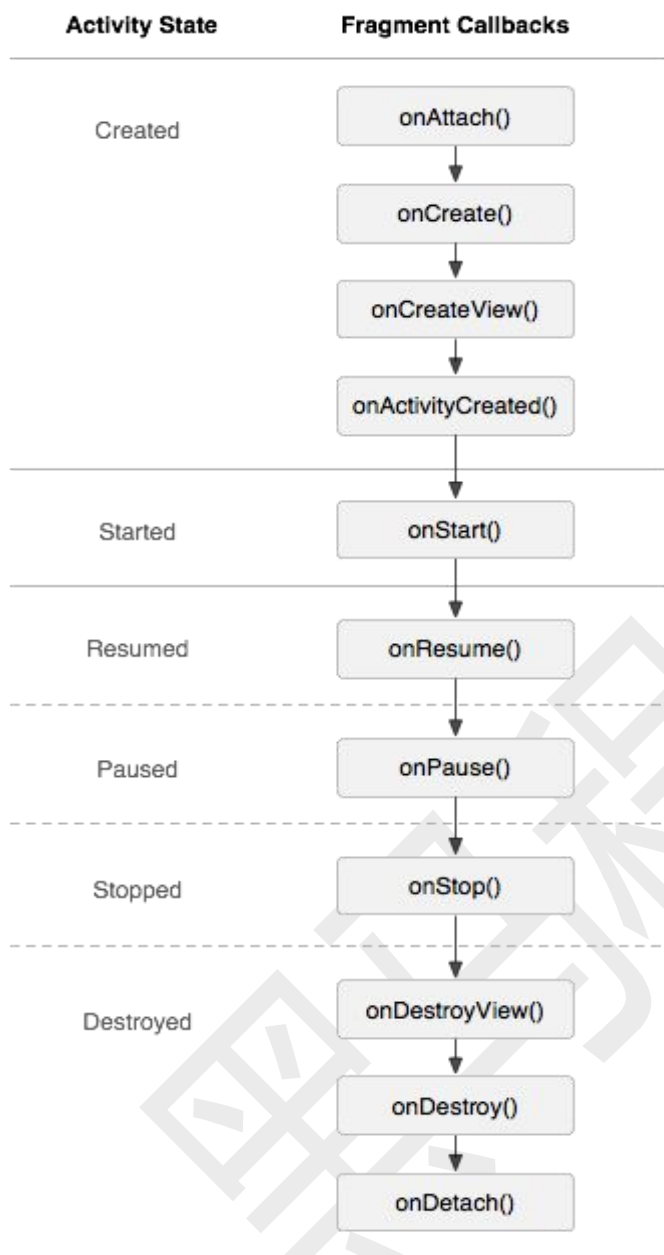
1) 当 Fragment 跟 Activity 绑定之后，在 Fragment 中可以直接通过 getActivity () 方法获取到其绑定的 Activity 对象，这样就可以调用 Activity 的方法了。在 Activity 中可以通过如下方法获取到 Fragment 实例

```
FragmentManager fragmentManager = getFragmentManager();  
Fragment fragment = fragmentManager.findFragmentByTag(tag);  
Fragment fragment = fragmentManager.findFragmentById(id);
```

获取到 Fragment 之后就可以调用 Fragment 的方法。也就实现了通信功能。

2) 另外也可以调用 fragment.setArguments(Bundle)方法，将数据绑定到 Fragment 域中。

2、描述一下 Fragment 的生命周期



3、Fragment 的 replace 和 add 方法的区别 (2015.8.30)

Fragment 本身并没有 `replace` 和 `add` 方法，这里的理解应该为使用 `FragmentManager` 的 `replace` 和 `add` 两种方法切换 Fragment 时有什么不同。

我们经常使用的一个架构就是通过 `RadioGroup` 切换 Fragment，每个 Fragment 就是一个功能模块。

```
        case R.id.rb_1:
            rb_1.setBackgroundColor(Color.RED);
            transaction.show(fragment1);
            transaction.replace(R.id.fl, fragment1, "Fragment1");
            break;
        case R.id.rb_2:
            rb_2.setBackgroundColor(Color.YELLOW);
            transaction.replace(R.id.fl, fragment2, "Fragment2");
            transaction.show(fragment2);
            break;
        case R.id.rb_3:
            rb_3.setBackgroundColor(Color.BLUE);
            transaction.replace(R.id.fl, fragment3, "Fragment3");
            transaction.show(fragment3);
            break;
```

实现这个功能可以通过 replace 和 add 两种方法。

Fragment 的容器一个 FrameLayout，add 的时候是把所有的 Fragment 一层一层的叠加到了 FrameLayout 上了，而 replace 的话首先将该容器中的其他 Fragment 去除掉然后将当前 Fragment 添加到容器中。

一个 Fragment 容器中只能添加一个 Fragment 种类，如果多次添加则会报异常，导致程序终止，而 replace 则无所谓，随便切换。

因为通过 add 的方法添加的 Fragment，每个 Fragment 只能添加一次，因此如果要想达到切换效果需要通过 Fragment 的 hide 和 show 方法结合者使用。将要显示的 show 出来，将其他 hide 起来。这个过程 Fragment 的生命周期没有变化。

通过 replace 切换 Fragment，每次都会执行上一个 Fragment 的 onDestroyView，新 Fragment 的 onCreateView、onStart、onResume 方法。

基于以上不同的特点我们在使用的使用一定要结合着生命周期操作我们的视图和数据。

4、Fragment 如何实现类似 Activity 栈的压栈和出栈效果的？（2015.8.30）

Fragment 的事物管理器内部维持了一个双向链表结构，该结构可以记录我们每次 add 的 Fragment 和 replace

的 Fragment，然后当我们点击 back 按钮的时候会自动帮我们实现退栈操作。

Add this transaction to the back stack. This means that the transaction will be remembered after it is committed, and will reverse its operation when later popped off the stack.

Parameters:

name An optional name for this back stack state, or null.

```
transaction.addToBackStack("name");
```

//实现源码 在 BackStackRecord 中

```
public FragmentTransaction addToBackStack(String name) {  
    if (!mAllowAddToBackStack) {  
        throw new IllegalStateException(  
            "This FragmentTransaction is not allowed to be added to the back  
stack.");  
    }  
    mAddToBackStack = true;  
    mName = name;  
    return this;  
}
```

//上面的源码仅仅做了一个标记

除此之外因为我们要使用 FragmentManger 用的是 FragmentActivity 因此 FragmentActivity 的 onBackPressed 方法必定重新覆写了。打开看一下，发现确实如此。

```
/**  
 * Take care of popping the fragment back stack or finishing the activity  
 * as appropriate.  
 */  
public void onBackPressed() {  
    if (!mFragments.popBackStackImmediate()) {  
        finish();  
    }  
}
```

//mFragments 的原型是 [FragmentManagerImpl](#)，看看这个方法都干嘛了


```
@Override
```

```
public boolean popBackStackImmediate() {  
    checkStateLoss();  
    executePendingTransactions();  
    return popBackStackState(mActivity.mHandler, null, -1, 0);  
}
```

//看看 popBackStackState 方法都干了啥，其实通过名称也能大概了解 只给几个片段吧，代码太多了

```
while (index >= 0) {  
    //从后退栈中取出当前记录对象  
    BackStackRecord bss = mBackStack.get(index);  
    if (name != null && name.equals(bss.getName())) {  
        break;  
    }  
    if (id >= 0 && id == bss.mIndex) {  
        break;  
    }  
    index--;  
}
```

5. Android 高级 (★★★★)

一、Android 性能优化

1、如何对 Android 应用进行性能分析

一款 App 流畅与否安装在自己的真机里，玩几天就能有个大概的感性认识。不过通过专业的分析工具可以使我们更好的分析我们的应用。

如果不考虑使用其他第三方性能分析工具的话，我们可以直接使用 ddms 中的工具，其实 ddms 工具已经非常的强大了。ddms 中有 traceview、heap、allocation tracker 等工具都可以帮助我们分析应用的方法执行时间效率和内存使用情况。

◆ [traceview](#) 工具在本文章中已经有详细的介绍，因此这里就不再赘述。

◆ heap

heap 工具可以帮助我们检查代码中是否存在会造成内存泄漏的地方。

用 heap 监测应用进程使用内存情况的步骤如下：

1. 启动 eclipse 后，切换到 DDMS 透视图，并确认 Devices 视图、Heap 视图都是打开的；
2. 点击选中想要监测的进程，比如 system_process 进程；
3. 点击选中 Devices 视图界面中最上方一排图标中的“Update Heap”图标；
6. 点击 Heap 视图中的“Cause GC”按钮；
7. 此时在 Heap 视图中就会看到当前选中的进程的内存使用量的详细情况。

说明：

- a) 点击“Cause GC”按钮相当于向虚拟机请求了一次 gc 操作；
- b) 当内存使用信息第一次显示以后，无须再不断的点击“Cause GC”，Heap 视图界面会定时刷新，在对应用的不断的操作过程中就可以看到内存使用的变化；
- c) 内存使用信息的各项参数根据名称即可知道其意思，在此不再赘述。

如何才能知道我们的程序是否有内存泄漏的可能性呢。这里需要注意一个值：Heap 视图中部有一个 Type 叫做 data object，即数据对象，也就是我们的程序中大量存在的类类型的对象。在 data object 一行中有一列是“Total Size”，其值就是当前进程中所有 Java 数据对象的内存总量，一般情况下，这个值的大小决定了是否有内存泄漏。可以这样判断：

- a) 不断的操作当前应用，同时注意观察 data object 的 Total Size 值；
- b) 正常情况下 Total Size 值都会稳定在一个有限的范围内，也就是说由于程序中的代码良好，没有造成对象不被垃圾回收的情况，所以说虽然我们不断的操作会不断的生成很多对象，而在虚拟机不断的进行 GC 的过程中，这些对象都被回收了，内存占用量会落到一个稳定的水平；
- c) 反之如果代码中存在没有释放对象引用的情况，则 data object 的 Total Size 值在每次 GC 后不会有明显的回落，随着操作次数的增多 Total Size 的值会越来越大，

直到到达一个上限后导致进程被 kill 掉。

d) 此处以 system_process 进程为例 ,在我的测试环境中 system_process 进程所占用的内存的 data object 的 Total Size 正常情况下会稳定在 2.2~2.8 之间，而当其值超过 3.55 后进程就会被 kill。

总之，使用 DDMS 的 Heap 视图工具可以很方便的确认我们的程序是否存在内存泄漏的可能性。

◆ allocation tracker

运行 DDMS，只需简单的选择应用进程并单击 Allocation tracker 标签，就会打开一个新的窗口，单击 “Start Tracing” 按钮；

然后，让应用运行你想分析的代码。运行完毕后，单击 “Get Allocations” 按钮，一个已分配对象的列表就会出现第一个表格中。

单击第一个表格中的任何一项，在表格二中就会出现导致该内存分配的栈跟踪信息。通过 allocation tracker，不仅知道分配了哪类对象，还可以知道在哪个线程、哪个类、哪个文件的哪一行。

2、什么情况下会导致内存泄露

Android 的虚拟机是基于寄存器的 Dalvik，它的最大堆大小一般是 16M，有的机器为 24M。因此我们所能利用的内存空间是有限的。如果我们的内存占用超过了一定的水平就会出现 OutOfMemory 的错误。

内存溢出的几点原因：

1、资源释放问题

程序代码的问题，长期保持某些资源，如 Context、Cursor、IO 流的引用，资源得不到释放造成内存泄露。

2、对象内存过大问题

保存了多个耗用内存过大的对象（如 Bitmap、XML 文件），造成内存超出限制。

3、static 关键字的使用问题

`static` 是 Java 中的一个关键字，当用它来修饰成员变量时，那么该变量就属于该类，而不是该类的实例。所以用 `static` 修饰的变量，它的生命周期是很长的，如果用它来引用一些资源耗费过多的实例（Context 的情况最多），这时就要谨慎对待了。

```
public class ClassName {  
  
    private static Context mContext;  
  
    //省略  
  
}
```

以上的代码是很危险的，如果将 Activity 赋值到 `mContext` 的话。那么即使该 Activity 已经 `onDestroy`，但是由于仍有对象保存它的引用，因此该 Activity 依然不会被释放。

我们举 Android 文档中的一个例子。

```
private static Drawable sBackground;  
@Override  
protected void onCreate(Bundle state) {  
    super.onCreate(state);  
    TextView label = new TextView(this); //getApplicationContext  
    label.setText("Leaks are bad");  
    if (sBackground == null) {  
        sBackground = getDrawable(R.drawable.large_bitmap);  
    }  
    label.setBackgroundDrawable(sBackground);  
    setContentView(label);  
}
```

`sBackground` 是一个静态的变量，但是我们发现，我们并没有显式的保存 Context 的引用，但是，当 `Drawable`

与 View 连接之后，Drawable 就将 View 设置为一个回调，由于 View 中是包含 Context 的引用的，所以，实际上我们依然保存了 Context 的引用。这个引用链如下：

Drawable->TextView->Context

所以，最终该 Context 也没有得到释放，发生了内存泄露。

◆ 针对 static 的解决方案

① 应该尽量避免 static 成员变量引用资源耗费过多的实例，比如 Context。

② Context 尽量使用 ApplicationContext，因为 Application 的 Context 的生命周期比较长，引用它不会出现内存泄露的问题。

③ 使用 WeakReference 代替强引用。比如可以使用 WeakReference<Context> mContextRef;

4、线程导致内存溢出

线程产生内存泄露的主要原因在于线程生命周期的不可控。我们来考虑下面一段代码。

```
public class MyActivity extends Activity {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);
        new MyThread().start();
    }
    private class MyThread extends Thread{
        @Override
        public void run() {
            super.run();
            //do something while(true)
        }
    }
}
```

这段代码很平常也很简单，是我们经常使用的形式。我们思考一个问题：假设 MyThread 的 run 函数是一个很费

时的操作，当我们开启该线程后，将设备的横屏变为了竖屏，一般情况下当屏幕转换时会重新创建 Activity，按照我们的想法，老的 Activity 应该会被销毁才对，然而事实上并非如此。

由于我们的线程是 Activity 的内部类，所以 MyThread 中保存了 Activity 的一个引用，当 MyThread 的 run 函数没有结束时，MyThread 是不会被销毁的，因此它所引用的老的 Activity 也不会被销毁，因此就出现了内存泄露的问题。

有些人喜欢用 Android 提供的 AsyncTask，但事实上 AsyncTask 的问题更加严重，Thread 只有在 run 函数不结束时才出现这种内存泄露问题，然而 AsyncTask 内部的实现机制是运用了 ThreadPoolExecutor，该类产生的 Thread 对象的生命周期是不确定的，是应用程序无法控制的，因此如果 AsyncTask 作为 Activity 的内部类，就更容易出现内存泄露的问题。

针对这种线程导致的内存泄露问题的解决方案：

第一、将线程的内部类，改为静态内部类（因为非静态内部类拥有外部类对象的强引用，而静态类则不拥有）。

第二、在线程内部采用弱引用保存 Context 引用。

3、如何避免 OOM 异常

想要避免 OOM 异常首先我们要知道什么情况下会导致 OOM 异常。

1、图片过大导致 OOM

Android 中用 bitmap 时很容易内存溢出，比如报如下错误：Java.lang.OutOfMemoryError : bitmap size exceeds VM budget。

解决方法：

方法 1：等比例缩小图片

```
BitmapFactory.Options options = new BitmapFactory.Options();
options.inSampleSize = 2;
//Options 只保存图片尺寸大小，不保存图片到内存
BitmapFactory.Options opts = new BitmapFactory.Options();
opts.inSampleSize = 2;
Bitmap bmp = null;
bmp = BitmapFactory.decodeResource(getResources(),
mImageIds[position],opts);
//回收
bmp.recycle();//
```

以上代码可以优化内存溢出，但它只是改变图片大小，并不能彻底解决内存溢出。

方法 2：对图片采用软引用，及时地进行 recycle()操作

```
SoftReference<Bitmap> bitmap = new SoftReference<Bitmap>(pBitmap);
if(bitmap != null){
    if(bitmap.get() != null && !bitmap.get().isRecycled()){
        bitmap.get().recycle();
        bitmap = null;
    }
}
```

2、界面切换导致 OOM

有时候我们会发现这样的问题，横竖屏切换 N 次后 OOM 了。

这种问题没有固定的解决方法，但是我们可以从以下几个方面下手分析。

1、看看页面布局当中有没有大的图片，比如背景图之类的。

去除 xml 中相关设置，改在程序中设置背景图（放在 onCreate()方法中）：

```
Drawable drawable = getResources().getDrawable(R.drawable.id);
ImageView imageView = new ImageView(this);
imageView.setBackgroundDrawable(drawable);
```

在 Activity destory 时注意，drawable.setCallback(null)；防止 Activity 得不到及时的释放。

2、跟上面方法相似，直接把 xml 配置文件加载成 view 再放到一个容器里，然后直接调用

this setContentView(View view);方法，避免 xml 的重复加载。

3、在页面切换时尽可能少地重复使用一些代码

比如：重复调用数据库，反复使用某些对象等等.....

常见的内存使用不当的情况

3、查询数据库没有关闭游标

程序中经常会进行查询数据库的操作，但是经常会有使用完毕 Cursor 后没有关闭的情况。如果我们的查询结果集比较小，对内存的消耗不容易被发现，只有在长时间大量操作的情况下才会出现内存问题，这样就会给以后的测试和问题排查带来困难和风险。

4、构造 Adapter 时，没有使用缓存的 convertView

在使用 ListView 的时候通常会使用 Adapter，那么我们应该尽可能的使用 convertView。

5、Bitmap 对象不再使用时调用 recycle() 释放内存

有时我们会手工的操作 Bitmap 对象，如果一个 Bitmap 对象比较占内存，当它不再被使用的时候，可以调用 Bitmap.recycle() 方法回收此对象的像素所占用的内存，但这不是必须的，视情况而定。

6、其他

Android 应用程序中最典型的需要注意释放资源的情况是在 Activity 的生命周期中，在 onPause()、onStop()、onDestroy() 方法中需要适当的释放资源的情况。

4、Android 中如何捕获未捕获的异常(重要)

- 1、自定义一个 Application，比如叫 MyApplication 继承 Application 实现 UncaughtExceptionHandler。
- 2、覆写 UncaughtExceptionHandler 的 onCreate 和 uncaughtException 方法。


```
@Override
public void onCreate() {
    super.onCreate();
    Thread.setDefaultUncaughtExceptionHandler(this);
}

@Override
public void uncaughtException(final Thread thread, final Throwable ex) {
    new Thread(new Runnable() {

        @Override
        public void run() {
            Looper.prepare();
            System.out.println(Thread.currentThread());
            Toast.makeText(getApplicationContext(), "thread="+thread.getId()+"
ex="+ex.toString(), 1).show();
            Looper.loop();
        }
    }).start();
    SystemClock.sleep(3000);
    android.os.Process.killProcess(android.os.Process.myPid());
}
}
```

注意：上面的代码只是简单的将异常打印出来。

在 onCreate 方法中我们给 Thread 类设置默认异常处理 handler，如果这句代码不执行则一切都是白搭。

在 uncaughtException 方法中我们必须新开辟个线程进行我们异常的收集工作，然后将系统给杀死。

3、在 AndroidManifest 中配置该 Application

```
<application
    android:name="com.example.uncatchexception.MyApplication"
```

4、blog 分享

关于异常数据的收集在网上有一篇不错的 blog 可以推荐给大家。

<http://blog.csdn.net/jdsjlzx/article/details/7606423>

5、Android 性能优化博客

给大家推荐一篇 Android 性能优化的 blog，其目录如下：

1.背景

2.应用 UI 性能问题分析

2-1 应用 UI 卡顿原理

2-2 应用 UI 卡顿常见原因

2-3 应用 UI 卡顿分析解决方法

2-3-1 使用 HierarchyViewer 分析 UI 性能

2-3-2 使用 GPU 过度绘制分析 UI 性能

2-3-3 使用 GPU 呈现模式图及 FPS 考核 UI 性能

2-3-4 使用 Lint 进行资源及冗余 UI 布局等优化

2-3-5 使用 Memory 监测及 GC 打印与 Allocation Tracker 进行 UI 卡顿分析

2-3-6 使用 Traceview 和 dmtracedump 进行分析优化

2-3-7 使用 Systrace 进行分析优化

2-3-8 使用 tracestxt 文件进行 ANR 分析优化

2-4 应用 UI 性能分析解决总结

3.应用开发 Memory 内存性能分析优化

3-1 Android 内存管理原理

3-2 Android 内存泄露性能分析

3-2-1 Android 应用内存泄露概念

3-2-2 Android 应用内存泄露察觉手段

3-2-3 Android 应用内存泄露 leakcanary 工具定位分析

3-2-4 Android 应用内存泄露 MAT 工具定位分析

3-2-5 Android 应用开发规避内存泄露建议

3-3 Android 内存溢出 OOM 性能分析

3-3-1 Android 应用内存溢出 OOM 概念

3-3-2 Android 应用内存溢出 OOM 性能分析

3-3-3 Android 应用规避内存溢出 OOM 建议

3-4 Android 内存性能优化总结

4.Android 应用 API 使用及代码逻辑性能分析

4-1 Android 应用 StringStringBuilderStringBuffer 优化建议

4-2 Android 应用 OnTrimMemory 实现性能建议

4-3 Android 应用 HashMap 与 ArrayMap 及 SparseArray 优化建议

4-4 Android 应用 ContentProviderOperation 优化建议

4-5 Android 应用其他逻辑优化建议

5.Android 应用移动设备电池耗电性能分析

5-1 Android 应用耗电量概念

5-2 Android 应用耗电量优化建议

6.Android 应用开发性能优化总结

blog 原文如下：

<http://blog.csdn.net/yanbober/article/details/48394201>

二、Android 屏幕适配

1、屏幕适配方式都有哪些

1.1 适配方式之 dp

名词解释：

◆ 分辨率：eg：480*800,1280*720。表示物理屏幕区域内像素点的总和。(切记：跟屏幕适配没有任何关系)

因为我们既可以把 1280*720 的分辨率做到 4.0 的手机上面。我也可以把 1280*720 的分辨率做到 5.0 英寸的手机上面，如果分辨率相同，手机屏幕越小清晰。

◆ px(pix)：像素，就是屏幕中最小的一个显示单元

◆ dpi (像素密度)：即每英寸屏幕所拥有的像素数，像素密度越大，显示画面细节就越丰富。

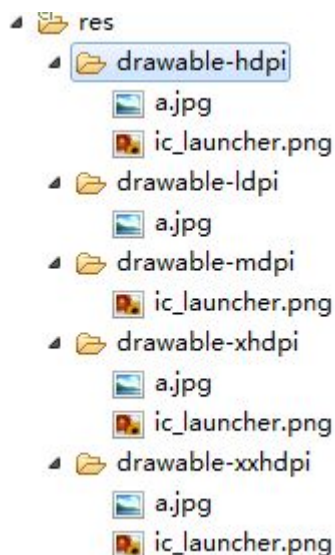
计算公式：像素密度= $\sqrt{(\text{长度像素数}^2 + \text{宽度像素数}^2)}$ / 屏幕尺寸

注：屏幕尺寸单位为英寸 例：分辨率为 1280*720 屏幕宽度为 6 英寸 计算所得像素密度约等于 245，屏幕尺寸指屏幕对角线的长度。

在 Android 手机中 dpi 分类：

ldpi	Resources for low-density (<i>ldpi</i>) screens (~120dpi).
mdpi	Resources for medium-density (<i>mdpi</i>) screens (~160dpi). (This is the baseline density.)
hdpi	Resources for high-density (<i>hdpi</i>) screens (~240dpi).
xhdpi	Resources for extra high-density (<i>xhdpi</i>) screens (~320dpi).

在我们的 Android 工程目录中有如下 drawable-*dpi 目录，这些目录是用来适配不同分辨率手机的。



Android 应用在查找图片资源时会根据其分辨率自动从不同的文件目录下查找（这本身就是 Android 系统的适配策略），如果在低分辨率的文件目录中比如 `drawable-mdpi` 中没有图片资源，其他目录中都有，当我们将该应用部署到 `mdpi` 分辨率的手机上时，那么该应用会查找分辨率较高目录下的资源文件，如果较高分辨率目录下也没有资源则只好找较低目录中的资源了。

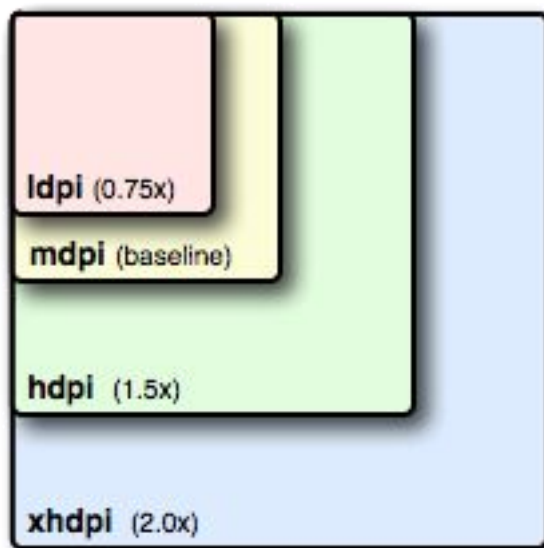
常见手机屏幕像素及对应分辨率级别：

- ◆ ldpi 320*240
- ◆ mdpi 480*320
- ◆ hdpi 800*480
- ◆ xhdpi 1280*720
- ◆ xxhdpi 1920*1080

dp 和 px 之间的简单换算关系：

- ◆ ldpi 的手机 1dp=0.75px
- ◆ mdpi 的手机 1dp=1.0px
- ◆ hdpi 的手机 1dp=1.5px
- ◆ xhdpi 的手机 1dp=2.0px

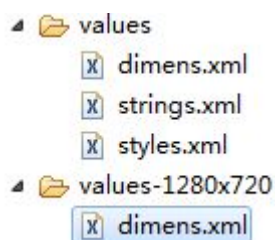
◆ xxhdpi 的手机 1dp=3.0px



Tips：根据上面的描述我们得出如下结论，对于 mdpi 的手机，我们的布局通过 dp 单位可以达到适配效果。

1.2 适配方式之 `dimens`

跟 `drawable` 目录类似的，在 Android 工程的 `res` 目录下有 `values` 目录，这个是默认的目录，同时为了适配不同尺寸手机我们可以创建一个 `values-1280x720` 的文件夹，同时将 `dimens.xml` 文件拷贝到该目录下。



在 `dimens.xml` 中定义一个尺寸，如下图所示。

```
dimens.xml
1 <resources>
2   <dimen name="width">160dp</dimen>
3 </resources>
```

在 `values-1280x720` 目录中的 `dimens.xml` 中定义同样的尺寸名称，但是使用不同的尺寸，如下图所示。

```
<resources>
  <dimen name="width">180dp</dimen>
</resources>
```

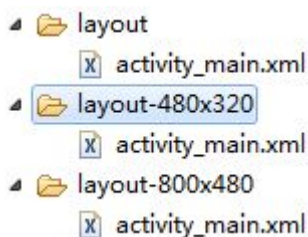
当我们在布局文件中使用长或者宽度单位时，比如下图所示，应该使用@dimen/width 来灵活的定义宽度。

```
<TextView
    android:background="#000000"
    android:layout_width="@dimen/width"
    android:layout_height="wrap_content"
    android:text="@string/hello_world" />
<LinearLayout
```

Tips：在 values-1280x720 中，中间的是大写字母 X 的小写形式 x，而不是加减乘除的乘号。如果我们在 values-1280x720 中放置了 dimens 常量，一定记得也将该常量的对应值在 values 目录下的 dimens.xml 中放一份，因为该文件是默认配置，当用户的手机不是 1280*720 的情况下系统应用使用的是默认 values 目录中的 dimens.xml。

1.3 适配方式之 layout

跟 values 一样，在 Android 工程目录中 layout 目录也支持类似 values 目录一样的适配，在 layout 中我们可以针对不同手机的分辨率制定不同的布局，如下图所示。



1.4 适配方式之 java 代码适配

为了演示用 java 代码控制适配的效果，因此假设有这样的需求，让一个 TextView 控件的宽和高分别为屏幕的宽和高的一半。

我们新创建一个 Android 工程，修改 main_activity.xml，布局文件清单如下：

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity" >
```

```
<!-- 当前控件宽高为屏幕宽度的各 50% -->
<TextView
    android:id="@+id/tv"
    android:background="#000000"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/hello_world" />
</RelativeLayout>
```

在 MainActivity.java 类中完成用 java 代码控制 TextView 的布局效果，其代码清单如下：

```
public class MainActivity extends Activity {

    private static final String tag = null;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        //去掉 title
        requestWindowFeature(Window.FEATURE_NO_TITLE);
        setContentView(R.layout.activity_main);
        //获取 TextView 控件
        TextView tv = (TextView) findViewById(R.id.tv);
        //找到当前控件的父控件(父控件上给当前的子控件去设定一个规则)
        DisplayMetrics metrics = new DisplayMetrics();
        //给当前 metrics 去设置当前屏幕信息(宽(像素)高(像素))
        getWindowManager().getDefaultDisplay().getMetrics(metrics);
        //获取屏幕的高度和宽度
        Constant.srceenHeight = metrics.heightPixels;
        Constant.srceenWidth = metrics.widthPixels;
        //日志输出屏幕的高度和宽度
        Log.i(tag, "Constant.srceenHeight = "+Constant.srceenHeight);
        Log.i(tag, "Constant.srceenWidth = "+Constant.srceenWidth);
        //宽高各 50%
        RelativeLayout.LayoutParams layoutParams = new RelativeLayout.LayoutParams(
            //数学角度上 四舍五入
            (int)(Constant.srceenWidth*0.5+0.5),
            (int)(Constant.srceenHeight*0.5+0.5));
        //给 tv 控件设置布局参数
        tv.setLayoutParams(layoutParams);
    }
}
```


其中 Constant 类是一个常量类，很简单，只有两个常量用来记录屏幕的宽和高，其代码清单如下：

```
public class Constant {  
    public static int srceenHeight;  
    public static int srceenWidth;  
}
```

1.5 适配方式之 weight 权重适配

在控件中使用属性 `android:layout_weight="1"` 可以起到适配效果，但是该属性的使用有如下规则：

- 1、只能用在线性控件中，比如 LinearLayout。
- 2、竖直方向上使用权重的控件高度必须为 0dp（Google 官方的推荐用法）
- 3、水平方向上使用权重的控件宽度必须为 0dp（Google 官方的推荐用法）

2、屏幕适配的处理技巧都有哪些

手机自适应主要分为两种情况：横屏和竖屏的切换，以及分辨率大小的不同。

2.1 横屏和竖屏的切换

1、Android 应用程序支持横竖屏幕的切换，Android 中每次屏幕的切换都会重启 Activity，所以应该在 Activity 销毁（执行 `onPause()` 方法和 `onDestroy()` 方法）前保存当前活动的状态；在 Activity 再次创建的时候载入配置，那样，进行中的游戏就不会自动重启了！有的程序适合从竖屏切换到横屏，或者反过来，这个时候怎么办呢？可以在配置 Activity 的地方进行如下的配置 `android:screenOrientation="portrait"`（landscape 是横向，portrait 是纵向）。这样就可以保证是竖屏总是竖屏了。

2、而有的程序是适合横竖屏切换的。如何处理呢？首先要在配置 Activity 的时候进行如下的配置：

`android:configChanges="keyboardHidden|orientation"`，另外需要重写 Activity 的 `onConfigurationChanged` 方法。实现方式如下：

```
@Override
public void onConfigurationChanged(Configuration newConfig){
    super.onConfigurationChanged(newConfig);
    if(this.getResources().getConfiguration().orientation==Configuration.ORIENTATION_
    LANDSCAPE){
        //TODO
    }else
    if(this.getResources().getConfiguration().orientation==Configuration.ORIENTATION_
    PORTRAIT){
        //TODO
    }
}
```

2.2 分辨率大小不同

对于分辨率问题，官方给的解决办法是创建不同的 layout 文件夹，这就需要对每种分辨率的手机都要写一个布局文件，虽然看似解决了分辨率的问题，但是如果其中一处或多处有修改了，就要每个布局文件都要做出修改，这样就造成很大的麻烦。那么可以通过以下几种方式解决：

一）使用 layout_weight

目前最为推荐的 Android 多屏幕自适应解决方案。

该属性的作用是决定控件在其父布局中的显示权重，一般用于线性布局中。其值越小，则对应的 layout_width 或 layout_height 的优先级就越高（一般到 100 作用就不太明显了）；一般横向布局中，决定的是 layout_width 的优先级；纵向布局中，决定的是 layout_height 的优先级。

传统的 layout_weight 使用方法是将当前控件的 layout_width 和 layout_height 都设置成 fill_parent,这样就可以把控件的显示比例完全交给 layout_weight；这样使用的话，就出现了 layout_weight 越小，显示比例越大的情况（即权重越大，显示所占的效果越小）。不过对于 2 个控件还好，如果控件过多，且显示比例也不相同的时候，控制起来就比较麻烦了，毕竟反比不是那么好确定的。于是就有了现在最为流行的 0px 设值法。看似让人难以理解的 layout_height=0px 的写法，结合 layout_weight，却可以使控件成正比例显示，轻松解决了当前 Android 开发最为头疼的碎片化问题之一。

二) 清单文件配置：【不建议使用这种方式，需要对不同的界面写不同的布局】

需要在 AndroidManifest.xml 文件的<manifest>元素如下添加子元素

```
<supports-screens android:largeScreens="true"
    android:normalScreens="true"
    android:anyDensity="true"
    android:smallScreens="true"
    android:xlargeScreens="true">
</supports-screens>
```

以上是为我们的屏幕设置多分辨率支持（更准确的说是适配大、中、小三种密度）。

Android:anyDensity="true"，这一句对整个的屏幕都起着十分重要的作用，值为 true，我们的应用程序当安装在不同密度的手机上时，程序会分别加载 hdpi,mdpi,ldpi 文件夹中的资源。相反，如果值设置为 false，即使我们在 hdpi,mdpi,ldpi，xdpi 文件夹下拥有同一种资源，那么应用也不会自动地去相应文件夹下寻找资源。而是会在大密度和小密度手机上加载中密度 mdpi 文件中的资源。

有时候会根据需要在代码中动态地设置某个值，可以在代码中为这几种密度分别设置偏移量，但是这种方法最好不要使用，最好的方式是在 xml 文件中不同密度的手机进行分别设置。这里地图的偏移量可以在 values-xpdi，values-hpdi,values-mdpi,values-ldpi 四种文件夹中的 dimens.xml 文件进行设置。

三)、其他：

说明：

在不同分辨率的手机模拟器下，控件显示的位置会稍有不同

通过在 layout 中定义的布局设置的参数，使用 dp (dip)，会根据不同的屏幕分辨率进行适配

但是在代码中的各个参数值，都是使用的像素 (px) 为单位的

技巧：

1、尽量使用线性布局，相对布局，如果屏幕放不下了，可以使用 ScrollView（可以上下拖动）

ScrollView 使用的注意：

在不同的屏幕上显示内容不同的情况，其实这个问题我们往往是用滚动视图来解决的，也就是 ScrollView；需要注意的是 ScrollView 中使用 layout_weight 是无效的，既然使用 ScrollView 了，就把它里面的控件的大小都设成固定的吧。

2、指定宽高的时候，采用 dip 的单位，dp 单位动态匹配

3、由于 android 代码中写的单位都是像素，所有需要通过工具类进行转化

4、尽量使用 9-patch 图，可以自动的依据图片上面显示的内容被拉伸和收缩。其中在编辑的时候，灰色区域是被拉伸的，上下两个点控制水平方向的拉伸，左右两点控制垂直方向的拉伸

3、dp 和 px 之间的关系

dp：是 dip 的简写，指密度无关的像素。

指一个抽象意义上的像素，程序用它来定义界面元素。一个与密度无关的，在逻辑尺寸上，与一个位于像素密度为 160dpi 的屏幕上的像素是一致的。要把密度无关像素转换为屏幕像素，可以用这样一个简单的公式： $\text{pixels} = \text{dips} * (\text{density} / 160)$ 。举个例子，在 DPI 为 240 的屏幕上，1 个 DIP 等于 1.5 个物理像素。

布局时最好使用 dp 来定义我们程序的界面，因为这样可以保证我们的 UI 在各种分辨率的屏幕上都可以正常显示。

```
/**
 * 根据手机的分辨率从 px(像素) 的单位 转成为 dp
 */
public static int px2dip(Context context, float pxValue) {
    final float scale = context.getResources().getDisplayMetrics().density;
    return (int) (pxValue / scale + 0.5f);
}

/**
 * 根据手机的分辨率从 dip 的单位 转成为 px(像素)
 */
public static int dip2px(Context context, float dpValue) {
    final float scale = context.getResources().getDisplayMetrics().density;
    return (int) (dpValue * scale + 0.5f);
}
```

三、AIDL

1、什么是 AIDL 以及如何使用

①aidl 是 Android interface definition Language 的英文缩写，意思 Android 接口定义语言。

②使用 aidl 可以帮助我们发布以及调用远程服务，实现跨进程通信。

③将服务的 aidl 放到对应的 src 目录，工程的 gen 目录会生成相应的接口类

我们通过 bindService (Intent , ServiceConnect , int) 方法绑定远程服务，在 bindService 中有一个 ServiceConnec 接口，我们需要覆写该类的 onServiceConnected(ComponentName,IBinder)方法，这个方法的第二个参数 IBinder 对象其实就是已经在 aidl 中定义的接口，因此我们可以将 IBinder 对象强制转换为 aidl 中的接口类。

我们通过 IBinder 获取到的对象（也就是 aidl 文件生成的接口）其实是系统产生的代理对象，该代理对象既可以跟我们的进程通信，又可以跟远程进程通信，作为一个中间的角色实现了进程间通信。

四、自定义控件

1、如何自定义一个控件

自定义控件可以分为两种自定义组合控件和自定义 view。

◆ 自定义组合控件

自定义组合控件就是把多个控件做为一个整体看待、处理。这样的好处不仅可以减轻 xml 的代码量，也提高了代码的复用性。

在手机卫士项目中我们第一次接触了自定义组合控件。

1. 声明一个 View 对象，继承相对布局，或者线性布局或者其他的 ViewGroup。
2. 在自定义的 View 对象里面重写它的构造方法，在构造方法里面就把布局都初始化完毕。
3. 根据业务需求添加一些 api 方法，扩展自定义的组合控件；
4. 希望在布局文件里面可以自定义一些属性。
5. 声明自定义属性的命名空间。

```
xmlns:itheima="http://schemas.android.com/apk/res/com.itheima.mobilesafe"
```

6. 在 res 目录下的 values 目录下创建 attrs.xml 的文件声明我们写的属性。
7. 在布局文件中写自定义的属性。
8. 使用这些定义的属性。自定义 View 对象的构造方法里面有一个带两个参数的构造方法布局文件里面定义的属性都放在 AttributeSet attrs，获取那些定义的属性。

◆ 自定义 view

自定义 View 首先要实现一个继承自 View 的类。添加类的构造方法，通常是三个构造方法，不过从 Android5.0 开始构造方法已经添加到 4 个了。override 父类的方法，如 onDraw，（onMeasure）等。如果自定义的 View 有自己的属性，需要在 values 下建立 attrs.xml 文件，在其中定义属性，同时代码也要做修改。

blog 分享：<http://blog.csdn.net/lmj623565791/article/details/24252901>

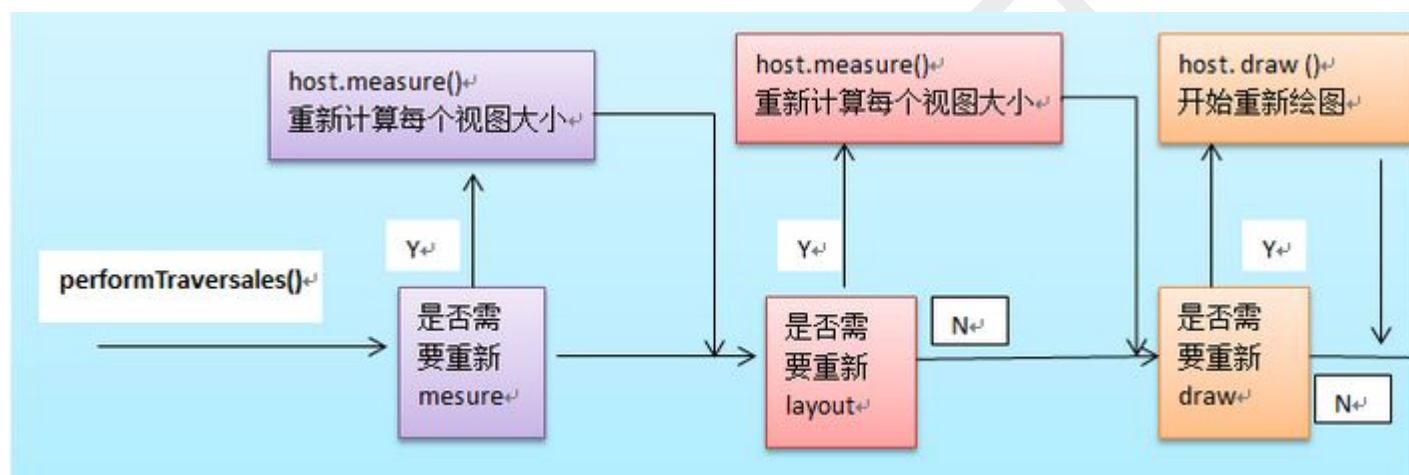
2、请描述一下 View 的绘制流程

整个 View 树的绘图流程是在 ViewRoot.java 类(该类位于 Android 源码下面：

D:\AndroidSource_GB\AndroidSource_GB\frameworks\base\core\java\android\view)的

performTraversals()函数展开的，该函数做的执行过程可简单概况为根据之前设置的状态，判断是否需要重新计

算视图大小(measure)、是否重新需要安置视图的位置(layout)、以及是否需要重绘 (draw)，其框架过程如下：



1、measure() 过程

主要作用：为整个 View 树计算实际的大小，即设置实际的高(对应属性:mMeasuredHeight)和宽(对应属性:mMeasureWidth)，每个 View 的控件的实际宽高都是由父视图和本身视图决定的。

具体的调用链如下：ViewRoot 根对象的属性 mView(其类型一般为 ViewGroup 类型)调用 measure()方法去计算 View 树的大小，回调 View/ViewGroup 对象的 onMeasure()方法，该方法实现的功能如下：

1、设置本 View 视图的最终大小，该功能的实现通过调用 setMeasuredDimension()方法去设置实际的高(对应属性：mMeasuredHeight)和宽(对应属性：mMeasureWidth)。

2、如果该 View 对象是个 ViewGroup 类型，需要重写该 onMeasure()方法，对其子视图进行遍历的 measure()过程。对每个子视图的 measure()过程，是通过调用父类 ViewGroup.java 类里的 measureChildWithMargins()方法去实现，该方法内部只是简单地调用了 View 对象的 measure()方法。

2、layout 布局过程

主要作用：为将整个根据子视图的大小以及布局参数将 View 树放到合适的位置上。

具体的调用链如下：

1、layout 方法会设置该 View 视图位于父视图的坐标轴，即 mLeft，mTop，mLeft，mBottom(调用 setFrame()函数去实现)接下来回调 onLayout()方法(如果该 View 是 ViewGroup 对象，需要实现该方法，对每个子视图进行布局)。

2、如果该 View 是个 ViewGroup 类型，需要遍历每个子视图 childView，调用该子视图的 layout()方法去设置它的坐标值。

3、draw() 绘图过程

由 ViewRoot 对象的 performTraversals()方法调用 draw()方法发起绘制该 View 树，值得注意的是每次发起绘图时，并不会重新绘制每个 View 树的视图，而只会重新绘制那些“需要重绘”的视图，View 类内部变量包含了一个标志位 DRAWN，当该视图需要重绘时，就会为该 View 添加该标志位。

调用流程：

- 1、绘制该 View 的背景
- 2、为显示渐变框做一些准备操作(大多数情况下，不需要改渐变框)
- 3、调用 onDraw()方法绘制视图本身(每个 View 都需要重载该方法，ViewGroup 不需要实现该方法)
- 4、调用 dispatchDraw ()方法绘制子视图(如果该 View 类型不为 ViewGroup，即不包含子视图，不需要重载该方法)

值得说明的是，ViewGroup 类已经为我们重写了 dispatchDraw ()的功能实现，应用程序一般不需要重写该方法，但可以重载父类函数实现具体的功能。

参考 blog 分享：<http://blog.csdn.net/qinjuning/article/details/7110211>

五、Android 中的事件处理

1、Handler 机制

Android 中主线程也叫 UI 线程，那么从名字上我们也知道主线程主要是用来创建、更新 UI 的，而其他耗时操作，比如网络访问，或者文件处理，多媒体处理等都需要在子线程中操作，之所以在子线程中操作是为了保证 UI 的流畅程度，手机显示的刷新频率是 60Hz，也就是一秒钟刷新 60 次，每 16.67 毫秒刷新一次，为了不丢帧，那么主线程处理代码最好不要超过 16 毫秒。当子线程处理完数据后，为了防止 UI 处理逻辑的混乱，Android 只允许主线程修改 UI，那么这时候就需要 Handler 来充当子线程和主线程之间的桥梁了。

我们通常将 Handler 声明在 Activity 中，然后覆写 Handler 中的 handleMessage 方法，当子线程调用 handler.sendMessage()方法后 handleMessage 方法就会在主线程中执行。

这里面除了 Handler、Message 外还有隐藏的 Looper 和 MessageQueue 对象。

在主线程中 Android 默认已经调用了 Looper.preper() 方法，调用该方法的目的是在 Looper 中创建 MessageQueue 成员变量并把 Looper 对象绑定到当前线程中。当调用 Handler 的 sendMessage (对象) 方法的时候就将 Message 对象添加到了 Looper 创建的 MessageQueue 队列中，同时给 Message 指定了 target 对象，其实这个 target 对象就是 Handler 对象。主线程默认执行了 Looper.looper () 方法，该方法从 Looper 的成员变量 MessageQueue 中取出 Message，然后调用 Message 的 target 对象的 handleMessage()方法。这样就完成了整个消息机制。

2、事件分发机制

2.1 事件分发中的 onTouch 和 onTouchEvent 有什么区别，又该如何使用？

这两个方法都是在 View 的 dispatchTouchEvent 中调用的，onTouch 优先于 onTouchEvent 执行。如果在 onTouch 方法中通过返回 true 将事件消费掉，onTouchEvent 将不会再执行。

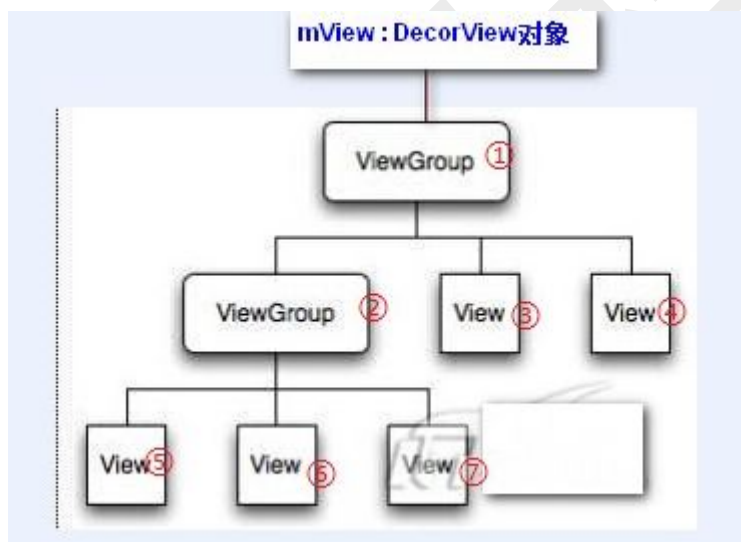
另外需要注意的是，onTouch 能够得到执行需要两个前提条件，第一 mOnTouchListener 的值不能为空，第二当前点击的控件必须是 enable 的。因此如果你有一个控件是非 enable 的，那么给它注册 onTouch 事件将永远得不到执行。对于这一类控件，如果我们想要监听它的 touch 事件，就必须通过在该控件中重写 onTouchEvent 方法来实现。

2.2 请描述一下 Android 的事件分发机制

Android 的事件分发机制主要是 Touch 事件分发，有两个主角:ViewGroup 和 View。Activity 的 Touch 事件事实上是调用它内部的 ViewGroup 的 Touch 事件，可以直接当成 ViewGroup 处理。

View 在 ViewGroup 内，ViewGroup 也可以在其他 ViewGroup 内，这时候把内部的 ViewGroup 当成 View 来分析。

先分析 ViewGroup 的处理流程：首先得有个结构模型概念：ViewGroup 和 View 组成了一棵树形结构，最顶层为 Activity 的 ViewGroup，下面有若干的 ViewGroup 节点，每个节点之下又有若干的 ViewGroup 节点或者 View 节点，依次类推。如图：



当一个 Touch 事件(触摸事件为例)到达根节点，即 Activity 的 ViewGroup 时，它会依次下发，下发的过程是调用子 View(ViewGroup)的 dispatchTouchEvent 方法实现的。简单来说，就是 ViewGroup 遍历它包含着的子 View，调用每个 View 的 dispatchTouchEvent 方法，而当子 View 为 ViewGroup 时，又会通过调用 ViewGroup 的 dispatchTouchEvent 方法继续调用其内部的 View 的 dispatchTouchEvent 方法。上述例子中的消息下发顺序是这样

的：①-②-⑤-⑥-⑦-③-④。dispatchTouchEvent 方法只负责事件的分发，它拥有 boolean 类型的返回值，当返回为 true 时，顺序下发会中断。在上述例子中如果⑤的 dispatchTouchEvent 返回结果为 true，那么⑥-⑦-③-④将都接收不到本次 Touch 事件。

1.Touch 事件分发中只有两个主角：ViewGroup 和 View。ViewGroup 包含 onInterceptTouchEvent、dispatchTouchEvent、onTouchEvent 三个相关事件。View 包含 dispatchTouchEvent、onTouchEvent 两个相关事件。其中 ViewGroup 又继承于 View。

2.ViewGroup 和 View 组成了一个树状结构，根节点为 Activity 内部包含的一个 ViewGroup。

3.触摸事件由 Action_Down、Action_Move、Action_Up 组成，其中一次完整的触摸事件中，Down 和 Up 都只有一个，Move 有若干个，可以为 0 个。

4.当 Activity 接收到 Touch 事件时，将遍历子 View 进行 Down 事件的分发。ViewGroup 的遍历可以看成是递归的。分发的目的是为了找到真正要处理本次完整触摸事件的 View，这个 View 会在 onTouchEvent 结果返回 true。

5.当某个子 View 返回 true 时，会中止 Down 事件的分发，同时在 ViewGroup 中记录该子 View。接下去的 Move 和 Up 事件将由该子 View 直接进行处理。由于子 View 是保存在 ViewGroup 中的，多层 ViewGroup 的节点结构时，上级 ViewGroup 保存的会是真实处理事件的 View 所在的 ViewGroup 对象：如 ViewGroup0-ViewGroup1-TextView 的结构中，TextView 返回了 true，它将被保存在 ViewGroup1 中，而 ViewGroup1 也会返回 true，被保存在 ViewGroup0 中。当 Move 和 UP 事件来时，会先从 ViewGroup0 传递至 ViewGroup1，再由 ViewGroup1 传递至 TextView。

6.当 ViewGroup 中所有子 View 都不捕获 Down 事件时，将触发 ViewGroup 自身的 onTouch 事件。触发的方式是调用 super.dispatchTouchEvent 函数，即父类 View 的 dispatchTouchEvent 方法。在所有子 View 都不处理的情况下，触发 Activity 的 onTouchEvent 方法。

7.onInterceptTouchEvent 有两个作用：1.拦截 Down 事件的分发。2.中止 Up 和 Move 事件向目标 View 传递，使得目标 View 所在的 ViewGroup 捕获 Up 和 Move 事件。

3、在 Android 中主线程如何给子线程发 Message？（2015-12-1）重要

这是一个很好玩的话题通常我们都是 Activity 中，让子线程执行耗时任务，执行完之后给主线程发送消息让主线程更新 UI。其实还有很多应用场景需要让主线程给子线程发送消息，该消息作为任务的载体，比如在 IntentService 中，主线程就给子线程发送了消息，让子线程干活。

给大家写个 Demo 演示主线程给子线程发送消息。

【文件 1-1】 MainActivity.java 代码片段

```
1. package com.yange.message2subthread;
2.
3. import android.os.Bundle;
4. import android.os.Handler;
5. import android.os.Looper;
6. import android.os.Message;
7. import android.util.Log;
8. import android.view.View;
9. import android.app.Activity;
10. /**
11.  *
12.  * @author wzy 2015-12-1
13.  *
14.  * 演示主线程给子线程发送 Message
15.  *
16.  */
17. public class MainActivity extends Activity {
18.     private Handler mMySubHandler;
19.
20.     @Override
21.     protected void onCreate(Bundle savedInstanceState) {
22.         super.onCreate(savedInstanceState);
23.         setContentView(R.layout.activity_main);
24.     }
25.     @Override
26.     protected void onStart() {
27.         super.onStart();
28.         /**
29.          * 当 Activity 启动的时候创建一个子线程, 并启动
30.          */
31.         MyThread thread = new MyThread();
32.         thread.start();
```

```
33. }
34. /**
35.  * 绑定布局中的点击按钮, 点击后给子线程发送消息
36.  *
37.  * @param view
38.  */
39. public void click(View view){
40.     Message msg = new Message();
41.     msg.obj = "MainActivity";
42.     mMySubHandler.sendMessage(msg);
43. }
44. /**
45.  * 绑定布局文件中的按钮 2, 点击后让子线程退出, 关闭子线程
46.  * 其实这里只需要让子线程的 Looper 对象退出即可, 因为 Looper.loop(); 是线程阻塞的.
47.  * @param view
48.  */
49. public void click2(View view){
50.     if (myLooper!=null) {
51.         myLooper.quit();
52.     }
53. }
54. /**
55.  * 将 Looper 对象声明为成员变量
56.  */
57. private Looper myLooper;
58. @Override
59. protected void onDestroy() {
60.     super.onDestroy();
61.     /**
62.      * 在退出的时候, 将子线程释放掉, 不然可能会导致内存泄露
63.      */
64.     if (myLooper!=null) {
65.         myLooper.quit();
66.     }
67. }
68. class MyThread extends Thread{
69.
70.     @Override
71.     public void run() {
72.         //1. 创建一个 Looper 对象 (内部创建了 MessageQueue, 并将 MessageQueue 作
73. 为 Looper 对象的成员, 然后将 Looper 对象绑定到 ThreadLocal 中)
74.         Looper.prepare();
75.         /**
76.          * 创建一个 Handler
```

```
77.         */
78.         mMySubHandler = new Handler() {
79.             @Override
80.             public void handleMessage(android.os.Message msg) {
81.                 //处理主线程发送的消息
82.                 Log.d("tag", "接收到信息"+msg);
83.             };
84.         };
85.         //2. 获取当前 Looper 对象
86.         myLooper = Looper.myLooper();
87.         //3. 让消息循环起来
88.         Looper.loop();
89.         Log.d("tag", "子线程退出");
90.     }
91. }
92.
93. }
94.
```

六、Android 签名

1、简单描述下 Android 数字签名

在 Android 系统中，所有安装到系统的应用程序都必有一个数字证书，此数字证书用于标识应用程序的作者和在应用程序之间建立信任关系。

Android 系统要求每一个安装进系统的应用程序都是经过数字证书签名的，数字证书的私钥则保存在程序开发者的手中。Android 将数字证书用来标识应用程序的作者和在应用程序之间建立信任关系，不是用来决定最终用户可以安装哪些应用程序。

这个数字证书并不需要权威的数字证书签名机构认证(CA)，它只是用来让应用程序包自我认证的。

同一个开发者的多个程序尽可能使用同一个数字证书，这可以带来以下好处。

(1)有利于程序升级，当新版程序和旧版程序的数字证书相同时，Android 系统才会认为这两个程序是同一个程序的不同版本。如果新版程序和旧版程序的数字证书不相同，则 Android 系统认为他们是不同的程序，并产生冲突，会

要求新程序更改包名。

(2)有利于程序的模块化设计和开发。Android 系统允许拥有同一个数字签名的程序运行在一个进程中，Android 程序会将他们视为同一个程序。所以开发者可以将自己的程序分模块开发，而用户只需要在需要的时候下载适当的模块。

在签名时，需要考虑数字证书的有效期：

- (1)数字证书的有效期要包含程序的预计生命周期，一旦数字证书失效，持有改数字证书的程序将不能正常升级。
- (2)如果多个程序使用同一个数字证书，则该数字证书的有效期要包含所有程序的预计生命周期。
- (3)Android Market 强制要求所有应用程序数字证书的有效期要持续到 2033 年 10 月 22 日以后。

Android 数字证书包含以下几个要点：

- (1)所有的应用程序都必须有数字证书，Android 系统不会安装一个没有数字证书的应用程序
- (2)Android 程序包使用的数字证书可以是自签名的，不需要一个权威的数字证书机构签名认证
- (3)如果要正式发布一个 Android ，必须使用一个合适的私钥生成的数字证书来给程序签名，而不能使用 adt 插件或者 ant 工具生成的调试证书来发布。
- (4)数字证书都是有有效期的，Android 只是在应用程序安装的时候才会检查证书的有效期。如果程序已经安装在系统中，即使证书过期也不会影响程序的正常功能。

2、使用 Eclipse 如何生成数字签名

可以通过 Eclipse 导出工程时为当前工程设置签名证书。File -> Export ->Export Android Application ->Create New keystore

七、Android 中的动画

1、Android 中的动画有哪几类，它们的特点和区别是什么

Android 中动画分为两种，一种是 Tween 动画、还有一种是 Frame 动画。

Tween 动画，这种实现方式可以使视图组件移动、放大、缩小以及产生透明度的变化；

Frame 动画，传统的动画方法，通过顺序的播放排列好的图片来实现，类似电影。

2、如何修改 Activity 进入和退出动画

可以通过两种方式，一是通过定义 Activity 的主题，二是通过覆写 Activity 的 `overridePendingTransition` 方法。

◆ 通过设置主题样式

在 `styles.xml` 中编辑如下代码：

```
<style name="AnimationActivity" parent="@android:style/Animation.Activity">
    <item name="android:activityOpenEnterAnimation">@anim/slide_in_left</item>
    <item name="android:activityOpenExitAnimation">@anim/slide_out_left</item>
    <item name="android:activityCloseEnterAnimation">@anim/slide_in_right</item>
    <item name="android:activityCloseExitAnimation">@anim/slide_out_right</item>
</style>
```

添加 `themes.xml` 文件：

```
<style name="ThemeActivity">
    <item name="android:windowAnimationStyle">@style/AnimationActivity</item>
    <item name="android:windowNoTitle">>true</item>
</style>
```

在 `AndroidManifest.xml` 中给指定的 Activity 指定 theme。

◆ 覆写 `overridePendingTransition` 方法

```
overridePendingTransition(R.anim.fade, R.anim.hold);
```

八、其他知识（集大众智慧）

1、AsyncTask 如何使用

AsyncTask 用于处理异步任务，该类是一个抽象的泛型类。类的签名如下：`public abstract class AsyncTask<Params, Progress, Result>`。

三种泛型类型分别代表“启动任务执行的输入参数”、“后台任务执行的进度”、“后台计算结果的类型”。在特定场合下，并不是所有类型都被使用，如果没有被使用，可以用 `java.lang.Void` 类型代替。

一个异步任务的执行一般包括以下几个步骤：

- 1.`execute(Params... params)`，执行一个异步任务，需要我们在代码中调用此方法，触发异步任务的执行。
- 2.`onPreExecute()`，在 `execute(Params... params)` 被调用后立即执行，一般用来在执行后台任务前对 UI 做一些标记。
- 3.`doInBackground(Params... params)`，在 `onPreExecute()` 完成后立即执行，用于执行较为费时的操作，此方法将接收输入参数和返回计算结果。在执行过程中可以调用 `publishProgress(Progress... values)` 来更新进度信息。
- 4.`onProgressUpdate(Progress... values)`，在调用 `publishProgress(Progress... values)` 时，此方法被执行，直接将进度信息更新到 UI 组件上。
- 5.`onPostExecute(Result result)`，当后台操作结束时，此方法将会被调用，计算结果将做为参数传递到此方法中，直接将结果显示到 UI 组件上。

在使用的时候，有几点需要格外注意：

- 1.异步任务的实例必须在 UI 线程中创建。
- 2.`execute(Params... params)` 方法必须在 UI 线程中调用。
- 3.不要手动调用 `onPreExecute()`，`doInBackground(Params... params)`，`onProgressUpdate(Progress... values)`，`onPostExecute(Result result)` 这几个方法。
- 4.不能在 `doInBackground(Params... params)` 中更改 UI 组件的信息。
- 5.一个任务实例只能执行一次，如果执行第二次将会抛出异常。

我写一个简单的例子来演示 AsyncTask 的用法。

MainActivity.java

```
public class MainActivity extends Activity {

    private TextView tv;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        tv = (TextView) findViewById(R.id.tv);
    }
    /**
     * 点击 button 绑定的事件
     */
    public void start(View view){
        AsyncTask<Integer,Integer,String> asyncTask = new MyAsyncTask();
        asyncTask.execute(100);
    }
    class MyAsyncTask extends AsyncTask<Integer, Integer, String>{

        /**
         * 该方法在子线程中运行，因此不能有任何修改 UI 操作
         */
        @Override
        protected String doInBackground(Integer... params) {
            for(int i=0;i<params[0];i++){
                try {
                    //模拟耗时操作
                    Thread.sleep(100);
                } catch (InterruptedException e) {
                    e.printStackTrace();
                }
                //发送进度
                publishProgress(i);
            }
            return "任务已经完成";
        }
        /**
         * 任务执行前在 UI 线程中调用
         */
    }
}
```

```
@Override
protected void onPreExecute() {
    Toast.makeText(MainActivity.this, "开始执行任务", 0).show();
    super.onPreExecute();
}
/**
 * 任务执行后在 UI 线程中调用<br>
 * @param result 正是 doInBackground 的返回值
 */
@Override
protected void onPostExecute(String result) {
    Toast.makeText(MainActivity.this, result, 0).show();
    super.onPostExecute(result);
}
/**
 * 在 UI 线程中执行
 * 当 doInBackground 执行 publishProgress 时调用该方法
 */
@Override
protected void onProgressUpdate(Integer... values) {
    super.onProgressUpdate(values);
    tv.setText("当前进度: "+values[0]);
}
}
}
```

activity_main.xml

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent" >

    <TextView
        android:id="@+id/tv"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_centerInParent="true"
        android:text="显示进度"
        android:textSize="30sp" />

    <Button
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentBottom="true"
        android:layout_margin="10dp"
        android:onClick="start"
        android:text="开始" />

</RelativeLayout>
```

运行效果图：

点击开始后，界面显示的数值不同的更改。



2、都使用过哪些框架、平台

2.1 EventBus（事件处理）

2.2 xUtils（网络、图片、ORM）

2.3 JPush（推送平台）

2.4 友盟（统计平台）

2.5 有米（优米）（广告平台）

2.6 百度地图

2.7 bmob（服务器平台、短信验证、邮箱验证、第三方支付）

2.8 阿里云 OSS（云存储）

2.9 ShareSDK（分享平台、第三方登录）

2.10 ImageLoader（图片加载器，支持多种缓存，避免 OOM）

3、Glide 原理（2015-11-29）

（Glide 是一个著名的网络访问框架，如果是面试初级程序员，可能只会问你会不会用，要是面试高级程序员可能会问 Glide 内部原理你懂不懂，会不会和懂不懂是不同层次的问题，希望大家从能会不会的层次提高到懂不懂原理的层次）

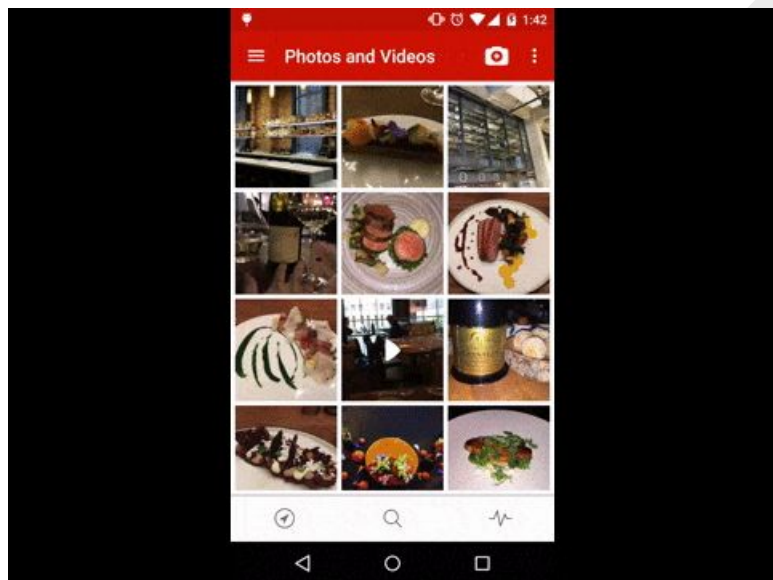
Glide 最简单的使用案例就是从远程服务器或者本地文件系统加载图片，把它们放在磁盘与内存缓存中，然后加载到 view 上。它可以用在全是图片的 app 中，Glide 为包含图片的滚动列表做了尽可能流畅的优化。

对象池

Glide 原理的核心是为 bitmap 维护一个对象池。对象池的主要目的是通过减少大对象的分配以重用来提高性能。

Dalvik 和 ART 虚拟机都没有使用 [compacting garbage collector](#) , compacting garbage collector 是一种模式，这种模式中 GC 会遍历堆，同时把活跃对象移到相邻内存区域，让更大的内存块可以用在后续的分配中。因为安卓没有这种模式，就可能会出现被分配的对象分散在各处，对象之间只有很小的内存可用。如果应用试图分配一个大于邻近的闲置内存块空间的对象，就会导致 OutOfMemoryError，然后崩溃，即使总的空余内存空间大于对象的大小。

使用对象池还可以帮助提高滚动的性能，因为重用 bitmap 意味着更少的对象被创建与回收。垃圾回收会导致“停止一切(Stop The World)”事件，这个事件指的是回收器执行期间，所有线程（包括 UI 线程）都会暂停。这个时候，图像帧无法被渲染同时 UI 可能会停滞，这在滚动期间尤其明显。



4、Android 四大著名图片处理框架 (2015-11-29)

	Imageloader	Picasso	Glide	Fresco
作者	nostra13	Square	Sam sjudd	Facebook
时间	2011/09	2013/02	2012/12	2015/03
Star/Issues Con/PR	10k/1k 35/120	7k/800 58/350	5k/600 22/61	6k/600 17/50
最新版本	1.9.5	2.5.3	4.0	0.7

Universal ImageLoader 是很早开源的图片缓存，在早期被很多应用使用。

Picasso 是 Square 开源的项目，且他的主导者是 JakeWharton，所以广为人知。

Glide 是 Google 员工的开源项目，被一些 Google App 使用，在去年的 Google I/O 上被推荐，不过目前国内资料不多。

Fresco 是 Facebook 在今年上半年开源的图片缓存，主要特点包括：

- (1) 两个内存缓存加上 Native 缓存构成了三级缓存
- (2) 支持流式，可以类似网页上模糊渐进式显示图片
- (3) 对多帧动画图片支持更好，如 Gif、WebP

鉴于 Fresco 还没发布正式的 1.0 版本，同时一直没太多时间熟悉 Fresco 源码，后面对比不包括 Fresco，以后有时间再加入对比。

4.1 基本概念

在正式对比前，先了解几个图片缓存通用的概念：

- (1) RequestManager：请求生成和管理模块

(2) Engine：引擎部分，负责创建任务(获取数据)，并调度执行

(3) GetDataInterface：数据获取接口，负责从各个数据源获取数据。

比如 MemoryCache 从内存缓存获取数据、DiskCache 从本地缓存获取数据，下载器从网络获取数据等。

(4) Displayer：资源(图片)显示器，用于显示或操作资源。

比如 ImageView，这几个图片缓存都不仅仅支持 ImageView，同时支持其他 View 以及虚拟的 Displayer 概念。

(5) Processor 资源(图片)处理器

负责处理资源，比如旋转、压缩、截取等。

以上概念的称呼在不同图片缓存中可能不同，比如 Displayer 在 ImageLoader 中叫做 ImageAware，在 Picasso 和 Glide 中叫做 Target。

4.2 共同优点

(1) 使用简单

都可以通过一句代码可实现图片获取和显示。

(2) 可配置度高，自适应程度高

图片缓存的下载器(重试机制)、解码器、显示器、处理器、内存缓存、本地缓存、线程池、缓存算法等大都可轻松配置。

自适应程度高，根据系统性能初始化缓存配置、系统信息变更后动态调整策略。

比如根据 CPU 核数确定最大并发数，根据可用内存确定内存缓存大小，网络状态变化时调整最大并发数等。

(3) 多级缓存

都至少有两级缓存、提高图片加载速度。

(4) 支持多种数据源

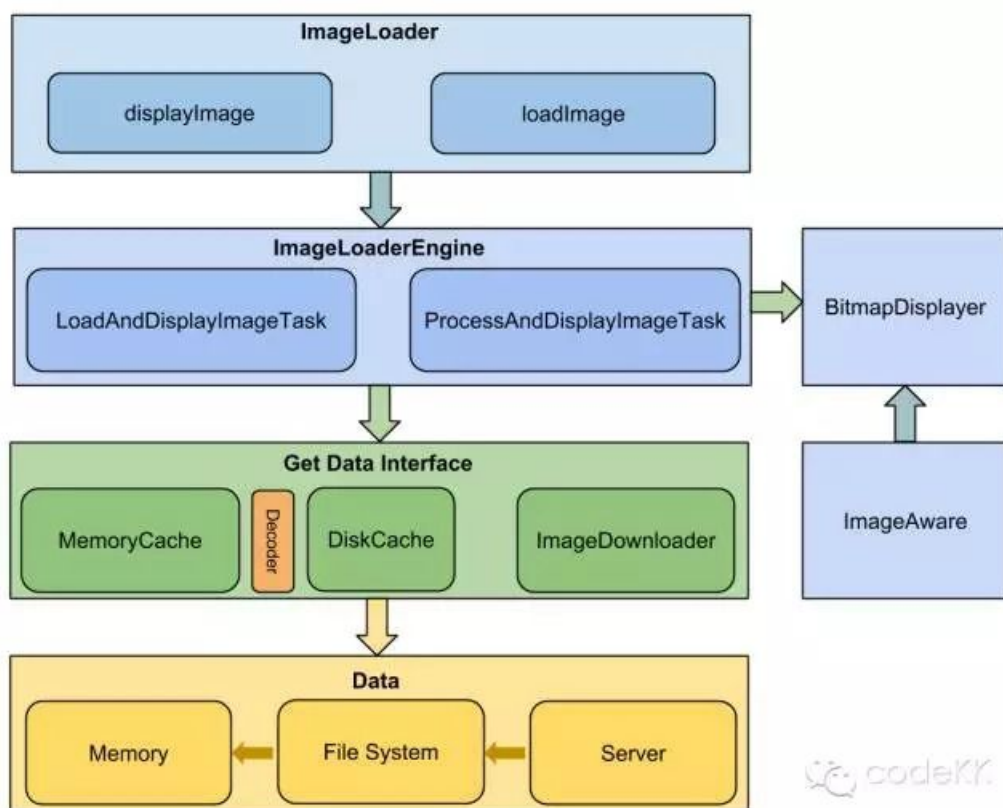
支持多种数据源，网络、本地、资源、Assets 等

(5) 支持多种 Displayer

不仅仅支持 ImageView，同时支持其他 View 以及虚拟的 Displayer 概念。

其他小的共同点包括支持动画、支持 transform 处理、获取 EXIF 信息等。

4.3 ImageLoader 设计及优点



4.3.1 总体设计及流程

上面是 ImageLoader 的总体设计图。整个库分为 ImageLoaderEngine，Cache 及 ImageDownloader，ImageDecoder，BitmapDisplayer，BitmapProcessor 五大模块，其中 Cache 分为 MemoryCache 和 DiskCache 两部分。

简单的讲就是 ImageLoader 收到加载及显示图片的任务，并将它交给 ImageLoaderEngine，ImageLoaderEngine 分发任务到具体线程池去执行，任务通过 Cache 及 ImageDownloader 获取图片，中间可能经过 BitmapProcessor 和 ImageDecoder 处理，最终转换为 Bitmap 交给 BitmapDisplay 在 ImageAware 中显示。

4.3.2 ImageLoader 优点

(1) 支持下载进度监听

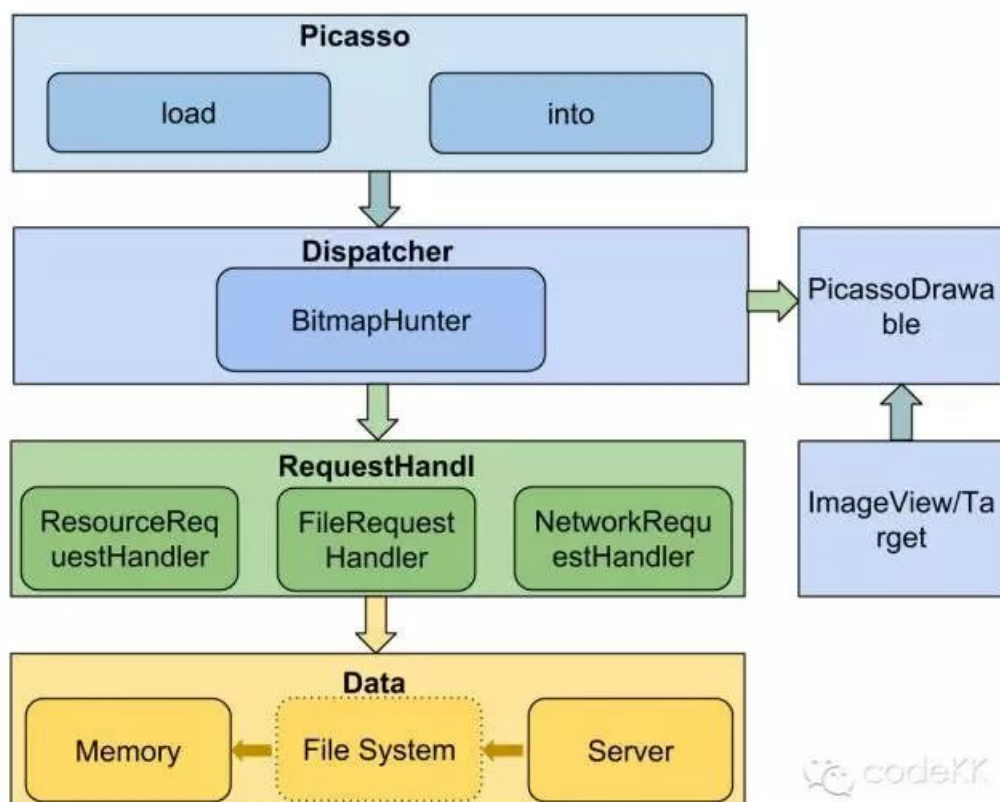
(2) 可以在 View 滚动中暂停图片加载

通过 PauseOnScrollListener 接口可以在 View 滚动中暂停图片加载。

(3) **默认实现多种内存缓存算法** 这几个图片缓存都可以配置缓存算法，不过 ImageLoader 默认实现了较多缓存算法，如 Size 最大先删除、使用最少先删除、最近最少使用、先进先删除、时间最长先删除等。

(4) 支持本地缓存文件名规则定义

4.4、Picasso 设计及优点



4.4.1 总体设计及流程

上面是 Picasso 的总体设计图。整个库分为 Dispatcher，RequestHandler 及 Downloader，PicassoDrawable 等模块。

Dispatcher 负责分发和处理 Action，包括提交、暂停、继续、取消、网络状态变化、重试等等。

简单的讲就是 Picasso 收到加载及显示图片的任务，创建 Request 并将它交给 Dispatcher，Dispatcher 分发任务到具体 RequestHandler，任务通过 MemoryCache 及 Handler(数据获取接口) 获取图片，图片获取成功后通过 PicassoDrawable 显示到 Target 中。

需要注意的是上面 Data 的 File system 部分，Picasso 没有自定义本地缓存的接口，默认使用 http 的本地缓存，API 9 以上使用 okhttp，以下使用 Urlconnection，所以如果需要自定义本地缓存就需要重定义 Downloader。

4.4.2 Picasso 优点

(1) 自带统计监控功能

支持图片缓存使用的监控，包括缓存命中率、已使用内存大小、节省的流量等。

(2) 支持优先级处理

每次任务调度前会选择优先级高的任务，比如 App 页面中 Banner 的优先级高于 Icon 时就很适用。

(3) 支持延迟到图片尺寸计算完成加载

(4) 支持飞行模式、并发线程数根据网络类型而变

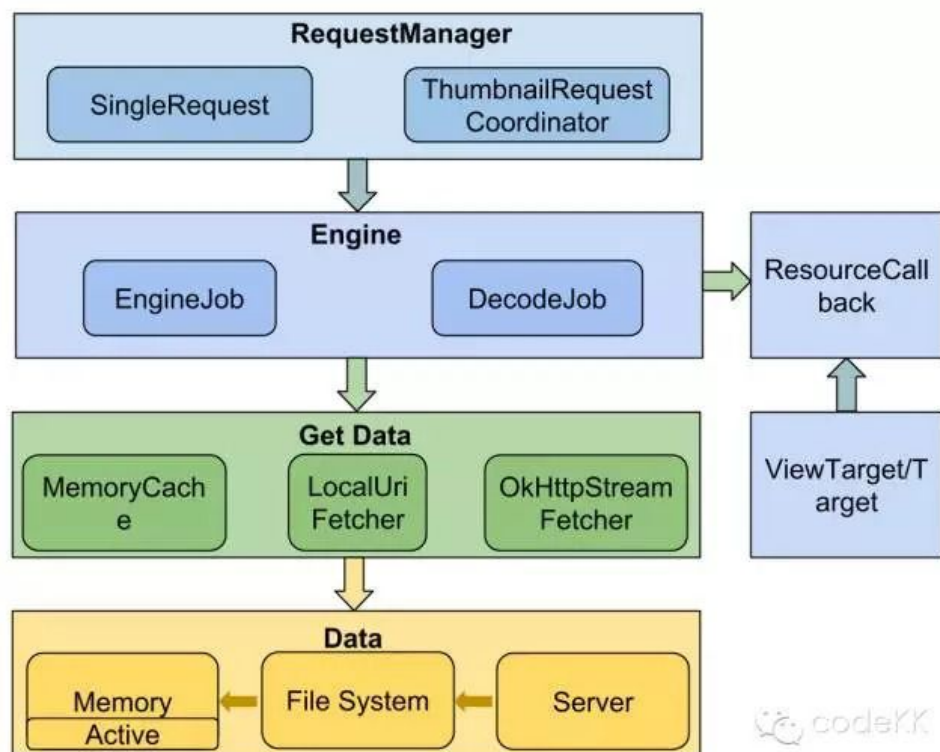
手机切换到飞行模式或网络类型变换时会自动调整线程池最大并发数，比如 wifi 最大并发为 4，4g 为 3，3g 为 2。

这里 Picasso 根据网络类型来决定最大并发数，而不是 CPU 核数。

(5) “无”本地缓存

无”本地缓存，不是说没有本地缓存，而是 Picasso 自己没有实现，交给了 Square 的另外一个网络库 okhttp 去实现，这样的好处是可以通过请求 Response Header 中的 Cache-Control 及 Expired 控制图片的过期时间。

4.5 Glide 设计及优点



4.5.1 总体设计及流程

上面是 Glide 的总体设计图。整个库分为 `RequestManager`(请求管理器)、`Engine`(数据获取引擎)、`Fetcher`(数据获取器)、`MemoryCache`(内存缓存)、`DiskLRUCache`、`Transformation`(图片处理)、`Encoder`(本地缓存存储)、`Registry`(图片类型及解析器配置)、`Target`(目标) 等模块。

简单的讲就是 Glide 收到加载及显示资源的任务，创建 `Request` 并将它交给 `RequestManager`，`Request` 启动 `Engine` 去数据源获取资源(通过 `Fetcher`)，获取到后 `Transformation` 处理后交给 `Target`。

Glide 依赖于 `DiskLRUCache`、`GifDecoder` 等开源库去完成本地缓存和 Gif 图片解码工作。

4.5.2 Glide 优点

(1) 图片缓存->媒体缓存

Glide 不仅是一个图片缓存，它支持 Gif、WebP、缩略图。甚至是 Video，所以更该当做一个媒体缓存。

(2) 支持优先级处理

(3) 与 Activity/Fragment 生命周期一致，支持 trimMemory

Glide 对每个 context 都保持一个 RequestManager，通过 FragmentTransaction 保持与 Activity/Fragment 生命周期一致，并且有对应的 trimMemory 接口实现可供调用。

(4) 支持 okhttp、Volley

Glide 默认通过 UrlConnection 获取数据，可以配合 okhttp 或是 Volley 使用。实际 ImageLoader、Picasso 也都支持 okhttp、Volley。

(5) 内存友好

① Glide 的内存缓存有个 active 的设计

从内存缓存中取数据时，不像一般的实现用 get，而是用 remove，再将这个缓存数据放到一个 value 为软引用的 activeResources map 中，并计数引用数，在图片加载完成后进行判断，如果引用计数为空则回收掉。

② 内存缓存更小图片

Glide 以 url、view_width、view_height、屏幕的分辨率等做为联合 key，将处理后的图片缓存在内存缓存中，而不是原始图片以节省大小

③ 与 Activity/Fragment 生命周期一致，支持 trimMemory

④ 图片默认使用默认 RGB_565 而不是 ARGB_888，虽然清晰度差些，但图片更小，也可配置到 ARGB_888。

其他：Glide 可以通过 signature 或不使用本地缓存支持 url 过期

4.5.3 Glide 的使用

Glide 使用起来很简单，而且不需要任何特别的配置就自动包含了 bitmap pooling。

```
1DrawableRequestBuilder requestBuilder = Glide.with(context).load(imageUrl);  
2requestBuilder.into(imageView);
```

这就是加载一张图片的全部要求。就像安卓中的很多地方一样，with() 方法中的 context 到底是哪种类型是不清楚的。有一点很重要需要记住，就是传入的 context 类型影响到 Glide 加载图片的优化程度，Glide 可以监视 activity 的生命周期，在 activity 销毁的时候自动取消等待中的请求。但是如果你使用 Application context，你就失去了这种优化效果。

译者注：其实以上的代码是一种比较规范的写法，我们更熟悉的写法是：

```
Glide.with(context)  
1    .load("http://inthecheesefactory.com/uploads/source/glidedepicasso/cover.j  
2    pg")  
3    .into(ivImg);
```

4.5.4 优化特性

类似的是，如果相关的 item 已经滚出了屏幕的范围，Glide 会自动取消列表中的悬着的图片请求。因为绝大多数开发者都会在 adapter 中利用 view 的回收，Glide 做到这点是通过在 ImageView 上设置一个 tag，在加载另外一张图片之前检查这个 tag，如果存在就取消第一次请求。

Glide 提供了几个让你感觉图片加载速度变快的特性。第一个就是在图片显示在屏幕上之前就预先取出图片。它提供了一个 ListPreloader 类，它被应该事先取出的 item 数目实例化。然后通过 setOnScrollListener(OnScrollListener) 被传递给 ListView。你希望在 ListView 之外也能预先取出图片吗？没问题，使用前面的 builder 对象就可以了，只需调用 builder.downloadOnly()。

4.6 汇总

	Imageloader	Picasso	Glide
作者	nostra13	Square	Sam sjudd
时间	2011/09	2013/02	2012/12
Star/Issues Con/PR	10k/1k 35/120	7k/800 58/350	5k/600 22/61
最新版本	1.9.5	2.5.3	4.0
功能	★★★★☆	★★★★☆	★★★★★
代码理解	★★★★☆	★★★★★	★★★★☆

三者总体上来说，ImageLoader 的功能以及代理容易理解长度都一般。

Picasso 代码虽然只在一个包下，没有严格的包区分，但代码简单、逻辑清晰，一两个小时就能叫深入的了解完。

Glide 功能强大，但代码量大、流转复杂。在较深掌握的情况下才推荐使用，免得出了问题难以下手解决。

5、都使用过哪些自定义控件

3.1 pull2RefreshListView

3.2 LazyViewPager

3.3 SlidingMenu

3.4 SmoothProgressBar

3.5 自定义组合控件

3.6 ToggleButton

3.7 自定义吐司

.....

6、Android 程序员进阶必备网站 (2015-11-29)

要想成为一名高级程序员，仅仅学会黑马的课程还是远远不够的。还需要我们平常的不断学习、总结、积累。推荐大家几个学习 Android 的网站。

4.1 黑马程序员官网 （编程，始于黑马）

<http://www.itheima.com/>


黑马程序员
www.itheima.com

为莘莘学子改变命运而讲课，为千万学生少走弯路而著书！

[新手指南](#)
[首页](#)
[课程介绍](#)
[视频教程](#)
[在线公开课](#)
[如何报名](#)
[黑马论坛](#)

[> 认识黑马，从这里开始！](#)
[> 我该选择学习哪门课程？](#)
[> 没有基础适不适合学编程？](#)
[> 我要实地考察，怎么走？](#)
[> 校区分布图，我该选择哪一所？](#)
[> 新生报到注意事项？](#)
[> 一部电影，看4个月的学习生活！](#)



Android 6.0 重磅推出

黑马程序员Android 6.0课程新品发布会
与2015年11月Android开发者大会同步

[▶ 抢先观看](#)

[我还有其他问题](#)
 一张贴看完
黑马所有微电影
  一张贴看完
黑马所有薪资
  一句话说出
你对黑马的印象
  登陆报名系统
开启黑马入学流程

4.2 CSDN （中国第一程序员社区）

<http://www.csdn.net/>




让移动连接你我

CSDN 移动客户端上线

[App store 下载](#)
[Android 市场下载](#)

[业界](#)
[云计算](#)
[移动](#)
[研发](#)
[学院](#)
[论坛](#)
[博客](#)
[下载](#)
[问答](#)
[ITeye](#)
[商城](#)
[CODE](#)
[活动](#)
[CTO](#)
[年费会员](#)
[外包](#)

[学院](#)
[线下培训](#)
[Java](#)
[iOS](#)
[C/C++](#)
[游戏开发](#)
[PHP](#)
[Azure](#)
[H3C](#)
[华为云计算](#)
[英特尔软件](#)
[IBM大学](#)
[异构开发](#)
[AWS](#)
[Qualcomm](#)
[腾讯云](#)
[容联](#)
[高校](#)



芯片巨头为何痴恋开源软件？ 英特尔Imad Sousou来解密

在10月底举办的OpenStack东京峰会期间，记者见到了首次面对中国媒体的英特尔软件与服务事业部副总裁、英特尔开源技术中心总经理Imad Sousou。通过他解密了英特尔的开源战略。

临阵磨枪，血拼季网站优化的最后三板斧

京东王晓雨：Apache Kylin在云海的实践

Java 8 vs. Scala (二)：Stream vs. Collection

ECMAScript标准制定过程展示及ES7新特性披露

CSDN学院移动客户端，打造指尖上的IT课堂

深度剖析C++对象池自动回收技术实现

SDCC2015前端专场札记



豆瓣音乐应用实战

HTML 5无处不在

快速入口 定制你的个性CSDN内容

推荐频道： BDTC 2015 SDCC 2015 微信开发学习路线 Rust路线图 CSDN学院 极客头条 Swift 智能硬件

推荐服务： C币兑换 博客专栏 精品资源

推荐专区： COSOS开源OS社区 触控科技 长跑节

4.3 51CTO（技术成就梦想）

<http://www.51cto.com/>

51CTO.com 技术成就梦想

社区 ▾ 学院 WOT MDSA 高招

输入您要搜索的内容

新闻 云计算 移动 开发 系统 网络 安全 数据库 服务器 虚拟化 博客 下载 论坛 读书 CIOAge

混合云社区 专题汇总 运维挑战 OpenStack五周年 开源PaaS方案 Python视频教程 下一代防火墙 Java20周年 安全报告 iOS实战开发 微

如何利用AngularJS打造一款简单Web应用 2/5

精选专题 | IBM Bluemix注册试用 | Linux运维高薪就业指导绝密资料 | Swift时代下iOS攻城狮学习路线图

最受管理员欢迎的顶级开源工具大盘点 | **血拼双十一 莫要购来了商品丢失了信息**

今日重点

- 原创 | WOT2015谢佳标：基于R语言的大数据处理及建模技术
- 原创 | WOT2015李大学专访：互联网+浪潮下的颠覆与重构
- 原创 | WOT2015章天锋：用大数据把夜晚还给分拣员
- 原创 | WOT2015 大数据技术峰会靠什么勾引技术宅
- 原创 | WOT2015 刘志军：互联网和大数据倒逼消费金融发展
- 原创 | WOT2015刘明浩：互联网金融安全形势不容乐观
- 原创 | WOT2015罗奇斌：互联网和基因行业的融合是未来大趋势
- 原创 | 心理大师于际敬：技术人群心理特征及压力管理

4.4 ITEye (CSDN 旗下网站)

<http://www.iteye.com/>

ITEYE 100offer 互联网人才拍卖 **告别盲目投简历** 让海量好机会主动来找你

首页 | 资讯 | 精华 | 论坛 | 问答 | 博客 | 专栏 | 群组 | 招聘 | 搜索

高手问答：Java多线程编程实战指南 (设计模式篇)

活动奖品：《Java多线程编程实战指南(设计模式篇)》

活动时间：11月23日~30日

高手问答：Java多线程编程实战指南 1 2 3

热点聚焦

十大算法，让你轻松进阶高手

快速排序算法、归并排序、二分查找算法、BFPRT(线性查找算法)、DFS(深度优先搜索)、BFS(广度优先搜索)、Dijkstra算法、动态规划算法、朴素贝叶斯分 [\[详情\]](#)

人人都该懂点儿TCP

即使你的工作也许不需要对TCP了如指掌，也不需要去了解具体的TCP/IP实例。你也应该懂一些基本的TCP知识，本文会告诉你为什么。 [\[详情\]](#)

致我们终将组件化的 Web

按模块划分“目录结构，把当前模块下的所有逻辑和资源都放一起了，这对于多人独自开发和维护个人模块不是很好吗？当然了，那争论的结果是我乖乖地改掉 [\[详情\]](#)

透明手机 **汽油价格查询**

推荐精华文章

- [编程语言 IDE 对比](#)
- [最全的静态网站生成器\(开源项目\)](#)
- [Java NIO 系列教程](#)
- [Java并发教程 \(Oracle官方资料\)](#)
- [编程精华资源 \(ITeye优秀专栏\)](#)

4.5 eoeAndroid

<http://www.eoeandroid.com/portal.php>



eoe Android 社区
让移动开发更简单，做最棒的开发者社区

用QQ帐号登录 只需一步，快速开始

帐号 用户名/Email 自动登录 找回密码
密码 登录 免费加入

首页 社区 新手入门 实例教程 资源下载 游戏开发 **Android源码** 招聘 快速导航

请输入搜索内容 帖子 热搜：二维码 定时器 手电筒 滑块 斗地主 书架 定位 买手机 聊天 游戏开发 游戏 股票查询

最新消息 完整安卓 Android开发视频教程共10季，迅雷第1季 - 《老罗Android开发视频教程》-android入门介绍-第1集：android学习路线图介绍

longchamp小羊皮 DuaAV pkaG 27hn gif显示的问题
3E世界游戏平台专业的游戏运营平台
签到不了了怎么办？
android studio安装在mac上，能否与手机进
android studio 如何打开eclipse创建的项目
如何让应用窗口全屏显示
Android 之 json数据的解析 (jsonReader)
AppCan 2015年度TOP10开发者团队评选

eoe推荐 8月7日，最新 TOP 10—Android精品源码
Android精品源码【免费下载】，包含日历效果实现、Fit Chart环形计数器。
PullLoadMoreRecyclerView
android 面试技巧，本人面试多家公司经验获
Android Studio 权威教程
eoe专家答疑索引（汇总）帖，一帖看尽精华
安卓面试题大集【内含javaweb，J2ee，数
小团队队长承接苏州，上海android项目
接android项目，demo，UI，html5壳子
BoreDream模板代码-社交应用-新浪微博
迪恩客服端（医疗项目源码）

赞助商
云·端·创变
2015 AppCan移动云大会
11月27日 中国·北京
20W+年薪，30+公司，寻Android大
【推荐】极客校园大使等你来！

4.6 Android 巴士

<http://www.apkbus.com/>



安卓巴士 您好，欢迎来到安卓巴士！ 登录 注册 | 切换到宽版

APKBUS HUAWEI | 华为云服务

首页 技术博客 开发者服务 开源代码 视频教程 线下活动 应聘招聘 开发工具

官方微博 巴友QQ群 精华专题 官方微信 认证开发

主题 回复 收藏 评论 动态 最新热门 精华区 我的安卓巴士

《Android开发工程师》微专业 够极客，真潮流 限时团购价，学费省到HIGH Android开发工程师微专业

想成为产品经理？ 网易五个亿级产品负责人亲授 限时团购价，学费省到high！

Android shape属性详细整理 有时候，为了满足一些需

【Android高级】Android系统以及Activity启动一、Android系统启动[/backc

史上最简单的短信接口接入方法 这是我见过的一个简单的短信

Android软件开发之盘点自定义View界面大合 自定义View界面大合集 今天我用自己写的一

Android Support Annotations 使用详解 在Android Support Library1

给 Android 开发者的 RxJava 详解 RxJava 到底是什么一个词：异

4.7 开源中国（有 apk 可下载）

<http://www.oschina.net/>

首页 开源项目 问答 代码 博客 翻译 资讯 专题 城市圈 当前访客身份: 游客 [登录] [加入开源中国]

open source china 开源中国社区 oschina.net 众包 悬赏 作品 整包 我的 码云 Git Team PaaS Sonar 招聘 职位 简历 招聘 在 39395 个开源项目中搜索 搜索

综合资讯 —— 源创会年终盛典 更多» 软件更新资讯 更多»

12月12日 北京 OSC 源创会 —— 国际会议中心
2015年源创会巡回的终点 - 北京，一场献给社区开发者的年度盛典正在酝酿中..... 源创会是开源中国坚持了...

- 众包翻译分享 —— 《Apache Mesos 官方文档》 11/29
- Fetchr —— web 应用通用数据访问层 11/29
- Linux 基金会说，没有 Linux 就没有 3D 电影 11/29
- OSChina 周日乱弹 —— 人生何必复杂 11/29
- Git 项目推荐 | 基于 Servlet 3 的 JFinal 改造版本 11/29
- 博客 | AFNetworking+NgInx+HTTPS 服务器通信 11/29
- Win 10 用户现在可以从桌面查看微软硬件产品 11/29
- VPN 协议漏洞暴露用户真实 IP 11/29
- OSChina 周六乱弹 —— 你敢把电脑借给父母用吗？ 11/28
- JTune : LinkedIn 的 Java CMS 高精度优化 11/28

Seafile 5.0.0 发布，开源文件云存储
Seafile 5.0.0 发布，该版本三大主要更新：Change most icons from png to icon font Most of the serv...

- mzhphp v2.0，一日开数站 PHP 极速框架 11/29
- Monkey 1.6.6 发布，Web 服务器 11/29
- CoreOS 877.1.0 Alpha 发布，服务器操作系统 11/29
- Birdfont 2.14 发布，字体编辑器 11/29
- GIMP 2.9.2 发布，开源图片处理软件 11/29
- TACTIC 4.4.0.v2 发布下载，生产资产管理系统 11/29
- Elive 2.6.12 (Beta) 发布 11/29
- FreeType 2.6.2 发布，字体工具 11/29
- Akka 2.4.1 发布，Actor 开发模型库 11/29
- Sequelize v3.14.2 发布，Node.js 的 ORM 11/29

七牛融合CDN全面降价，降幅达42%
今天你动弹了吗？热门头条大战！
今天你动弹了吗？ 动弹

小十郎：学了两天；发现打轮都不会；突然发现我的智商有点问题了
1分钟前 (0评) 0

javacc：周日第一次登陆os
12分钟前 (0评) 0

OSC大胖森：说实话，你有多久没吃过麻麻做的饭了？
17分钟前 (1评) iPhone 0

4.8 apk 源码（安卓源码服务专家，有上百个 apk 源码）

<http://www.javaapk.com/>

首页 VIP源码 例子大全11.23更新 基友群 运行不了？ 例如:2048 搜索

今天 2015年11月23日 星期一
您还没有登录，[登录] [注册] [找回密码]

最新发布

Android项目源码仿微信登录注册聊天换肤二维码扫描
JavaApk 2015年11月23日 0 3次 交友

本项目是一个基于安卓的Android仿微信客户端-猫友。是一个csdn上的朋友的原创项目，原帖可以看看这里<http://blog.csdn.net/ericfantastic/article/details/49451249> 实现了微信的登录注册、主界面、聊天会话、通讯录、发现界面、个人设置、添加好友.....

Android项目源码使用百度翻译接口的在线翻译
JavaApk 2015年11月22日 0 2次 翻译

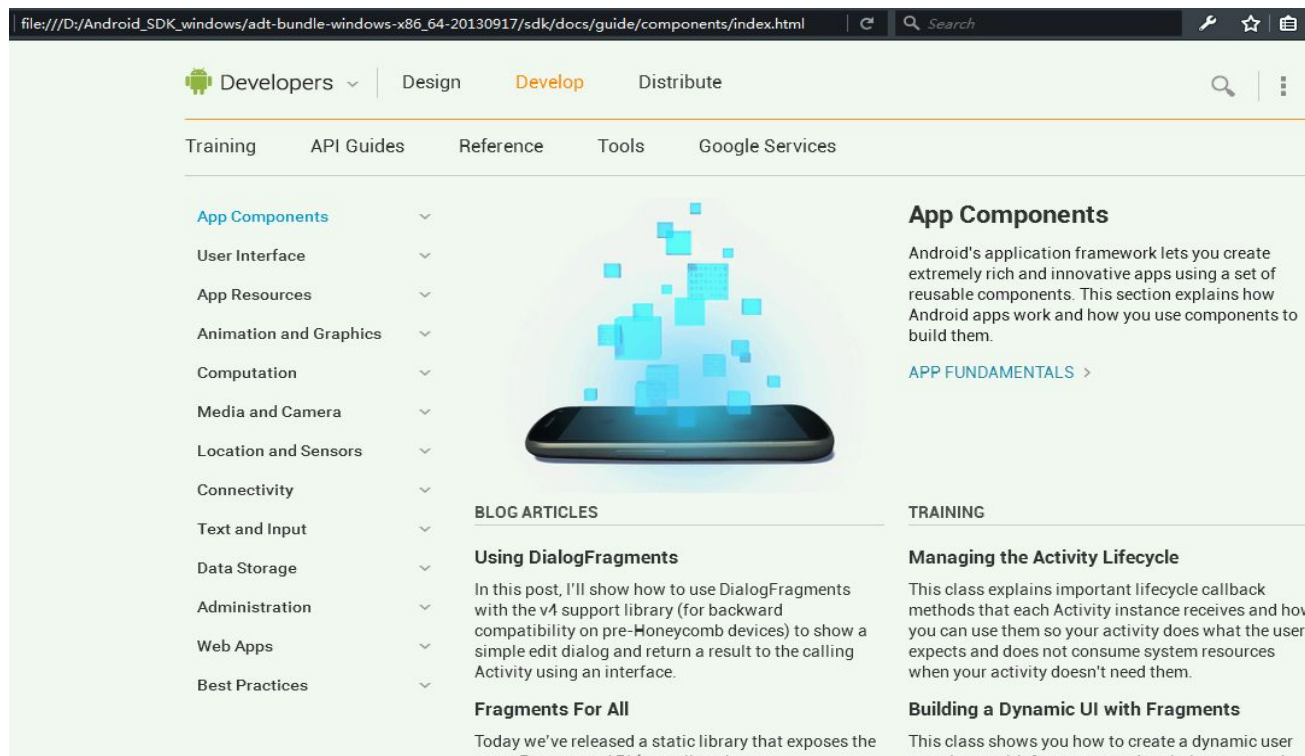
本项目是一个基于安卓的简单在线翻译app应用项目源码，试用了百度翻译的接口，通过直接带入字符串的方式进行翻译，可以翻译单个文字也可以翻译一段话，www.javaapk.com之前也介绍过很多关于翻译的项目源码，需要的可以在www.javaapk.com搜索“翻译”就可以找到下载。.....

例子源码

aidl	arduino	Bmob	EditText
GIF显示	GPS	IOS风格	JSON
ListView	MD5	NFC	ocr
SD卡	Socket	sqlite	TextView
UDP	ViewPager	VPN	webservice
wifi	WIN8风格	xml操作	Zip操作
上传下载	下拉刷新	九宫格锁屏	书籍翻页
产品介绍	仪表盘	传感器	侧滑菜单
其他	内存管理	列表控件	动态布局
动画特效	可穿戴	各种菜单	后台服务
图片剪裁	图片滤镜	图片轮播	地图定位
天气查询	字体使用	字母索引	屏幕截图
应用管理	开发框架	异步图片	引导页面

4.9 Android 官方文档

在 sdk 目录下的 docs 目录中。最好的文档，没有之一。很遗憾的就是无法访问其网站，只能离线查看。



4.10 泡在网上的日子

<http://www.jcodecraeer.com/>



7、volley 的原理 (2015-11-29)

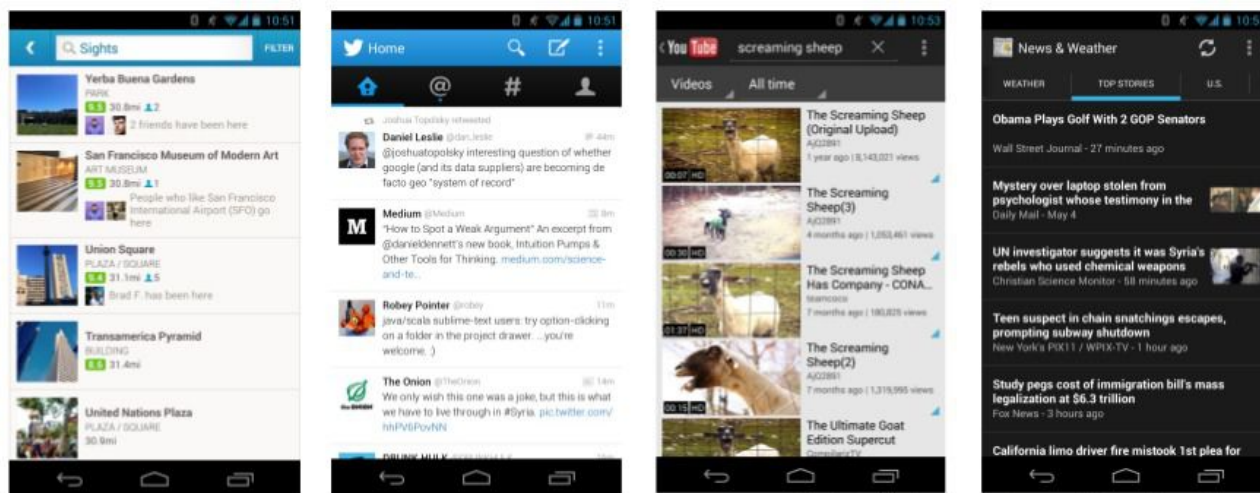
7.1 Volley 简介

我们平时在开发 Android 应用的时候不可避免地都需要用到网络技术，而多数情况下应用程序都会使用 HTTP 协议来发送和接收网络数据。Android 系统中主要提供了两种方式进行 HTTP 通信，HttpURLConnection 和 HttpClient，几乎在任何项目的代码中我们都能看到这两个类的身影，使用率非常高。

不过 HttpURLConnection 和 HttpClient 的用法还是稍微有些复杂的，如果不进行适当封装的话，很容易就会写出不少重复代码。于是乎，一些 Android 网络通信框架也就应运而生，比如说 AsyncHttpClient，它把 HTTP 所有的通信细节全部封装在了内部，我们只需要简单调用几行代码就可以完成通信操作了。再比如 Universal-Image-Loader，它使得在界面上显示网络图片的操作变得极度简单，开发者不用关心如何从网络上获取图片，也不用关心开启线程、回收图片资源等细节，Universal-Image-Loader 已经把一切都做好了。

Android 开发团队也是意识到了有必要将 HTTP 的通信操作再进行简单化，于是在2013年 Google I/O 大会上推出了一个新的网络通信框架——Volley。Volley 可是说是把 AsyncHttpClient 和 Universal-Image-Loader 的优点集于了一身，既可以像 AsyncHttpClient 一样非常简单地 HTTP 通信，也可以像 Universal-Image-Loader 一样轻松加载网络上的图片。除了简单易用之外，Volley 在性能方面也进行了大幅度的调整，它的设计目标就是非常适合去进行数据量不大，但通信频繁的网络操作，而对于大数据量的网络操作，比如说下载文件等，Volley 的表现就会非常糟糕。

下图所示的这些应用都是属于数据量不大，但网络通信频繁的，因此非常适合使用 Volley。



http://blog.csdn.net/guolin_blog

7.2 下载 Volley

介绍了这么多理论的东西，下面我们就准备开始进行实战了，首先需要将 Volley 的 jar 包准备好，如果你的电脑上装有 Git，可以使用如下命令下载 Volley 的源码：

下载完成后将它导入到你的 Eclipse 工程里，然后再导出一个 jar 包就可以了。如果你的电脑上没有 Git，那么也可以直接使用我导出好的 jar 包，下载地址是：http://www.kwstu.com/ResourcesView/kwstu_201441183330928。

新建一个 Android 项目，将 volley.jar 文件复制到 libs 目录下，这样准备工作就算是做好了。

7.3 StringRequest 的用法

前面已经说过，Volley 的用法非常简单，那么我们就从最基本的 HTTP 通信开始学习吧，即发起一条 HTTP 请求，然后接收 HTTP 响应。首先需要获取到一个 RequestQueue 对象，可以调用如下方法获取到：

```
RequestQueue mQueue = Volley.newRequestQueue(context);
```

注意这里拿到的 RequestQueue 是一个请求队列对象，它可以缓存所有的 HTTP 请求，然后按照一定的算法并发地发出这些请求。RequestQueue 内部的设计就是非常合适高并发的，因此我们不必为每一次 HTTP 请求都创建一个 RequestQueue 对象，这是非常浪费资源的，基本上在每一个需要和网络交互的 Activity 中创建一个 RequestQueue 对象就足够了。

接下来为了要发出一条 HTTP 请求，我们还需要创建一个 `StringRequest` 对象，如下所示：

```
StringRequest stringRequest = new StringRequest("http://www.baidu.com",  
  
        new Response.Listener<String>() {  
  
            @Override  
  
            public void onResponse(String response) {  
  
                Log.d("TAG", response);  
  
            }  
  
        }, new Response.ErrorListener() {  
  
            @Override  
  
            public void onErrorResponse(VolleyError error) {  
  
                Log.e("TAG", error.getMessage(), error);  
  
            }  
  
        });
```

可以看到，这里 `new` 出了一个 `StringRequest` 对象，`StringRequest` 的构造函数需要传入三个参数，第一个参数就是目标服务器的 URL 地址，第二个参数是服务器响应成功的回调，第三个参数是服务器响应失败的回调。其中，目标服务器地址我们填写的是百度的首页，然后在响应成功的回调里打印 出服务器返回的内容，在响应失败的回调里打印出失败的详细信息。

最后，将这个 `StringRequest` 对象添加到 `RequestQueue` 里面就可以了，如下所示：

```
mQueue.add(stringRequest);
```

另外，由于 `Volley` 是要访问网络的，因此不要忘记在你的 `AndroidManifest.xml` 中添加如下权限：

```
<uses-permission android:name="android.permission.INTERNET" />
```

好了，就是这么简单，如果你现在运行一下程序，并发出这样一条 HTTP 请求，就会看到 `LogCat` 中会打印出如

下图所示的数据。

```

Text
<!DOCTYPE html PUBLIC "-//WAPFORUM//DTD XHTML Mobile 1.0//EN" "http://www.wapforum.org/DTD/xhtml-mobile10.dtd">

<html xmlns="http://www.w3.org/1999/xhtml"><!--STATUS OK--><head><meta http-equiv="Content-Type" content="text/
r: #262626;}form {position: relative;margin: 12px 15px 91px;height: 41px;}img {border: 0}.wordWrap{margin-right:
{background-color: #F5F5F5;border: 1px solid #787878;font-size: 16px;font-weight: bold;height: 41px;letter-spac:
0}.lg {margin-top: 30px}a {text-decoration: none;color: #46446d;}.a {margin-top: 20px}.d {margin: 58px 0 8px;}.l
ding-left: 18px;border-left: 1px solid #dadada;}</style><title>百度一下,你就知道</title><script>function form_subm
"http://m.baidu.com/static/index/u.png" alt="百度首页"/></div><form action="/from=844b/s" method="get" onsubmit="
nput type="hidden" value="0" name="ts"/><input type="hidden" name="ms" value="1"/></div><input type="submit" va:
p;news?idx=30000&itj=311">文库</a><a href="http://m.baidu.com/img?idx=30000&ssid=0&from=844b&bd_
4%2Cta%40utouch_2_4.1__&itj=34&device=mobile">知道</a><a href="http://m.baidu.com/ssid=0/from=844b/bd_pa
40utouch_2_4.1__&itj=31">百科</a></div><div class="a"><a href="http://m.baidu.com/ssid=0/from=844b/bd_page_t
321_1004%2Cta%40utouch_2_4.1__&idx=30000&itj=35&wtj=wi">地图</a><a href="http://tieba.baidu.com/?idx
type=1&uid=0&pu=sz%401321_1004%2Cta%40utouch_2_4.1__&idx=30000&itj=39">hao123</a><a href="http://
lass="d"><div class="b"><a href="http://duokoo.baidu.com/novel/?fr=home&ssid=0&from=844b&bd_page_ty
http://blog.csdn.net/guolin_blog

```

没错，百度返回给我们的就是这样一长串的 **HTML** 代码，虽然我们看起来会有些吃力，但是浏览器却可以轻松地对这段 **HTML** 代码进行解析，然后将百度的首页展现出来。

这样的话，一个最基本的 **HTTP** 发送与响应的功能就完成了。你会发现根本还没写几行代码就轻易实现了这个功能，主要就是进行了以下三步操作：

1. 创建一个 `RequestQueue` 对象。
2. 创建一个 `StringRequest` 对象。
3. 将 `StringRequest` 对象添加到 `RequestQueue` 里面。

不过大家都知道，**HTTP** 的请求类型通常有两种，**GET** 和 **POST**，刚才我们使用的明显是一个 **GET** 请求，那么如果想要发出一条 **POST** 请求应该怎么做 呢？**StringRequest** 中还提供了另外一种四个参数的构造函数，其中第一个参数就是指定请求类型的，我们可以使用如下方式进行指定：

```
StringRequest stringRequest = new StringRequest(Method.POST, url, listener, errorListener);
```

可是这只是指定了 HTTP 请求方式是 POST，那么我们要提交给服务器的参数又该怎么设置呢？很遗憾，

`StringRequest` 中并没有提供设置 `POST` 参数的方法，但是当发出 `POST` 请求的时候，`Volley` 会尝试调用 `StringRequest` 的父类——`Request` 中的 `getParams()` 方法来获取 `POST` 参数，那么解决方法自然也就有了，我们只需要在 `StringRequest` 的匿名类中重写 `getParams()` 方法，在这里设置 `POST` 参数就可以了，代码如下所示：

```
StringRequest stringRequest = new StringRequest(Method.POST, url, listener, errorListener) {
```

```
@Override
```

```
protected Map<String, String> getParams() throws AuthFailureError {
```

```
    Map<String, String> map = new HashMap<String, String>();
```

```
    map.put("params1", "value1");
```

```
    map.put("params2", "value2");
```

```
    return map;
```

```
}
```

```
};
```

你可能会说，每次都这样用起来岂不是很累？连个设置 **POST** 参数的方法都没有。但是不要忘记，**Volley** 是开源的，只要你愿意，你可以自由地在里面添加和修改任何的方法，轻松就能定制出一个属于你自己的 **Volley** 版本。

7.4. JsonRequest 的用法

学完了最基本的 **StringRequest** 的用法，我们再来进阶学习一下 **JsonRequest** 的用法。类似于 **StringRequest**，**JsonRequest** 也是继承自 **Request** 类的，不过由于 **JsonRequest** 是一个抽象类，因此我们无法直接创建它的实例，那么只能从它的子类入手了。**JsonRequest** 有两个直接子类，**JsonObjectRequest** 和 **JSONArrayRequest**，从名字上你应该能就看出它们的区别了吧？一个是用于请求一段 **JSON** 数据的，一个是用于请求一段 **JSON** 数组的。

至于它们的用法也基本上没有什么特殊之处，先 **new** 出一个 **JsonObjectRequest** 对象，如下所示：

```
JsonObjectRequest jsonObjectRequest = new JsonObjectRequest("http://m.weather.com.cn/data/
```

```
101010100.html", null, new Response.Listener<JSONObject>() {
```

```
    @Override
```

```
    public void onResponse(JSONObject response) {
```

```
        Log.d("TAG", response.toString());
```

```
    }
```

```
    }, new Response.ErrorListener() {  
  
        @Override  
  
        public void onErrorResponse(VolleyError error) {  
  
            Log.e("TAG", error.getMessage(), error);  
  
        }  
  
    });
```

可以看到，这里我们填写的 URL 地址是 <http://m.weather.com.cn/data/101010100.html>，这是中国天气网提供的一个查询天气信息的接口，响应的数据就是以 JSON 格式返回的，然后我们在 `onResponse()` 方法中将返回的数据打印出来。

最后再将这个 `JsonObjectRequest` 对象添加到 `RequestQueue` 里就可以了，如下所示：

```
mQueue.add(jsonObjectRequest);
```

这样当 HTTP 通信完成之后，服务器响应的天气信息就会回调到 `onResponse()` 方法中，并打印出来。现在运行一下程序，发出这样一条 HTTP 请求，就会看到 LogCat 中会打印出如下图所示的数据。

由此可以看出，服务器返回给我们的数据确实是 JSON 格式的，并且 `onResponse()` 方法中携带的参数也正是一个 `JSONObject` 对象，之后只需要从 `JSONObject` 对象取出我们想要得到的那部分数据就可以了。

你应该发现了吧，`JsonObjectRequest` 的用法和 `StringRequest` 的用法基本上是完全一样的，Volley 的易用之处也在这里体现出来了，会了一种就可以让你举一反三，因此关于 `JsonArrayRequest` 的用法相信已经不需要我再去讲解了吧。

8、okhttp 的原理 (2015-11-29)

主要是通过 Dispatcher 不断从 `RequestQueue` 中取出请求 (Call)，根据是否已缓存调用 Cache 或 Network 这两类数据获取接口之一，从内存缓存或是服务器取得请求的数据。

该引擎有同步和异步请求，同步请求通过 `Call.execute()` 直接返回当前的 `Response`，而异步请求会把当前的请

求 Call.enqueue 添加 (AsyncCall) 到请求队列中，并通过回调 (Callback) 的方式来获取最后结果。

参考博客：<http://www.jcodecraeer.com/a/anzhuokaifa/androidkaifa/2015/0326/2643.html>

9、ViewPagerindicator 的原理 (2015-11-29)

该项目总体设计非常简单，一个 pageIndicator 接口类，具体样式的导航类实现该接口，然后根据具体样式去实现相应的逻辑。IcsLinearLayout：LinearLayout 的扩展，支持了 4.0 以上的 divider 特性。

CirclePageIndicator、LinePageIndicator、UnderlinePageIndicator、TitlePagerIndicator 继承自 View。
TabPageIndicator、IconPageIndicator 继承自 HorizontalScrollView。

CirclePageIndicator、LinePageIndicator、UnderlinePageIndicator 继承自 View 的原因是它们样式相对简单，继承 View，自己去定制一套测量和绘制逻辑更简单，而且免去了 Measure 部分繁琐的步骤，效率更高。

TitlePagerIndicator 相对复杂，Android 系统提供的控件中没有类似的，而且实现底部线条的精准控制也复杂，所以只能继承自 View，实现绘制逻辑，达到理想的 UI 效果。

TabPageIndicator、IconPageIndicator 继承自 HorizontalScrollView 是由于它们各自的 ChildView 较多，而且具有相似性些，继承自 LinearLayout，通过 for 循环一个个 add 上去更简单，而且 HorizontalScrollView 具有水平滑动的功能，当 tab 比较多时，可以左右滑动。

参考博客：<http://blog.csdn.net/ljx19900116/article/details/43482083>

10、slidingmenu 的原理 (2015-11-29)

SlidingMenu 设计思路 三个主要的 ViewGroup，主 ViewGroup 里面包含两个重叠的 ViewGroup，盖在上面的就

是显示内容,而下面的就是菜单上面的侧滑以后露出后面的 View

主 ViewGroup 作为一个自定义控件,里面的内容和菜单利用自定义属性设置 SlidingMenu(继承 RelativeLayout),就是这个主 ViewGroup,其中又包含两个,内容 CustomViewAbove 和 菜单 CustomViewBehind(都继承 ViewGroup),SlidingMenu 提供两个自定义属性,供使用者传入两个布局 id,对应主体内容和菜单布局,此外还有其他一些属性供开发者设置菜单效果。

参考博客：<http://www.tuicool.com/articles/eErqY3>

6. Android 项目 (★★★★)

1、如何让 LinearLayout 自动换行如下图的颜色分类所示。

颜色分类:

米黄色

粉红色

浅灰色

黑色

浅蓝色

尺码:

均码

数量:

-

1

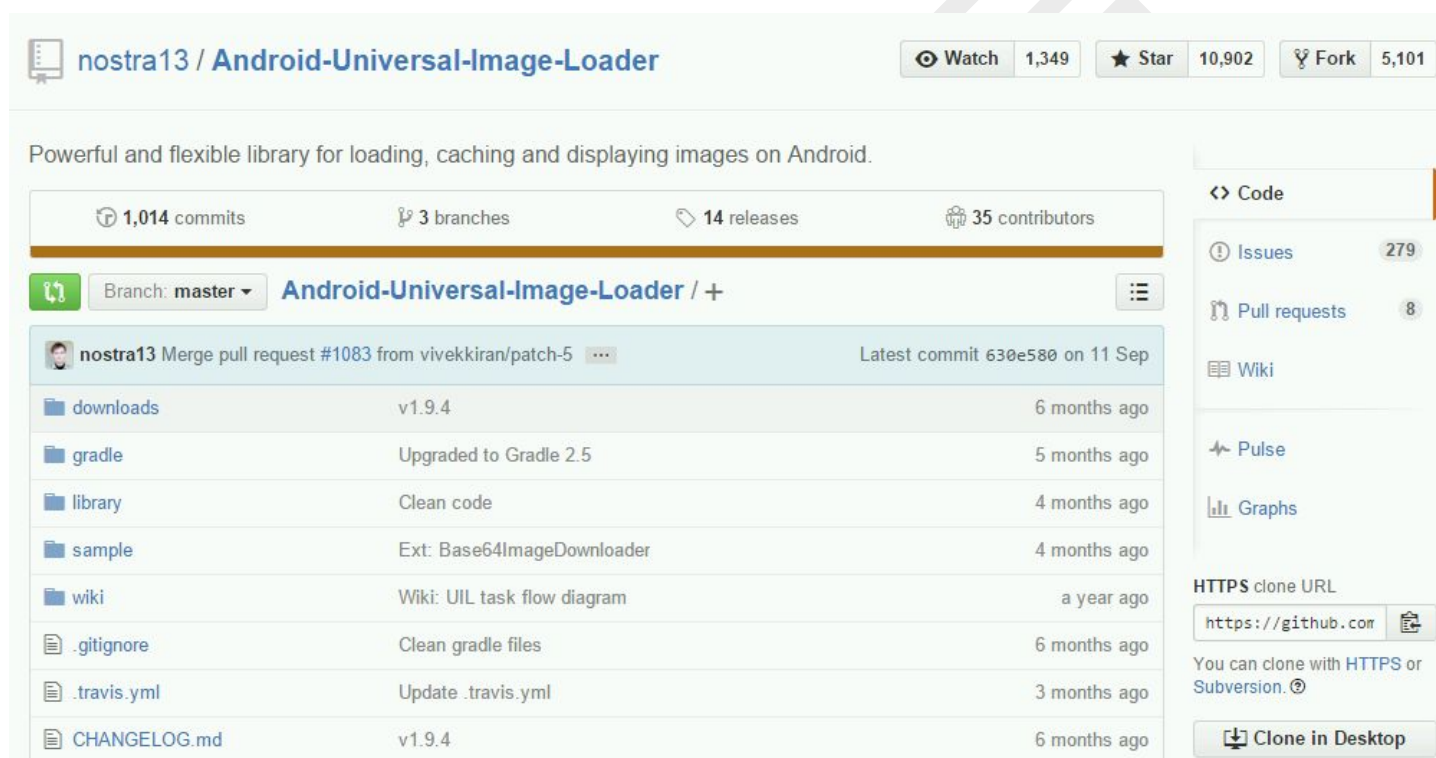
+

<http://yanmingming.sinaapp.com/?p=1205#more-1205>

2、ImageLoader 在项目中的使用

几乎所有的 App 都有图片的展示，这些图片或来自网络或来自本地存储，在使用图片的时候，我们不仅要考虑避免内存的 OOM，又要考虑加载的效率。ImageLoader 是一个比较好用的开源框架，代码托管在 github 上，地址如下：

下：<https://github.com/nostra13/Android-Universal-Image-Loader>



nostra13 / **Android-Universal-Image-Loader**

Powerful and flexible library for loading, caching and displaying images on Android.

1,014 commits 3 branches 14 releases 35 contributors

Branch: master **Android-Universal-Image-Loader** / +

nostra13 Merge pull request #1083 from vivekkiran/patch-5 Latest commit 630e580 on 11 Sep

File	Commit	Time
downloads	v1.9.4	6 months ago
gradle	Upgraded to Gradle 2.5	5 months ago
library	Clean code	4 months ago
sample	Ext: Base64ImageDownloader	4 months ago
wiki	Wiki: UIL task flow diagram	a year ago
.gitignore	Clean gradle files	6 months ago
.travis.yml	Update .travis.yml	3 months ago
CHANGELOG.md	v1.9.4	6 months ago

Code

Issues 279

Pull requests 8

Wiki

Pulse

Graphs

HTTPS clone URL

<https://github.com>

You can clone with HTTPS or Subversion.

Clone in Desktop

Android 开源框架 Universal-Image-Loader 完全解析 (一) --- 基本介绍及使用

<http://blog.csdn.net/xiaanming/article/details/26810303>

Android 开源框架 Universal-Image-Loader 完全解析 (二) --- 图片缓存策略详解

<http://blog.csdn.net/xiaanming/article/details/27525741>

Android 开源框架 Universal-Image-Loader 完全解析 (三) --- 源代码解读

<http://blog.csdn.net/xiaanming/article/details/39057201>

3、Java 和 JavaScript 互相调用（webview 和 js 的互相调用）

目前越来越多的 Android App 都采用了混合编程的架构，也就是 Android 原生控件+html5，当然 html 页面也必须运行在 webview 控件中，我们可以简单吧 webview 看成内置到 app 中的小型浏览器，这里面最核心的问题就是 html 中的 js 代码如何调用 java 代码，比如 js 调用 java 代码，然后打开一个新的 Activity 界面。比如淘宝已经大量采用了 h5 来实现其客户端功能，商品列表是 Android 端控件，点击商品列表进入某个商品详情可能就是 h5 界面，这里就存在 java 给 js 传递数据的问题，点击 h5 的商品详情界面的立即购买，可能就又从 h5 跳转到了 Activity 界面，这里又牵扯到 js 给 java 传递数值的问题。

看了下面的两篇博客，然后在写一个 demo 加深印象之后，相信大家会对 js 和 java 直接的纠缠搞的更明白。

android webview js 交互 第一节（java 和 js 交互）

<http://blog.csdn.net/wangtingshuai/article/details/8631835>

android webview js 交互，响应 webview 中的图片点击事件

<http://blog.csdn.net/wangtingshuai/article/details/8635787>

4、PopupWindow 弹出层在项目中的使用



在绝大多数 app 中都用到了 popupWindow，因此掌握 popupWindow 是 Android 程序员必备技能之一。

上图中 popupWindow 的实现博客如下：

<http://www.linuxidc.com/Linux/2012-05/61519.htm>

<http://www.linuxidc.com/Linux/2012-05/61519p2.htm>

<http://www.linuxidc.com/Linux/2012-05/61519p3.htm>

5、Notification 在 Android 中的使用

几乎所有的 app 都会使用状态的通知，如下图所示。



如果你现在还不知道 Notification 如何使用，那么请看完下面这篇博客，看完之后再建议你写一个自己的 demo，这样到公司工作的时候代码也可以直接拿过来使用。

【Android】状态栏通知 Notification、NotificationManager 详解

<http://blog.csdn.net/feng88724/article/details/6259071>

6、带索引的 ListView 在 Android 中的应用



如上图所示，在很多应用中都有的效果。上图实现如下博客地址：

<http://blog.csdn.net/xiaanming/article/details/12684155>

7、随手势滑动而消失 Activity 的使用

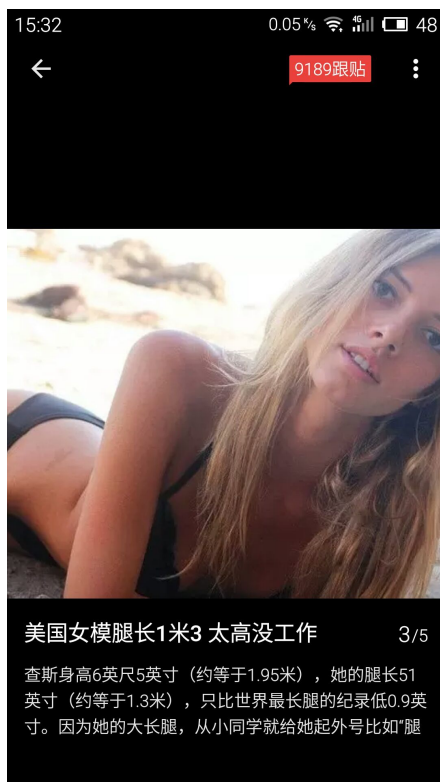
在部分 app 中，Activity 的销毁可以随着手势的滑动而销毁，给人一种很炫酷的感觉，其实现原理如下博客所示：

<http://blog.csdn.net/xiaanming/article/details/20934541>

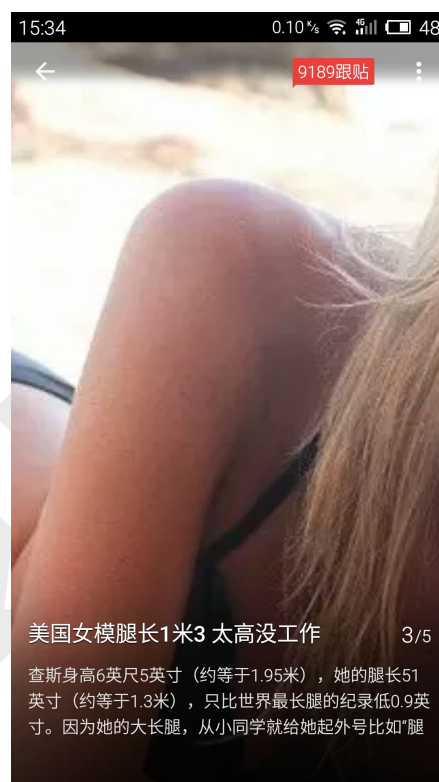


向右滑动消失。

8、TouchGallery 在 Android 中的应用



用两个手向外滑动后的效果



在很多应用中都使用了可以手势识别的画廊，比如网易新闻以及其他所有的新闻客户端。这里使用到最核心的代码托管在 github 上。地址如下：

<https://github.com/Dreddik/AndroidTouchGallery>

9、TextView 显示富文本

部分 app 使用 TextView 显示出来很绚丽的效果，这是利用了 TextView 对 Html 的支持做到的。只不过 TextView 支持的 Html 是有限的。

下面博客实现了如下功能：

实现 TextView 里的文字有不同颜色

TextView 显示 html 文件中的图片

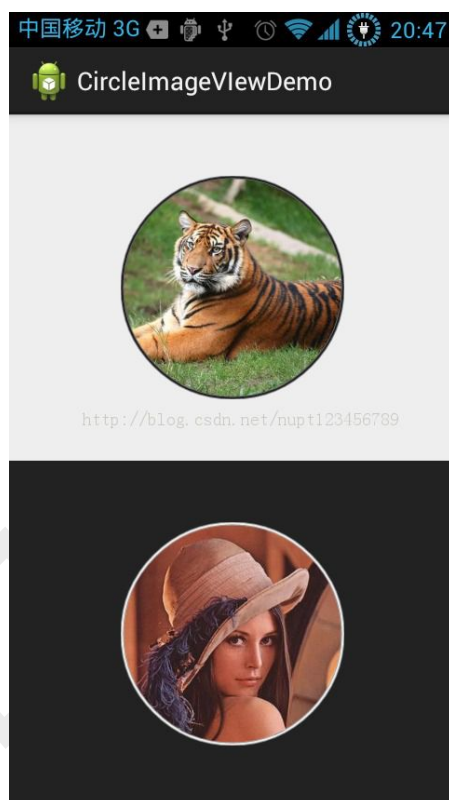
Android---文字中插入表情

android 短信字符转表情显示过程

Android TextView 支持的 HTML 标签

<http://www.cnblogs.com/playing/archive/2011/03/17/1987033.html>

10、CircleImageView 实现圆形图片



如上图所示，很多 app 在用户登录后都会使用圆形图片展示用户的头像。其实现原理如下博客所示：

<http://blog.csdn.net/nupt123456789/article/details/24886451>

11、网易新闻客户端频道管理的实现 (2015-11-25)



如上图所示，很多新闻资讯类客户端都用这种功能。其实现思路：

实现思路：

1. 获取数据库中频道的列表，如果为空，赋予默认列表，并存入数据库，之后通过对应的适配器赋给对应的 GridView
2. 2 个 GridView-- (1.DragGrid 2. OtherGridView)

DragGrid 用于显示我的频道，带有长按拖拽效果

OtherGridView 用于显示更多频道，不带推拽效果

注：由于屏幕大小不一定，外层使用 ScrollView，所以 2 者都要重写计算高度

3. 点击 2 个 GridView 的时候，根据点击的 Item 对应的 position，获取 position 对应的 view，进行创建一层移动

的动画层

起始位置 : 点击的 `position` `getLocationInWindow()` 获取。终点位置 : 另一个 `GridView` 的最后一个 `ITEM` 的 `position + 1` 的位置。

并赋予移动动画，等动画结束后对 2 者对应的频道列表进行数据的 `remove` 和 `add` 操作。

4. 设置点击和拖动的限制条件，如 推荐 这个 `ITEM` 是不允许用户操作的。

5. 拖动的 `DragGrid` 的操作：

(1) 长按获取长按的 `ITEM` 的 `position` -- `dragPosition` 以及对应的 `view`，手指触摸屏幕的时候，调用 `onInterceptTouchEvent` 来获取 `MotionEvent.ACTION_DOWN` 事件，获取对应的数据。由于这里是继承了 `GridView`，所以长按时间可以通过 `setOnItemLongClickListener` 监听来执行，或则你也可以通过计算点击时间来监听是否长按。

(2) 通过 `onTouchEvent(MotionEvent ev)` 来监听手指的移动和抬起动作。当它移动到 其它的 `item` 下面，并且下方的 `item` 对应的 `position` 不等于 `dragPosition`，进行数据交换，并且 2 者之间的所有 `item` 进行移动动画，动画结束后，数据更替刷新界面。

(3) 抬起手后，清除掉拖动时候创建的 `view`，让 `GridView` 中的数据显示。

6. 退出时候，将改变后的频道列表存入数据库。

其实现原理如下：

<http://www.javaapk.com/source/4529.html>

本人网盘下载地址：<http://pan.baidu.com/s/1dD3Bo6t>

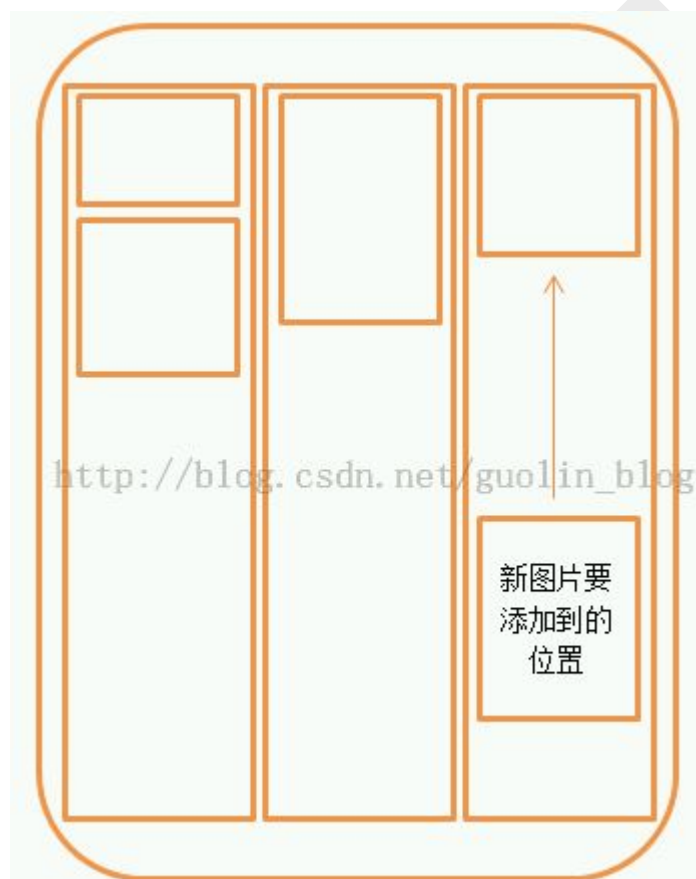
12、Android 瀑布流的实现

如下图所示，不规则也是一种美。下面的瀑布流在很多 `Android` 应用中都有使用。

传统界面的布局方式总是行列分明、坐落有序的，这种布局已是司空见惯，在不知不觉中大家都已经对它产生了审美疲劳。这个时候瀑布流布局的出现，就给人带来了耳目一新的感觉，这种布局虽然看上去貌似毫无规律，但是却有一种说不上来的美感，以至于涌现出了大批的网站和应用纷纷使用这种新颖的布局来设计界面。

实现原理：

瀑布流的布局方式虽然看起来好像排列的很随意，其实它是有很科学的排列规则的。整个界面会根据屏幕的宽度划分成等宽的若干列，由于手机的屏幕不是很大，这里我们就分成三列。每当需要添加一张图片时，会将这张图片的宽度压缩成和列一样宽，再按照同样的压缩比例对图片的高度进行压缩，然后在这三列中找出当前高度最小的一列，将图片添加到这一列中。之后每当需要添加一张新图片时，都去重复上面的操作，就会形成瀑布流格局的照片墙，示意图如下所示。



听我这么说完后，你可能会觉得瀑布流的布局非常简单嘛，只需要使用三个 `LinearLayout` 平分整个屏幕宽度，然后动态地 `addView()` 进去就好了。确实如此，如果只是为了实现功能的话，就是这么简单。可是别忘了，我们是在手

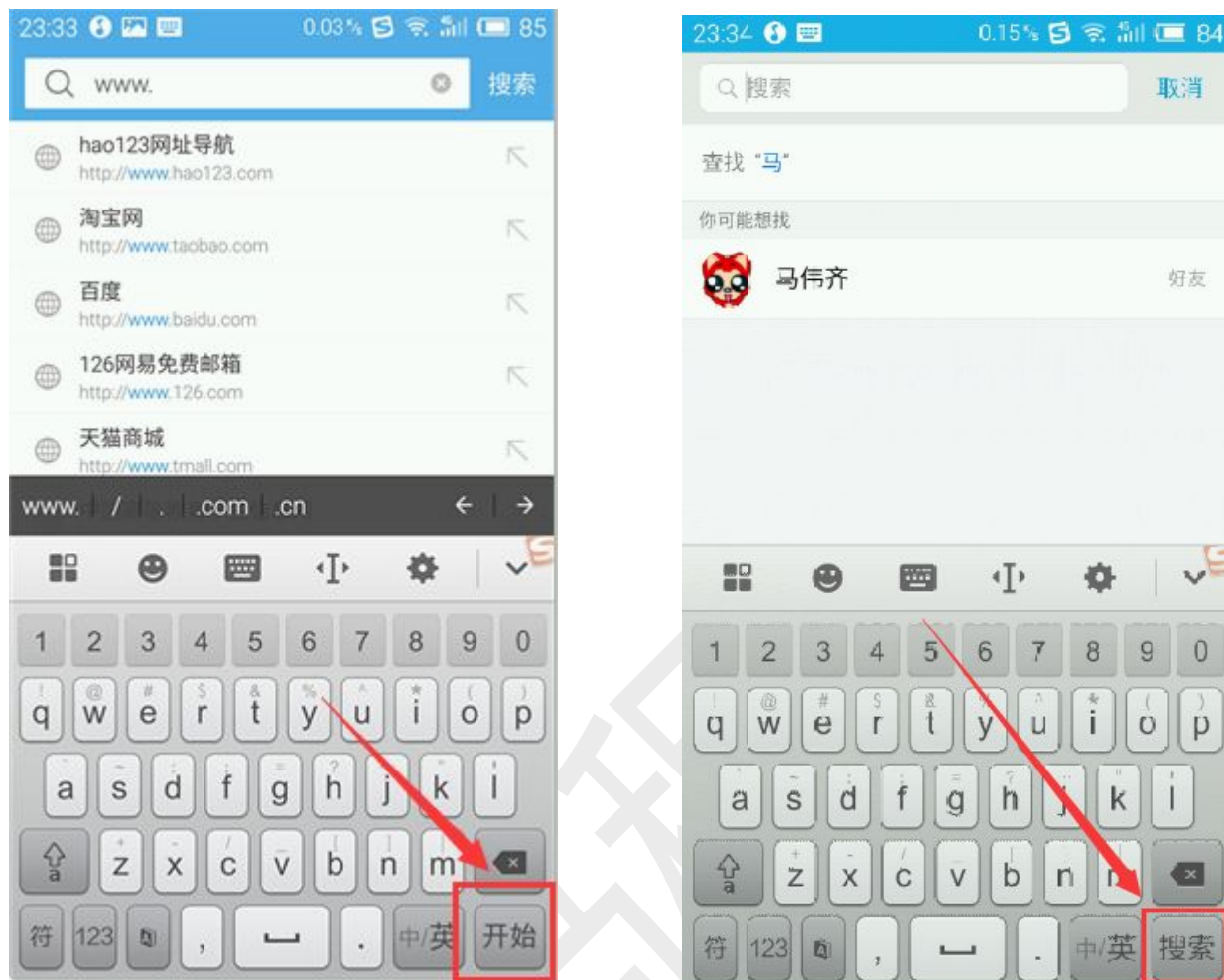
机上进行开发，如果不停地往 LinearLayout 里添加图片，程序很快就会 OOM。因此我们还需要一个合理的方案来对图片资源进行释放，这里仍然是准备使用 LruCache 算法。



其实现博客如下所示：

http://blog.csdn.net/guolin_blog/article/details/10470797

13、监听键盘事件 (2015-11-29)



如上图所示一个是 UC 浏览器，一个是 QQ 的界面，如何动态修改输入法键盘红色框中（右下角）的文字，以及如何监听右下角按钮的点击事件？

上面其实是两个问题：

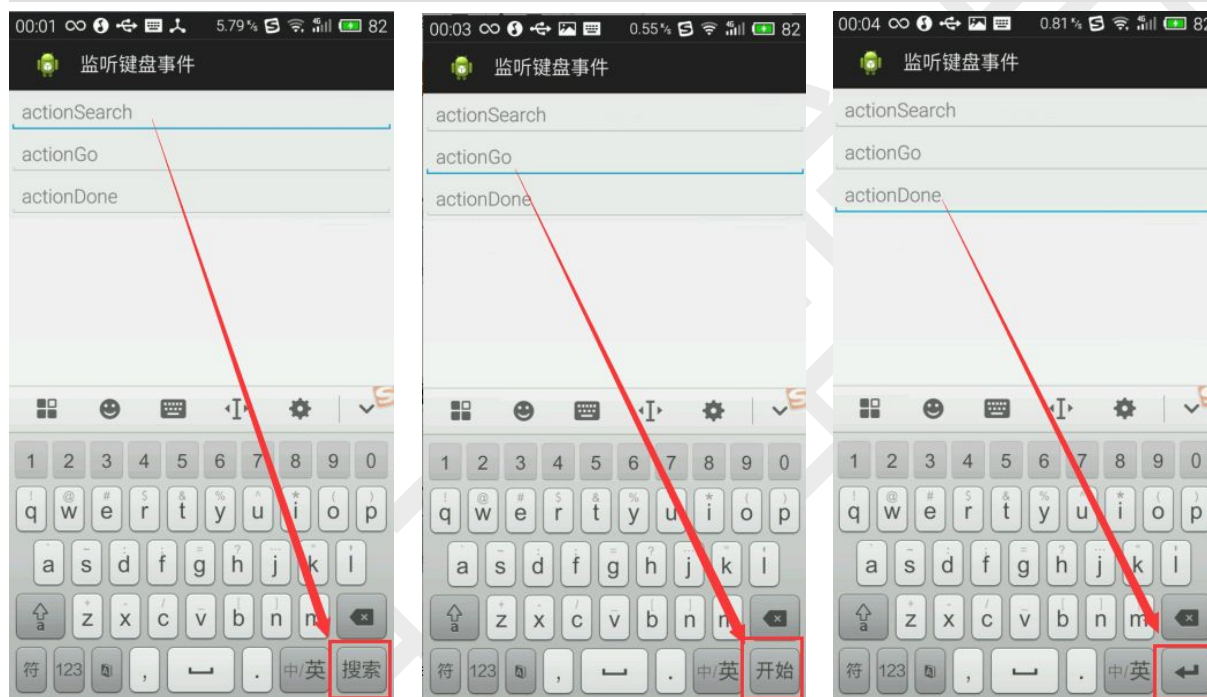
1、如果修改右下角的文字

EditText 有 android:imeOptions 属性，只有修改下面的属性即可。比如给出实例代码如下：

```
1. <EditText
2.     android:layout_width="match_parent"
3.     android:layout_height="wrap_content"
4.     android:hint="actionSearch"
5.     android:inputType="text"
6.     android:imeOptions="actionSearch"
7. />
8. </EditText
```



```
9.         android:layout_width="match_parent"
10.         android:layout_height="wrap_content"
11.         android:hint="actionGo"
12.         android:inputType="text"
13.         android:imeOptions="actionGo"
14.     />
15.     <EditText
16.         android:hint="actionDone"
17.         android:inputType="text"
18.         android:layout_width="match_parent"
19.         android:layout_height="wrap_content"
20.         android:imeOptions="actionDone"
21.     />
```



上面三种属性值，对应的效果如上图所示。

但是仅仅是显示不同的样式而已，还没有响应点击事件。接下来就需要通过代码监听红色框中按钮的点击事件。

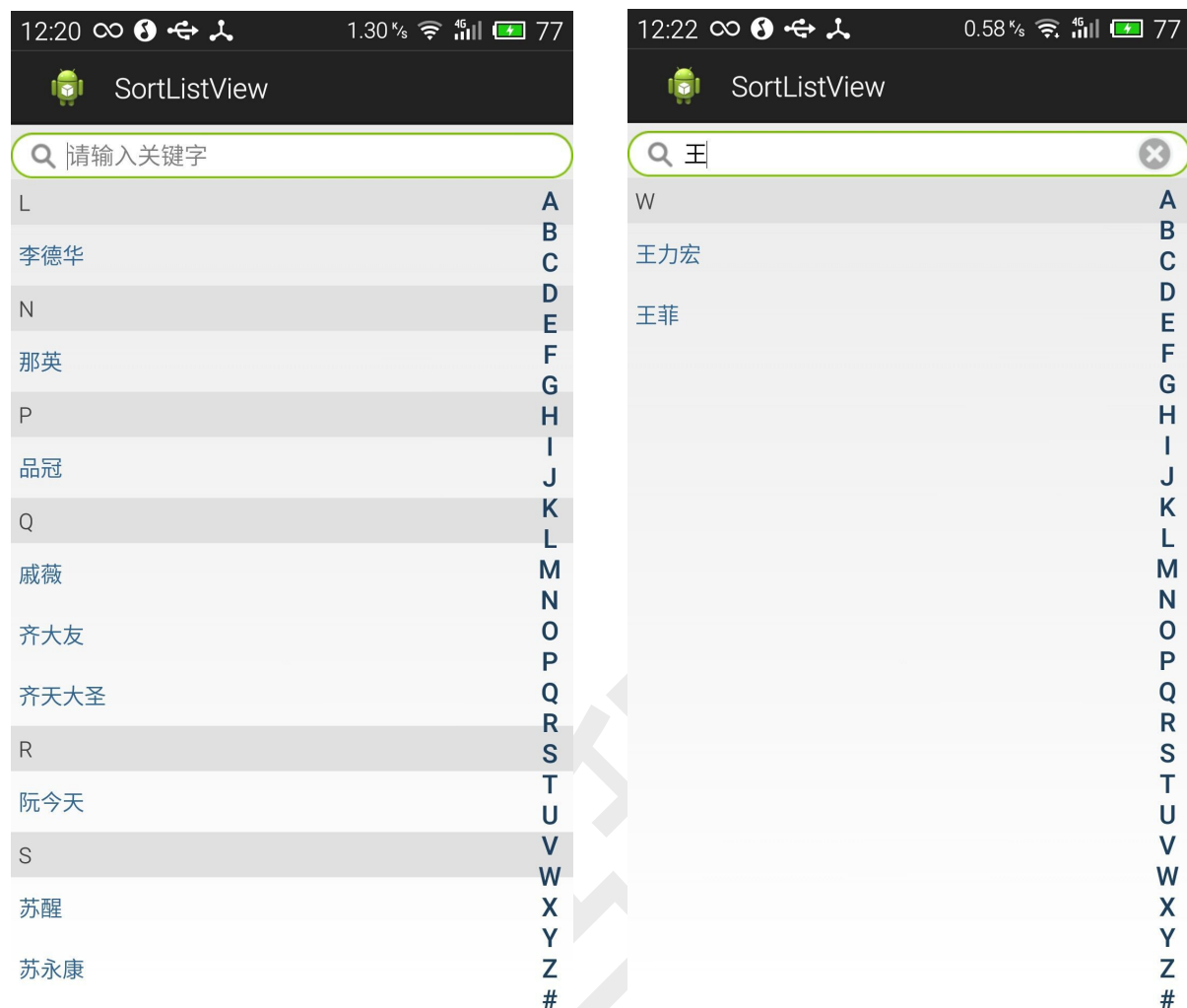
2、如何监听右下角按钮的点击事件

如下代码所示，只需要给 EditText 对象设置一个 `setOnEditorActionListener` 监听器即可。注意：该监听器只能监听上图中右下角红框中的按键动作。

```
1. /**
2.  * 监听输入法键盘的事件
3.  *
4.  * @author wzy 2015-11-28
```

```
5.  *
6.  */
7.  public class MainActivity extends Activity {
8.
9.      private EditText et_search;
10.
11.  @Override
12.  protected void onCreate(Bundle savedInstanceState) {
13.      super.onCreate(savedInstanceState);
14.      setContentView(R.layout.activity_main);
15.      et_search = (EditText) findViewById(R.id.et_search);
16.      //给 EditText 设置按键（注意：只能监听右下角按键）的监听
17.      et_search.setOnEditorActionListener(new OnEditorActionListener() {
18.
19.          @Override
20.          public boolean onEditorAction(TextView v, int actionId, KeyEvent event) {
21.              Toast.makeText(MainActivity.this, "点击按键了", Toast.LENGTH_SHORT).show();
22.              return true;
23.          }
24.      });
25.  }
26. }
```


14、可以按照字母排序的 ListView (2015-11-29)



如上图所示，很多应用都有的效果。代码比较多，为了方便大家学习，我把其 eclipse 工程上传到我的百度网盘上供大家下载。

地址：<http://pan.baidu.com/s/1kTEout5>

15、省市区三级联动



地址：<http://pan.baidu.com/s/1eQ39oCq>

16、购物客户端二级菜单



地址：<http://pan.baidu.com/s/1qWAA1NQ>

17、二维码扫描

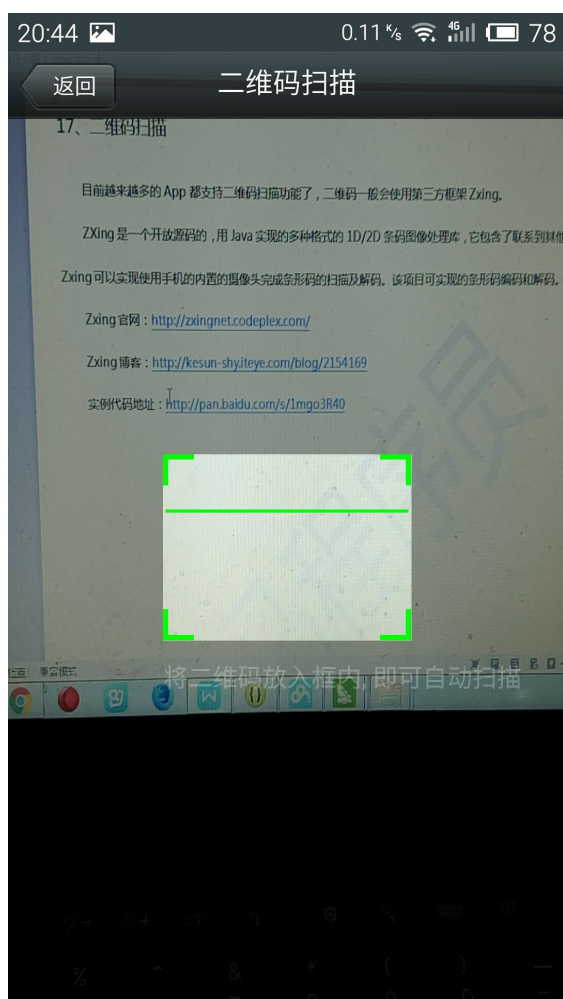
目前越来越多的 App 都支持二维码扫描功能了，二维码一般会使用第三方框架 Zxing。

ZXing 是一个开放源码的，用 Java 实现的多种格式的 1D/2D 条码图像处理库，它包含了联系到其他语言的端口。

Zxing 可以实现使用手机的内置的摄像头完成条形码的扫描及解码。该项目可实现的条形码编码和解码。

Zxing 官网：<http://zxingnet.codeplex.com/>

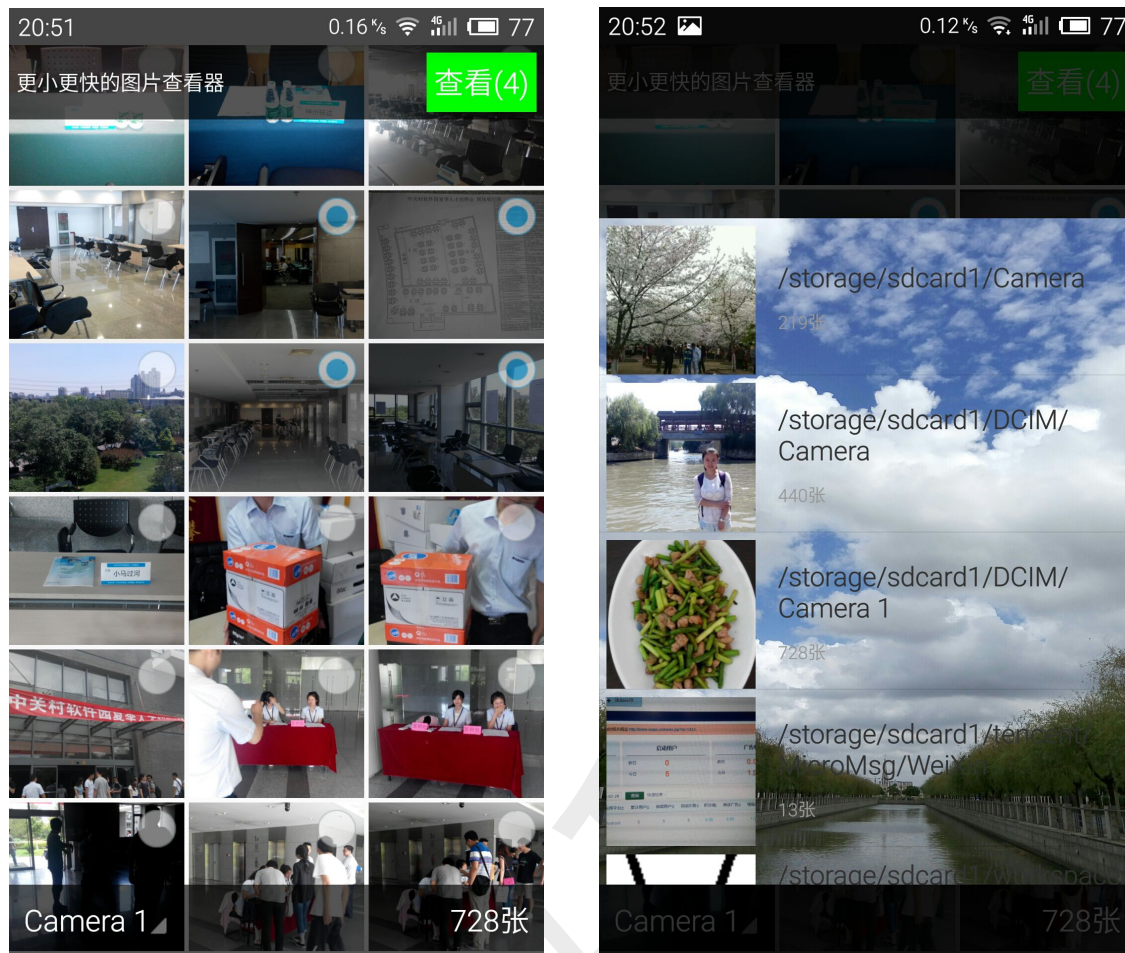
Zxing 博客：<http://kesun-shy.iteye.com/blog/2154169>



实例代码地址：<http://pan.baidu.com/s/1mgo3R40>

18、微信图片选择器

微信图片选择器项目在慕课网上有非常好视频，地址：<http://www.imooc.com/learn/489>。



如上图所示是自己做的仿微信图片选择器，主界面是一个 GridView，展示手机中某个文件夹的图片，点击最下侧后会从底部弹出一个 popwindow，popwindow 里面是一个 ListView 列出所有图片的路径。选中某一个条目后，会回到主界面展示选中文件夹中所有的图片，选择图片后，图片会变暗，然后点击查看会跳转到一个画廊 Activity，该 Activity 支持手势识别，可以放大缩小图片查看。

源码地址：<http://pan.baidu.com/s/1jGvt5aA>

7. 面试实战记录 (★★)

(本章节的内容记录了本人于 2015 年 5 月份在上海的面试的真实情况。在这里分享给大家，希望对大家有帮助。

该部分内容网络连接：<http://bbs.itheima.com/thread-196394-1-1.html>)

挑战公司 No. 1：上海创梯信息科技有限公司

公司地址：上海杨浦区昆明路 1209 号尚凯大厦副楼 504 室



面试时间：5 月 12 日 10:00 AM.

面试结果：顺利砍下 Android 技术总监，15K offer:victory:

面试过程：

10:00 到公司，前台 MM 给了张面试人员登记表，10 分钟搞定表格。

10:30 由于公司 BOSS 正在面试其他人，因此，又等了几分钟。

10:40 在 BOSS 办公室与 BOSS 斗智斗勇，聊了有接近 1 个小时。

疯狂的笔试+面试记录（前方内容“高能”：funk：，请准备高度精神集中前行）：

1. 笔试

由于 BOSS 正在打造公司新业务，刚刚开始组建公司技术团队，公司没有人懂 IT 技术，故本次面试没有笔试题，额。。。

2. 面试过程问答精选：

旁白：进入办公室后，我淡定等待，约莫几分钟后，一位 35 岁左右，看起来非常老道的 BOSS 走了进来。几句寒暄之后，问了我一些关于我的学校、专业、工作经验、接触的 Android 项目等普通流水线式的没有营养问题，轻松搞定！接着，BOSS 终于切入到了重点：

BOSS 问：其实我们想做一个快递业务的 APP，类似于滴滴打车。用户如果想要寄快递，只需要打开 APP，查找附近都有哪些快递员，然后直接跟附近的快递员联系。如此，既能方便用户，又能提高快递员的收入。

我（阳哥）答：我有一个问题想要问一下，咱们公司（注意，一定要说“咱们”，让别人感觉你已经融入他们的公司团队了）不像顺丰，不是典型的物流公司，为什么我们要做快递类的 APP 呢（主动沟通交流，有问题就问，这样面试官才能跟你聊起来！）？

BOSS 答：你知道的那些物流公司现在都是各自为政。例如，你用顺丰快递，你可以下载个顺丰的 APP。但是，你以后可能还要用中通、申通、圆通等等这些物流公司，难道你要下载十几个 APP 吗？而我们要做通用的快递类 APP！

BOSS 问：你知道一个 APP 重要的是什么吗？

我（阳哥）答：用户体验！

BOSS 说：不对，是流量，也就是用户数量（好吧，阳哥作为技术屌丝，关注多的就是 APP 用户体验，所以，不敢去反驳他，先顺着他的意思来）。

BOSS 问：如果让你去做一个类似于滴滴打车的 APP，你认为自己做的出来吗？

我（阳哥）答：像滴滴打车这样的 APP，不仅仅只是客户端的问题。它不是由几个简单的客户端程序员就能做出来的，实际上，它应该是由一个技术团队完成的大型项目。您目前能够看得见的仅仅是用户客户端，看不见的是庞大的后台系统。说详细点就是：滴滴打车这样的项目应该分 3 部分：服务器端，用户客户端，出租车司机客户端。服务端考虑的是数据的存储，业务的调度处理，以及与各个客户端的数据交互。而用户客户端我个人理解主要是一个 UI 的显示和与用户数据交互的功能。比如，用户将自己的当前坐标发送到服务器端（当前坐标可以通过百度地图、高德地图获取），服务器根据用户的坐标查找附近的所有空闲状态的出租车，然后将用户的用车需求推送到出租车司机客户端。出租车司机接收到信息后，如果愿意接这笔单子，就向服务器发送同意信息。服务器就是作为一个中介将用户与出租车司机联系起来。大概逻辑就是这样一个过程，中间的技术细节比较多。总之，一个人做滴滴打车是不太现实的（大家可以看出来，这一段，阳哥是傻瓜式教学，没有说的太复杂，好让 BOSS 能够听明白。不然，技术说的太深，BOSS 就不知道说什么了，还怎么聊的开心。面试的最终目的就是让对方觉得看你很顺眼，这是关键！）。

（顺便啰嗦一下，黑马的 Android 课程中有部分 JavaWEB 课程。我们做为 APP 开发人员，不管是服务器端还是移动端都应该有所了解。如果仅仅会移动端开发技术而不懂服务端知识，长久看来，确实会很影响 Android 程序员向更高层次发展。）

BOSS 问：那么，如果我让你负责客户端的开发呢？

我（阳哥）答：客户端 Android 开发并不难，比如支付宝是一个很强大的金融系统。但是，你让我做它的 Android 端开发，我感觉我没问题。毕竟，Android 客户端重点是信息的展示和与用户的交互。所以一个人做客户端是没有问题的。但是，目前市场上的企业很少有人做一个客户端项目的。市场竞

争如此激烈，一个人做耗时太长，可能你还没有做出来，产品已经被淘汰了。因此，应该多招几个 Android 程序员，这样团队就算有人员流失也不会影响整个公司的发展（展示自己对于项目组人数把控的个人看法）。

旁白：几轮 PK 下来，BOSS 基本上对我已经很信服了。

BOSS 说：其实，在面试你之前，我已经招聘到了两个移动开发的程序员，一个是 Android 程序员（6K），一个是 iOS 程序员（8K），你可以看一下他们两个的面试登记信息和他们的简历。

我（阳哥）问：根据这两份简历可以看出来，这两个人的技术有些一般（不是装逼，他们的简历写的确实有点 LOW）。

BOSS 问：是的，我看的出来，你的经验要比他们丰富地多。所以，我准备让你出任我们公司的技术总监，带着他们两个做客户端开发，你认为你有信心做好吗？

我（阳哥）答：这个职位我以前没有做过，所以心里没有多大的底（阳哥认为这样的问题确实比较考验人，你直接回答“可以，没问题”，就会显得你浮夸、不谨慎、办事不牢靠。如果你直接回答“不能”，又显得你不敢担当，没有上进和挑战的精神。因此回答这样的问题必须两者兼顾，既要谦虚谨慎又要表现出富有挑战精神）。

BOSS 说：我知道你没有做过，但是，一个人挑战任何一样新工作之前都是没有做过的。你有没有想过一旦你能够胜任我们的公司的技术总监，你以后的身价将和现在完全不同，所以，对自己要有一些信心（BOSS 故意画大饼，就说明你有戏了！）。当然，这是一个管理岗位，需要什么样的技术人员你可以招。

我（阳哥）说：好吧，其实我希望挑战一下，但是我不一定能够达到您对我那么高的期望（始终保持谦虚的态度很重要！你的态度决定别人对待你的态度！）。

BOSS 问：你尽力就好，请问你的期望月薪是多少？

我（阳哥）答：15k（第一次面试，我心里也不太清楚上海的市场，15k 一般是黑马班里的大神级同学要的薪资，所以，试水一下）。

BOSS 说：好的，没问题（竟然没有砍价:L，额，土豪公司）。

旁白：随后，阳哥又和 BOSS 聊了一些其他内容，由于 BOSS 马上要开会，就告诉阳哥如果你同接受我们的邀请，那么我希望我们明天下午可以好好聊聊。。。之后，一堆寒暄，不列出来了。虽然，第一家面试就这样结束了，没有技术上太多的 PK。但是，却使阳哥在精神上却得到了巨大的鼓励，面试第二家上海公司有了更大的信心！

面试吐槽：

阳哥的上海处女面就这样丢掉了！遗憾的是 BOSS 不太懂技术，没有碰撞出技术的火花。但是，让人幸运的是他提供给我了一个技术总监的岗位。整个过程可以看到，面试的时候，面试官一方面会明中考察我们的技术。另一方面，在暗中实际上也在考察我们的沟通能力、思维逻辑能力、管理能力等这些软实力。所以，黑马的每一位达人，在黑马拼命码代码的同事，别忘了多和你的小伙伴分享技术，不要忘记中午的时候抓住每一次的公开演讲机会。有时候，一个人综合实力远大于单纯的技术竞争力！加油吧，骚年们！

挑战公司 No.2：上海复深蓝信息技术有限公司

公司地址：上海市徐汇区漕河泾开发区虹梅路 2007 号 1 号楼



面试时间：5月12日 14:00 PM.

面试结果：顺利砍下 Android 工程师，17K offer+1K 补助 = 18K 月薪:victory:

面试过程：

14:00 到公司，前台 MM 给了一张面试登记表和一张 Android 笔试试题，然后一位和蔼的大叔（技术大拿）将我带公司接待处，让我开始笔试。

14:20 笔试完成，笔试很简单，做完后把试题交给前台，前台帮我联系面试官。

14:25 跟一个年龄大概在 25~30 岁之间的年轻技术官进行了接近 1 个小时的面试（通过交谈阳哥感觉面试官应该是项目经理或者开发组组长）。

15:20 人事面试，人事面试后提前跟我说 CTO 会对我进行一个电话复试。

第二天晚上大概 19:00，公司 CTO 对我进行了技术复试，总共持续了 31 分钟。电话复试题要比初试难的多了，姜还是老的辣。我把复试的对话录音保存下来了，作为以后上海黑马就业指导课程的案例讲解（内容何止“高能”，简直就是高达！）。

疯狂的笔试+面试记录（前方内容“残酷” :funk:，请准备受虐心态前行）：

1. 笔试

笔试时出现了一个小插曲，和大家分享一下。阳哥笔试所在的接待区是一个半开放式的区域，共有 3 张桌子，每张桌子上都有一支签字笔。我先用第一张桌子，发现笔是坏的（郁闷），然后换到第二张桌子，发现笔还是坏的（心想：尼玛，这么霉，顺便汗一下），最后只能换到第三张桌子，你猜怎么滴，对，笔还是坏的（想要骂娘了，FUCK！）。没辙了，翻遍我的背包，自己也没带笔。难道企业是用这种方式考察一个应聘者的吗？这怎么办？

我想去前台借吧，前台肯定有（不要去人事那里借，前台毕竟只是前台，不会管你太多的细节，人事可就不一样了，他们会认为一个面试者来比试不带笔是极不严肃的表现）。热心的前台 MM 很乐意地把自己的笔借给了我（心里一阵暖啊！这家必须拿下！）。

为了方便大家更清晰的看到笔试题目，阳哥手录笔试题和答案，看你能做对多少？！

Q：Android 的四大组件有哪些？

A：Activity、Service、ContentProvider、BroadcastReceiver。

Q：请描述下 Activity 的生命周期？

A：onCreate、onStart、onResume、onPause、onStop、onDestroy、onRestart。

Q：如何将一个 Activity 设置成窗口模式？

A：将 Activity 的样式设置成：android:theme="@android:style/Theme.Dialog

Q：如何退出 Activity？如何安全退出已调用多个 Activity 的 Application？

A：调用 Activity 的 finish 方法可以退出当前 Activity。可以自定义一个 Application，在 Application 中声明一个成员变量 ArrayList 用于存放打开的 Activity，当退出时遍历 ArrayList，依次调用 Activity 的 finish 方法。

Q：请介绍下 Android 的五种布局。

A：LinearLayout、RelativeLayout、FrameLayout、RelativeLayout、TableLayout

Q：请介绍下 Android 的数据存储方式。

A：SharedPreferences、XML、SQLite、文件系统

Q：DDMS 和 TraceView 的区别。

A：DDMS 的全称是 Dalvik Debug Monitor Service，是 Android 开发环境中的 Dalvik 虚拟机调试监控服务。TraceView 是程序性能分析器

Q：说说 Activity、Intent、Service 以及他们之间的关系。

A：Activity 负责界面的显示和用户的交互，Intent 封装了数据，可以实现 Activity 之间以及 Activity 和 Service 之间数据的传递。Service 运行在后台进程，一般我们会让给其运行一些后台任务，Activity 通过 StartService (Intent) 或者 BindService (Intent) 可以启动 Service。

Q：请介绍一下 ContentProvider 是如何实现数据共享的。

A：我们可以定义一个类继承 ContentProvider，然后覆写该类的 insert、delete、update 等方法，在这些方法里访问数据库等资源。同时将我们 ContentProvider 注册在 AndroidManifest 文件中，其他应用需要使用的时候只需获取 ContentResolver，然后通过 ContentResolver 访问即可。

2. ROUND 1：PK 项目经理技术面试问答精选

项目经理问：Activity 都有哪些生命周期？

我（阳哥）答：这个问题其实在笔试题中我已经给出答案了（此时，阳哥表示非常疑惑~）。

项目经理说：我知道，你再说一遍。

旁白：当时阳哥我真没搞明白，为何笔试上的题目他还是问我一遍，而且笔试上的好几道题他都重复问了一遍。现在想想应该是他怀疑我笔试的时候作弊了，比如我可以百度什么的。好吧，我不就是笔试的时候出去跟前台妹子借了一支笔，然后又把笔试题给拍了个照片而已嘛。而且，我那 3G 的联通定制机安装的移动 4G 的卡，只能享受 2G 的网速，我打开个百度都得几分钟。算了，不能和面试官较劲！我忍！

我（阳哥）答：Activity 有以下生命周期回调方法，比如常用的有 onCreate、onStart、onResume、onPause、onStop、onDestroy、onRestart。默认情况下，如果我们不给 Activity 设置横竖屏配置信息的话，在横竖屏切换时会将一个 Activity 销毁掉然后重新创建。

项目经理问：Fragment 你用过吗？

我（阳哥）答：这个当然用过呀。现在的应用中基本都有 Fragment 的应用，Fragment 比较小巧灵活。

项目经理问：那 Fragment 跟 Activity 之间是如何实现值传递的？

旁白：其实项目经理在问我会不会 Fragment 的时候 我已经预料到接下来会问我 Fragment 跟 Activity 直接的值传递问题。对于前面的问题，如果我说不会，就不会有下面的问题，但是如果说不会的话，那么项目经理很可能因为这一个问题而把我 PASS 掉，因为 Fragment 是 Android 中一个非常重要的知识，这个是必须会的（黑马的 Android 基础课程中有一天就是讲 Fragment 的）。如果连这个都不会，那么再好的面试技巧也挽救不了同学们的！

我（阳哥）答：Activity 可以先获取 FragmentSupportManager 或者 FragmentManager，前者是 v4 包下的，向下兼容因此用的比较多。然后这些 Manager 通过 Fragment 的 tag 或者 id 调用 findFragmentByTag("tag")，findFragmentById("id")找到我们需要的 Fragment 对象，然后通过调用 Fragment 对象的方法来进行值的传递。

项目经理问：Android 中都有哪些组件需要在清单文件中注册？

旁白：四大组件是学习 Android 的必会知识点，也是黑马 Android 基础中的重点内容，因此这个问题同学们其实应该是很好回答的！

我（阳哥）答：一般来说四大组件都需要在 AndroidManifest.xml 中进行注册，不过其中 Activity、Service、ContentResolver 是必须注册的，而 BroadcastReceiver 可以在清单文件中注册，也可以不注册，这也分别叫做静态注册和动态注册。

旁白：在 Android 中一般通过 XML 文件注册的组件，我们叫静态注册，而通过代码注册或者创建的组件我们叫做动态注册。动态注册和静态注册这两个名称听起来很高大山，其实理解起来 so easy 滴！

项目经理问：你自己有做过自定义控件吗？

我 阳哥 答：自定义控件做过，比如我们项目中的 SlideMenu，LazyViewPager，Pull2RefreshListView，VerticalSeekBar，RandomLayout 等都是自定义控件。

项目经理问：那你说说 View 的绘制过程？

我（阳哥）答：View 绘制是从根节点（Activity 是 DecorView）开始，他是一个自上而下的过程。View 的绘制经历三个过程：Measure、Layout、Draw。

旁白：黑马的老版本课程体系中有 2 天的自定义控件，在这两天课程中同学们能够学会 SlideMenu，Pull2RefreshListView，优酷菜单等自定义控件，目前的黑马课程又对自定义控制进行了加强，添加了多种 QQ5.0 新特性内容，当然难度其实也不小。

项目经理问：ListView 你们应该有用过吧？

我（阳哥）答：这个在我们的项目中应用的很多，其实在如今所有流行的 App 中，ListView 都有一个大量的应用，说 ListView 是一个使用率高的控件都不为过。

项目经理问：ListView 的优化你们是怎么做的？

我（阳哥）答：ListView 的优化有多种多样的策略。在我们的项目中主要做了如下优化。1、重用 ConvertView，2、给 ConvertView 绑定 ViewHolder，3、分页加载数据，4、使用缓存。前两个是通

用的解决方案，后两个是针对我们业务的个性化解决方案。我们的数据来自服务端，如果服务端有 1000 条数据的话，我们客户端不可能傻瓜式的一次性用 ListView 把这些数据全部加载进来，因此我们就用分页加载数据，每次加载 20 页，当用户请求更多的时候再获取更多数据，网络的访问就算网速再快也多多少少会有一定的延迟，因此我们的网络请求是异步处理的，同时从网络加载来的数据使用了 2 级缓存来处理，第一级是内存级别的缓存，第二级是本地文件的缓存。当 ListView 加载数据的时候首先从内存中找，如果找不到再去本地文件中找，只有都找不到的情况下才去请求网络。

旁白：ListView 的优化是黑马课程一个重要的知识点，因此大家上课的时候这个必须得学会，不然在以后的面试中肯定会栽跟头的。

3. ROUND 2：第二轮 PK CTO（非人类）技术面试问答精选

旁白：第二轮技术复试是在第二天晚上 7 点时开始的，当时，阳哥我刚从外面面试完回到家中。跟我聊的是他们公司的 CTO，通过聊天也能感受到他的技术非同寻常，前几个问题问我时感觉多方有种咄咄逼人的气势（貌似面试的人太多了，对于被面试人员都是不屑的口气）。不过几轮技术 PK 后，发现原来 CTO 对人类也可以这么温柔和气的（尼玛，技术好才能被人看得起啊！心底话！）。

CTO 问：说说你对泛型的了解？

我（阳哥）答：泛型是 jdk5.0 版本出来的新特性，他的引入主要有两个好处，一是提高了数据类型的安全性，可以将运行时异常提高到编译时期，比如 ArrayList 类就是一个支持泛型的类，这样我们给 ArrayList 声明成什么泛型，那么他只能添加什么类型的数据。第二，也是我个人认为意义远远大于第一个的就是他实现了我们代码的抽取，大大简化了代码的抽取，提高了开发效率。比如我们对数据的操作，如果我们有 Person、Department、Device 三个实体，每个实体都对应数据库中的一张表，每个实体都有增删改查方法，这些方法基本都是通用的，因此我们可以抽取出一个 BaseDao<T>，里面提供 CRUD 方法，这样我们操作谁只需要将我之前提到的三个类作为泛型值传递进去就 OK 了。而数据的

安全性，其实程序员本身通过主观意识是完全可以避免的，何况某些情况下，我们还真的想在 ArrayList 中既添加 String 类型的数据又添加 Integer 类型的数据。

CTO 问：你知道 Java 的继承机制吗？

我（阳哥）答：知道呀，这个问题很简单呀！java 是单继承多实现呀。

CTO 问：那你知道 java 为何这样设计吗？

旁白：从上面的问题也可以看出越是资历老的程序猿越喜欢刨根问底，因此如果同学们面试的时候遇到一个年龄稍微大点的程序员，那么一定要提前做好思想准备了，他可能先问你一个看似很简单的问题，然后再追问一个很深的思想或者原理。

我（阳哥）答：为何 Java 这样设计，其实这也是我一直的一个小疑惑。不过我是这样理解的。我只能用反证法，如果一个类继承了类 A 和类 B，A 和 B 都有一个 C 方法，那么当我们用这个子类对象调用 C 方法的时候，jvm 就晕了，因为他不能确定你到底是调用 A 类的 C 方法还是调用了 B 类的 C 方法。而多实现就不会出现这样的问题，假设 A 和 B 都是接口，都有 C 方法，那么问题就能解决了，因为接口里的方法仅仅是个方法的声明，并没有实现，子类实现了 A 和 B 接口只需要实现一个 C 方法就 OK 了，这样调用子类的 C 方法时，Java 不至于神志不清。从另外一个方面考虑的话应该就是 Java 是严格的面向对象思想的语言，一个孩子只能有一个亲爸爸。

CTO 问：Java 的异常体系你知道吗？

我（阳哥）答：知道呀，顶层是 Throwable 接口，往下分了两大类，一个 RuntimeException 另一个是普通的 Exception。

CTO 问：那你知道这两类异常的区别吗？

我（阳哥）答：当然知道，java 的命名是见名知意的。从名字上我们也知道 RuntimeException 就是运行时异常，在运行的时候才能被 jvm 发现导致程序的终止，而普通 Exception 必须进行 try、catch 处理，或者在方法上用 throws 声明。

CTO 问：那你的期望薪资是多少？

我（阳哥）答：我期望的薪资已经给贵公司人事说过了，是 17k。

CTO 问：你这么年轻，就想要到 17k 呀！

我（阳哥）问：对的，我是还年轻，高中同样是学习 3 年，有的考上了重点大学，有的却只考上了个大专院校，甚至落榜，不同的人学习能力是完全不一样的，甚至可以用天壤之别来形容，因此如果只简单的用时间来衡量一个人的价值显然就是不太合理的，比尔盖茨跟我这样大年龄的时候已经是亿万富翁了，而我还在找 17、18k 薪水的工作（前面的问题已经回答的这么漂亮了，谈薪水的时候一定要表现出绝对的自信！）。

CTO 问：你说的对，不过我还得问你几个问题，你说你们项目中有用到图片吗？

我（阳哥）答：这个当然有呀，我们新闻客户端基本上每条新闻都有图片，只有图文并茂的新闻才会有人看。

CTO 问：那你说你们遇到 OOM 异常吗？

我（阳哥）说：这个前期的时候我们的 APP 确实遇到过这样的问题，不过现在新的版本早就把这些题给解决了。

CTO 问：那你们是怎么解决的？

我（阳哥）答：OOM 异常是 Android 中经常遇到的一个问题，程序员稍微不注意可能就导致其产生。因为 Android 的每一个应用都是一个 Dalvik 虚拟机，该虚拟机的默认堆内存只有 16M，远远无法跟我们的 PC 机比较，因此容易导致 OOM（Out Of Memory）异常的产生。导致这样的异常主要有以下原因：1、加载大图片或数量过多的图片，因为图片是超级耗内存的，2、操作数据库的时候 Cursor 忘记关闭，3、资源未释放，比如 io 流，file 等，4、内存泄露。我们用的 OOM 主要是加载图片导致的。因为后面的三种原因都是可以通过约束程序员的编码规范来进行预防，或者使用性能分析工具来检查。

CTO 问：好的，那你们的图片是怎么处理的？

旁白：随后这就是 CTO 面试应聘者的一个习惯，他会追着一个知识点往死里问，直到问到系统的底层，或者他能理解的底。

我（阳哥）答：图片的处理主要用两种方式。我们的应用中有两处用到了图片，一个是 ListView 中展示的图片缩略图，这种情况的特点是数量大，但是单个图片内存小，只有几 kb，另外一种是大图片，就是用户通过手机拍摄的图片，然后通过 http 的 post 提交的方式提交到服务器上。然后在客户端将这个图片也展示出来。对于第一种情形，我们是通过三种技术手段来解决问题的，一是图片的缓存策略，二是 ListView 的优化，其实在上面我已经讲过，三是 WeakReference(弱引用)的使用。对于第二种情形，我们主要是首先通过 BitmapFactory.Options 参数获取图片的宽和高，然后再根据我们 ImageView 的宽高对图片进行一个很大比例压缩。

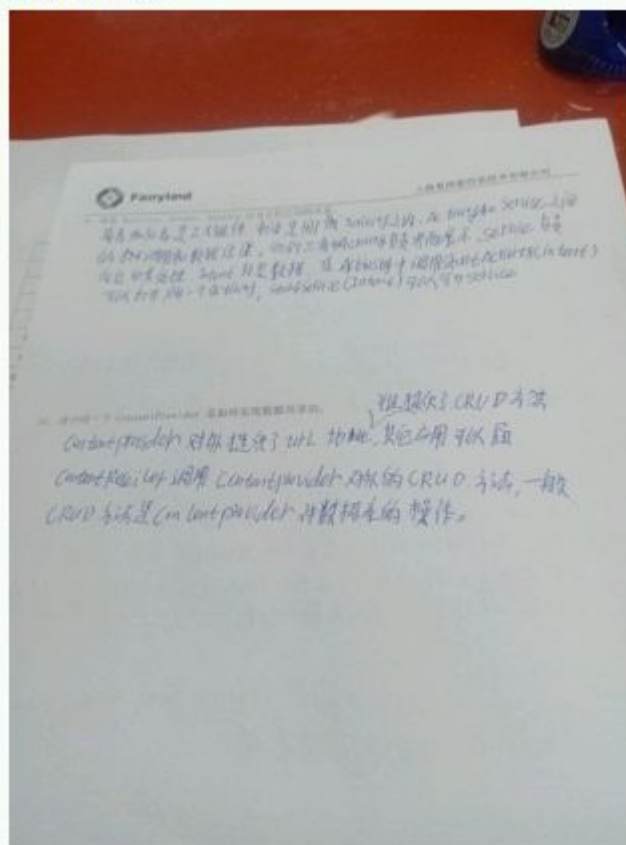
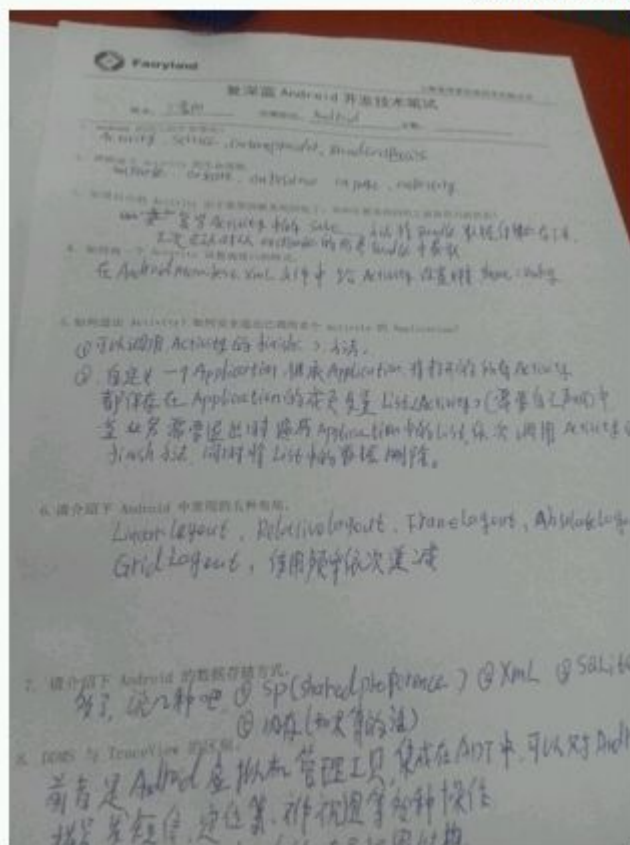
CTO 问：那你说说弱引用是怎么使用的？

我（阳哥）答：WeakReference 是一个类，在 ArrayList 中我们把这个类作为对象传递进去，把我们的图片放在 WeakReference 里面，这样当 davlik 虚拟机内存不够用的时候，就会把 WeakReference 对象回收掉，这样我们在 WeakReference 里面保存的数据也被回收了。

面试吐槽：

阳哥在上海的第二面终于遇到懂技术的人，把笔试、技术一面、人事、技术二面一气呵成的通了个关，不拖泥带水，最终人事给了基本薪资 17k+1k 多补助的 offer，这种感觉也许只有你经历过了才能体会吧！相信黑马学子经过四个月的刻苦磨练也能远远超过我的水平。加油吧，骚年们！

阳哥冒死偷拍的笔试题！



阳哥录用通知邮件！

2015年新员工报到须知（王阳）.docx

尊敬的王阳先生：

您好！

非常感谢您应聘上海...技术有限公司的职位！您在面试中的出色表现给我们留下了深刻的印象。本公司拟同意录用您，我们诚挚的邀请您于2015年5月18日上午9:00到公司办理具体录用事宜。但是无论在任何情况下，您必须在收到本通知3天内书面回复本公司（书面回复包括短信、特快专递、通过面试登记表上记载的电子邮箱发出的电子邮件或登记的手机号码发出的短信），且在指定时间内到公司，否则本通知自始无效。本公司录用您的条件是：您提供的入职申请材料合法、真实、完整、有效，且您具备胜任本职位工作的身体健康状况。

请您一并携带以下材料、证件及报告：

1. 身份证复印件、学历（学位）证明复印件及其他资质证书复印件（验原件）（应届毕业生须携带学生证）；
2. 原公司有效离职单、劳动手册或离职证明（应届毕业生可忽略此项）；
3. 累计就业工龄证明（社会保险累计缴费记录证明，可网上截图（详情见附件一），应届毕业生可忽略此项）；
4. 技术资质证书或其他认证类证书（需携带原件或复印件扫描件）；
5. 公积金账号（应届毕业生可忽略此项）；

1) 请提供上家单位全称及个人公积金账号；

挑战公司 No.3：中阜投融资资产管理股份有限公司---中投融

公司地址：上海市静安区威海路 228 号招商局广场南楼 21 楼



面试时间：5 月 12 日 16:30 PM.

面试结果：顺利拿下 Android 工程师，15K offer+1k 住房补贴=16k:victory:

面试过程：

16:30 到公司，因为这一天安排了 3 家面试，这家本来是安排的 16:00 的，但是迟到了半个小时，不过企业招人都比较急，这个可以理解。

16:30~16:40 填写面试登记表，这家没有笔试题，填完后坐在公司前台附件的沙发上等了几分钟，桌子上放了一盘水果糖，诱人，也不敢吃，哈哈。

16:40~17:15 跟一位 Android 程序员进行第一轮 PK。

17:20~17:50 跟项目经理进行第二轮 PK。

17:50~18:00 跟人事谈薪资。

疯狂的笔试+面试记录（前方内容“高能”：funk:，请准备高度精神集中前行）：

1. 笔试

这一家没有笔试题，其实阳哥发现一个规律，一般 Android 开发团队刚刚组建的，或者 Android 开发人员不多的企业基本都没有笔试这一环节，这可能是他们公司的 Android 开发团队各方面还没有形成一套标准的流程原因吧。

2. 第一轮技术面试过程问答精选：

旁白：面试我的哥们儿比较腼腆，也许是他也刚进公司没多久的缘故，好像才 4 个月的样子。他主要问了我都做过哪些 Android 项目，怎么学习的 Android，都做过哪些 Android 控件，然后他又问了一个他们公司目前正在做的项目遇到的一个难题，问我怎么解决。

PS：最后这个问题，阳哥感觉他并不是在考察我的技术，而是以请教我问题的态度在向我咨询了。面试能面到这种程度，基本已经有戏了。

面试官问：介绍下你都做过哪些 Android 项目？

我（阳哥）答：这自己主要做过 3 个项目，新闻类的，应用平台类的，手机管理类的，其中自己参与多也做的成熟的是新闻类的一款 App。

旁白：上面的三种类型的项目，都是黑马 Android 课程体系中的教案，因此介绍项目基本没压力。

面试官问：那你在项目的开发中担任一个什么角色？

旁白：这样的问题其实很多面试官都问我了，感觉被问到的概率有 80%，此种问题显然是想考察我们的担当能力，技术担当和责任担当。在上海黑马 66 期的开班典礼上，我是这样给大家说的，我们 66 期有 70 多位同学，我们班分成 10 个小组，每个小组选出一名组长，组长负责全组的学习，组长不是固定不变的，每周考试一次，每次考试成绩高者为组长。每次考试如果这个组的平均分在班排名低的，组长得

无条件接受惩罚。在黑马，没有个人排名，只有团队排名，我们更注重团队的协同能力，而不注重个人的表现。

我（阳哥）答：我在这个项目中属于主要参与者之一吧，或者说是主要负责人，我们 Android 项目不像是 javaweb 项目那样需要大量的程序员，像我们的项目，总共 2 到 3 个程序员 3 个月功夫就能搞定，因此我们几个人也没有绝对的说谁领导谁。不过这样的项目，让我们任何一个人去开发都是很 OK 的，只是时间问题。

面试官问：你们用什么代码管理软件？

我（阳哥）答：SVN。

旁白：在黑马有专门的一节课是讲 SVN 和 Git 的，并且后面的项目也是用 SVN 跟大家进行代码共享的。

面试官问：你学习 Android 的途径都有哪些？

我（阳哥）答：现在是互联网时代了，不像我们大学时代主要通过书本学习。在大学的时候选修的 Java 课程，那时候还是诺基亚时代，图书馆的移动开发书架基本是被诺基亚的塞班占据的。自己大学毕业的时候 Android 已经开始风靡全国了，那时候自己从网络上看到的 Android 开发的各种视频，然后开始学习的 Android。自己在 Android 的开发中遇到的各种问题一般都是靠百度解决的，说度娘是好老师真不为过，其实也用过 Google，不过被屏蔽了，也懒得翻墙，百度现在做的也不差，百度出来的 Android 技术主要来自如下专业网站，比如：CSDN、51CTO、ITEye、AndroidBus、EOEAndroid 等，对了还有一个国外的没有被屏蔽的网站是 Github，我们项目中的很多控件其实都是从 Github 上学习过来的。

面试官问：那你从 Github 上都用到过什么控件？

我（阳哥）答：Github 上的开源项目非常的多，基本上你想用的东西都有，比如 xUtils、HelloCharts、Clander、SlideMenu、SeatTable、LDrawer、SmoothProgress、Touch Gallery、ViewPagerIndicator。

面试官问：你自己有做过自定义控件吗？

我（阳哥）答：自定义控件做过，比如我们项目中的 SlideMenu ,LazyViewPager ,Pull2RefreshListView , VerticalSeekBar,RandomLayout 等都是自定义控件。哦，对了，最近我刚给我女朋友还做了一个 HideSlideBar 控件，我可以让你看一看。里面主要由 VerticalSeekBar+ProgressBar+NineOldAnimation 完成的。

旁白：我把我自己做的自定义控件给他演示了一遍，这个是我女朋友项目中需要用的一个需求，她不会做，我帮她写了一个 Demo，没想到正好用上了。

面试官问：哦，不错，你这个动画用的是属性动画吧？

我（阳哥）答：对的，这个属性动画，用了 JakeWharton 大神的开源框架。不过这个属性动画其实自己写也可以，内部原理比较简单，就是给控件设置一个开始位置，一个结束位置，设置一个延时时间，然后不停的更改控件的 layout 位置即可。

旁白：这个东西在黑马的项目中都有讲解，回答不难，在 Android 应用中我们会用到很多第三方的框架，我们不仅仅要会用别人的东西还必须知道别人写这个东西的内部原理。

面试官问：那好，现在我们有个项目遇到一个问题，你看如果让你做你应该怎么去解决？

旁白：这时候，阳哥已经感觉到自己已经 PK 成功了。不过面试官性格也不错，遇到问题善于寻求外界的任何帮助。

我（阳哥）答：可以的，什么需求您讲吧，我听听。

面试官问：我们的项目中有多个 Fragment，Fragment A 跳转到 Fragment B，Fragment B 跳转到 FragmentC，那么这时候我多次按返回键，如何能让 Fragment 跟 Activity 的任务栈一样，依次从 FragmentC 跳转到 FragmentB，再跳转到 FragmentA？

旁白：这个确实是需要比较棘手的问题。其实面试官也知道你没有做过类似需求的开发，事实也是这样。这就比较考验临场发挥能力了。这种问题可以从模仿 Activity 任务栈考虑解决。任务栈是一种数据结构，

我们可以自定义这种数据结构，然后管理这个数据结构，但是每个 Fragment 都是独立的，如何把这些独立的 Fragment 关联起来，这都是需要考虑的问题。

我(阳哥)答：这个需求应该很好实现。我们可以这样，首先定义一个 BaseFragment，让 FragmentA、FragmentB、FragmentC 都继承 BaseFragment，第二在 BaseFragment 中定义一个 ArrayList，每打开一个 Fragment，把这个 Fragment 对象添加到 ArrayList 中，这样这个 ArrayList 就可以当做一个栈结构，第三我们需要设置返回键监听，当监听到返回键的时候，查看当前 ArrayList 中倒数第二个 Fragment 有没有 Fragment，如果有则取出该 Fragment 并把 ArrayList 中末尾 Fragment 删除，然后用 FragmentManager 的 Replace 方法，将当前 Fragment 替换成最新 Fragment 即可，如果 ArrayList 中只有一个 Fragment，且监听到了返回键，那就不对 Fragment 做处理，同时也不拦截该事件，这样也不会影响其他 Activity 之间的切换。

旁白：回答了上面的问题后，面试官说他回去试试，然后就去叫他们的领导了。

3. 第二轮技术复试过程问答精选：

旁白：第二轮面试我的是个技术大拿，主要负责公司整个软件架构的设计，这家公司之前只有 web 端，全都是他干的，现在公司向移动端发展，因此最近一直在招 Android 和 iOS 开发人员，他不太懂 Android，不然估计他一个人就能把一个 Android 项目搞定了。他问了我一些关于 http、数据安全的知识，然后就开始跟我谈人生了，一直到他们快下班，他才把人事叫来跟我谈了谈薪资。

经理问：你做的 Android 的项目跟服务器交互都有哪些接口？

旁白：大家不要被接口这个词给迷惑了。这里的接口不是 java 中的接口类，也不是很高大上的东西，其实换成大白话就是你们的客户端跟服务端是如何实现数据的交互的。

我(阳哥)答：我们的接口有多种形式，第一种是 http 的形式，客户端跟服务器通过 http 协议传输数据，比如我们的新闻列表的请求都是给服务器发送的 get 请求，然后服务器把数据发给我们，我们上传

给服务的图片是通过 http 的 post 请求方式完成的。第二种是 socket 完成的，服务端开启 ServerSocket，客户端开启 socket，然后客户端跟服务端建立长连接，这样实现了客户端跟服务端数据的即时通信，通信的协议是我们公司按照 xmpp 开放协议的基础上修改的，其实 xmpp 协议就是一个 xml 格式的数据。第三种是集成了第三方接口，比如分享功能用的是 ShareSDK，消息推送用的是 JPush，内置广告用的是万普世纪。

经理问：你们 http 传输数据的时候安全是怎么保证的？

我（阳哥）答：我们的数据有些是需要安全设置的有些不需要，我们的新闻类数据不需要特殊的添加安全设置，而用户注册，用户登录以及用户隐私数据保存是考虑安全性的。用户的密码等信息肯定不能进行明文传输的，我们将用户的密码在本地进行了 MD5 算法的加密，然后再传输。同时保存在本地的時候也是加密后的数据。还有需要安全性更高的数据需要通过我们自定义协议通过 Socket 传输。

经理问：那你知道 MD5 的原理吗？

旁白：老程序员就是喜欢刨根问底，如果技术功底不雄厚的话确实这个问题很难应付，因为对于大多数人只需要用一个技术就行了，不会去关注他的内部原理。

我（阳哥）答：MD5 算以 512 位分组来处理输入的信息，且每一分组又被划分为 16 个 32 位子分组，经过了复杂处理后，输出由四个 32 位分组组成，将这四个 32 位分组合级联后将生成一个 128 位散列值。这个过程是不可逆的，我们也把他叫做数据指纹，但是我们依然对用户的输入进行安全校验，如果是纯数字类型密码，是不允许注册的，因此就算你 MD5 加密了，黑客也可以通过彩虹碰撞的形式进行暴力破解。

旁白：接下来，经理问了我几个问题可能能难住我，关键是他也不懂 Android，只能问我 javaSE 和思想上的一些东西，之后就开始跟我聊人生了，说他们公司是国企背景，自己在公司发展多么好，公司未来要做一个什么样的产品，公司弹性工作制，每年至少 14 薪，又问我女朋友在哪，什么时候考虑结婚，在哪买房，老家是哪的，爸妈是干什么工作的，你是如何规划未来的等等。。。。。

4. 人事面试：

这家的人事面试与其说是人事面试还不如说成人事咨询，人事跟我介绍了公司背景，跟我谈了薪资，跟我谈了入职安排，又谈公司发展，公司的福利待遇等等。人事的最后一个问题都是出奇的相似，

人事：我该问你的问题都问完了，你还有什么要问我的吗？

PS：这些问题千万不要一个都不问，这样人事会认为你这个应聘者对我们公司不感兴趣。如果有其他人也去面试，那么你就可以能被 PASS 了。其实我们真的想去这家公司的话，肯定会有一大堆问题的，但是也不要问的太多，问的太多显得你很啰嗦，人事的耐心也是有限的，同时也不要问太低级的问题。对于我们程序员来说，应该最关注如下两大类问题，一是自己将要加入的开发团队的情况，二是公司的薪资待遇以及员工培训发展情况。第一类问题显得我们技术专业，技术屌丝的特性被发挥的淋漓尽致，第二类问题很现实，我们不仅要眼前的工资还要考虑未来的发展。

我（阳哥）答：咱们公司的技术团队目前有多少人，都做哪些项目？

人事：公司目前技术团队还在不停的扩建，目前总共有十几名，不过服务端人比较多。现在主要做 p2p 理财类产品，这也是很火的一个趋势。

我（阳哥）答：咱们公司给开发人员有定期的培训吗？

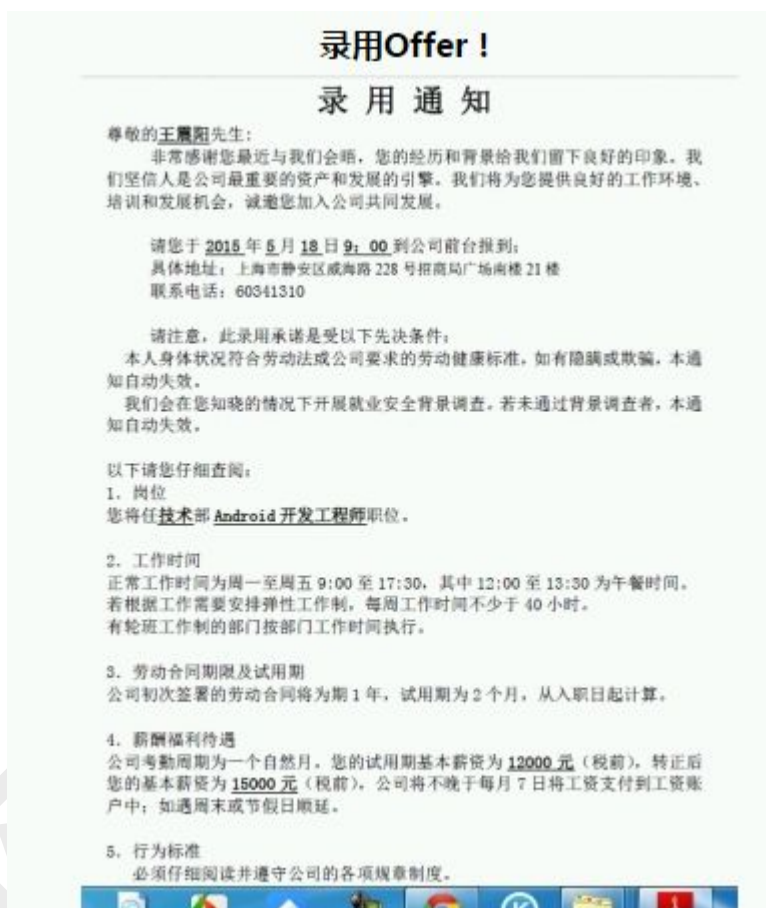
人事：有的，公司每年都有拓展培训，拓展培训是针对全体员工的，技术团队的话每周或者每个月可能有技术分享活动，每周一名技术人员进行分享，同时可以获 100 元/次的奖励。

面试吐槽：

阳哥在上海第一天竟然面试了 3 家，上海这么大，每个公司都在不同的区，回到家已经晚上 7 点多了，我的感受只有一个字儿累，躺在床上就起不来了，不过还好今天本来是抱着被面试官“虐”的心

态去的，结果没有被“虐”，而且还能旗开得胜。这给了我很多自信，不仅仅是对自己技术的自信，更是对黑马 Android 课程的自信。

最后送大家一句话：没有企业给不了的薪资，只有自己掌握不了的技术。没有自己掌握不了的技术，只有不够努力的自己。



挑战公司 No.4：上海游竞网络科技有限公司---PLU

公司地址：上海市闸北区广中西路 777 弄 99 号江裕大厦 10 层



面试时间：5月13日 14:00 PM.

面试结果：Android 工程师，16K offer+500 元住房补贴+490 元交通和饭补+200 元全勤奖
=17.1k:victory:

面试过程：

16:00 到公司，这家公司在闸北区，比较远，不过办公楼很高大上。

16:00~16:30 填写面试登记表和笔试题。

16:30~17:15 跟面试官进行技术 PK（这家面试流程比较简洁，技术只考察一轮就让人事谈薪资了）。

17:20~17:40 跟人事谈人生谈薪资。

疯狂的笔试+面试记录（前方内容“高能”：funk:，请准备高度精神集中前行）：

1. 笔试

PS : 笔试是在公司前台旁白的桌子上做的。阳哥本来胆子就小，这一下搞的偷拍都没自信了，导致对焦不成功，拍出来的照片很模糊，大家凑合着看吧，我把图片上的试题以文本的形式列出来，如下。

Q : Activity 的生命周期？

PS : 我晕，跟复深蓝的笔试题如此的雷同，难道他们两家公司技术人员互相抄袭的吗？忍住，不笑！

A : onCreate、onStart、onResume、onPause、onStop、onDestroy、onRestart。

Q : Activity 销毁前，如何保存 Activity 的状态？

A : 可以使用 onSaveInstanceState (Bundle) 方法将 Activity 中需要的数据保存起来，当下次重新启动 Activity 的时候在 onCreate (Bundle) 中获取 Bundle 数据。

Q : 请介绍下 Android 中常用的 5 种布局？

A : LinearLayout、RelativeLayout、FrameLayout、RelativeLayout、TableLayout。

Q : 请介绍下 Android 的数据存储方式？

A : SharedPreferences、XML、SQLite、文件系统、内存（如果算的话）。

Q : AIDL 的全称是什么？如何工作？能处理哪些类型的数据？

A :

① Android Interface Definition Language

② AIDL 一般用于远程服务，也就是进程间通信。我们可以分服务端和客户端，服务端声明 AIDL 文件，该文件命名为 xxx.aidl,ADT 会自动将 xxx.aidl 生成代码文件，代码文件提供了 aidl 中接口的实现。客户端如果要使用服务端提供的服务需要将 xxx.aidl 文件放到客户端源代码目录下 然后生成 xxx.java 类，客户端通过 bindService 的形参 ServiceConnection 的 onServiceConnected 获取到 Service 对象，这个对象通过 Stub.asInterface (service) 返回 aidl 的实现类。之后我们就可用调用这个 aidl 的实现类。

③ 基本数据类型都可以，复杂对象也可以，只不过需要实现 Parcelable 接口。

Q：请介绍一下 handler 机制？

A：Android 中 handler 多用于主线程和子线程之间的通信，比如在 Android 中子线程是不允许修改 UI 的，如果修改只能让子线程给主线程通过 handler 发送 message，然后主线程进行修改。Handler 整个机制的实现，还依赖 Looper、Message 两个核心内容。在主线程中 Android 默认给我们创建了 Looper

Q：java 如何调用 c、c++ 语言？

A：java 通过 JNI 调用 C/C++ 代码，在使用的时候首先通过 `System.loadLibrary("xxx")` 将 xxx.so 文件加载到 jvm 中，同时在类中必须对 so 文件中的方法进行生命，格式：`public native void test();`

Q：Android 分几层，分别是什么？

A：四层。Linux Core、Libraries (Android Runtime)、Application Framework、Applications。

Q：final、finally、finalize 的区别？

A：第一个是关键字最终，用 final 修饰的类为最终类，不能被继承，修饰的方法不能被覆写，修饰的变量不能被改变。finally 是异常体系中的关键字，当系统遇到异常是，在进行 trycatch 的时候，finally 代码块里的代码是必须被执行的。finalize 是 Object 类中的方法，当 GC 回收对象时回调的方法。

Q：heap 和 stack 的区别？

A：堆和栈。栈存放对象的引用，堆存放对象实体。堆中的对象是有 jvm 的垃圾回收器负责回收。

2. 第一轮技术面试过程问答精选：

旁白：面试我的是 85 年出生的 Android 组组长（这是后来他们公司人事给我打电话让我入职的时候，跟我说的）。他们公司是做游戏视频直播平台的，因此对 app 的性能要求比较高，他用手机让我看了一段代码。代码（记得不是很清楚了）大概如下：

```
1. public class MainActivity extends Activity {  
2.     @Override
```

```
3.         protected void onCreate(Bundle savedInstanceState) {
4.             super.onCreate(savedInstanceState);
5.             setContentView(R.layout.activity_main);
6.             new Thread(new Runnable() {
7.                 @Override
8.                 public void run() {
9.                     while (true) {
10.                        SystemClock.sleep(1000);
11.                    }
12.                }
13.            }).start();
14.        }
```

面试官问：上面的代码有问题吗？

我（阳哥）答：当然有问题呀，这个 Activity 在启动的时候开启了一个子线程，但是当 Activity 退出的时候该子线程还在运行，并没有停止。子线程运行在进程中的，Activity 退出的时候进程并没有退出，而由前台进程变为后台进程。

面试官问：那应该怎么解决？

我（阳哥）答：我们可以这样，把 while (true) 改成 while(flag)，flag 是一个 boolean 类型的变量，通过改变 flag 的 true 或者 false 来终止子线程的运行，当 Activity 退出时会调用 onDestroy 方法，因此在该方法中我们可以把 flag 设置为 false。

旁白：面试官听了我的答案，从他的表情上来看好像他不是很满意。但是他也没说我说的对不对，而是给我说了他心中的答案，不过他给的答案我当时是虚心接受了，不过到现在我还没明白他说的道理在哪，我写了测试代码也没测试出来啥。

面试官问：这个代码有问题就出在这个 Thread 是一个匿名的，而且没有声明为静态成员变量？

我（阳哥）答：哦，这样子呀，这个我真不知道。

旁白：其实我内心是很想问他为什么呢？但是如果他答不上来就会让他很难堪，如果回答上来了倒没啥问题。从他也不自信的言语中我感觉还是不问他比较好？

面试官问：你对 RTSP 流媒体协议有了解吗？有没有做过手机播放器的应用？

我（阳哥）答：这个知道，自己做过流媒体播放器。我使用过开源的 Vitamio 开源库。

旁白：黑马课程中有一个项目是手机影音。因此这个东西自己是知道的。不懂的人可能以为 RSTP、ffmpeg 这些专业名词都多么的高大上，其实在黑马手机影音中，我们会做一个视频播放器，视频播放器分两种，一种是 Android 自带解码，第二种是万能播放器，所谓万能播放器就是绝大多数流媒体格式都支持。

面试官问：那你知道代码测试怎么测？

旁白：关于测试，阳哥真心不太懂，怎么办？？尽量回答一些自己知道的，同时把测试方面的话题给关闭掉，省得面试官追问我。

我（阳哥）答：您说的是 Monkey Test 吗？

面试官问：恩。。。也算吧？还有其他吗？

我（阳哥）答：我们程序员在写代码的时候都是有规范的，优良的代码规范是规避 bug 重要的一步，同时我们公司每周都有一个代码走读会议，所谓的代码走读，就是开发人员互相看对方的代码，检查代码是否规范以及是否存在 bug。当然也有测试，不过我们的测试都是黑盒测试。

面试官问：哦，代码测试，就是可以通过测试一段代码来分析这段代码有没有问题，性能是否可靠。

我（阳哥）答：对的，Android 自带了 AndroidTest 功能，可以对一个类一个方法进行测试。

面试官问：那好，图片你有处理过吧？

我（阳哥）答：这个肯定有呀，哪个 Android 应用没有图片。比如图片缓存呀，图片缩放呀，图片缓存呀。

旁白：其实自己对图片处理这块儿，还是比较了解的，可惜他也没怎么深问我，没有给我很大的发挥机会。

面试官问：我们现在要做一个视频在线播放的 app，播放的都是游戏直播或者录像视频，iOS 已经做好了，Android 还没有做好，你要是进来的话你就是做视频直播这一块儿。

我（阳哥）答：这个已经做好的能让我看一下吗？

旁白：面试官拿来水果机让我看了一下，主界面就是一个 ListView，每个 ListView 的一行都有大概 4 个视频预览图，点击之后可以播放。

我（阳哥）问：这个切图，设计啥的都做好了吧，剩下的就是编码了其实？

面试官答：是的，后期我们 Android 团队还会不停的夸大，我们的发展重心已经偏向移动端。

旁白：接下来就是跟面试官侃大山了，从天文到地理，从 BAT 到初创型公司，阳哥也不能甘拜下风。

我（阳哥）答：恩，现在是移动互联网时代了，移动端肯定得做好，不管什么样的公司都特别重视移动端。咱们公司现在才开始研发移动端，其实已经稍微有点儿晚了，就得赶紧拼命追赶了。

3. 人事面试：

跟面试官聊完后，技术官对我的评价是，我挺喜欢你，你跟人事好好聊聊吧，哈哈，我肯定会跟人事 MM 好好聊天的。人事问的问题太老套路了，感觉全上海人事问的问题都一样，难道他们都是从同一个地方培训出来的？人事也有培训吗？其实应付人事的问题不难，难的是如何回答人事的问题。答题的方式直接决定了我们在人事那里的印象。我把人事问的几个问题大概列一下。

人事：你为什么离职来上海？

PS：人事问这个问题的主要目的就是看你的动机纯不纯，有没有不良倾向。

我（阳哥）答：主要是两个方面吧，第一个是我的女朋友在这边工作，我年龄也不小了，也到了谈婚论嫁的时候了，我不着急，家里父母亲着急呀，来上海工作的话离女朋友比较近，结婚就比较好办，第二个主要还是。。怎么说呢。。。高大上的说就是为了自己事业更高的发展吧，说的直接点就是上海毕竟

国际化都市，发展机会很多，自己也想来上海工作个 3~5 年挣钱买个房子首付啥的，如果在上海的这几年发展的好可以考虑再公司附近买房，如果自己发展的不好，在上海周边买个房也行。

人事：你女朋友一直在上海，还是刚回来上海？

我（阳哥）答：她上一年来的这边，之前是我大学同学，主要是她家有亲戚帮她找了一份设计院的工作，而我只能靠自己找工作。

人事：你以后打算长期留在上海了吗？

旁白：上面的问题其实是人事在考察你的稳定性，公司可不想招到一个人刚培养出来就走掉了。但是如果你直接肯定的回答：我会一直留在上海，又显得很假。“装”的高境界就是让对方感觉你很不会“装”。

我（阳哥）答：这个有想过，自己其实蛮喜欢上海的，可是计划赶不上变化，来上海的前一年我也从来没想到我会来，因此不能说一辈子都在上海，不过目前这几年是打算在上海好好发展一下。上海的房价、户口政策都是问题，现在不是我选不选择上海，而是上海选择不选择我的问题，呵呵，其实谁都不傻，都想来好地方，关键就看自己有没有本事扎根于此了。

人事：你能承受加班吗？

旁白：其实互联网企业加班是很常见的现象，尤其是像上海这种生活节奏比较快的城市，加班肯定是避免不了的，但是我们也有我们的底线，不能无条件无休止的为公司加班，但是也不能一点儿班都不能加，毕竟有时候公司忙的话，比如新产品上线，新 bug 的解决都是需要加班的，这个时候我们还真的义不容辞的为公司付出，毕竟我们的薪水是公司给的。在公司工作一方面是我们为公司付出，另一方面我也要知道感恩公司。

我（阳哥）答：加班很正常，自己之前的公司也经常加班。不过公司加班一般都不会让员工平白无辜的加班的，我们加班的时候公司会发晚餐补助，打车补助等，如果加班超过半天会计入到我们的存休中，等活不忙的时候我们可以申请休息。因此只要不是高频度的天天加班到凌晨应该没啥问题的。

人事：你的五年规划是啥？

我（阳哥）答：自己还是想往技术方向发展，自己也比较喜欢代码，愿意去研究技术。人往高处走水往低处流，经过自己几年的积累后自己想当一名技术总监或者项目经理类的技术管理类岗位。不过这些都得靠自己的努力以及对机遇的把握情况了。

面试吐槽：

这家企业各个方面的条件真心不错，想拒绝都很难找到合适的理由！我要了 15k 的薪资，人家给了 16k，还有各种诱人的福利。感觉还是游戏类的公司土豪。你以为你要 15k，人家给你 16k 已经很高了吗？请让我把话说完，告诉你啥叫游戏公司。该入职的时候，当然我没有去入职（入职是上午 10:00），上午 10:20 的公司人事给我打电话，问我迟到了还是怎么会儿事儿，我说对不起，感谢贵公司对我的认可，自己已经有了新的选择。把他们给拒绝了。下午 1 点多的时候，我正在吃饭，结果这家公司的技术官又给我打电话来了，在电话里跟我聊了很久，他说如果再给你多几 k 你会考虑过来上班呢？我哑巴了，自己从来就没有享受过这么好的待遇，受宠若惊的阳哥无言以对！自己的内心像打翻了五味瓶一样，这种感觉估计也没有几个人理解。后来还是被我给拒绝了，内心在流泪！

这篇文章阳哥已经写到尾声了，但是想给大家说的话却依然很多很多。

亲爱的黑马同学们：

如果你还在撸着代码，看着视频，再苦再难，请你一定要坚持，今天你付出的一点一滴都将会在明天变成千千万万倍的回报。

加油&坚持！

挑战公司 No.5：一号店旗下壹药网

公司地址：上海市浦东新区碧波路 572 弄 115 号 10 幢

(偷拍前台妹子~天热，穿的少其实是正常的，可惜桌子挡住了该看的，哦，不该看的:lol)



面试时间：5月14日 10:00 AM.

面试结果：Android 工程师，15K offer&14个月薪资:victory:

面试过程：

10:10 到公司，这就是传说中的一号店！，公司总共三栋独栋楼，两个在装修，一个在用，因此工位紧张，导致我面试都是在前台所在的大厅里进行的。

10:20~10:50 填写面试登记表和技术官面试。

10:50~11:10 跟研发部 Leader 面试。

11:10~11:30 跟人事谈薪资以及入职事宜。

疯狂的笔试+面试记录（前方内容“高能” :funk:，请准备高度精神集中前行）：

PS : 笔试跟面试是一起的，一个技术官拿着一张正反面都写满题的 A4 纸，从第一题到最后一题，他指一题我回答一题，我回答一题，他指下一题。配合的天意无缝，哈哈~~~

1. 笔试

1) java 基础部分

Q : Java 面向对象有哪些特征？

A : 封装、继承、多态。

Q : `short s1=1;s1=s1+1` 有什么错？

`short s1=1;s1+=1` 有什么错？

A : 第一个是有错的，`short` 在内存中占 2 个字节，而整数 1 默认为 `int` 型占 4 个字节，`s1+1` 其实这个时候就向上转型为 `int` 类型了，因此第一行代码必须强转才行。第二个之所以可以是以为这句话翻译过来就是 `s1++`，也就是 `short` 类型的数据自身加增 1，因此不会有问题。

Q : 静态成员类、非静态成员类有什么区别？什么是匿名内部类？

A : 静态成员类相当于外部类的静态成员，是外部类在加载的时候进行初始化，非静态成员类相当于外部类的普通成员，当外部类创建对象的时候才会初始化。匿名内部一般都是在方法里面直接通过 `new ClassName(){};` 形式的类。比如我们 `new Thread ( new Runnable(){} ).start()`；就用到了匿名内部类。

Q : `abstract class` 和 `interface` 有什么区别？

A : 前者是抽象类，可以有抽象方法，也可以没有。后者是接口，只能有抽象方法。他们都不能创建对象，需要被继承。

Q : `ArrayList` 是不是线程安全的？如果不是，如何是 `ArrayList` 成为线程安全的？

A : 不安全的。可以使用 `Collections.synchronizedList(list)` 将 `list` 变为线程安全的。

Q : 是否可以继承 `String` 类？

A : 不可以，因为 String 类是 final 类。为啥不解释了吧。

Q : 以下两条语句返回值为 true 的有：

A : "yiyaowang"=="yiyaowang";

B: "yiyaowang".equals(new String("yiyaowang"));

A : 第一个返回 true，都是字符串常量，存储在字符串常量池中，且只有一份。第二个返回 true，用 equals 比较的是字符串内容。

Q : 当一个对象被当做参数传递到一个方法后，此方法可以改变这个对象的属性，并可返回变化后的结果，那么这里到底是值传递还是引用传递？

A : java 中只有值传递，没有引用传递。这里的引用本身就是值，传递的是引用这个值。

Q : 定义类 A 和类 B 如下：

```
1. class A{
2.     int a =1;
3.     double d = 2.0;
4.     void show(){
5.         System.out.println("Class A:a="+a+"\td="+d);
6.     }
7. }
8. class B extends A{
9.     float a = 3.0f;
10.    String d = "Hello World!";
11.    void show(){
12.        super.show();
13.        System.out.println("Class B:a="+a+"\td="+d);
14.    }
15. }
```

(1)若在应用程序的 main 方法中有以下语句：

A a = new A();

a.show();

则输出结果是？

(2)若在应用程序的 main 方法中定义类 B 的对象 b:

```
A b = new B();
```

```
b.show();
```

则输出结果是？

A : 第一个是 ClassA : a=1 d=2.0

第二个是 ClassA : a =1 d=2.0

ClassB : a=3.0 d=Hello World!

Q : heap 和 stack 有什么区别？

A : 堆和栈。栈存放对象的引用，堆存放对象实体。堆中的对象是有 jvm 的垃圾回收器负责回收。

Q : 请描述下 JVM 加载 class 文件的原理机制。

A : JVM 加载 class 是动态性的，也就是当“需要”的时候才会加载，这也是为节约 JVM 内存来考虑的。同时 JVM 的类加载是父类委托机制，这个机制简单来讲，就是“类装载机有载入类的需求时，会先请示其 Parent 使用其搜索路径帮忙载入，如果 Parent 找不到，那么才由自己依照自己的搜索路径搜索类”。

2) Android 基础部分

Q : 如何适配不同屏幕分辨率的机型？

A : 屏幕适配方式就多了去了。Android 系统本身提供了很多适配方法，比如存放图片资源的 drawable 目录根据不同分辨率的手机提供了 drawable-hdpi、drawable-ldpi、drawable-mdpi、drawable-xhdpi 等多中目录。我们只需把适应不同分辨率的多套图片分别放到对应的目录中即可。Android 的 layout、values 目录也提供了类似 drawable 的适配功能。

但是在开发中，不可能针对不同的手机分辨率提供多种图片资源，这太耗费资源了。我们一般在写控件宽高的时候都会用 dp 单位取代 pix 单位。因为 dp 是一个相对单位，pix 是绝对单位，使用 dp 替代 pix 也可以解决很多适配问题。dp 跟 pix 之间可以通过公式进行转换。

Q : View 和 ViewGroup 的关系是什么？View 的绘制过程（主要方法）有哪些？

A : ViewGroup 继承了 View。onMeasure、onLayout、onDraw。

Q : Activity 和 Task 的区别及启动模式有哪些？

A : Activity 运行 Task 中。Activity 有四种启动模式。standard、singleTop、singleTask、singleInstance。
standard:默认的启动模式，多个 Activity 位于同一 Task 中。singleTop，顾名思义就是 Task 栈顶只能有一个相同的 Activity，singleTask 就是一个 Task 中只有一个 Activity，singleInstance 就是一个 Activity 独享一个 Task。

PS : 关于 Activity 的四种启动模式其实还有更详细的说法，我在上面就简单介绍一下了，如果面试官需要问的更详细再往深处介绍就行了。

Q : 如何注册 BroadcastReceiver 和 Service？Service 有什么特征，哪些情况会用到 Service？

A : 都可以通过 AndroidManifest.xml 进行静态注册。不过 BroadcastReceiver 可以在代码中通过 registerReceiver 方法来注册。Service 运行在后台进程中，一般需要在后台一直运行的任务会让 Service 来完成。比如我们的 telephoneService、locationService 等等。

Q : Android 有哪些安全机制？

A : 权限机制。我们的应用只要涉及到了用户的隐私、网络都需要在 AndroidManifest.xml 中进行声明，这样用户在安装的时候可以根据你申请的权限进行判断是否允许应用的某些行为。

Q : Handler 机制的原理，内部是如何实现的？

A : Android 中 handler 多用于主线程和子线程之间的通信，比如在 Android 中子线程是不允许修改 UI 的，如果修改只能让子线程给主线程通过 handler 发送 message，然后主线程进行修改。Handler 整个机制的实现，还依赖 Looper、Message 两个核心内容。在主线程中 Android 默认给我们创建了 Looper,当我们通过 handler.sendMessage()后,该消息被添加到 MessageQueue 中,Looper.looper

中有个 while(true)的循环不停的从消息队列中取消息。取消息的过程是线程阻塞的，这样不至于在没有消息的时候过多的耗费 CPU 资源。

Q：Thread 和 AsyncTask 的区别是什么？

A：AsyncTask 是封装好的线程池，比起 Thread+Handler 的方式，AsyncTask 在操作 UI 线程上更方便，因为 onPreExecute()、onPostExecute()及更新 UI 方法 onProgressUpdate()均运行在主线程中，这样就不用 Handler 发消息处理了；

Q：说说 MVC 模式的原理，在 Android 中的运用。

A：MVC 是 Model、View、Controller 三部分组成的。其中 View 主要由 xml 布局文件，或者用代码编写动态布局来体现。Model 是数据模型，其实类似 javabean，不过这些 JavaBean 封装了对数据库、网络等的操作。Controller 一般由 Activity 负责，它根据用户的输入，控制用户界面数据的显示及更新 model 对象的状态，它通过控制 View 和 Model 跟用户进行交互。

Q：如何加载 ndk 库？如何在 jni 中注册 native 函数，有几种注册方式？

A：通过 System.loadLibrary("xxx")进行加载。其实 native 有几种注册方式，自己当时并不知道，自己只知道一种注册方法，就是首先根据 native 方法名，生成 Java_com_xxx_MethodName(xxx,xxx); 当然这个 c/c++源码文件中需要引入 jni.h，然后把这个 c/c++源码编译成 so 文件。

自己后来百度了一下，网上有人数还有一种注册方式是动态注册，我就把关于动态注册的东西直接拷贝过来：

JNI 允许你提供一个函数映射表，注册给 Java 虚拟机，这样 Jvm 就可以用函数映射表来调用相应的函数，就可以不必通过函数名来查找需要调用的函数了。Java 与 JNI 通过 JNINativeMethod 的结构来建立联系，它在 jni.h 中被定义，其结构内容如下：

```
1. typedef struct {
2.     const char* name; //Java 中函数的名字
3.     const char* signature; //用字符串描述的函数的参数和返回值
4.     void* fnPtr; //指向 C 函数的函数指针
```



```
5. } JNINativeMethod;
```

第一个变量 name 是 Java 中函数的名字。

第二个变量 signature，用字符串是描述了函数的参数和返回值

第三个变量 fnPtr 是函数指针，指向 C 函数。

当 java 通过 System.loadLibrary 加载完 JNI 动态库后，紧接着会查找一个 JNI_OnLoad 的函数，如果有，就调用它，

而动态注册的工作就是在这里完成的。

1)JNI_OnLoad()函数

JNI_OnLoad()函数在 VM 执行 System.loadLibrary(xxx)函数时被调用，它有两个重要的作用：

指定 JNI 版本：告诉 VM 该组件使用那一个 JNI 版本(若未提供 JNI_OnLoad()函数，VM 会默认该使用最老的 JNI 1.1 版)，如果要使用新版本的 JNI，

例如 JNI 1.4 版，则必须由 JNI_OnLoad()函数返回常量 JNI_VERSION_1_4(该常量定义在 jni.h 中) 来告知 VM。

初始化设定，当 VM 执行到 System.loadLibrary()函数时，会立即先呼叫 JNI_OnLoad()方法，因此在该方法中进行各种资源的初始化操作很恰当，

2)RegisterNatives

RegisterNatives 在 AndroidRunTime 里定义

syntax:

```
jint RegisterNatives(jclass clazz, const JNINativeMethod* methods,jint nMethods)
```

Q：App 在什么情况下会出现内存泄露？如何避免这些情况？

A：造成内存泄露的可能性有很多，我说几种吧，1) 资源未及时释放，比如引用的 io 流资源、网络资源、数据库游标 Cursor 等没有释放 2) 注册的监听器、广播等未及时取消 3) 集合对象没有及时清理 4) 不良代码

避免上述问题，主要还看程序员知识掌握的程度和编码经验的多少，但是从技术角度考虑我们需要注意一些细节，比如，重复使用的资源可以考虑使用缓存技术、池技术。使用的任何资源都记得关闭或者异常处理，保证在恶劣的情况下也能使资源得到释放。对于图片的操作要注意缓存的使用，同时要记住对图片对象进行及时的回收。使用 ListView 的时候，尽量让 convertView 得到复用。

3) 逻辑思考

Q: 你让工人为你工作 7 天，给你工人的回报是一根金条，金条评分成 7 段，你必须在每天结束时给他们一段金条，如果只许你两次把金条弄断，如何给你的工人付费？

PS: 因为上面的问题自己回答的都比较溜，所有这个逻辑思考题，面试官直接说我想着你也会就不做吧。哈哈~面试官已经放弃继续考察我了，嗨皮~~~然后他去找老大去了，趁机我赶紧把笔试题给偷拍了下来（是不是阳哥胆子越来越大呀？！），这样才能大家看到真实的笔试题是什么样子滴。

哦，对了，上面的逻辑思考题我面试的时候是免试的，亲爱的黑马童鞋们你们知道答案吗？

2. 第二轮面试过程问答精选：

旁白: 面试我的应该是开发组部门的老大。胖胖的，黑黑的，矮矮的，从开始面试到结束，一直面带微笑。因为之前那个技术官已经面试过我的技术了，因此他也没问我技术，就是跟我瞎聊了一些关于 Android 方面的东西。因此详细内容这里就省略了，请大家直接进入下一关~duang~

3. 人事面试：

人事面试的问题，其实我在 V4 版本中也说过，基本所有的人事问的问题都大同小异，跟这个人事聊的唯一有趣的就是，由于公司正在装修，找不到面试我的办公地点，然后她竟然把我带到公司旁边的凉亭子下面，我们两个并排坐着，就聊起来了，知道的知道我们是在面试，不知道的估计还以为我们在谈恋爱呢。:#

面试吐槽：

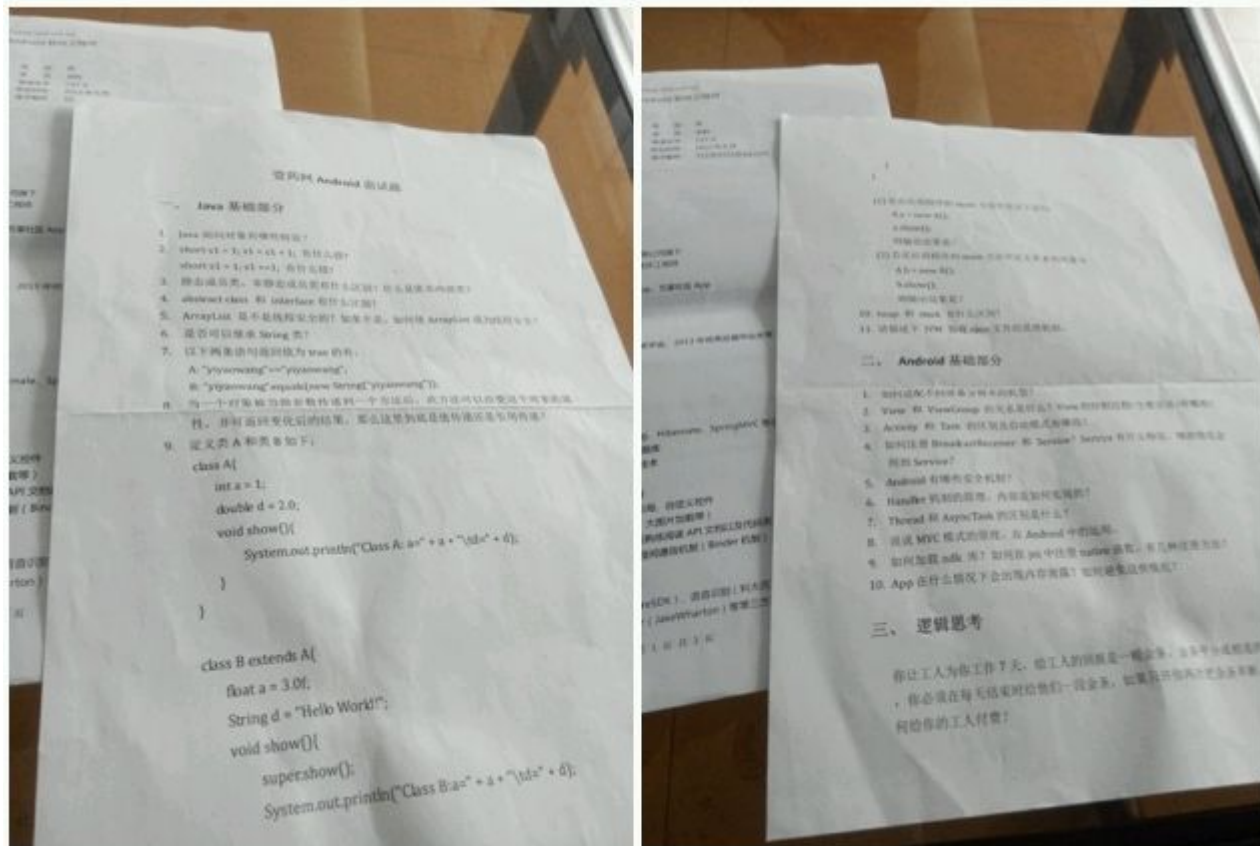
在前台我填写面试登记表的时候写的期望薪资是 15k，他们也没有压低我工资，我知道我要少了。不过，没办法，谁让阳哥不懂行情，在投递简历的时候都是写着期望 15k 呢！想写个 16、17、18 都不好意思了，因此大家以后找工作可别学阳哥这么傻，能多要一定多要。

跟大家分享一个励志语句以结束本篇文章吧：

相信梦想是价值的源泉，相信眼光决定未来的一切，相信成功的信念比成功本身更重要，相信人生有挫折没有失败，相信生命的质量来自决不妥协的信念。



笔试题（在前台眼皮底下偷拍~）



挑战公司 No.6：聚信租赁

公司地址：上海市徐汇区漕溪北路 398 号汇智大厦 28 楼



面试时间：5月14日 14:00 PM.

面试结果：Android 工程师，16K + 入职送 iPhone 6 + 每年出国旅游一次 + 补充住房公积金 + 至少 14

薪:victory:

面试过程：

14:00 到公司前台，领了一大堆资料，然后让我在一间办公室做题。

14:00~14:30 填写面试登记表和性格测试。

14:30~15:20 两个技术官坐在我对面，同时面试我。

15:20~15:30 人事妹子跟我单聊公司薪资福利以及入职事宜。

疯狂的笔试+面试记录（前方内容“高能”：funk:，请准备高度精神集中前行）：

1. 笔试

这家比较奇怪，是给了 6 页的笔试题，打开一看全是性格测试题。性格测试题只拍了一张图片，在该篇文章的结果。估计大多数童鞋都想遇到阳哥这样的狗屎运吧;P~阳哥的性格绝对没问题的，这个我可以保证哈~:P

2. 技术面试

竟然同时来了两个技术官面我，一大一小，一高一矮，并排坐在一起，一对二，好吧，阳哥还是首次遇到 1:2 的阵容，这下可有好戏看了。**PS**：阳哥是个没有见过世面的人，面试的时候上个 31 层高的楼都兴奋的不得了，上去以后，发现上错楼了~糗事说多了都是泪:#，要不是阳哥有着过五关斩六将身经百战的成功经验，估计要吓尿:loveliness:。好吧开始吧，阳哥命中注定必有次劫，看来是躲不过了。

Q：你做一个自我介绍吧？

旁白：其实遇到好几家面试官都让我做自我介绍了，该如何自我介绍阳哥估计都会背了，好玩（**恶心**）的是在万达信息面试，面试了 3 个技术官，每个人都分别让我做了自我介绍，尼玛，他们 3 个就不会沟通一下要问我啥吗，一个问题至于问我 3 遍吗~:funk:阳哥是敢怒不敢言，毕竟在人家的地盘。

PS：自我介绍的内容就不说了，每个人都是独特的，我就跟大家说一下应该如何自我介绍吧。

一个优良的自我介绍会给面试官留下深刻的印象，大部分情况下，所谓的面试好坏其实看的就是你给面试官留下的印象怎么样了，我们用俗语叫感觉。

自我介绍应该分以下几个部分，按照一定的逻辑连贯起来。如果连贯不起来，或者不够熟练一定在台下多背几遍，多讲几遍，但是面试的时候不要说的跟背过似的，高境界就是让面试官感觉你是临场发挥的，却又比背的都好。

1) 个人基本信息（姓名、年龄、老家、居住地等）

2) 自己来自哪里（工作地点），是干什么的（给自己一个清晰的定位，比如：我是一名 Android 开发工程师），担任过什么职务、做过什么样的项目

3) 自己为何来贵公司面试

4) 最后祝愿 (希望能得到贵公司的认可等等, 不用太多, 一两句话就 ok)

Q: 介绍一下你做过的项目吧?

PS: 黑马那么多项目, 随便准备 3 个就 ok 了。

介绍项目大概的思路如下:

1) 这个项目是干什么的 (比如是一个类似网易新闻的地方新闻客户端, 或者类似美团的 o2o, 或者类似豌豆荚的一个应用市场, 或者类似淘宝的购物平台)? 解释就是拿一个市场上耳熟能详的应用跟自己的应用做类比, 省的面试官听的云里雾里的。

2) 自己负责了哪些模块 (功能) 的职责 (比如负责系统的架构, 核心代码的编写, xx 功能模块的开发等等)

3) 自己在这个项目中担当的责任 (比如, 这个项目是自己独立开发的, 这个项目是和另外一个同事一起架构一起开发的, 这个项目是自己负责了几个核心模块)

4) 项目中都用到了哪些技术

5) 从项目中学到了哪些东西 (可以从技术方向和业务两个方向入手)

旁白: 面试官问的很多技术性问题跟之前问的都大同小异, 因此这里只给出有特色且技术含量高的。阳哥正在写面试宝典, 该宝典核心内容针对的还是技术问题, 阳哥会从 javase 基础到 javase 高级, 从 Android 基础到 Android 高级以及到 Android 项目依次展开分析, 其次也会写一些常见的非技术性问题, 敬请期待~

Q: ①在 ListView 的优化中, 我们为何使用 ConvertView? ②为何使用 ViewHolder? ③你认为哪个更能解决问题? ④你认为 view.inflate 和 view.findViewById 哪个更耗时, 为什么? ⑤如果这两个 AP 让你重新写, 你怎么写?

PS :上面的问题，阳哥认为是面试以来遇到很难的一个，也是很有技术含量的一道题。前一半问题还好回答，最后一个问题真的需要发挥想象了。

A :①使用 ConvertView 可以实现对 view 的复用，这样大大节约了每次创建对象的时间，提升了 ListView 的显示效率。②使用 ViewHolder 作为内部类，可以将 view 的子控件封装在 ViewHolder 类中，然后通过 View.setTag(ViewHolder)将 view 和 ViewHolder 进行绑定，这样我们就不用每次都调用 view 的 findViewById(id)方法来查找控件。③使用 ConvertView 解决了一大部分问题，使用 ViewHolder 实现了控件换时间的问题，因为给 View 对象设置一个 Tag 本身就是占用内存的，因此 ViewHolder 的使用还是需要区分不同的应用场景的，没有绝对的好与不好。如果内存足够需要高效则 ViewHolder 建议使用，否则不建议使用。④当然是 view.inflate 耗时，这个函数完成的功能是把 xml 布局文件通过 pullParser 的形式给解析到内存中，需要 io，需要递归子节点。⑤我其实还不太相信我写出来的代码比 Google 官方写的好，如果让我写的话我可能会这样考虑，当用户在使用 view.inflate 的时候将多个 id 作为数组添加到形参中，这样在初始化 view 的使用我就可以给这个 view 直接调用 setTag 方法绑定需要的子控件。不过这个原生方法其实也应该保留共不同的需求使用。

PS :技术面试时间并不长，我回答了几个之后，他们两个大眼瞪小眼，A 看看 B 问：你还有什么问的吗？B 说我没有，你还有吗？A 说我也没了。那行，接下来，他们就让我等人事了。

3. 人事面试：

人事问的问题都差不多，我在上一篇也说过，这里就不说人事的问题了。唯一要给大家爆料的就是人事给我讲的他们公司的福利待遇，可以用土豪不差钱形容:)~

人事说：入职后转正即送 iPhone 6 一部。

我（阳哥）说：我是做 Android 开发的，给我 iPhone 6 干嘛

旁白：你认为阳哥真不想要 iPhone 6 吗？为了显示咱的高度敬业精神，毕竟咱是做 Android 开发的，更需要的是 Android 手机，决不能像 iPhone 低腰（ni dao shi gei wo ya）！

人事说：我们 Android 开发人员也会额外配 Android 机的，iPhone6 可以生活用。

旁白：我去~这就是没见过世面的阳哥，当时的表情。



人事说：转正满一年，每年都有一次出国旅游。

我（阳哥）说：哦，咱们公司还挺不错的嘛！

旁白：不知说啥好了，只能用表情来形容了，阳哥至今没出过国，国外啥个样子嘛，



人事说：我们一般一年更少发 14 薪，都是介于 14~16 薪之间，具体发多少要看个人平时的一个 KPI 考核了。

我（阳哥）说：哦，这样呀，还行吧。



面试吐槽：

在家公司的人事跟我算了基本工资 16k+各种补助 1k=17k+。每年出国旅游，入职送 iPhone，补充住房公积金，好吧，阳哥受宠若惊了。自己回来会百度了一下，终于搞明白这家公司为何这么奢侈！不多说，看图吧。

聚信租赁获13亿银团贷款 IPO仍在排队中

2015-01-08 17:26:42 来源: 网易财经



16

网易财经1月8日讯 今日，聚信国际租赁股份有限公司（以下简称“**聚信租赁**”）在上海召开发布会，正式与农行上海静安支行、建行上海分行、交行上海分行以及南京银行上海分行共同签订13亿元银团贷款协议。

去年10月17日聚信租赁正式发布招股说明书预披露，目前公司仍处于排队审核，等待上市中。公告称，公司拟赴上海证券交易所上市，拟发行新股10000万股，包括公开发行的新股及公司股东公开发售的股份。发行后总股本为44285.71万股，预计融资额12.5亿元。募集资金在扣除发行费用后，将用于充实发行人净资产，补充自有资金以支持未来业务发展。

据介绍，本次银团贷款项目融资由农行上海静安支行作为牵头行，建行上海分行和交行

跟人事面试的时候，人事说他们在上海总部目前有大概 100 多名员工，阳哥不懂经济，不知道 100 多人的公司能融资 13 个亿是个什么样的概念？这再次验证了阳哥的那句千古流传的话语，只要技术好，公司不差钱！

聚信租赁
JUXIN LEASING

应聘登记表

部门及职位: _____ 可到职日期: _____

个人概况: 本表所有内容为个人信息, 我们将为您严格保密, 所有内容为必填项。

性别:	出生日期:	民族:	身份证号码:
☐ 未婚 ☐ 已婚 ☐ 其他			
户口所在地:	籍贯:	E-mail 地址:	
家庭住址:	邮编:		
户口地址:	邮编:		
现所在地:			
固定电话:	紧急联系人:		
手机:	紧急联系人电话:		
特长:			
语种和读写程度)			
使用的软件)			

性格测试

指导语:
本问卷中的所有问题都和你下人们的生活, 婚姻状况以及婚姻满意度密切相关, 所有问题都至关重要, 请认真作答, 不要有任何顾虑, 并选择一个答案, 把所选答案的序号写在右边的答案纸上。答题时不必对每道题都加考虑, 只要按你的第一印象进行作答即可。
请注意: 每道题都加考虑, 只选一个答案, 不要多选。同时, 测试中有两道题, 如果答案没有选项, 则请手写答案。
声明: 请如实填写您的基本信息, 我们对此保密!

1. 我倾向于从何处得到力量:
(E) 别人。
(D) 我自己的思想。

2. 当我参加一个社交聚会时, 我倾向于有更多的力气:
(E) 在夜色很深时, 一旦我开始投入, 也许就是最晚离开的那一个。
(D) 在夜刚开始的时候, 我就疲倦了并且想回家。

3. 下列哪一种听起来比较吸引人?
(E) 与我的情人到有很多人且社交活动频繁的地方。
(D) 待在家中与我的情人做一些特别的事情, 例如说观赏一部有趣的录像带并享用我带食物。

4. 在约会中, 我通常:
(E) 整体来说蛮健谈的。
(D) 较安静并保留, 直到我觉得舒服。

5. 过去, 我倾向于遇见我大部分情人:
(E) 在宴会中, 夜总会, 工作上, 休闲活动中, 会议上, 或当朋友介绍我给过私人的方式, 例如个人广告, 录像约会, 或是由亲密的朋友和家人

附录：更新记录

时间	类型	操作人	备注
2015 年 5 月 20 号	创建文档	王震阳	新建
2015 年 8 月 7 号	完成文档	同上	第一版收尾
2015 年 8 月 8 号	公布网络	同上	地址： http://bbs.itheima.com/thread-223247-1-1.html
2015 年 10 月 13 号	添加试题	同上	改动较大，以此版为准
2015 年 11 月 24 号	添加 20 余道上海 Android2 期学生面试题	同上	改动很大升级版本号为 Version 2.0
2015 年 11 月 25 号	添加面试实战记录	同上	新添加面试实战内容
2015 年 11 月 26 号	添加项目技能点	同上	新添加 Android 项目知识
2015 年 11 月 29 号	添加开源框架原理	高大冬、徐鑫	修改版本号为 Version 2.5

至此，本文档完！

2015 年 11 月 26 日 星期四 小雨

上海市闵行区联航路 1588 号坤大科技楼 5 层 501